PennOS

Fall 2025

Generated by Doxygen 1.9.1

1 Data Structure Index 1

1.1 Data Structures . . . 1

2 File Index 1

2.1 File List . . . 1

3 Data Structure Documentation 3

3.1 busy_args_t Struct Reference . . . 3

3.1.1 Detailed Description . . . 3

3.1.2 Field Documentation . . . 4

3.2 directory_entry_t Struct Reference . . . 4

3.2.1 Detailed Description . . . 4

3.2.2 Field Documentation . . . 5

3.3 global_fd_entry_t Struct Reference . . . 6

3.3.1 Detailed Description . . . 7

3.3.2 Field Documentation . . . 7

3.4 parsed_command Struct Reference . . . 8

3.4.1 Detailed Description . . . 9

3.4.2 Field Documentation . . . 9

3.5 pcb Struct Reference . . . 10

3.5.1 Detailed Description . . . 11

3.5.2 Field Documentation . . . 11

3.6 spawn_args_t Struct Reference . . . 16

3.6.1 Detailed Description . . . 16

3.6.2 Field Documentation . . . 16

3.7 spthread_fwd_args_st Struct Reference . . . 17

3.7.1 Detailed Description . . . 17

3.7.2 Field Documentation . . . 17

3.8 spthread_meta_st Struct Reference . . . 19

3.8.1 Detailed Description . . . 19

3.8.2 Field Documentation . . . 19

3.9 spthread_signal_args_st Struct Reference . . . 20

3.9.1 Detailed Description . . . 20

3.9.2 Field Documentation . . . 20

3.10 spthread_st Struct Reference . . . 21

3.10.1 Detailed Description . . . 21

3.10.2 Field Documentation . . . 21

3.11 vec_st Struct Reference . . . 22

3.11.1 Detailed Description . . . 22

3.11.2 Field Documentation . . . 22

4 File Documentation **23**

4.1 src/FAT/commands.c File Reference . . . 23
4.1.1 Function Documentation . . . 24
4.1.2 Variable Documentation . . . 27
4.2 src/kernel/commands.c File Reference . . . 29
4.2.1 Function Documentation . . . 30
4.2.2 Variable Documentation . . . 47
4.3 src/FAT/commands.h File Reference . . . 48
4.3.1 Detailed Description . . . 49
4.3.2 Function Documentation . . . 49
4.3.3 Variable Documentation . . . 52
4.4 src/kernel/commands.h File Reference . . . 54
4.4.1 Function Documentation . . . 55
4.4.2 Variable Documentation . . . 77
4.5 src/FAT/fat_structs.h File Reference . . . 78
4.5.1 Detailed Description . . . 79
4.5.2 Macro Definition Documentation . . . 79
4.6 src/FAT/fd_table.c File Reference . . . 83
4.6.1 Function Documentation . . . 84
4.6.2 Variable Documentation . . . 85
4.7 src/FAT/fd_table.h File Reference . . . 86
4.7.1 Detailed Description . . . 87
4.7.2 Macro Definition Documentation . . . 87
4.7.3 Function Documentation . . . 88
4.7.4 Variable Documentation . . . 90
4.8 src/FAT/k_functions.c File Reference . . . 90
4.8.1 Function Documentation . . . 91
4.9 src/FAT/k_functions.h File Reference . . . 110
4.9.1 Function Documentation . . . 112
4.10 src/FAT/s_functions.c File Reference . . . 130
4.10.1 Function Documentation . . . 131
4.11 src/FAT/s_functions.h File Reference . . . 143
4.11.1 Function Documentation . . . 143
4.12 src/FAT/shell_commands.c File Reference . . . 151
4.12.1 Function Documentation . . . 152
4.13 src/FAT/shell_commands.h File Reference . . . 161
4.13.1 Detailed Description . . . 162
4.13.2 Function Documentation . . . 162
4.14 src/kernel/init.c File Reference . . . 170

4.14.1 Function Documentation . . . 170
4.14.2 Variable Documentation . . . 173
4.15 src/kernel/init.h File Reference . . . 173
4.15.1 Function Documentation . . . 174
4.16 src/kernel/kernel.c File Reference . . . 176
4.16.1 Function Documentation . . . 177
4.16.2 Variable Documentation . . . 193
4.17 src/kernel/kernel.h File Reference . . . 194
4.17.1 Macro Definition Documentation . . . 195
4.17.2 Function Documentation . . . 196
4.18 src/kernel/logging.c File Reference . . . 212
4.18.1 Function Documentation . . . 213
4.18.2 Variable Documentation . . . 225
4.19 src/kernel/logging.h File Reference . . . 226
4.19.1 Function Documentation . . . 227
4.20 src/kernel/pcb.c File Reference . . . 240
4.20.1 Function Documentation . . . 241
4.21 src/kernel/pcb.h File Reference . . . 249
4.21.1 Macro Definition Documentation . . . 251
4.21.2 Typedef Documentation . . . 252
4.21.3 Enumeration Type Documentation . . . 252
4.21.4 Function Documentation . . . 253
4.22 src/kernel/scheduler.c File Reference . . . 262
4.22.1 Macro Definition Documentation . . . 264
4.22.2 Function Documentation . . . 264
4.22.3 Variable Documentation . . . 274
4.23 src/kernel/scheduler.h File Reference . . . 275
4.23.1 Macro Definition Documentation . . . 276
4.23.2 Function Documentation . . . 276
4.24 src/kernel/shell.c File Reference . . . 285
4.24.1 Function Documentation . . . 285
4.25 src/kernel/shell.h File Reference . . . 292
4.25.1 Function Documentation . . . 292
4.26 src/kernel/syscall.c File Reference . . . 294
4.26.1 Function Documentation . . . 295
4.27 src/kernel/syscall.h File Reference . . . 304
4.27.1 Function Documentation . . . 304
4.28 src/kernel/user_processes.c File Reference . . . 313
4.28.1 Function Documentation . . . 314

4.29 src/kernel/user_processes.h File Reference . . . 333
4.29.1 Function Documentation . . . 335
4.30 src/parser/parser.c File Reference . . . 356
4.30.1 Macro Definition Documentation . . . 357
4.30.2 Function Documentation . . . 357
4.31 src/parser/parser.h File Reference . . . 361
4.31.1 Macro Definition Documentation . . . 363
4.31.2 Function Documentation . . . 364
4.32 src/pennfat.c File Reference . . . 367
4.32.1 Macro Definition Documentation . . . 367
4.32.2 Function Documentation . . . 368
4.33 src/pennos.c File Reference . . . 370
4.33.1 Function Documentation . . . 371
4.33.2 Variable Documentation . . . 372
4.34 src/utils/input_processing.c File Reference . . . 373
4.34.1 Macro Definition Documentation . . . 373
4.34.2 Function Documentation . . . 374
4.35 src/utils/input_processing.h File Reference . . . 374
4.35.1 Function Documentation . . . 375
4.36 src/utils/panic.c File Reference . . . 376
4.36.1 Function Documentation . . . 377
4.37 src/utils/panic.h File Reference . . . 377
4.37.1 Macro Definition Documentation . . . 378
4.37.2 Function Documentation . . . 378
4.38 src/utils/redirection.c File Reference . . . 378
4.38.1 Function Documentation . . . 379
4.39 src/utils/redirection.h File Reference . . . 380
4.39.1 Function Documentation . . . 381
4.40 src/utils/spthread.c File Reference . . . 382
4.40.1 Macro Definition Documentation . . . 383
4.40.2 Typedef Documentation . . . 384
4.40.3 Function Documentation . . . 385
4.40.4 Variable Documentation . . . 395
4.41 src/utils/spthread.h File Reference . . . 396
4.41.1 Macro Definition Documentation . . . 397
4.41.2 Typedef Documentation . . . 397
4.41.3 Function Documentation . . . 397
4.42 src/utils/Vec.c File Reference . . . 405
4.42.1 Function Documentation . . . 406

4.43 src/utils/Vec.h File Reference 412
4.43.1 Macro Definition Documentation 413
4.43.2 Typedef Documentation 413
4.43.3 Function Documentation 414

Index **421**

1 Data Structure Index

1.1 Data Structures

Here are the data structures with brief descriptions:

busy_args_t
Argument structure for busy process **3**

directory_entry_t
Directory entry structure (64 bytes) **4**

global_fd_entry_t
Global file descriptor entry **6**

parsed_command **8**

pcb
Process Control Block (PCB) **10**

spawn_args_t
Wrapper structure for passing arguments to spawned thread **16**

spthread_fwd_args_st **17**

spthread_meta_st **19**

spthread_signal_args_st **20**

spthread_st **21**

vec_st **22**

2 File Index

2.1 File List

Here is a list of all files with brief descriptions:

src/pennfat.c **367**

src/pennos.c 370

src/FAT/commands.c 23

src/FAT/commands.h
PennFAT filesystem commands and global state 48

src/FAT/fat_structs.h
PennFAT filesystem data structures and constants 78

src/FAT/fd_table.c 83

src/FAT/fd_table.h
Global file descriptor table for PennFAT 86

src/FAT/k_functions.c 90

src/FAT/k_functions.h 110

src/FAT/s_functions.c 130

src/FAT/s_functions.h 143

src/FAT/shell_commands.c 151

src/FAT/shell_commands.h
PennFAT shell commands 161

src/kernel/commands.c 29

src/kernel/commands.h 54

src/kernel/init.c 170

src/kernel/init.h 173

src/kernel/kernel.c 176

src/kernel/kernel.h 194

src/kernel/logging.c 212

src/kernel/logging.h 226

src/kernel/pcb.c 240

src/kernel/pcb.h 249

src/kernel/scheduler.c 262

src/kernel/scheduler.h 275

src/kernel/shell.c 285

src/kernel/shell.h 292

src/kernel/syscall.c 294

src/kernel/syscall.h 304

src/kernel/user_processes.c 313

src/kernel/user_processes.h 333

src/parser/parser.c 356

src/parser/parser.h 361

src/utils/input_processing.c 373

src/utils/input_processing.h 374

src/utils/panic.c 376

src/utils/panic.h 377

src/utils/redirection.c 378

src/utils/redirection.h 380

src/utils/spthread.c 382

src/utils/spthread.h 396

src/utils/Vec.c 405

src/utils/Vec.h 412

3 Data Structure Documentation

3.1 busy_args_t Struct Reference

Argument structure for busy process.

`#include <user_processes.h>`

Data Fields

- char name [64]
- bool is_background

3.1.1 Detailed Description

Argument structure for busy process.

Contains the process name and background flag for the busy command.

Definition at line 12 of file user_processes.h.

3.1.2 Field Documentation

3.1.2.1 is_background `bool busy_args_t::is_background`

Whether process runs in background

Definition at line 14 of file user_processes.h.

3.1.2.2 name `char busy_args_t::name[64]`

Name of the busy process

Definition at line 13 of file user_processes.h.

The documentation for this struct was generated from the following file:

- src/kernel/user_processes.h

3.2 directory_entry_t Struct Reference

Directory entry structure (64 bytes)

`#include <fat_structs.h>`

Data Fields

- char name [32]
- uint32_t size
- uint16_t firstBlock
- uint8_t type
- uint8_t perm
- time_t mtime
- char reserved [16]

3.2.1 Detailed Description

Directory entry structure (64 bytes)

Represents a file's metadata in the root directory. Each directory occupies exactly one block and contains fixed-size entries.

Definition at line 67 of file fat_structs.h.

3.2.2 Field Documentation

3.2.2.1 firstBlock `uint16_t directory_entry_t::firstBlock`

First block in FAT chain (0xFFFF if empty)

Definition at line 70 of file fat_structs.h.

Referenced by create_directory_entry_at_block(), k_close(), k_open(), k_unlink(), k_write(), and print_directory_entry().

3.2.2.2 mtime `time_t directory_entry_t::mtime`

Last modification time (Unix timestamp)

Definition at line 73 of file fat_structs.h.

Referenced by create_directory_entry_at_block(), k_close(), k_open(), k_write(), and print_directory_entry().

3.2.2.3 name `char directory_entry_t::name[32]`

Filename (null-terminated)

Definition at line 68 of file fat_structs.h.

Referenced by create_directory_entry_at_block(), k_unlink(), mkfs(), mv(), print_directory_entry(), and user_mv().

3.2.2.4 perm `uint8_t directory_entry_t::perm`

Permission bits

Definition at line 72 of file fat_structs.h.

Referenced by cmd_chmod(), create_directory_entry_at_block(), execute_command(), k_open(), print_directory_entry(), and user_chmod().

3.2.2.5 reserved `char directory_entry_t::reserved[16]`

Reserved for future use (padding to 64 bytes)

Definition at line 74 of file fat_structs.h.

3.2.2.6 size `uint32_t directory_entry_t::size`

File size in bytes

Definition at line 69 of file fat_structs.h.

Referenced by create_directory_entry_at_block(), execute_command(), k_close(), k_open(), k_write(), and print_directory_entry().

3.2.2.7 type `uint8_t directory_entry_t::type`

File type (TYPE_REGULAR, TYPE_DIRECTORY, etc.)

Definition at line 71 of file fat_structs.h.

Referenced by create_directory_entry_at_block(), and print_directory_entry().

The documentation for this struct was generated from the following file:

- src/FAT/fat_structs.h

3.3 global_fd_entry_t Struct Reference

Global file descriptor entry.

`#include <fd_table.h>`

Data Fields

- char filename [32]
- int mode
- off_t offset
- int refcount
- uint16_t first_block
- uint32_t size
- uint8_t perm

3.3.1 Detailed Description

Global file descriptor entry.

Represents an open file in the system. Shared across all processes via reference counting. When a process forks, child inherits parent's FD table, incrementing refcounts.

Definition at line 37 of file fd_table.h.

3.3.2 Field Documentation

3.3.2.1 filename `char global_fd_entry_t::filename[32]`

Name of the open file

Definition at line 38 of file fd_table.h.

Referenced by k_open().

3.3.2.2 first_block `uint16_t global_fd_entry_t::first_block`

First block in FAT chain

Definition at line 42 of file fd_table.h.

Referenced by k_close(), k_open(), k_read(), and k_write().

3.3.2.3 mode `int global_fd_entry_t::mode`

Open mode (F_WRITE, F_READ, F_APPEND)

Definition at line 39 of file fd_table.h.

Referenced by k_open().

3.3.2.4 offset `off_t global_fd_entry_t::offset`

Current file position for read/write

Definition at line 40 of file fd_table.h.

Referenced by k_lseek(), k_open(), k_read(), and k_write().

3.3.2.5 perm `uint8_t global_fd_entry_t::perm`

File permissions

Definition at line 44 of file fd_table.h.

Referenced by k_open().

3.3.2.6 refcount `int global_fd_entry_t::refcount`

Number of processes with this FD open

Definition at line 41 of file fd_table.h.

Referenced by decrement_refcount(), increment_refcount(), init_global_fd_table(), k_close(), and k_open().

3.3.2.7 size `uint32_t global_fd_entry_t::size`

Current file size in bytes

Definition at line 43 of file fd_table.h.

Referenced by k_close(), k_lseek(), k_open(), k_read(), and k_write().

The documentation for this struct was generated from the following file:

- src/FAT/fd_table.h

3.4 parsed_command Struct Reference

`#include <parser.h>`

Data Fields

- bool is_background
- bool is_file_append
- const char * stdin_file
- const char * stdout_file
- size_t num_commands
- char ** commands [ ]

3.4.1 Detailed Description

struct parsed_command stored all necessary information needed for penn-shell.

Definition at line 36 of file parser.h.

3.4.2 Field Documentation

3.4.2.1 commands `char** parsed_command::commands[]`

Definition at line 56 of file parser.h.

Referenced by execute_command(), main(), parse_command(), and print_parsed_command().

3.4.2.2 is_background `bool parsed_command::is_background`

Definition at line 39 of file parser.h.

Referenced by execute_command(), parse_command(), and print_parsed_command().

3.4.2.3 is_file_append `bool parsed_command::is_file_append`

Definition at line 43 of file parser.h.

Referenced by execute_command(), get_redirected_fds(), parse_command(), and print_parsed_command().

3.4.2.4 num_commands `size_t parsed_command::num_commands`

Definition at line 52 of file parser.h.

Referenced by parse_command(), and print_parsed_command().

3.4.2.5 stdin_file `const char* parsed_command::stdin_file`

Definition at line 46 of file parser.h.

Referenced by get_redirected_fds(), parse_command(), and print_parsed_command().

3.4.2.6 stdout_file `const char* parsed_command::stdout_file`

Definition at line 49 of file parser.h.

Referenced by execute_command(), get_redirected_fds(), parse_command(), and print_parsed_command().

The documentation for this struct was generated from the following file:

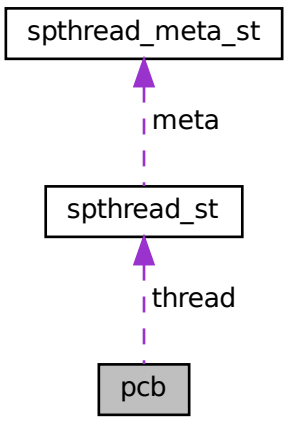
- src/parser/parser.h

3.5 pcb Struct Reference

Process Control Block (PCB)

`#include <pcb.h>`

Collaboration diagram for pcb:



Data Fields

- spthread_t thread
- pid_t pid
- pid_t ppid
- char name [256]
- int priority
- process_state_t state
- int exit_status
- pid_t children [128]
- int num_children
- pid_t zombie_children [128]
- int exit_statuses [128]
- int num_zombies
- pid_t stopped_children [128]
- int num_stopped
- int pending_signal
- unsigned int sleep_ticks_remaining
- int fd_table [MAX_FDS]
- bool is_waiting
- pid_t waiting_for_pid
- bool thread_created
- bool is_background

3.5.1 Detailed Description

Process Control Block (PCB)

Contains all state information for a single process in PennOS. Each process has a unique PID, parent PID, priority level, and tracks its children, zombies, and stopped children for proper process lifecycle management.

Definition at line 51 of file pcb.h.

3.5.2 Field Documentation

3.5.2.1 children `pid_t pcb::children[128]`

Array of living child PIDs

Definition at line 62 of file pcb.h.

Referenced by cmd_bg(), cmd_fg(), cmd_info(), cmd_jobs(), k_reparent_children_to_init(), pcb_add_child(), pcb_↩ remove_child(), s_waitpid(), and shell_main().

3.5.2.2 exit_status `int pcb::exit_status`

Exit status (encoded for s_waitpid)

Definition at line 60 of file pcb.h.

Referenced by cmd_info(), k_handle_pending_signals(), pcb_create(), and s_exit().

3.5.2.3 exit_statuses `int pcb::exit_statuses[128]`

Exit statuses of zombie children

Definition at line 66 of file pcb.h.

Referenced by cmd_info(), k_reparent_children_to_init(), pcb_add_zombie(), and pcb_get_zombie().

3.5.2.4 fd_table `int pcb::fd_table[MAX_FDS]`

File descriptor table

Definition at line 75 of file pcb.h.

Referenced by k_proc_create(), pcb_create(), s_close(), s_lseek(), s_open(), s_read(), s_spawn(), s_unlink(), s_write(), and user_echo().

3.5.2.5 is_background `bool pcb::is_background`

True if started as background job

Definition at line 81 of file pcb.h.

Referenced by cmd_bg(), cmd_fg(), cmd_jobs(), execute_command(), and pcb_create().

3.5.2.6 is_waiting `bool pcb::is_waiting`

True if blocked in s_waitpid

Definition at line 77 of file pcb.h.

Referenced by cmd_info(), k_notify_parent(), pcb_create(), and s_waitpid().

3.5.2.7 name `char pcb::name[256]`

Process name (from argv[0])

Definition at line 56 of file pcb.h.

Referenced by cmd_bg(), cmd_fg(), cmd_info(), cmd_jobs(), k_handle_pending_signals(), k_notify_parent(), k_↩ reparent_children_to_init(), main(), pcb_create(), ps_process(), s_exit(), s_nice(), s_sleep(), s_spawn(), s_waitpid(), scheduler(), and scheduler_insert().

3.5.2.8 num_children `int pcb::num_children`

Number of living children

Definition at line 63 of file pcb.h.

Referenced by cmd_bg(), cmd_fg(), cmd_info(), cmd_jobs(), k_handle_pending_signals(), k_proc_cleanup(), k_↩ reparent_children_to_init(), pcb_add_child(), pcb_create(), pcb_remove_child(), s_exit(), s_waitpid(), and shell_main().

3.5.2.9 num_stopped `int pcb::num_stopped`

Number of stopped children

Definition at line 70 of file pcb.h.

Referenced by cmd_bg(), cmd_info(), pcb_add_stopped_child(), pcb_create(), pcb_remove_stopped_child(), and s_↩ waitpid().

3.5.2.10 num_zombies `int pcb::num_zombies`

Number of zombie children

Definition at line 67 of file pcb.h.

Referenced by cmd_info(), k_handle_pending_signals(), k_proc_cleanup(), k_reparent_children_to_init(), pcb_add_↩ zombie(), pcb_create(), pcb_get_zombie(), s_exit(), and s_waitpid().

3.5.2.11 pending_signal `int pcb::pending_signal`

Pending signal (-1 if none, else P_SIG∗)

Definition at line 72 of file pcb.h.

Referenced by cmd_info(), k_handle_pending_signals(), pcb_create(), and s_kill().

3.5.2.12 pid `pid_t pcb::pid`

Process ID (unique identifier)

Definition at line 54 of file pcb.h.

Referenced by cmd_bg(), cmd_fg(), k_add_process(), k_handle_pending_signals(), k_notify_parent(), k_proc_↩ cleanup(), k_proc_create(), k_reparent_children_to_init(), main(), pcb_add_child(), pcb_add_stopped_child(), pcb↩ _add_zombie(), pcb_create(), ps_process(), s_exit(), s_nice(), s_sleep(), s_spawn(), s_waitpid(), scheduler(), scheduler_insert(), and scheduler_remove().

3.5.2.13 ppid `pid_t pcb::ppid`

Parent process ID

Definition at line 55 of file pcb.h.

Referenced by cmd_info(), k_handle_pending_signals(), k_proc_cleanup(), k_reparent_children_to_init(), pcb_create(), ps_process(), and s_exit().

3.5.2.14 priority `int pcb::priority`

Priority level: 0 (high), 1 (medium), 2 (low)

Definition at line 58 of file pcb.h.

Referenced by cmd_info(), k_handle_pending_signals(), k_notify_parent(), k_reparent_children_to_init(), main(), pcb↩ _create(), ps_process(), s_exit(), s_nice(), s_sleep(), s_spawn(), s_waitpid(), scheduler(), scheduler_insert(), and scheduler_remove().

3.5.2.15 sleep_ticks_remaining `unsigned int pcb::sleep_ticks_remaining`

Remaining sleep ticks (0 if not sleeping)

Definition at line 73 of file pcb.h.

Referenced by cmd_info(), pcb_create(), s_sleep(), and scheduler().

3.5.2.16 state process_state_t pcb::state

Current process state

Definition at line 59 of file pcb.h.

Referenced by cmd_bg(), cmd_fg(), cmd_info(), cmd_jobs(), cmd_zombify(), k_handle_pending_signals(), k_notify_↩ parent(), main(), pcb_create(), ps_process(), s_exit(), s_sleep(), s_waitpid(), scheduler(), and scheduler_insert().

3.5.2.17 stopped_children pid_t pcb::stopped_children[128]

Array of stopped child PIDs

Definition at line 69 of file pcb.h.

Referenced by cmd_bg(), cmd_info(), pcb_add_stopped_child(), pcb_remove_stopped_child(), and s_waitpid().

3.5.2.18 thread spthread_t pcb::thread

Thread handle for this process

Definition at line 52 of file pcb.h.

Referenced by k_proc_cleanup(), main(), s_sleep(), s_spawn(), s_waitpid(), scheduler(), and scheduler_cleanup().

3.5.2.19 thread_created bool pcb::thread_created

True if thread has been created

Definition at line 80 of file pcb.h.

Referenced by k_proc_cleanup(), and main().

3.5.2.20 waiting_for_pid pid_t pcb::waiting_for_pid

PID being waited for (-1 for any child)

Definition at line 78 of file pcb.h.

Referenced by cmd_info(), k_notify_parent(), pcb_create(), and s_waitpid().

3.5.2.21 zombie_children `pid_t pcb::zombie_children[128]`

Array of zombie child PIDs

Definition at line 65 of file pcb.h.

Referenced by cmd_info(), k_reparent_children_to_init(), pcb_add_zombie(), pcb_get_zombie(), and s_waitpid().

The documentation for this struct was generated from the following file:

- src/kernel/pcb.h

3.6 spawn_args_t Struct Reference

Wrapper structure for passing arguments to spawned thread.

Data Fields

- void ∗(∗ func)(void ∗)
- char ∗∗ argv
- pid_t pid

3.6.1 Detailed Description

Wrapper structure for passing arguments to spawned thread.

Definition at line 14 of file syscall.c.

3.6.2 Field Documentation

3.6.2.1 argv `char** spawn_args_t::argv`

Definition at line 16 of file syscall.c.

Referenced by s_spawn(), and spawn_wrapper().

3.6.2.2 func `void*(* spawn_args_t::func) (void *)`

Definition at line 15 of file syscall.c.

Referenced by s_spawn(), and spawn_wrapper().

3.6.2.3 pid `pid_t spawn_args_t::pid`

Definition at line 17 of file syscall.c.

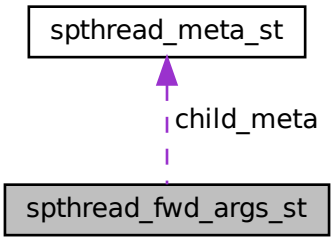
Referenced by s_spawn(), and spawn_wrapper().

The documentation for this struct was generated from the following file:

- src/kernel/syscall.c

3.7 spthread_fwd_args_st Struct Reference

Collaboration diagram for spthread_fwd_args_st:



Data Fields

- pthread_fn actual_routine
- void * actual_arg
- bool setup_done
- pthread_mutex_t setup_mutex
- pthread_cond_t setup_cond
- spthread_meta_t * child_meta

3.7.1 Detailed Description

Definition at line 26 of file spthread.c.

3.7.2 Field Documentation

3.7.2.1 actual_arg `void* spthread_fwd_args_st::actual_arg`

Definition at line 31 of file spthread.c.

Referenced by spthread_start().

3.7.2.2 actual_routine `pthread_fn spthread_fwd_args_st::actual_routine`

Definition at line 30 of file spthread.c.

Referenced by spthread_create(), and spthread_start().

3.7.2.3 child_meta `spthread_meta_t* spthread_fwd_args_st::child_meta`

Definition at line 44 of file spthread.c.

Referenced by spthread_start().

3.7.2.4 setup_cond `pthread_cond_t spthread_fwd_args_st::setup_cond`

Definition at line 41 of file spthread.c.

Referenced by spthread_create(), and spthread_start().

3.7.2.5 setup_done `bool spthread_fwd_args_st::setup_done`

Definition at line 39 of file spthread.c.

Referenced by spthread_create(), and spthread_start().

3.7.2.6 setup_mutex `pthread_mutex_t spthread_fwd_args_st::setup_mutex`

Definition at line 40 of file spthread.c.

Referenced by spthread_create(), and spthread_start().

The documentation for this struct was generated from the following file:

- src/utils/spthread.c

3.8 spthread_meta_st Struct Reference

Data Fields

- sigset_t suspend_set
- volatile sig_atomic_t state
- pthread_mutex_t meta_mutex

3.8.1 Detailed Description

Definition at line 61 of file spthread.c.

3.8.2 Field Documentation

3.8.2.1 meta_mutex `pthread_mutex_t spthread_meta_st::meta_mutex`

Definition at line 75 of file spthread.c.

Referenced by spthread_join(), and spthread_start().

3.8.2.2 state `volatile sig_atomic_t spthread_meta_st::state`

Definition at line 72 of file spthread.c.

Referenced by mark_self_terminated(), sigpthd_handler(), spthread_continue(), spthread_start(), spthread_suspend(), and spthread_suspend_self().

3.8.2.3 suspend_set `sigset_t spthread_meta_st::suspend_set`

Definition at line 63 of file spthread.c.

Referenced by sigpthd_handler(), spthread_start(), and spthread_suspend_self().

The documentation for this struct was generated from the following file:

- src/utils/spthread.c

3.9 spthread_signal_args_st Struct Reference

Data Fields

- const int signal
- volatile sig_atomic_t ack
- pthread_mutex_t shutup_mutex

3.9.1 Detailed Description

Definition at line 50 of file spthread.c.

3.9.2 Field Documentation

3.9.2.1 ack `volatile sig_atomic_t spthread_signal_args_st::ack`

Definition at line 52 of file spthread.c.

Referenced by sigpthd_handler(), spthread_continue(), and spthread_suspend().

3.9.2.2 shutup_mutex `pthread_mutex_t spthread_signal_args_st::shutup_mutex`

Definition at line 53 of file spthread.c.

Referenced by sigpthd_handler(), spthread_continue(), and spthread_suspend().

3.9.2.3 signal `const int spthread_signal_args_st::signal`

Definition at line 51 of file spthread.c.

Referenced by sigpthd_handler(), spthread_continue(), and spthread_suspend().

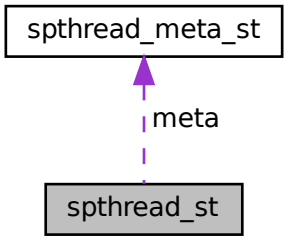
The documentation for this struct was generated from the following file:

- src/utils/spthread.c

3.10 spthread_st Struct Reference

`#include <spthread.h>`

Collaboration diagram for spthread_st:



Data Fields

- pthread_t thread
- spthread_meta_t * meta

3.10.1 Detailed Description

Definition at line 28 of file spthread.h.

3.10.2 Field Documentation

3.10.2.1 meta `spthread_meta_t* spthread_st::meta`

Definition at line 30 of file spthread.h.

Referenced by spthread_continue(), spthread_equal(), spthread_join(), and spthread_suspend().

3.10.2.2 thread `pthread_t spthread_st::thread`

Definition at line 29 of file spthread.h.

Referenced by spthread_cancel(), spthread_continue(), spthread_create(), spthread_equal(), spthread_join(), spthread_self(), and spthread_suspend().

The documentation for this struct was generated from the following file:

- src/utils/spthread.h

3.11 vec_st Struct Reference

`#include <Vec.h>`

Data Fields

- ptr_t * data
- size_t length
- size_t capacity
- ptr_dtor_fn ele_dtor_fn

3.11.1 Detailed Description

Definition at line 10 of file Vec.h.

3.11.2 Field Documentation

3.11.2.1 capacity `size_t vec_st::capacity`

Definition at line 13 of file Vec.h.

Referenced by vec_new().

3.11.2.2 data `ptr_t* vec_st::data`

Definition at line 11 of file Vec.h.

Referenced by vec_new().

3.11.2.3 ele_dtor_fn `ptr_dtor_fn vec_st::ele_dtor_fn`

Definition at line 14 of file Vec.h.

Referenced by vec_new().

3.11.2.4 length `size_t vec_st::length`

Definition at line 12 of file Vec.h.

Referenced by vec_new().

The documentation for this struct was generated from the following file:

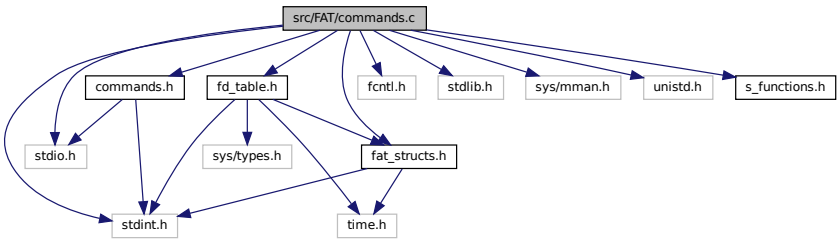
- src/utils/Vec.h

4 File Documentation

4.1 src/FAT/commands.c File Reference

```
#include "commands.h"
#include <fcntl.h>
#include <stdint.h>
#include <stdio.h>
#include <stdlib.h>
#include <sys/mman.h>
#include <unistd.h>
#include "fat_structs.h"
#include "fd_table.h"
#include "s_functions.h"
```

Include dependency graph for commands.c:



Functions

- void mkfs (const char *fs_name, int blocks_in_fat, int block_size_config)

  Create a new FAT filesystem.
- int mount (const char *fs_name)

  Mount an existing FAT filesystem.
- int unmount ()

  Unmount the currently mounted filesystem.

Variables

- uint16_t * _fat = NULL

  Global FAT table (in-memory copy)
- int _block_size = 0

  Block size in bytes (configured at mkfs time)
- int _fat_size = 0

  Number of entries in the FAT (number of data blocks)
- FILE * _current_mount = NULL

  File handle for the currently mounted filesystem.
- int _fs_fd = -1

  File descriptor for the currently mounted filesystem.
- const int BLOCK_SIZES [ ] = {256, 512, 1024, 2048, 4096}

4.1.1 Function Documentation

4.1.1.1 mkfs()

```
void mkfs (
        const char * fs_name,
        int blocks_in_fat,
        int block_size_config )
```

Create a new FAT filesystem.

Initializes a new FAT filesystem file with the specified parameters. Creates the FAT table, initializes all blocks to free (0x0000), and sets up an empty root directory.

Block size options:

- 0: 256 bytes
- 1: 512 bytes
- 2: 1024 bytes
- 3: 2048 bytes
- 4: 4096 bytes

Parameters

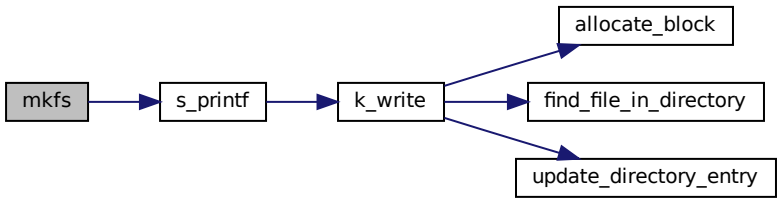
fs_name	Filename for the new filesystem
blocks_in_fat	Number of data blocks in the FAT
block_size_config	Block size configuration (0-4)

Definition at line 26 of file commands.c.

References _current_mount, BLOCK_SIZES, directory_entry_t::name, and s_printf().

Referenced by main().

Here is the call graph for this function:



Here is the caller graph for this function:



4.1.1.2 mount()

```
int mount (
        const char * fs_name )
```

Mount an existing FAT filesystem.

Opens the filesystem file and loads the FAT table into memory. Sets up global state (_fat, _block_size, _current_mount, etc.). Must be called before any file operations.

Parameters

fs_name	Filename of the filesystem to mount

Returns

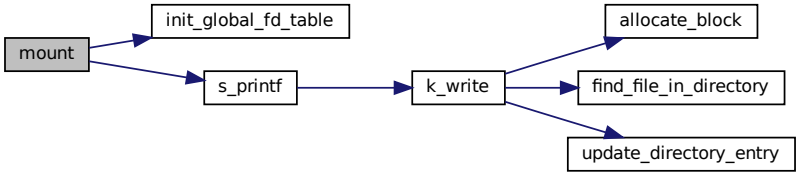
0 on success, -1 on error

Definition at line 122 of file commands.c.

References _block_size, _current_mount, _fat, _fat_size, _fs_fd, BLOCK_SIZES, init_global_fd_table(), and s_printf().

Referenced by main().

Here is the call graph for this function:



Here is the caller graph for this function:



4.1.1.3 unmount() `int unmount ( )`

Unmount the currently mounted filesystem.

Writes the FAT table back to disk, closes the filesystem file, and frees allocated memory. Should be called before program exit.

Returns

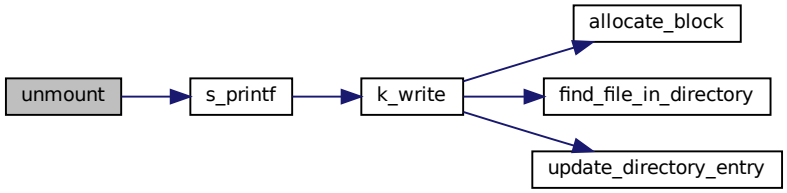
0 on success, -1 on error

Definition at line 171 of file commands.c.

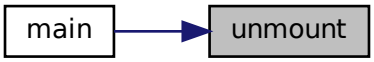
References _block_size, _current_mount, _fat, _fat_size, and s_printf().

Referenced by main().

Here is the call graph for this function:



Here is the caller graph for this function:



4.1.2 Variable Documentation

4.1.2.1 _block_size `int _block_size = 0`

Block size in bytes (configured at mkfs time)

Definition at line 12 of file commands.c.

Referenced by allocate_block(), create_directory_entry_at_block(), find_file_in_directory(), k_ls(), k_read(), k_write(), mount(), print_fat_table(), unmount(), and update_directory_entry().

4.1.2.2 _current_mount `FILE* _current_mount = NULL`

File handle for the currently mounted filesystem.

Definition at line 14 of file commands.c.

Referenced by k_ls(), k_open(), k_unlink(), main(), mkfs(), mount(), and unmount().

4.1.2.3 _fat `uint16_t* _fat = NULL`

Global FAT table (in-memory copy)

Array of 16-bit FAT entries. Each entry either:

- 0x0000: Free block
- 0xFFFF: End of chain
- Other value: Next block in chain

Definition at line 11 of file commands.c.

Referenced by allocate_block(), create_directory_entry(), find_file_in_directory(), k_ls(), k_open(), k_read(), k_unlink(), k_write(), mount(), print_fat_table(), unmount(), and update_directory_entry().

4.1.2.4 _fat_size `int _fat_size = 0`

Number of entries in the FAT (number of data blocks)

Definition at line 13 of file commands.c.

Referenced by allocate_block(), create_directory_entry_at_block(), find_file_in_directory(), k_ls(), k_read(), k_write(), mount(), print_fat_table(), unmount(), and update_directory_entry().

4.1.2.5 _fs_fd `int _fs_fd = -1`

File descriptor for the currently mounted filesystem.

Definition at line 15 of file commands.c.

Referenced by allocate_block(), create_directory_entry_at_block(), find_file_in_directory(), k_ls(), k_read(), k_write(), mount(), and update_directory_entry().

4.1.2.6 BLOCK_SIZES `const int BLOCK_SIZES[] = {256, 512, 1024, 2048, 4096}`

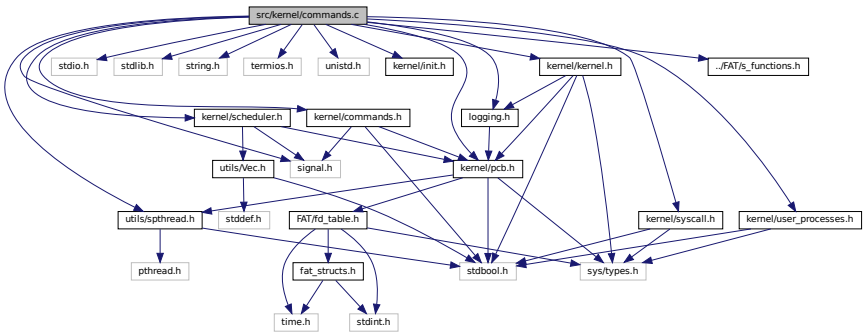
Definition at line 19 of file commands.c.

Referenced by mkfs(), and mount().

4.2 src/kernel/commands.c File Reference

```
#include "kernel/commands.h"
#include <signal.h>
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <termios.h>
#include <unistd.h>
#include "kernel/init.h"
#include "kernel/kernel.h"
#include "kernel/logging.h"
#include "kernel/pcb.h"
#include "kernel/scheduler.h"
#include "kernel/syscall.h"
#include "kernel/user_processes.h"
#include "utils/spthread.h"
#include "../FAT/s_functions.h"
```

Include dependency graph for commands.c:



Functions

- void sigint_handler (int sig)

  Signal handler for SIGINT (Ctrl-C)
- void print_help (void)

  Print help information for shell commands.
- void cmd_ps (void)

  Display all running processes (ps command)
- void cmd_info (pid_t pid)

Display detailed information about a specific process.

- void cmd_spawn (const char *type)

  Spawn a test process.
- void cmd_waitpid (pid_t pid)

  Non-blocking wait for a specific child process.
- void cmd_waitpid_blocking (pid_t pid)

  Blocking wait for a specific child process.
- void cmd_waitany_blocking (void)

  Blocking wait for any child process.
- void cmd_waitany (void)

  Non-blocking wait for any child process.
- void cmd_kill (pid_t pid, int signal)

  Send a signal to a process.
- void cmd_nice (pid_t pid, int priority)

  Change process priority.
- void cmd_zombify (bool background)

  Create a zombie process for testing.
- void cmd_orphanify (void)

  Create an orphan process for testing.
- void cmd_fg (pid_t pid)

  Move a stopped background job to foreground.
- void cmd_bg (pid_t pid)

  Resume a stopped background job.
- void cmd_logout (void)

  Exits the shell and shuts down PennOS.
- void cmd_jobs (void)

  Lists all jobs.

Variables

- volatile pid_t foreground_pid = -1

  PID of the current foreground process.
- volatile bool sigint_received = false

  Flag indicating whether SIGINT (Ctrl-C) was received.

4.2.1 Function Documentation

4.2.1.1 cmd_bg() `void cmd_bg ( pid_t pid )`

Resume a stopped background job.

Sends P_SIGCONT to the process and allows it to run in background.

Parameters

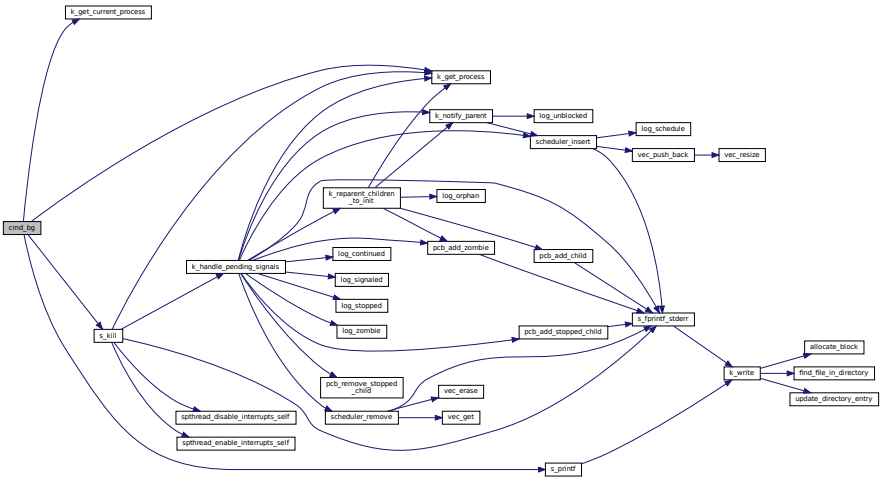
pid	Process ID to resume in background

Definition at line 564 of file commands.c.

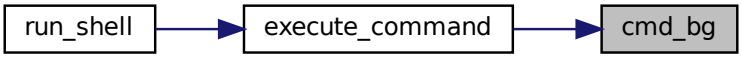
References pcb::children, pcb::is_background, k_get_current_process(), k_get_process(), pcb::name, pcb::num_children, pcb::num_stopped, P_SIGCONT, pcb::pid, s_kill(), s_printf(), pcb::state, STOPPED, and pcb::stopped_children.

Referenced by execute_command().

Here is the call graph for this function:



Here is the caller graph for this function:



4.2.1.2 cmd_fg() `void cmd_fg ( pid_t pid )`

Move a stopped background job to foreground.

Sends P_SIGCONT to the process and waits for it in foreground.

Parameters

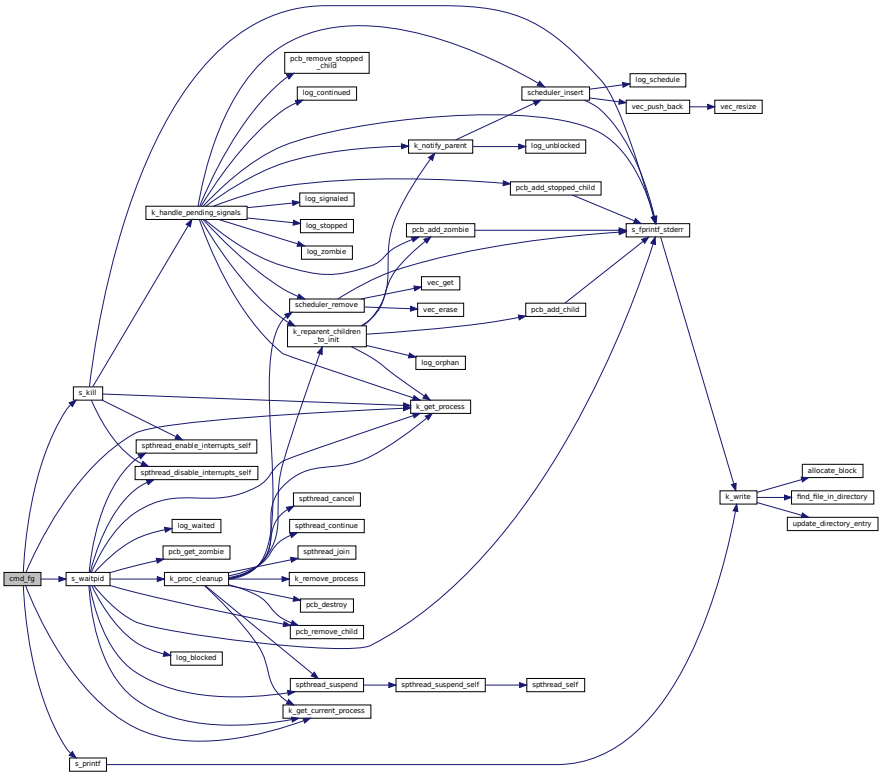
pid	Process ID to bring to foreground

Definition at line 486 of file commands.c.

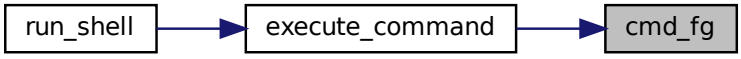
References pcb::children, foreground_pid, pcb::is_background, k_get_current_process(), k_get_process(), pcb::name, pcb::num_children, P_SIGCONT, P_WIFEXITED, P_WIFSIGNALED, pcb::pid, s_kill(), s_printf(), s_waitpid(), pcb::state, STOPPED, and ZOMBIE.

Referenced by execute_command().

Here is the call graph for this function:



Here is the caller graph for this function:



4.2.1.3 cmd_info() `void cmd_info ( pid_t pid )`

Display detailed information about a specific process.

Shows comprehensive information including PID, name, parent, priority, state, children, zombies, and stopped children.

Parameters

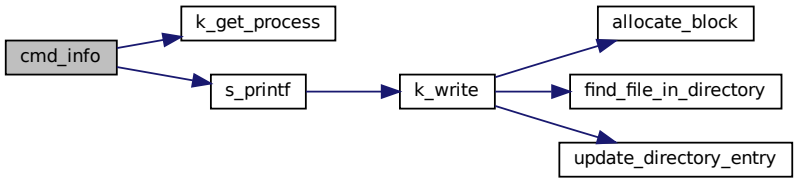
pid	Process ID to query

Definition at line 95 of file commands.c.

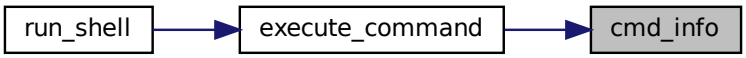
References BLOCKED, pcb::children, pcb::exit_status, pcb::exit_statuses, pcb::is_waiting, k_get_process(), pcb::name, pcb::num_children, pcb::num_stopped, pcb::num_zombies, pcb::pending_signal, pcb::ppid, pcb::priority, RUNNING, s↩ _printf(), pcb::sleep_ticks_remaining, pcb::state, STOPPED, pcb::stopped_children, pcb::waiting_for_pid, ZOMBIE, and pcb::zombie_children.

Referenced by execute_command().

Here is the call graph for this function:



Here is the caller graph for this function:



4.2.1.4 cmd_jobs() `void cmd_jobs ( void )`

Lists all jobs.

Display all background jobs.

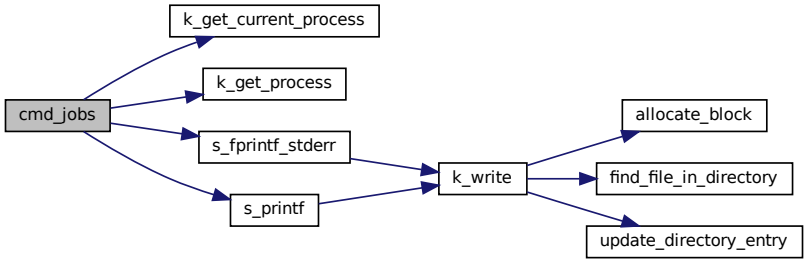
Example Usage: jobs

Definition at line 639 of file commands.c.

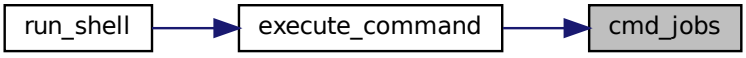
References BLOCKED, pcb::children, pcb::is_background, k_get_current_process(), k_get_process(), pcb::name, pcb::num_children, RUNNING, s_fprintf_stderr(), s_printf(), pcb::state, STOPPED, and ZOMBIE.

Referenced by execute_command().

Here is the call graph for this function:



Here is the caller graph for this function:



4.2.1.5 cmd_kill() `void cmd_kill ( pid_t pid, int signal )`

Send a signal to a process.

Parameters

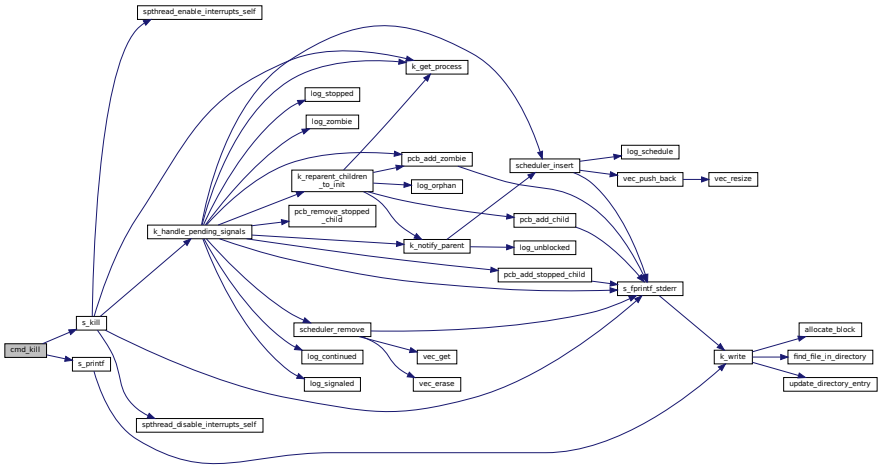
pid	Target process ID
signal	Signal to send (P_SIGSTOP, P_SIGCONT, or P_SIGTERM)

Definition at line 350 of file commands.c.

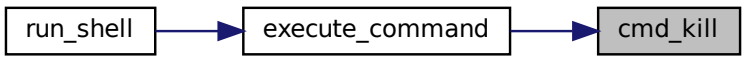
References P_SIGCONT, P_SIGSTOP, P_SIGTERM, s_kill(), and s_printf().

Referenced by execute_command().

Here is the call graph for this function:



Here is the caller graph for this function:



4.2.1.6 cmd_logout() `void cmd_logout ( void )`

Exits the shell and shuts down PennOS.

Exit the shell and shutdown the system.

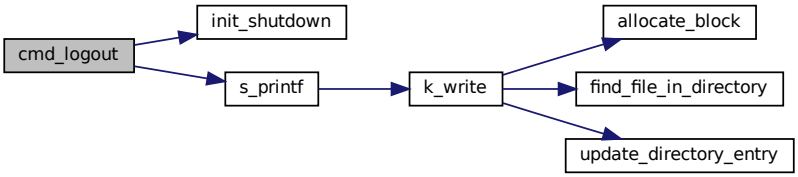
Example Usage: logout

Definition at line 629 of file commands.c.

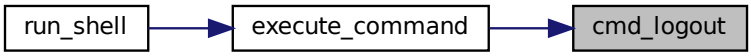
References init_shutdown(), and s_printf().

Referenced by execute_command().

Here is the call graph for this function:



Here is the caller graph for this function:



4.2.1.7 cmd_nice() `void cmd_nice ( pid_t pid, int priority )`

Change process priority.

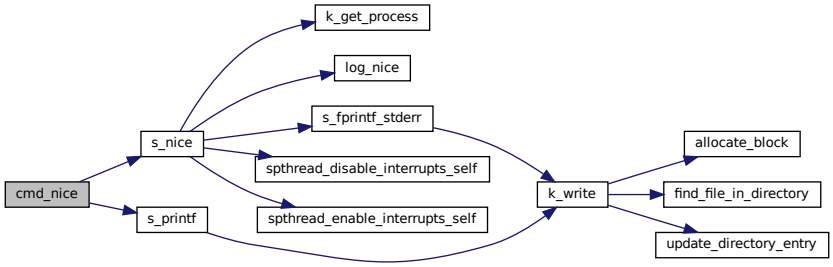
Parameters

pid	Target process ID
priority	New priority level (0=high, 1=medium, 2=low)

Definition at line 378 of file commands.c.

References s_nice(), and s_printf().

Here is the call graph for this function:



4.2.1.8 cmd_orphanify() `void cmd_orphanify ( void )`

Create an orphan process for testing.

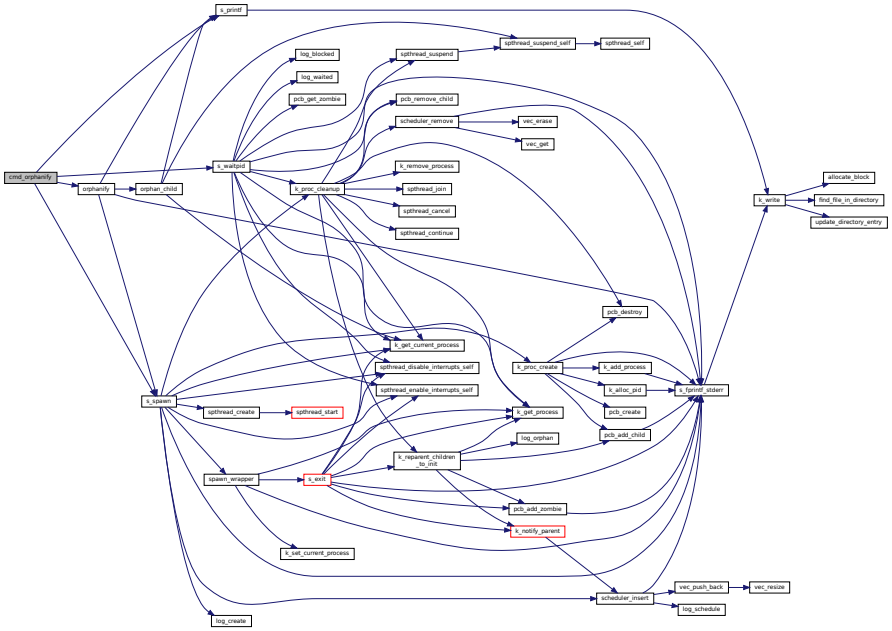
Spawns a parent that creates a child then exits, orphaning the child which gets reparented to INIT.

Definition at line 459 of file commands.c.

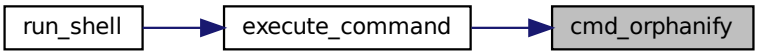
References orphanify(), s_printf(), s_spawn(), and s_waitpid().

Referenced by execute_command().

Here is the call graph for this function:



Here is the caller graph for this function:



4.2.1.9 cmd_ps() `void cmd_ps ( void )`

Display all running processes (ps command)

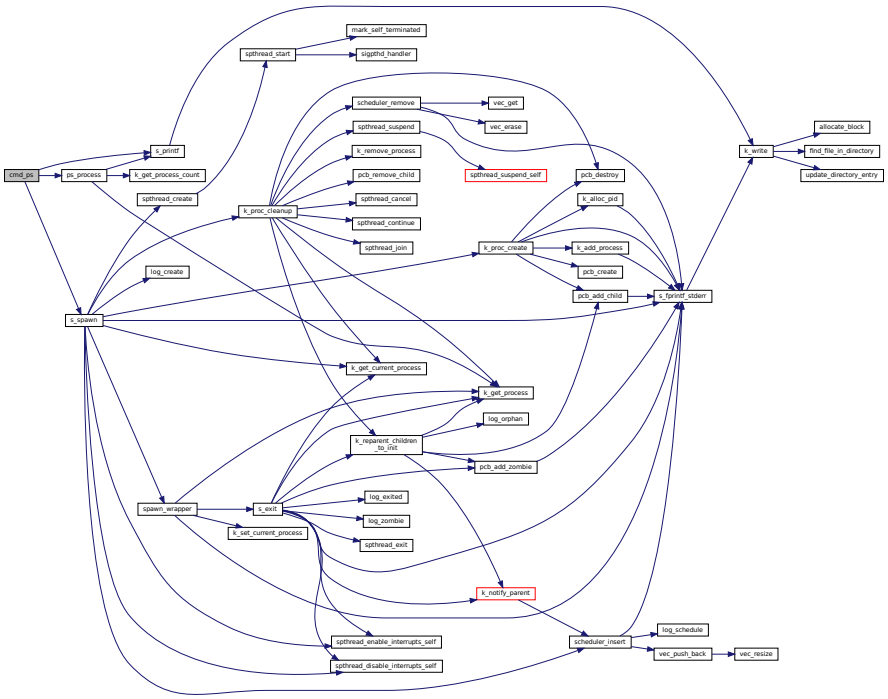
Prints a table showing PID, PPID, priority, and state for all processes.

Definition at line 78 of file commands.c.

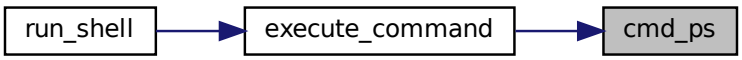
References foreground_pid, ps_process(), s_printf(), and s_spawn().

Referenced by execute_command().

Here is the call graph for this function:



Here is the caller graph for this function:



4.2.1.10 cmd_spawn() `void cmd_spawn ( const char * type )`

Spawn a test process.

Creates various test processes for demonstration purposes (hello, counting, orphan, etc.).

Parameters

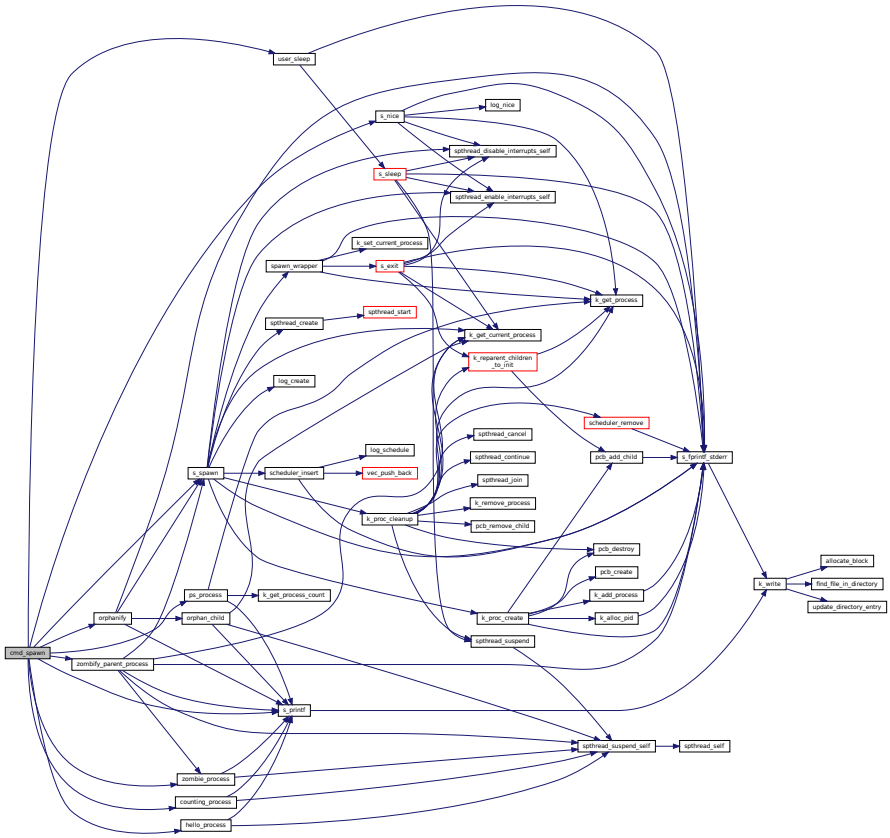
type	Type of process to spawn (e.g., "hello", "counting")

Definition at line 150 of file commands.c.

References counting_process(), hello_process(), orphanify(), ps_process(), s_nice(), s_printf(), s_spawn(), user_↩ sleep(), zombie_process(), and zombify_parent_process().

Referenced by execute_command().

Here is the call graph for this function:



Here is the caller graph for this function:



4.2.1.11 cmd_waitany() `void cmd_waitany ( void )`

Non-blocking wait for any child process.

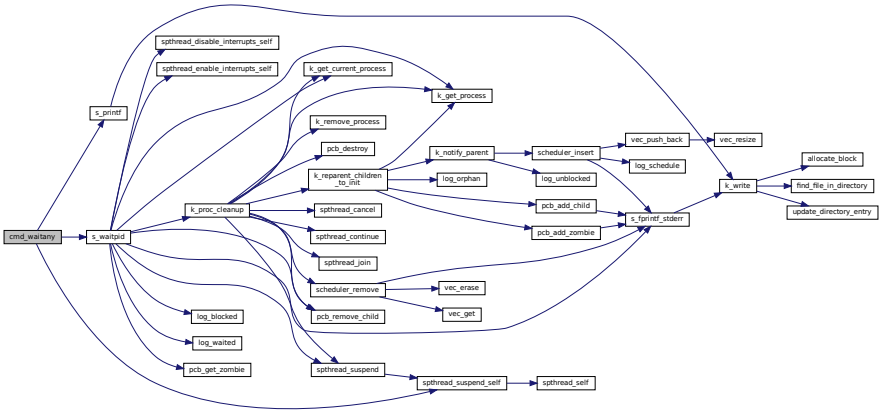
Equivalent to waitpid(-1, &status, WNOHANG) in UNIX. Returns immediately if no child has changed state.

Definition at line 316 of file commands.c.

References P_WIFEXITED, P_WIFSIGNALED, P_WIFSTOPPED, s_printf(), s_waitpid(), and spthread_suspend_self().

Referenced by execute_command().

Here is the call graph for this function:



Here is the caller graph for this function:



4.2.1.12 cmd_waitany_blocking() `void cmd_waitany_blocking ( void )`

Blocking wait for any child process.

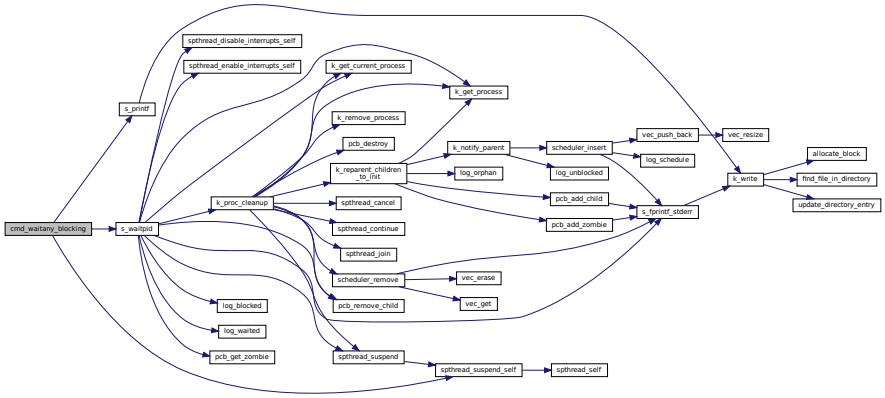
Equivalent to waitpid(-1, &status, 0) in UNIX. Blocks until any child changes state.

Definition at line 275 of file commands.c.

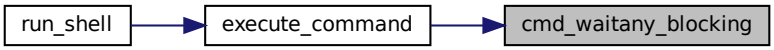
References P_WIFEXITED, P_WIFSIGNALED, P_WIFSTOPPED, s_printf(), s_waitpid(), sigint_received, and spthread_suspend_self().

Referenced by execute_command().

Here is the call graph for this function:



Here is the caller graph for this function:



4.2.1.13 cmd_waitpid() `void cmd_waitpid ( pid_t pid )`

Non-blocking wait for a specific child process.

Equivalent to waitpid(pid, &status, WNOHANG) in UNIX. Returns immediately if child has not changed state.

Parameters

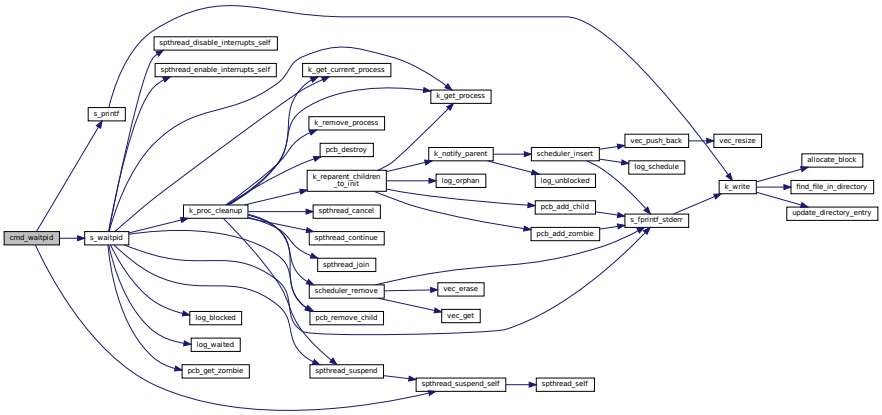
pid	Process ID to wait for

Definition at line 194 of file commands.c.

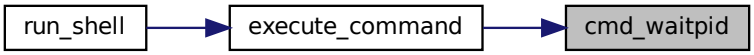
References P_EXITSTATUS, P_WIFEXITED, P_WIFSIGNALED, P_WIFSTOPPED, s_printf(), s_waitpid(), and spthread_suspend_self().

Referenced by execute_command().

Here is the call graph for this function:



Here is the caller graph for this function:



4.2.1.14 cmd_waitpid_blocking() `void cmd_waitpid_blocking ( pid_t pid )`

Blocking wait for a specific child process.

Equivalent to waitpid(pid, &status, 0) in UNIX. Blocks until the specified child changes state.

Parameters

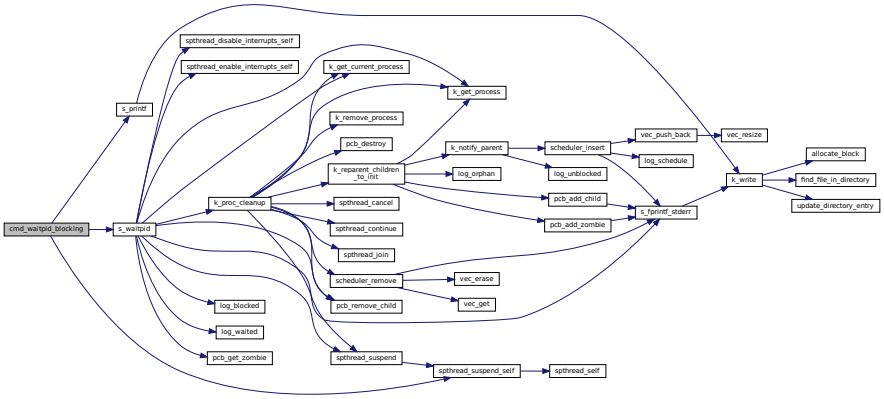
pid	Process ID to wait for

Definition at line 231 of file commands.c.

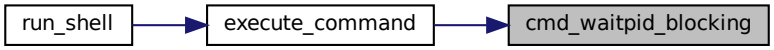
References P_EXITSTATUS, P_WIFEXITED, P_WIFSIGNALED, P_WIFSTOPPED, s_printf(), s_waitpid(), sigint_↩ received, and spthread_suspend_self().

Referenced by execute_command().

Here is the call graph for this function:



Here is the caller graph for this function:



4.2.1.15 cmd_zombify() `void cmd_zombify ( bool background )`

Create a zombie process for testing.

Spawns a child that immediately exits to become a zombie.

Parameters

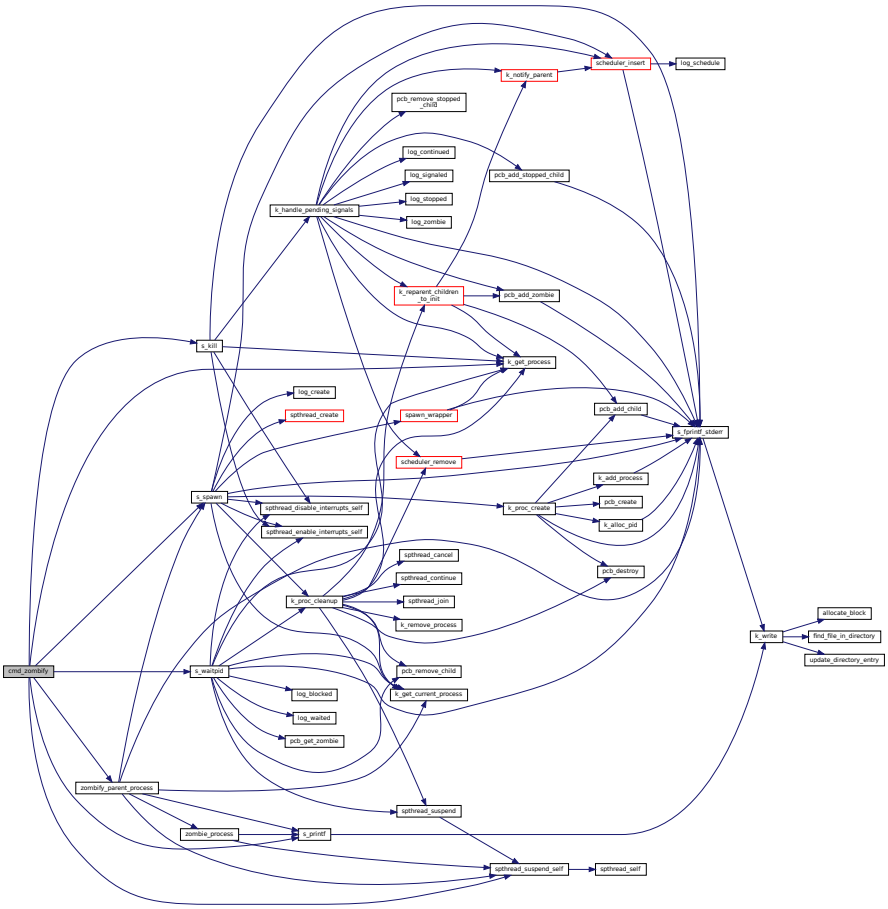
background	Whether to run the zombie process in background

Definition at line 395 of file commands.c.

References foreground_pid, k_get_process(), P_SIGTERM, s_kill(), s_printf(), s_spawn(), s_waitpid(), sigint_received, spthread_suspend_self(), pcb::state, ZOMBIE, and zombify_parent_process().

Referenced by execute_command().

Here is the call graph for this function:



Here is the caller graph for this function:



4.2.1.16 print_help() `void print_help ( void )`

Print help information for shell commands.

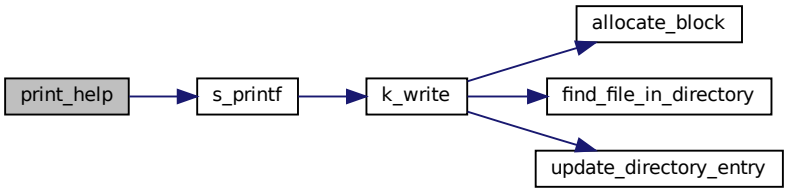
Displays usage information for all built-in shell commands.

Definition at line 35 of file commands.c.

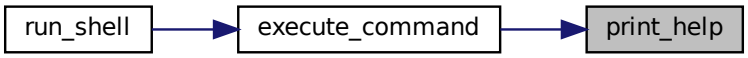
References s_printf().

Referenced by execute_command().

Here is the call graph for this function:



Here is the caller graph for this function:



4.2.1.17 sigint_handler() `void sigint_handler ( int sig )`

Signal handler for SIGINT (Ctrl-C)

Sends P_SIGTERM to the current foreground process and sets the sigint_received flag.

Parameters

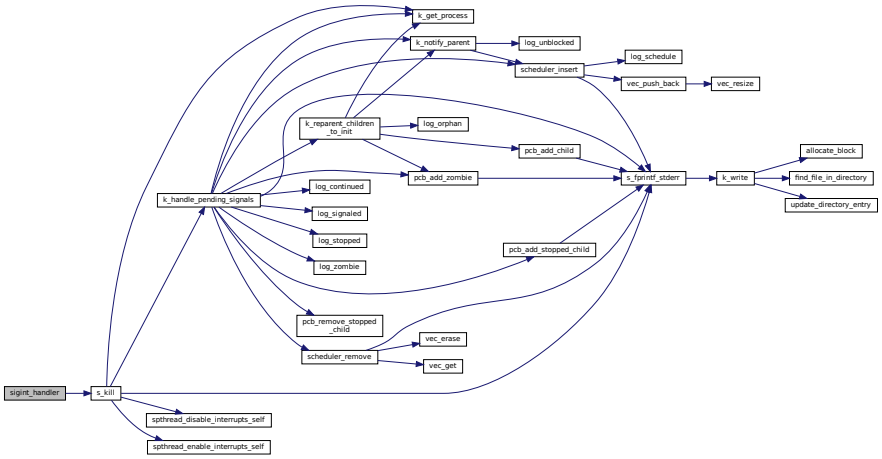
sig	Signal number (SIGINT)

Definition at line 23 of file commands.c.

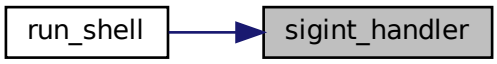
References foreground_pid, P_SIGTERM, s_kill(), and sigint_received.

Referenced by run_shell().

Here is the call graph for this function:



Here is the caller graph for this function:



4.2.2 Variable Documentation

4.2.2.1 foreground_pid `volatile pid_t foreground_pid = -1`

PID of the current foreground process.

Shared state used by signal handler to send SIGTERM to foreground process on Ctrl-C. Set to -1 when no foreground process is running.

Definition at line 19 of file commands.c.

Referenced by cmd_fg(), cmd_ps(), cmd_zombify(), execute_command(), and sigint_handler().

4.2.2.2 sigint_received `volatile bool sigint_received = false`

Flag indicating whether SIGINT (Ctrl-C) was received.

Set by signal handler and checked by shell to determine if user interrupted command.

Definition at line 20 of file commands.c.

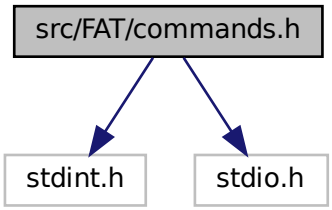
Referenced by cmd_waitany_blocking(), cmd_waitpid_blocking(), cmd_zombify(), and sigint_handler().

4.3 src/FAT/commands.h File Reference

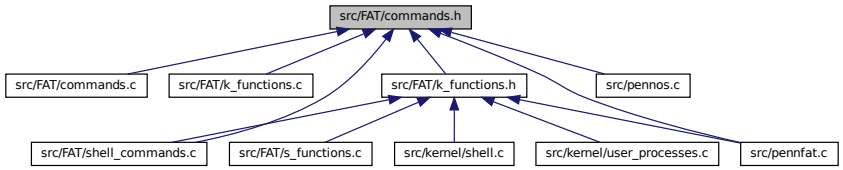
PennFAT filesystem commands and global state.

```
#include <stdint.h>
#include <stdio.h>
```

Include dependency graph for commands.h:



This graph shows which files directly or indirectly include this file:



Functions

- void mkfs (const char *fs_name, int blocks_in_fat, int block_size_config)

  Create a new FAT filesystem.
- int mount (const char *fs_name)

  Mount an existing FAT filesystem.
- int unmount ()

  Unmount the currently mounted filesystem.

Variables

- uint16_t * _fat

  Global FAT table (in-memory copy)
- int _block_size

  Block size in bytes (configured at mkfs time)
- int _fat_size

  Number of entries in the FAT (number of data blocks)
- FILE * _current_mount

  File handle for the currently mounted filesystem.
- int _fs_fd

  File descriptor for the currently mounted filesystem.

4.3.1 Detailed Description

PennFAT filesystem commands and global state.

Provides functions for creating, mounting, and unmounting FAT filesystems. Also exposes global state variables for the currently mounted filesystem.

4.3.2 Function Documentation

4.3.2.1 mkfs()

```
void mkfs (
        const char * fs_name,
        int blocks_in_fat,
        int block_size_config )
```

Create a new FAT filesystem.

Initializes a new FAT filesystem file with the specified parameters. Creates the FAT table, initializes all blocks to free (0x0000), and sets up an empty root directory.

Block size options:

- 0: 256 bytes
- 1: 512 bytes
- 2: 1024 bytes
- 3: 2048 bytes
- 4: 4096 bytes

Parameters

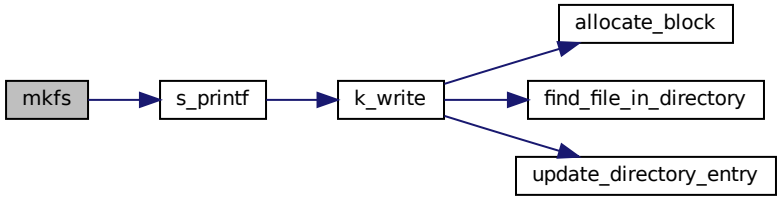
fs_name	Filename for the new filesystem
blocks_in_fat	Number of data blocks in the FAT
block_size_config	Block size configuration (0-4)

Definition at line 26 of file commands.c.

References _current_mount, BLOCK_SIZES, directory_entry_t::name, and s_printf().

Referenced by main().

Here is the call graph for this function:



Here is the caller graph for this function:



4.3.2.2 mount()

```
int mount (
        const char * fs_name )
```

Mount an existing FAT filesystem.

Opens the filesystem file and loads the FAT table into memory. Sets up global state (_fat, _block_size, _current_mount, etc.). Must be called before any file operations.

Parameters

fs_name	Filename of the filesystem to mount

Returns

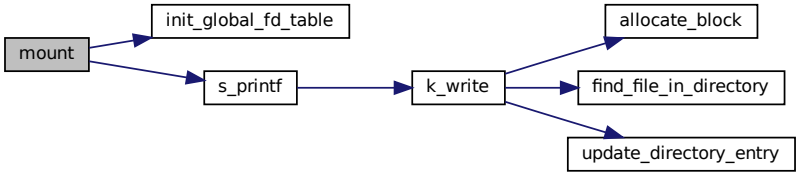
0 on success, -1 on error

Definition at line 122 of file commands.c.

References _block_size, _current_mount, _fat, _fat_size, _fs_fd, BLOCK_SIZES, init_global_fd_table(), and s_printf().

Referenced by main().

Here is the call graph for this function:



Here is the caller graph for this function:



4.3.2.3 unmount() `int unmount ( )`

Unmount the currently mounted filesystem.

Writes the FAT table back to disk, closes the filesystem file, and frees allocated memory. Should be called before program exit.

Returns

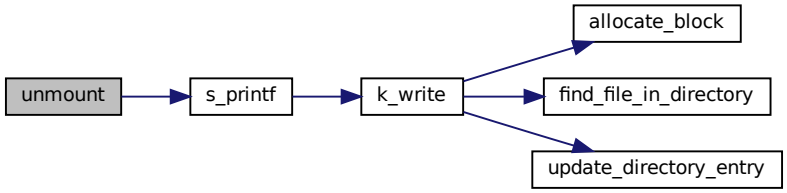
0 on success, -1 on error

Definition at line 171 of file commands.c.

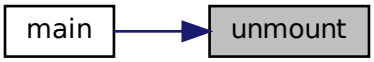
References _block_size, _current_mount, _fat, _fat_size, and s_printf().

Referenced by main().

Here is the call graph for this function:



Here is the caller graph for this function:



4.3.3 Variable Documentation

4.3.3.1 _block_size `int _block_size [extern]`

Block size in bytes (configured at mkfs time)

Definition at line 12 of file commands.c.

Referenced by allocate_block(), create_directory_entry_at_block(), find_file_in_directory(), k_ls(), k_read(), k_write(), mount(), print_fat_table(), unmount(), and update_directory_entry().

4.3.3.2 _current_mount `FILE* _current_mount [extern]`

File handle for the currently mounted filesystem.

Definition at line 14 of file commands.c.

Referenced by k_ls(), k_open(), k_unlink(), main(), mkfs(), mount(), and unmount().

4.3.3.3 _fat `uint16_t* _fat [extern]`

Global FAT table (in-memory copy)

Array of 16-bit FAT entries. Each entry either:

- 0x0000: Free block
- 0xFFFF: End of chain
- Other value: Next block in chain

Definition at line 11 of file commands.c.

Referenced by allocate_block(), create_directory_entry(), find_file_in_directory(), k_ls(), k_open(), k_read(), k_unlink(), k_write(), mount(), print_fat_table(), unmount(), and update_directory_entry().

4.3.3.4 _fat_size `int _fat_size [extern]`

Number of entries in the FAT (number of data blocks)

Definition at line 13 of file commands.c.

Referenced by allocate_block(), create_directory_entry_at_block(), find_file_in_directory(), k_ls(), k_read(), k_write(), mount(), print_fat_table(), unmount(), and update_directory_entry().

4.3.3.5 _fs_fd `int _fs_fd [extern]`

File descriptor for the currently mounted filesystem.

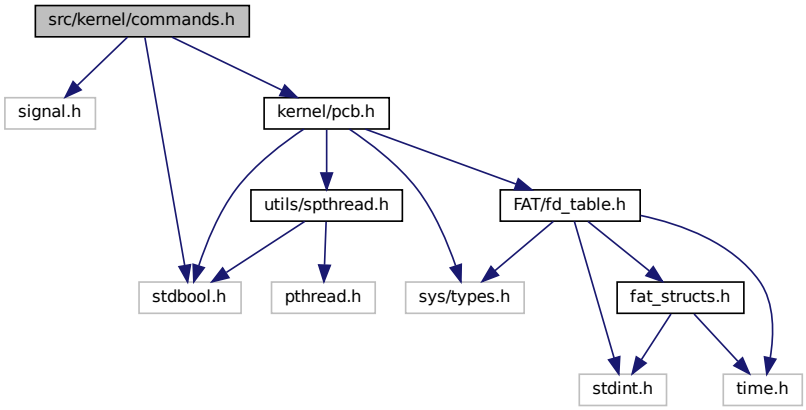
Definition at line 15 of file commands.c.

Referenced by allocate_block(), create_directory_entry_at_block(), find_file_in_directory(), k_ls(), k_read(), k_write(), mount(), and update_directory_entry().

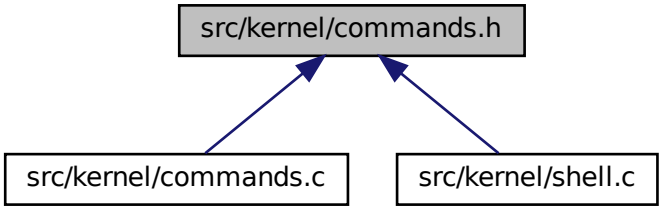
4.4 src/kernel/commands.h File Reference

```
#include <signal.h>
#include <stdbool.h>
#include "kernel/pcb.h"
```

Include dependency graph for commands.h:



This graph shows which files directly or indirectly include this file:



Functions

- void print_help (void)

  Print help information for shell commands.
- void cmd_ps (void)

  Display all running processes (ps command)
- void cmd_info (pid_t pid)

  Display detailed information about a specific process.

- void cmd_spawn (const char *type)
  Spawn a test process.
- void cmd_waitpid (pid_t pid)
  Non-blocking wait for a specific child process.
- void cmd_waitany (void)
  Non-blocking wait for any child process.
- void cmd_waitpid_blocking (pid_t pid)
  Blocking wait for a specific child process.
- void cmd_waitany_blocking (void)
  Blocking wait for any child process.
- void cmd_kill (pid_t pid, int signal)
  Send a signal to a process.
- void cmd_nice (pid_t pid, int priority)
  Change process priority.
- void cmd_zombify (bool background)
  Create a zombie process for testing.
- void cmd_orphanify (void)
  Create an orphan process for testing.
- void cmd_fg (pid_t pid)
  Move a stopped background job to foreground.
- void cmd_bg (pid_t pid)
  Resume a stopped background job.
- void cmd_jobs (void)
  Display all background jobs.
- void cmd_logout (void)
  Exit the shell and shutdown the system.
- void run_shell (void)
  Run the interactive shell in the current process context.
- void * shell_main (void *arg)
  Thread wrapper function for running the shell.
- void sigint_handler (int sig)
  Signal handler for SIGINT (Ctrl-C)

Variables

- volatile pid_t foreground_pid
  PID of the current foreground process.
- volatile bool sigint_received
  Flag indicating whether SIGINT (Ctrl-C) was received.

4.4.1 Function Documentation

4.4.1.1 cmd_bg() `void cmd_bg ( pid_t pid )`

Resume a stopped background job.

Sends P_SIGCONT to the process and allows it to run in background.

Parameters

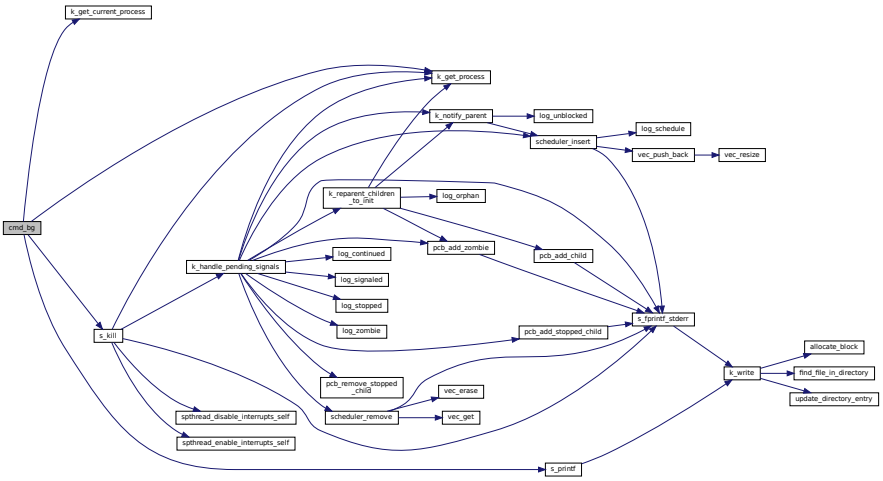
pid	Process ID to resume in background

Definition at line 564 of file commands.c.

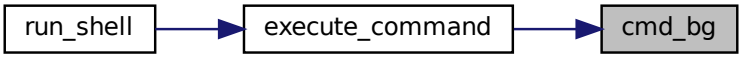
References pcb::children, pcb::is_background, k_get_current_process(), k_get_process(), pcb::name, pcb::num_children, pcb::num_stopped, P_SIGCONT, pcb::pid, s_kill(), s_printf(), pcb::state, STOPPED, and pcb::stopped_children.

Referenced by execute_command().

Here is the call graph for this function:



Here is the caller graph for this function:



4.4.1.2 cmd_fg() `void cmd_fg ( pid_t pid )`

Move a stopped background job to foreground.

Sends P_SIGCONT to the process and waits for it in foreground.

Parameters

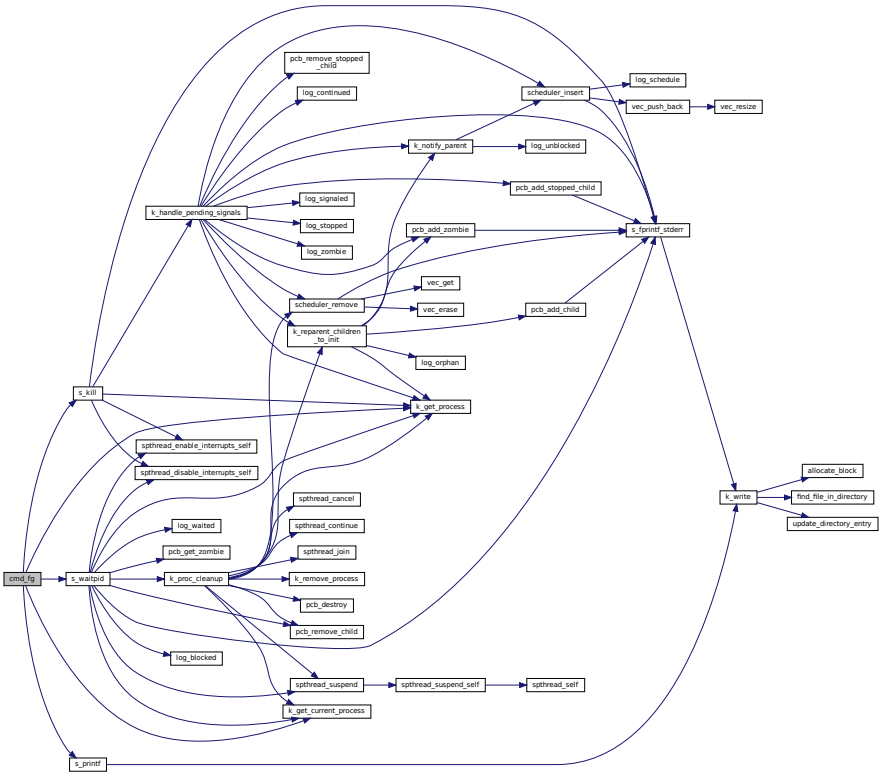
pid	Process ID to bring to foreground

Definition at line 486 of file commands.c.

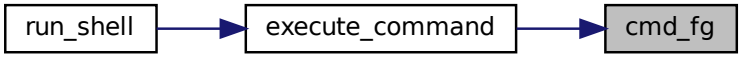
References pcb::children, foreground_pid, pcb::is_background, k_get_current_process(), k_get_process(), pcb::name, pcb::num_children, P_SIGCONT, P_WIFEXITED, P_WIFSIGNALED, pcb::pid, s_kill(), s_printf(), s_waitpid(), pcb::state, STOPPED, and ZOMBIE.

Referenced by execute_command().

Here is the call graph for this function:



Here is the caller graph for this function:



4.4.1.3 cmd_info() `void cmd_info ( pid_t pid )`

Display detailed information about a specific process.

Shows comprehensive information including PID, name, parent, priority, state, children, zombies, and stopped children.

Parameters

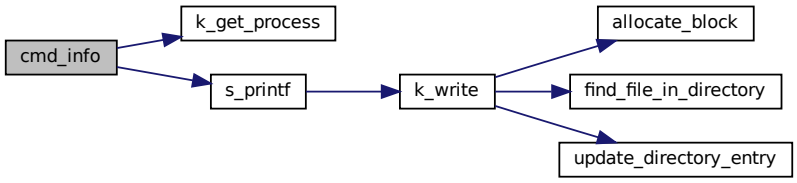
pid	Process ID to query

Definition at line 95 of file commands.c.

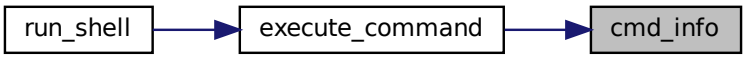
References BLOCKED, pcb::children, pcb::exit_status, pcb::exit_statuses, pcb::is_waiting, k_get_process(), pcb::name, pcb::num_children, pcb::num_stopped, pcb::num_zombies, pcb::pending_signal, pcb::ppid, pcb::priority, RUNNING, s↩ _printf(), pcb::sleep_ticks_remaining, pcb::state, STOPPED, pcb::stopped_children, pcb::waiting_for_pid, ZOMBIE, and pcb::zombie_children.

Referenced by execute_command().

Here is the call graph for this function:



Here is the caller graph for this function:



4.4.1.4 cmd_jobs() `void cmd_jobs ( void )`

Display all background jobs.

Lists stopped and running background processes.

Display all background jobs.

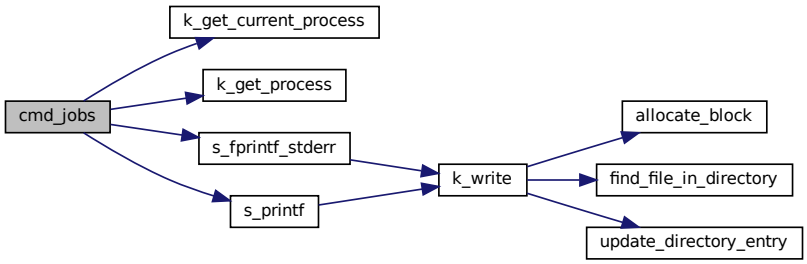
Example Usage: jobs

Definition at line 639 of file commands.c.

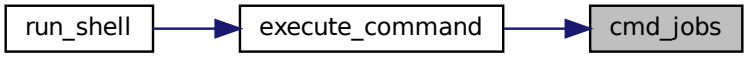
References BLOCKED, pcb::children, pcb::is_background, k_get_current_process(), k_get_process(), pcb::name, pcb::num_children, RUNNING, s_fprintf_stderr(), s_printf(), pcb::state, STOPPED, and ZOMBIE.

Referenced by execute_command().

Here is the call graph for this function:



Here is the caller graph for this function:



4.4.1.5 cmd_kill() `void cmd_kill ( pid_t pid, int signal )`

Send a signal to a process.

Parameters

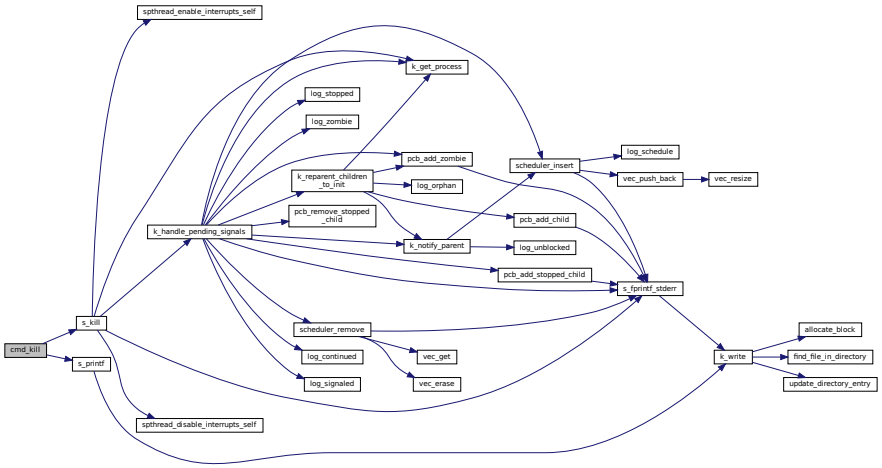
pid	Target process ID
signal	Signal to send (P_SIGSTOP, P_SIGCONT, or P_SIGTERM)

Definition at line 350 of file commands.c.

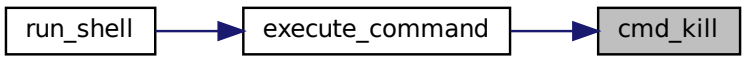
References P_SIGCONT, P_SIGSTOP, P_SIGTERM, s_kill(), and s_printf().

Referenced by execute_command().

Here is the call graph for this function:



Here is the caller graph for this function:



4.4.1.6 cmd_logout() `void cmd_logout ( void )`

Exit the shell and shutdown the system.

Triggers INIT shutdown sequence, stopping scheduler and cleaning up all processes.

Exit the shell and shutdown the system.

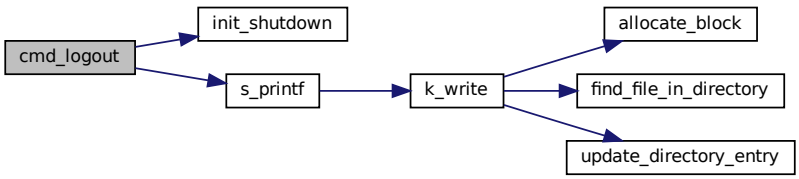
Example Usage: logout

Definition at line 629 of file commands.c.

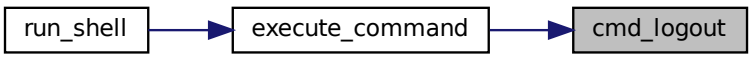
References init_shutdown(), and s_printf().

Referenced by execute_command().

Here is the call graph for this function:



Here is the caller graph for this function:



4.4.1.7 cmd_nice() `void cmd_nice ( pid_t pid, int priority )`

Change process priority.

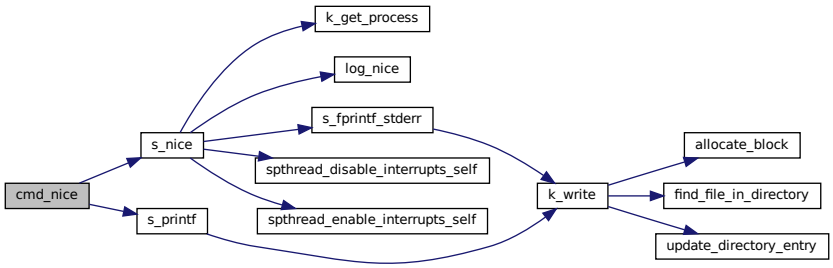
Parameters

pid	Target process ID
priority	New priority level (0=high, 1=medium, 2=low)

Definition at line 378 of file commands.c.

References s_nice(), and s_printf().

Here is the call graph for this function:



4.4.1.8 cmd_orphanify() `void cmd_orphanify ( void )`

Create an orphan process for testing.

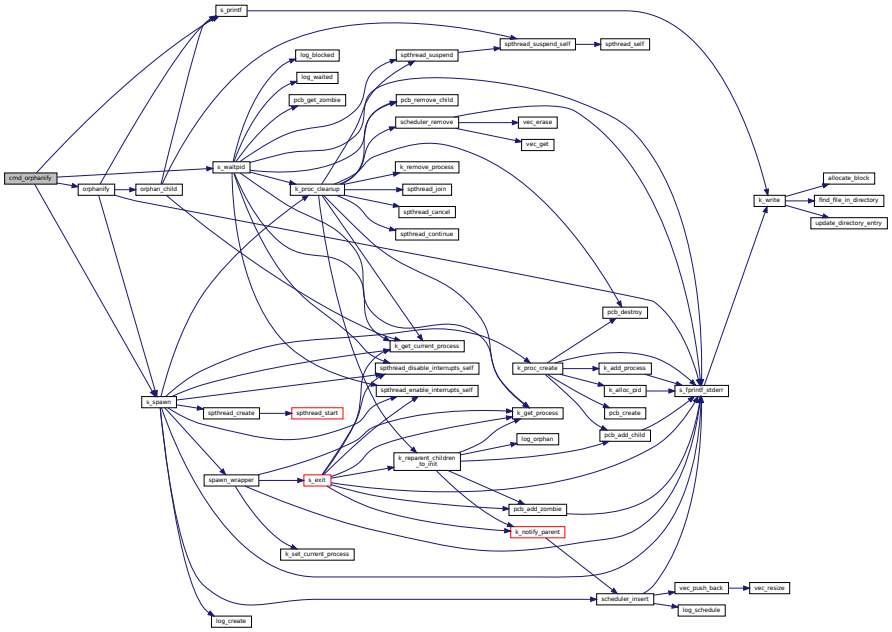
Spawns a parent that creates a child then exits, orphaning the child which gets reparented to INIT.

Definition at line 459 of file commands.c.

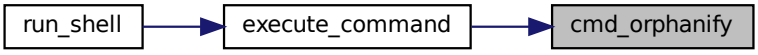
References orphanify(), s_printf(), s_spawn(), and s_waitpid().

Referenced by execute_command().

Here is the call graph for this function:



Here is the caller graph for this function:



4.4.1.9 cmd_ps() `void cmd_ps ( void )`

Display all running processes (ps command)

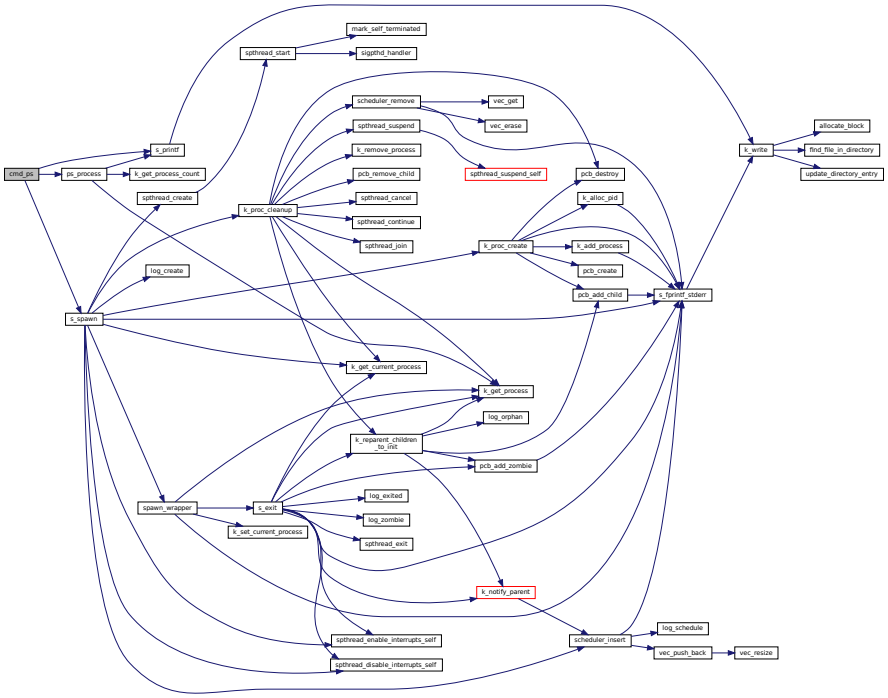
Prints a table showing PID, PPID, priority, and state for all processes.

Definition at line 78 of file commands.c.

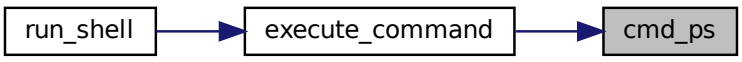
References foreground_pid, ps_process(), s_printf(), and s_spawn().

Referenced by execute_command().

Here is the call graph for this function:



Here is the caller graph for this function:



4.4.1.10 cmd_spawn() `void cmd_spawn ( const char * type )`

Spawn a test process.

Creates various test processes for demonstration purposes (hello, counting, orphan, etc.).

Parameters

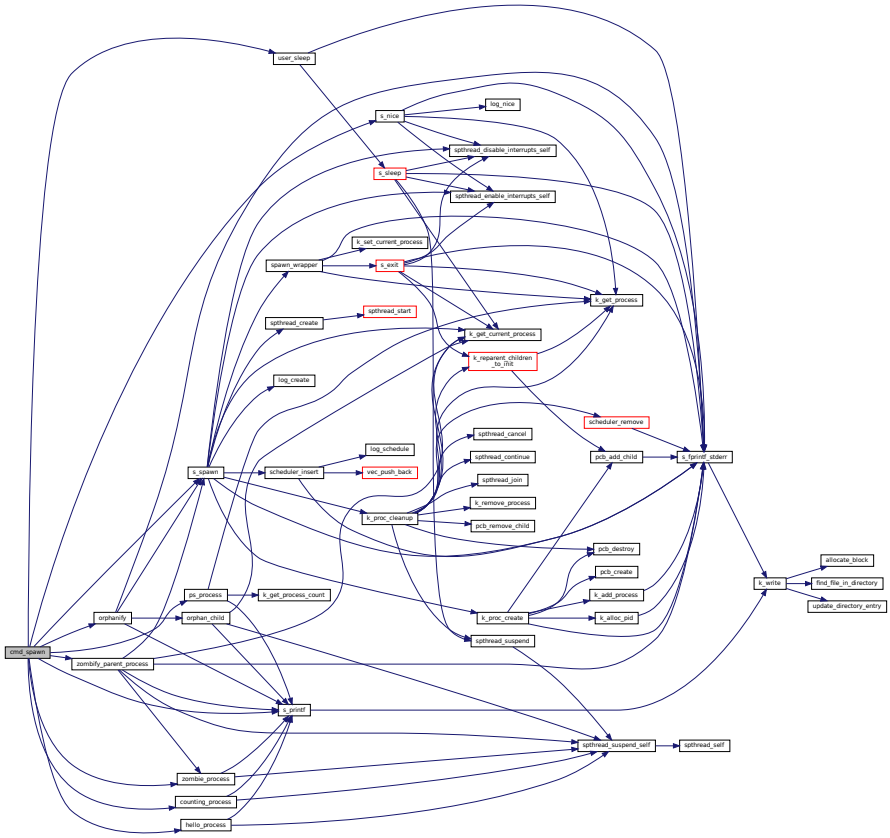
type	Type of process to spawn (e.g., "hello", "counting")

Definition at line 150 of file commands.c.

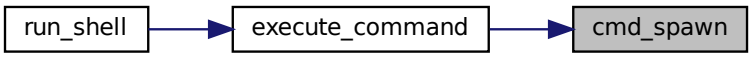
References counting_process(), hello_process(), orphanify(), ps_process(), s_nice(), s_printf(), s_spawn(), user_↩ sleep(), zombie_process(), and zombify_parent_process().

Referenced by execute_command().

Here is the call graph for this function:



Here is the caller graph for this function:



4.4.1.11 cmd_waitany() `void cmd_waitany ( void )`

Non-blocking wait for any child process.

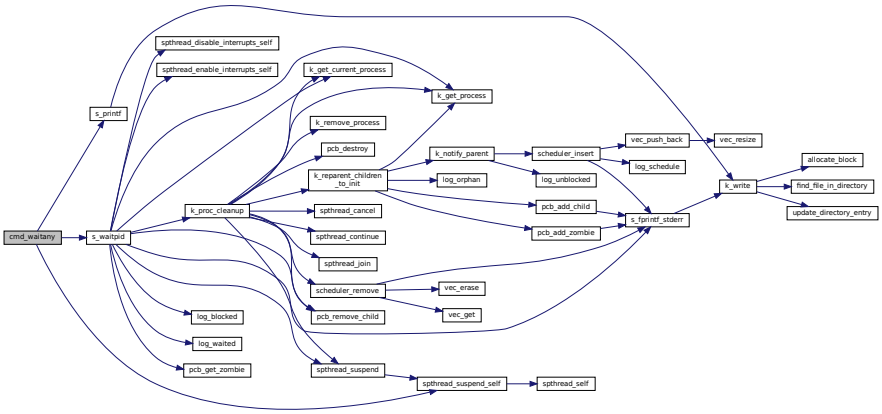
Equivalent to waitpid(-1, &status, WNOHANG) in UNIX. Returns immediately if no child has changed state.

Definition at line 316 of file commands.c.

References P_WIFEXITED, P_WIFSIGNALED, P_WIFSTOPPED, s_printf(), s_waitpid(), and spthread_suspend_self().

Referenced by execute_command().

Here is the call graph for this function:



Here is the caller graph for this function:



4.4.1.12 cmd_waitany_blocking() `void cmd_waitany_blocking ( void )`

Blocking wait for any child process.

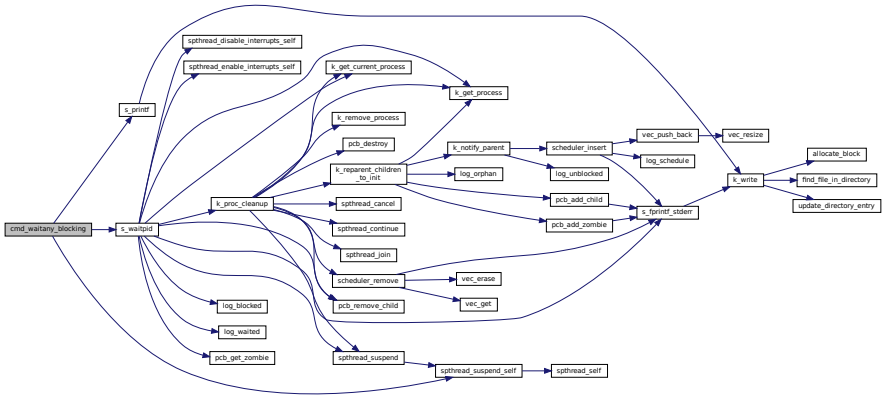
Equivalent to waitpid(-1, &status, 0) in UNIX. Blocks until any child changes state.

Definition at line 275 of file commands.c.

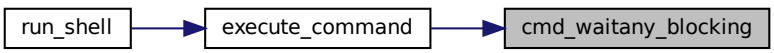
References P_WIFEXITED, P_WIFSIGNALED, P_WIFSTOPPED, s_printf(), s_waitpid(), sigint_received, and spthread_suspend_self().

Referenced by execute_command().

Here is the call graph for this function:



Here is the caller graph for this function:



4.4.1.13 cmd_waitpid() `void cmd_waitpid ( pid_t pid )`

Non-blocking wait for a specific child process.

Equivalent to waitpid(pid, &status, WNOHANG) in UNIX. Returns immediately if child has not changed state.

Parameters

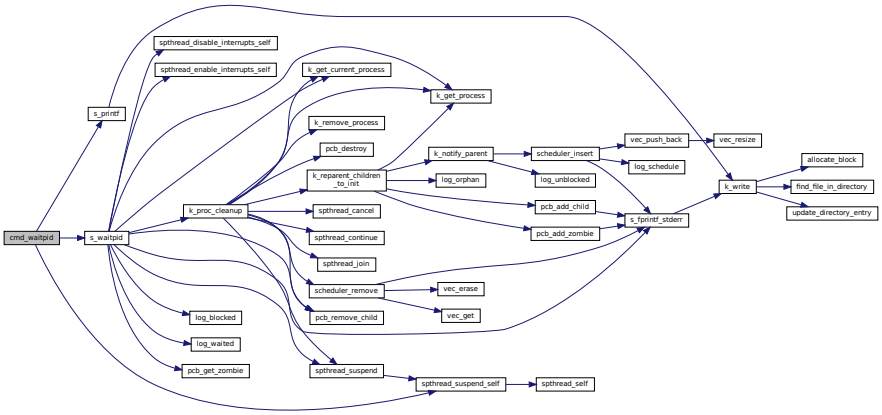
pid	Process ID to wait for

Definition at line 194 of file commands.c.

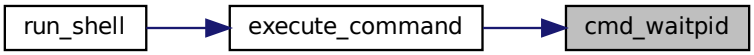
References P_EXITSTATUS, P_WIFEXITED, P_WIFSIGNALED, P_WIFSTOPPED, s_printf(), s_waitpid(), and spthread_suspend_self().

Referenced by execute_command().

Here is the call graph for this function:



Here is the caller graph for this function:



4.4.1.14 cmd_waitpid_blocking() `void cmd_waitpid_blocking ( pid_t pid )`

Blocking wait for a specific child process.

Equivalent to waitpid(pid, &status, 0) in UNIX. Blocks until the specified child changes state.

Parameters

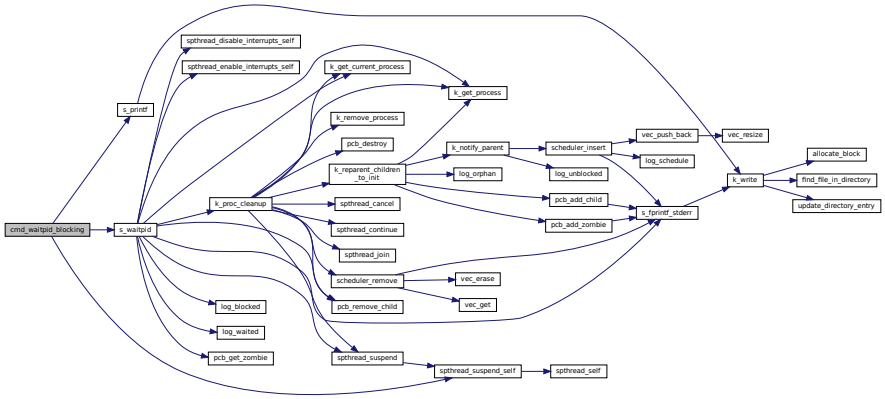
pid	Process ID to wait for

Definition at line 231 of file commands.c.

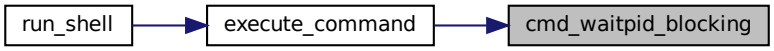
References P_EXITSTATUS, P_WIFEXITED, P_WIFSIGNALED, P_WIFSTOPPED, s_printf(), s_waitpid(), sigint_↩ received, and spthread_suspend_self().

Referenced by execute_command().

Here is the call graph for this function:



Here is the caller graph for this function:



4.4.1.15 cmd_zombify() `void cmd_zombify ( bool background )`

Create a zombie process for testing.

Spawns a child that immediately exits to become a zombie.

Parameters

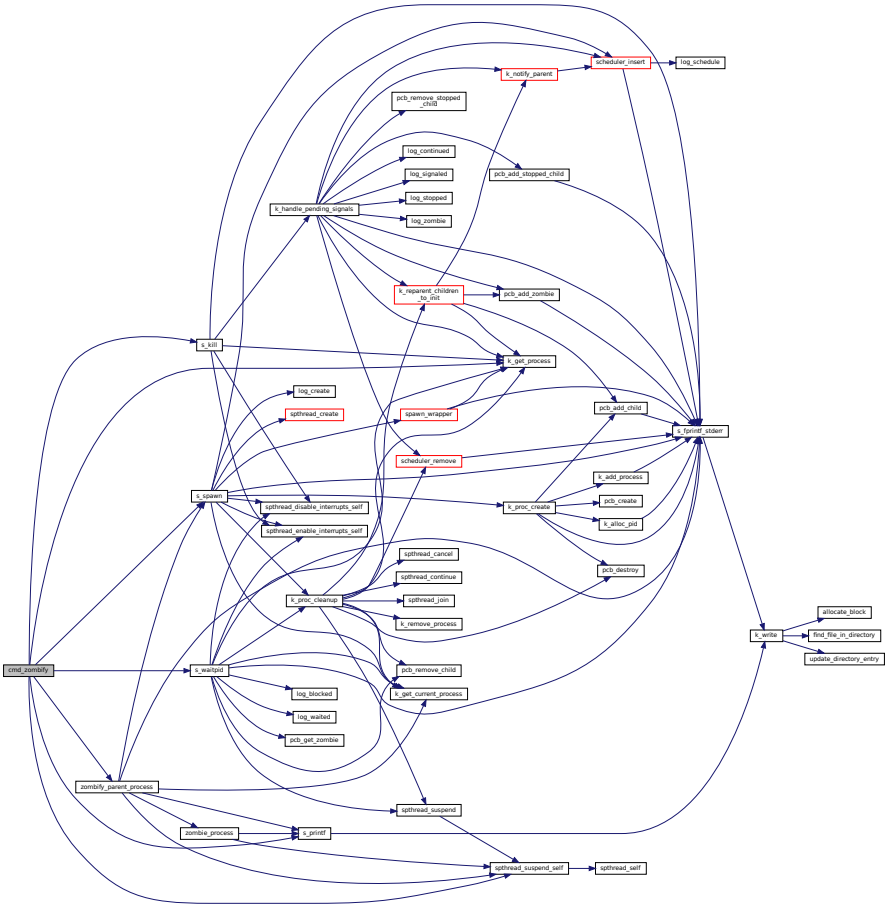
background	Whether to run the zombie process in background

Definition at line 395 of file commands.c.

References foreground_pid, k_get_process(), P_SIGTERM, s_kill(), s_printf(), s_spawn(), s_waitpid(), sigint_received, spthread_suspend_self(), pcb::state, ZOMBIE, and zombify_parent_process().

Referenced by execute_command().

Here is the call graph for this function:



Here is the caller graph for this function:



4.4.1.16 print_help() `void print_help ( void )`

Print help information for shell commands.

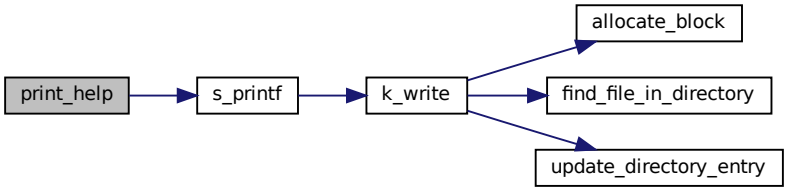
Displays usage information for all built-in shell commands.

Definition at line 35 of file commands.c.

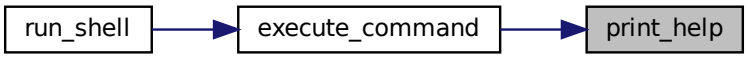
References s_printf().

Referenced by execute_command().

Here is the call graph for this function:



Here is the caller graph for this function:



4.4.1.17 run_shell() `void run_shell ( void )`

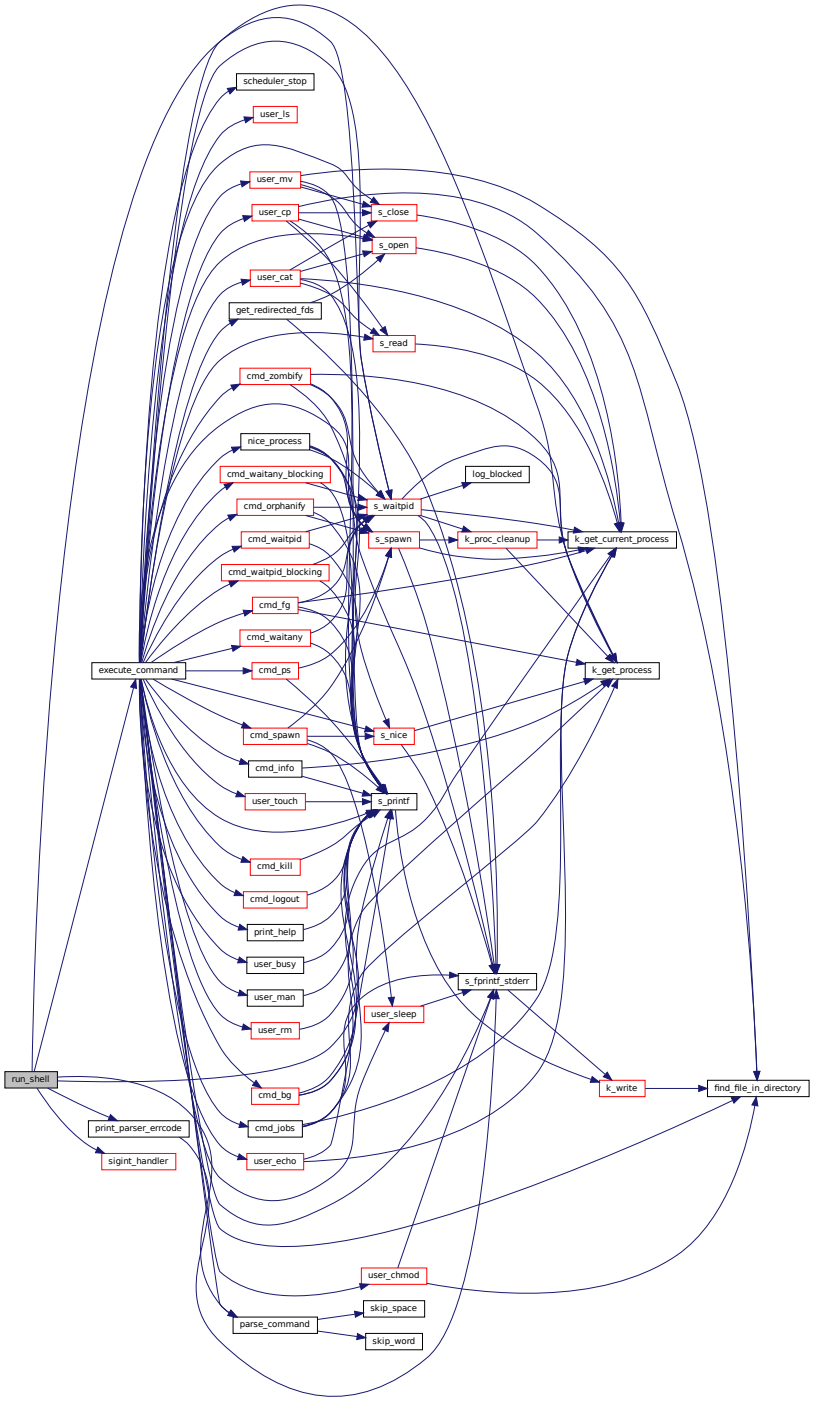
Run the interactive shell in the current process context.

Main shell loop - reads commands, parses them, and executes them.

Definition at line 751 of file shell.c.

References execute_command(), P_WIFSTOPPED, parse_command(), print_parser_errcode(), s_printf(), s_waitpid(), and sigint_handler().

Here is the call graph for this function:



4.4.1.18 shell_main() void* shell_main (void * *arg*)

Thread wrapper function for running the shell.

Parameters

arg	Unused argument (required for thread function signature)

Returns

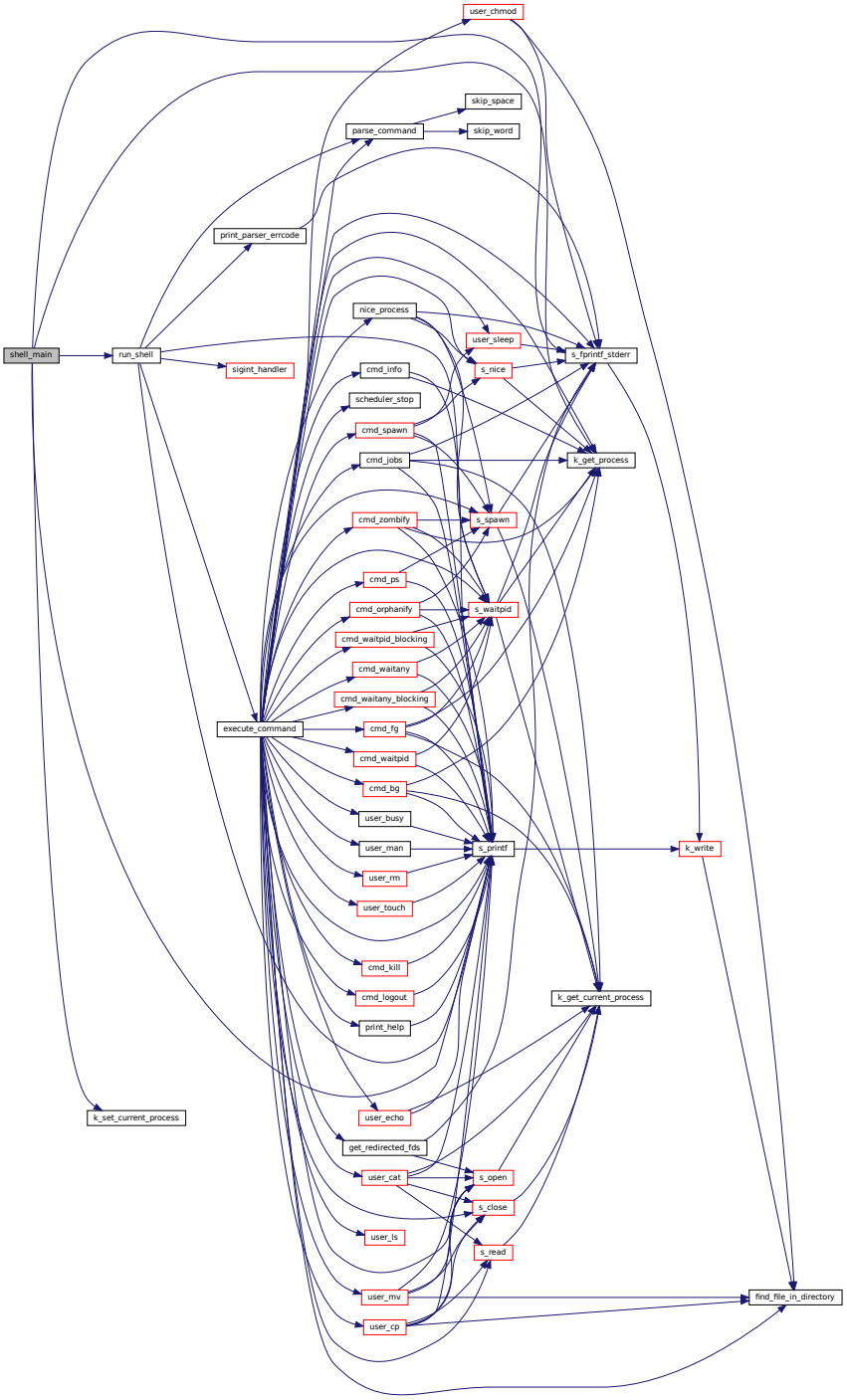
NULL when shell exits

Definition at line 840 of file shell.c.

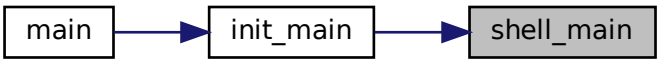
References pcb::children, k_get_process(), k_set_current_process(), pcb::num_children, run_shell(), s_fprintf_stderr(), and s_printf().

Referenced by init_main().

Here is the call graph for this function:



Here is the caller graph for this function:



4.4.1.19 sigint_handler() `void sigint_handler ( int sig )`

Signal handler for SIGINT (Ctrl-C)

Sends P_SIGTERM to the current foreground process and sets the sigint_received flag.

Parameters

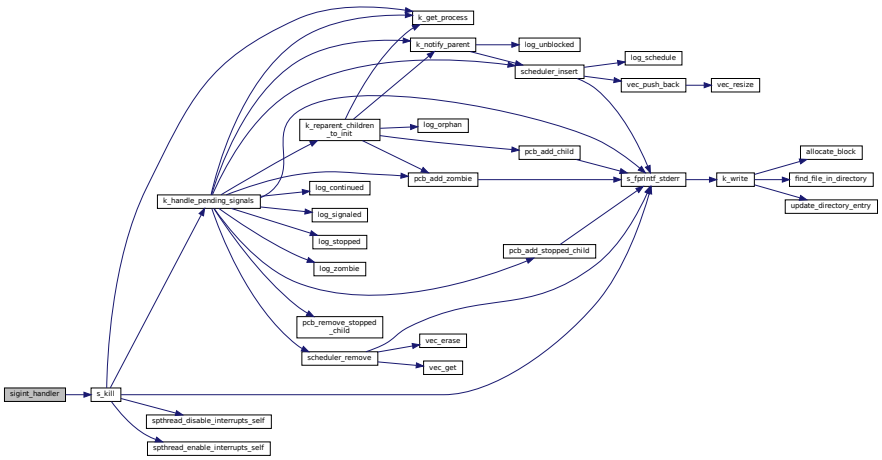
sig	Signal number (SIGINT)

Definition at line 23 of file commands.c.

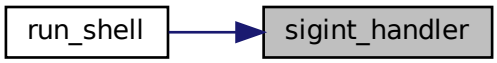
References foreground_pid, P_SIGTERM, s_kill(), and sigint_received.

Referenced by run_shell().

Here is the call graph for this function:



Here is the caller graph for this function:



4.4.2 Variable Documentation

4.4.2.1 foreground_pid `volatile pid_t foreground_pid [extern]`

PID of the current foreground process.

Shared state used by signal handler to send SIGTERM to foreground process on Ctrl-C. Set to -1 when no foreground process is running.

Definition at line 19 of file commands.c.

Referenced by cmd_fg(), cmd_ps(), cmd_zombify(), execute_command(), and sigint_handler().

4.4.2.2 sigint_received `volatile bool sigint_received [extern]`

Flag indicating whether SIGINT (Ctrl-C) was received.

Set by signal handler and checked by shell to determine if user interrupted command.

Definition at line 20 of file commands.c.

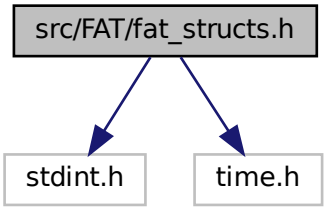
Referenced by cmd_waitany_blocking(), cmd_waitpid_blocking(), cmd_zombify(), and sigint_handler().

4.5 src/FAT/fat_structs.h File Reference

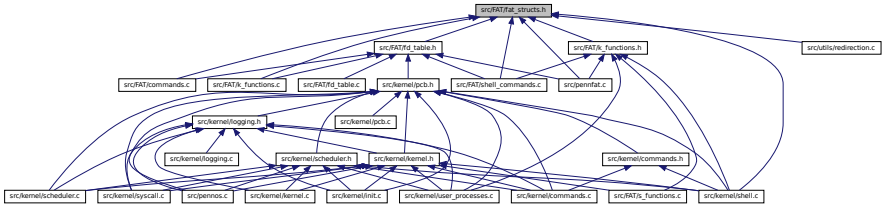
PennFAT filesystem data structures and constants.

```
#include <stdint.h>
#include <time.h>
```

Include dependency graph for fat_structs.h:



This graph shows which files directly or indirectly include this file:



Data Structures

- struct directory_entry_t

  Directory entry structure (64 bytes)

Macros

- #define F_WRITE 0

  File open modes.
- #define F_READ 1
- #define F_APPEND 2
- #define F_SEEK_SET 0

  File seek reference points (whence parameter for lseek)
- #define F_SEEK_CUR 1

- #define F_SEEK_END 2
- #define PERM_NONE 0

  File permission bits (Unix-style)
- #define PERM_WRITE 2
- #define PERM_READ 4
- #define PERM_READ_EXEC 5
- #define PERM_READ_WRITE 6
- #define PERM_ALL 7
- #define TYPE_UNKNOWN 0

  File type identifiers.
- #define TYPE_REGULAR 1
- #define TYPE_DIRECTORY 2
- #define TYPE_SYMLINK 4
- #define MAX_OPEN_FILES 256

  Maximum number of simultaneously open files in the system.
- #define FS_NOT_MOUNTED -1

  PennFAT error codes.
- #define FILE_NOT_FOUND -2
- #define FILE_ALREADY_OPEN -3
- #define NO_FREE_FD -4
- #define INVALID_MODE -5

4.5.1 Detailed Description

PennFAT filesystem data structures and constants.

Defines the core data structures, file modes, permissions, and error codes for the PennFAT filesystem implementation.

4.5.2 Macro Definition Documentation

4.5.2.1 F_APPEND `#define F_APPEND 2`

Append mode - create if not exists, seek to end

Definition at line 20 of file fat_structs.h.

4.5.2.2 F_READ `#define F_READ 1`

Read mode - file must exist

Definition at line 19 of file fat_structs.h.

4.5.2.3 F_SEEK_CUR `#define F_SEEK_CUR 1`

Seek from current position

Definition at line 26 of file fat_structs.h.

4.5.2.4 F_SEEK_END `#define F_SEEK_END 2`

Seek from end of file

Definition at line 27 of file fat_structs.h.

4.5.2.5 F_SEEK_SET `#define F_SEEK_SET 0`

File seek reference points (whence parameter for lseek)

Seek from beginning of file

Definition at line 25 of file fat_structs.h.

4.5.2.6 F_WRITE `#define F_WRITE 0`

File open modes.

Write mode - truncate file if exists, create if not

Definition at line 18 of file fat_structs.h.

4.5.2.7 FILE_ALREADY_OPEN `#define FILE_ALREADY_OPEN -3`

File is already open

Definition at line 57 of file fat_structs.h.

4.5.2.8 FILE_NOT_FOUND `#define FILE_NOT_FOUND -2`

Requested file does not exist

Definition at line 56 of file fat_structs.h.

4.5.2.9 FS_NOT_MOUNTED `#define FS_NOT_MOUNTED -1`

PennFAT error codes.

No filesystem is currently mounted

Definition at line 55 of file fat_structs.h.

4.5.2.10 INVALID_MODE `#define INVALID_MODE -5`

Invalid file open mode specified

Definition at line 59 of file fat_structs.h.

4.5.2.11 MAX_OPEN_FILES `#define MAX_OPEN_FILES 256`

Maximum number of simultaneously open files in the system.

Definition at line 50 of file fat_structs.h.

4.5.2.12 NO_FREE_FD `#define NO_FREE_FD -4`

No free file descriptors available

Definition at line 58 of file fat_structs.h.

4.5.2.13 PERM_ALL `#define PERM_ALL 7`

All permissions (read, write, execute)

Definition at line 37 of file fat_structs.h.

4.5.2.14 PERM_NONE `#define PERM_NONE 0`

File permission bits (Unix-style)

No permissions

Definition at line 32 of file fat_structs.h.

4.5.2.15 PERM_READ `#define PERM_READ 4`

Read permission

Definition at line 34 of file fat_structs.h.

4.5.2.16 PERM_READ_EXEC `#define PERM_READ_EXEC 5`

Read and execute permissions

Definition at line 35 of file fat_structs.h.

4.5.2.17 PERM_READ_WRITE `#define PERM_READ_WRITE 6`

Read and write permissions

Definition at line 36 of file fat_structs.h.

4.5.2.18 PERM_WRITE `#define PERM_WRITE 2`

Write permission

Definition at line 33 of file fat_structs.h.

4.5.2.19 TYPE_DIRECTORY `#define TYPE_DIRECTORY 2`

Directory (not fully supported)

Definition at line 44 of file fat_structs.h.

4.5.2.20 TYPE_REGULAR `#define TYPE_REGULAR 1`

Regular file

Definition at line 43 of file fat_structs.h.

4.5.2.21 TYPE_SYMLINK `#define TYPE_SYMLINK 4`

Symbolic link (not fully supported)

Definition at line 45 of file fat_structs.h.

4.5.2.22 TYPE_UNKNOWN `#define TYPE_UNKNOWN 0`

File type identifiers.

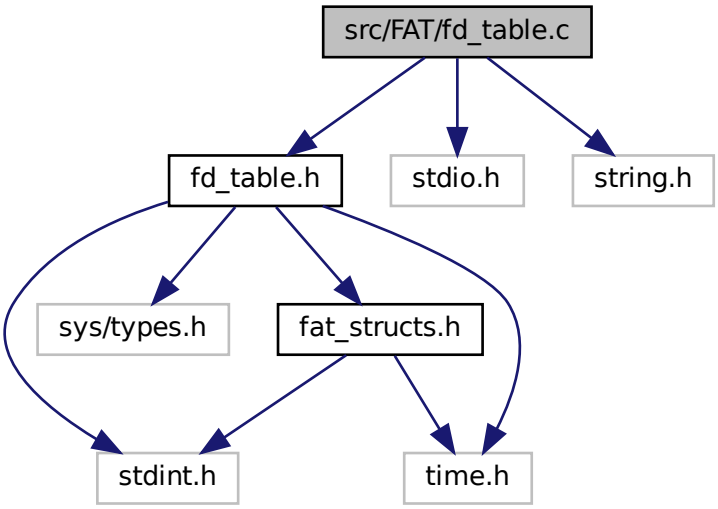
Unknown file type

Definition at line 42 of file fat_structs.h.

4.6 src/FAT/fd_table.c File Reference

```
#include "fd_table.h"
#include <stdio.h>
#include <string.h>
```

Include dependency graph for fd_table.c:



Functions

- void init_global_fd_table ()

  Initialize the global file descriptor table.
- void increment_refcount (int global_index)

  Increment the reference count for a global FD.
- void decrement_refcount (int global_index)

  Decrement the reference count for a global FD.

Variables

- global_fd_entry_t global_fd_table [256]

  Global file descriptor table (MAX_OPEN_FILES entries)

4.6.1 Function Documentation

4.6.1.1 decrement_refcount() `void decrement_refcount ( int global_index )`

Decrement the reference count for a global FD.

Called when a process closes a file descriptor. If refcount reaches zero, the entry becomes available for reuse.

Parameters

global_index	Index into global_fd_table

Definition at line 27 of file fd_table.c.

References global_fd_table, MAX_OPEN_FILES, and global_fd_entry_t::refcount.

4.6.1.2 increment_refcount() `void increment_refcount ( int global_index )`

Increment the reference count for a global FD.

Called when a process inherits or duplicates a file descriptor (e.g., via fork).

Parameters

global_index	Index into global_fd_table

Definition at line 21 of file fd_table.c.

References global_fd_table, MAX_OPEN_FILES, and global_fd_entry_t::refcount.

4.6.1.3 init_global_fd_table() `void init_global_fd_table ( )`

Initialize the global file descriptor table.

Sets all entries to unused (refcount=0). Must be called once during system initialization before any file operations.

Definition at line 8 of file fd_table.c.

References global_fd_table, MAX_OPEN_FILES, and global_fd_entry_t::refcount.

Referenced by mount().

Here is the caller graph for this function:



4.6.2 Variable Documentation

4.6.2.1 global_fd_table `global_fd_entry_t global_fd_table[256]`

Global file descriptor table (MAX_OPEN_FILES entries)

Shared across all processes in the system. Indexed by global FD number.

Definition at line 6 of file fd_table.c.

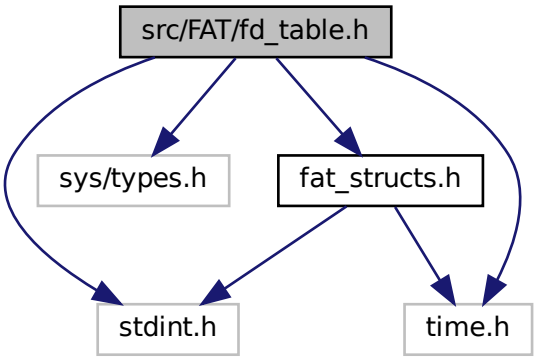
Referenced by decrement_refcount(), increment_refcount(), init_global_fd_table(), k_close(), k_lseek(), k_open(), k_↩ read(), k_unlink(), k_write(), and main().

4.7 src/FAT/fd_table.h File Reference

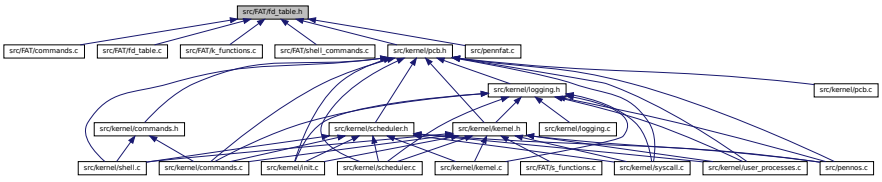
Global file descriptor table for PennFAT.

```
#include <stdint.h>
#include <sys/types.h>
#include <time.h>
#include "fat_structs.h"
```

Include dependency graph for fd_table.h:



This graph shows which files directly or indirectly include this file:



Data Structures

- struct global_fd_entry_t

  Global file descriptor entry.

Macros

- #define MAX_FDS 16

  Maximum number of file descriptors per process.
- #define F_WRITE 0

  File open modes (duplicated from fat_structs.h for convenience)
- #define F_READ 1
- #define F_APPEND 2
- #define IS_FD_IN_USE(fd) (global_fd_table[fd].refcount $>$ 0)

  Check if a file descriptor is in use.

Functions

- void init_global_fd_table ()

  Initialize the global file descriptor table.
- void increment_refcount (int global_index)

  Increment the reference count for a global FD.
- void decrement_refcount (int global_index)

  Decrement the reference count for a global FD.

Variables

- global_fd_entry_t global_fd_table [256]

  Global file descriptor table (MAX_OPEN_FILES entries)

4.7.1 Detailed Description

Global file descriptor table for PennFAT.

Manages open file descriptors across all processes. Each process has a local FD table (fd_table[MAX_FDS]) that indexes into this global table. Multiple processes can share file descriptors via fork/inheritance.

4.7.2 Macro Definition Documentation

4.7.2.1 F_APPEND `#define F_APPEND 2`

Append mode

Definition at line 28 of file fd_table.h.

4.7.2.2 F_READ `#define F_READ 1`

Read mode

Definition at line 27 of file fd_table.h.

4.7.2.3 F_WRITE `#define F_WRITE 0`

File open modes (duplicated from fat_structs.h for convenience)

Write mode

Definition at line 26 of file fd_table.h.

4.7.2.4 IS_FD_IN_USE `#define IS_FD_IN_USE( fd ) (global_fd_table[fd].refcount > 0)`

Check if a file descriptor is in use.

Returns true if the global FD entry has a non-zero reference count.

Definition at line 52 of file fd_table.h.

4.7.2.5 MAX_FDS `#define MAX_FDS 16`

Maximum number of file descriptors per process.

Definition at line 21 of file fd_table.h.

4.7.3 Function Documentation

4.7.3.1 decrement_refcount() `void decrement_refcount ( int global_index )`

Decrement the reference count for a global FD.

Called when a process closes a file descriptor. If refcount reaches zero, the entry becomes available for reuse.

Parameters

global_index	Index into global_fd_table

Definition at line 27 of file fd_table.c.

References global_fd_table, MAX_OPEN_FILES, and global_fd_entry_t::refcount.

4.7.3.2 increment_refcount() `void increment_refcount ( int global_index )`

Increment the reference count for a global FD.

Called when a process inherits or duplicates a file descriptor (e.g., via fork).

Parameters

global_index	Index into global_fd_table

Definition at line 21 of file fd_table.c.

References global_fd_table, MAX_OPEN_FILES, and global_fd_entry_t::refcount.

4.7.3.3 init_global_fd_table() `void init_global_fd_table ( )`

Initialize the global file descriptor table.

Sets all entries to unused (refcount=0). Must be called once during system initialization before any file operations.

Definition at line 8 of file fd_table.c.

References global_fd_table, MAX_OPEN_FILES, and global_fd_entry_t::refcount.

Referenced by mount().

Here is the caller graph for this function:



4.7.4 Variable Documentation

4.7.4.1 global_fd_table `global_fd_entry_t global_fd_table[256] [extern]`

Global file descriptor table (MAX_OPEN_FILES entries)

Shared across all processes in the system. Indexed by global FD number.

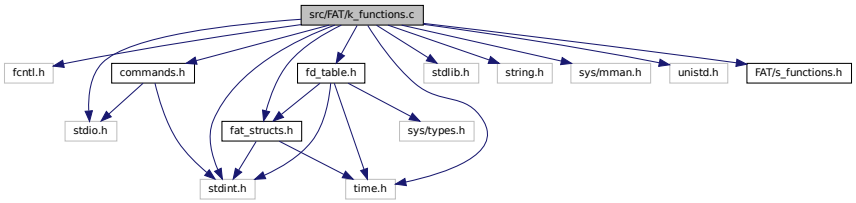
Definition at line 6 of file fd_table.c.

Referenced by decrement_refcount(), increment_refcount(), init_global_fd_table(), k_close(), k_lseek(), k_open(), k_↩ read(), k_unlink(), k_write(), and main().

4.8 src/FAT/k_functions.c File Reference

```
#include <fcntl.h>
#include <stdint.h>
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <sys/mman.h>
#include <time.h>
#include <unistd.h>
#include "commands.h"
#include "fat_structs.h"
#include "fd_table.h"
#include "FAT/s_functions.h"
```

Include dependency graph for k_functions.c:



Functions

- int update_directory_entry (const char *fname, directory_entry_t *updated_entry)

  Update an existing directory entry.
- int allocate_block ()

  Allocate a free block from the FAT.
- void print_directory_entry (directory_entry_t *entry)

  Print a directory entry in human-readable format.
- void print_fat_table ()

  Print the entire FAT table for debugging.
- int find_file_in_directory (const char *fname, directory_entry_t *entry)

  Find a file in the directory and retrieve its entry.
- int allocate_fd ()

  Allocate a new file descriptor.
- int create_directory_entry_at_block (int block, const char *fname, directory_entry_t *entry)
- int create_directory_entry (const char *fname, directory_entry_t *entry)

  Create a new directory entry for a file.
- int k_open (const char *fname, int mode)

  Open a file in the FAT filesystem.
- int k_write (int fd, const char *str, int n)

  Write data to an open file.
- int k_read (int fd, int n, char *buf)

  Read data from an open file.
- int k_close (int fd)

  Close an open file descriptor.
- int k_lseek (int fd, int offset, int whence)

  Reposition the file offset.
- int k_ls (const char *filename)

  List directory contents or show file info.
- int k_unlink (const char *fname)

  Delete a file from the filesystem.

4.8.1 Function Documentation

4.8.1.1 allocate_block() `int allocate_block ( )`

Allocate a free block from the FAT.

Scans the FAT for an available block (value 0) and marks it as end-of-chain (0xFFFF).

Returns

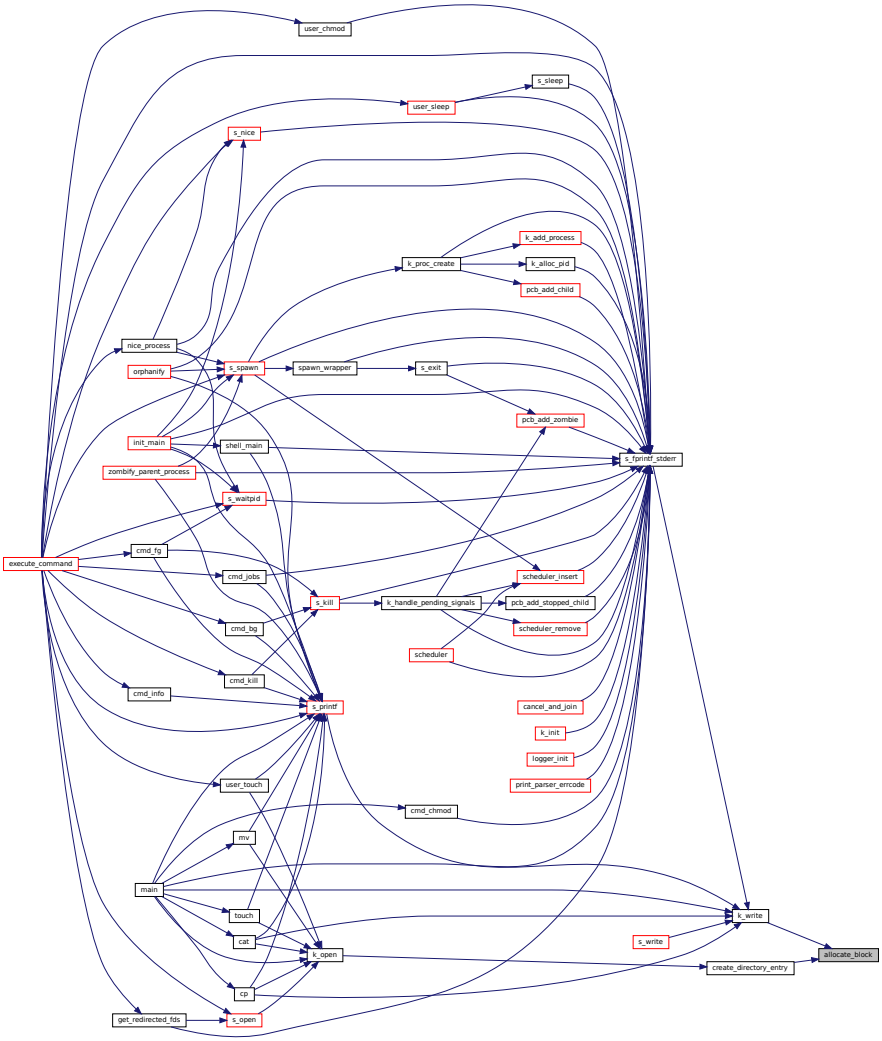
Block number, or -1 if no free blocks available

Definition at line 19 of file k_functions.c.

References _block_size, _fat, _fat_size, and _fs_fd.

Referenced by create_directory_entry(), and k_write().

Here is the caller graph for this function:



4.8.1.2 allocate_fd() `int allocate_fd ( )`

Allocate a new file descriptor.

Finds an available slot in the global file descriptor table and returns its index.

Returns

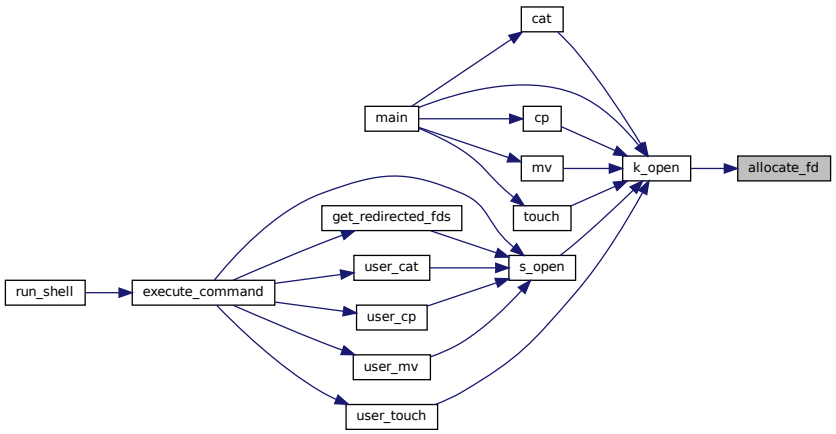
File descriptor number, or -1 if no slots available

Definition at line 141 of file k_functions.c.

References IS_FD_IN_USE, and MAX_OPEN_FILES.

Referenced by k_open().

Here is the caller graph for this function:



4.8.1.3 create_directory_entry() `int create_directory_entry ( const char * fname, directory_entry_t * entry )`

Create a new directory entry for a file.

Allocates space in the root directory and initializes a new entry with the specified filename. Sets initial metadata (size=0, type, permissions, modification time).

Parameters

fname	Filename for the new entry
entry	Output parameter for the created entry

Returns

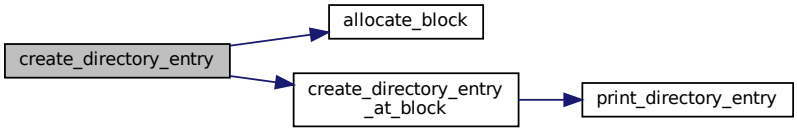
0 on success, -1 on failure

Definition at line 203 of file k_functions.c.

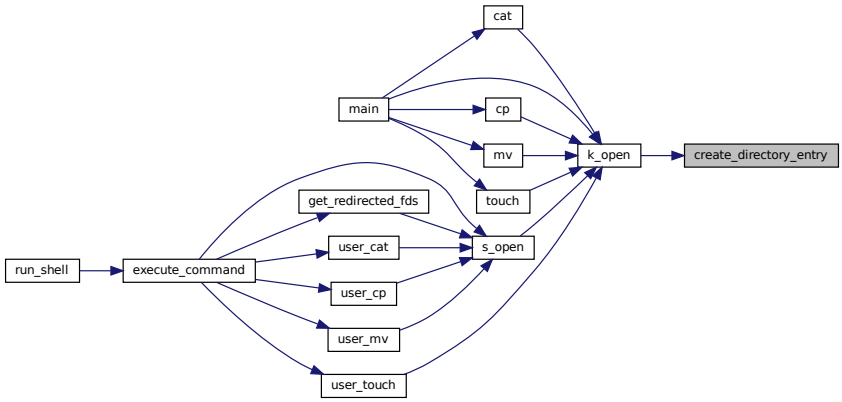
References _fat, allocate_block(), and create_directory_entry_at_block().

Referenced by k_open().

Here is the call graph for this function:



Here is the caller graph for this function:



4.8.1.4 create_directory_entry_at_block()

```
int create_directory_entry_at_block (
        int block,
        const char * fname,
        directory_entry_t * entry )
```

< Regular file

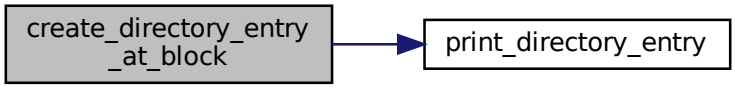
< Read and write permissions

Definition at line 151 of file k_functions.c.

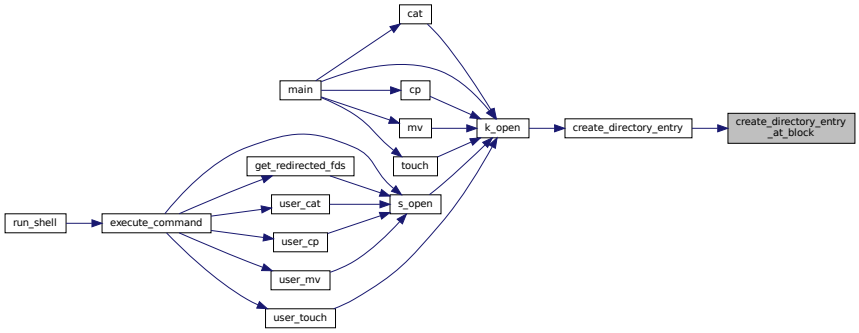
References _block_size, _fat_size, _fs_fd, directory_entry_t::firstBlock, directory_entry_t::mtime, directory_entry_t::name, directory_entry_t::perm, PERM_READ_WRITE, print_directory_entry(), directory_entry_t::size, directory_entry_t::type, and TYPE_REGULAR.

Referenced by create_directory_entry().

Here is the call graph for this function:



Here is the caller graph for this function:



4.8.1.5 find_file_in_directory()

```
int find_file_in_directory (
        const char * fname,
        directory_entry_t * entry )
```

Find a file in the directory and retrieve its entry.

Searches the root directory for a file with the specified name and populates the entry structure with its metadata if found.

Parameters

fname	Filename to search for
entry	Output parameter for directory entry

Returns

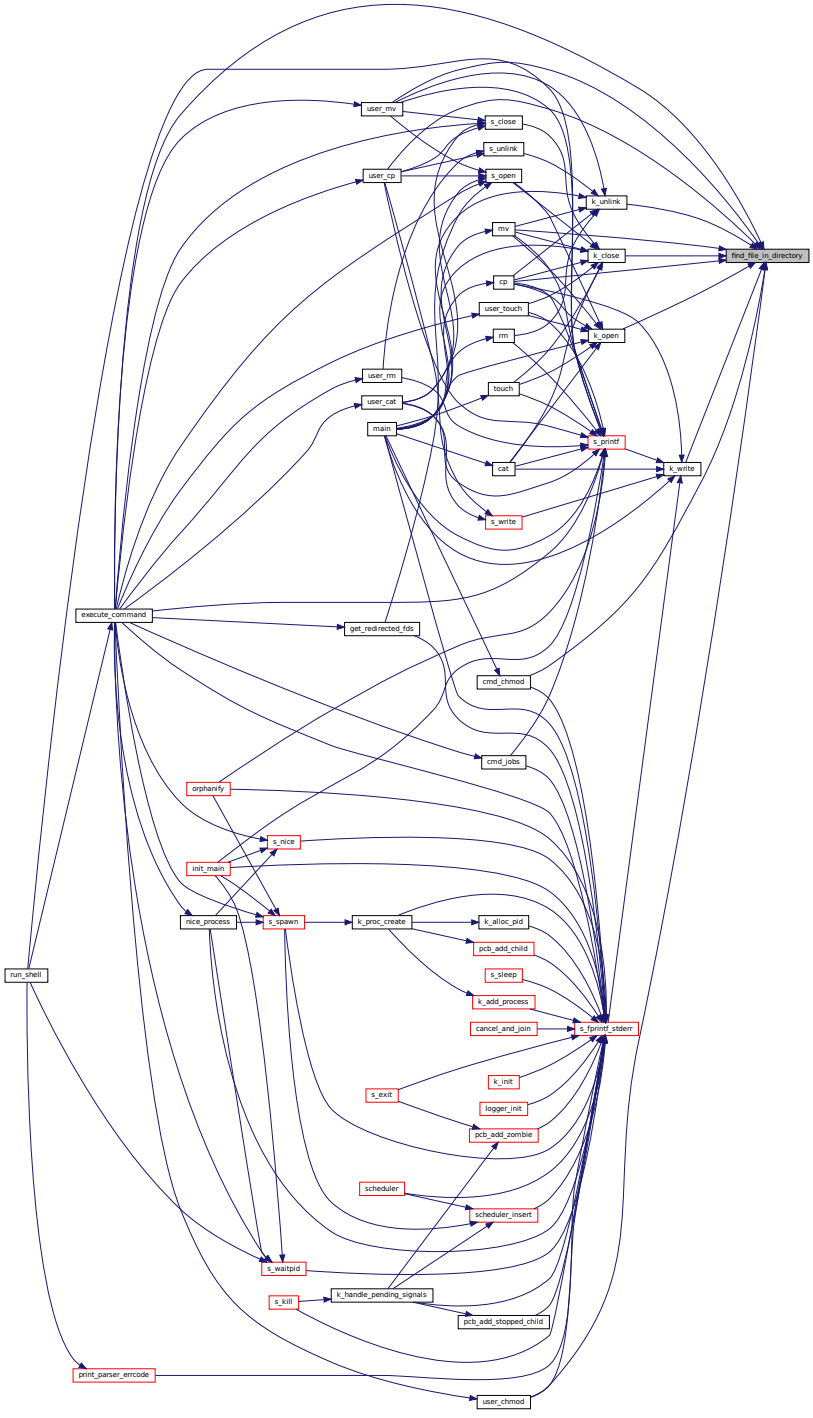
0 on success, -1 if file not found

Definition at line 94 of file k_functions.c.

References _block_size, _fat, _fat_size, and _fs_fd.

Referenced by cmd_chmod(), cp(), execute_command(), k_close(), k_open(), k_unlink(), k_write(), mv(), user_chmod(), user_cp(), and user_mv().

Here is the caller graph for this function:



4.8.1.6 k_close() int k_close (int *fd*)

Close an open file descriptor.

Releases the file descriptor and updates the reference count in the global FD table.

Parameters

fd	File descriptor to close

Returns

0 on success, -1 on error

< Write mode - truncate file if exists, create if not

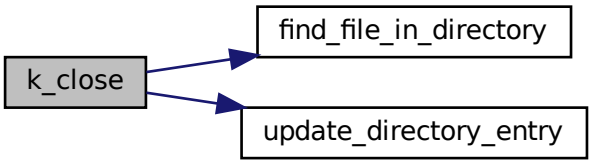
< Append mode - create if not exists, seek to end

Definition at line 502 of file k_functions.c.

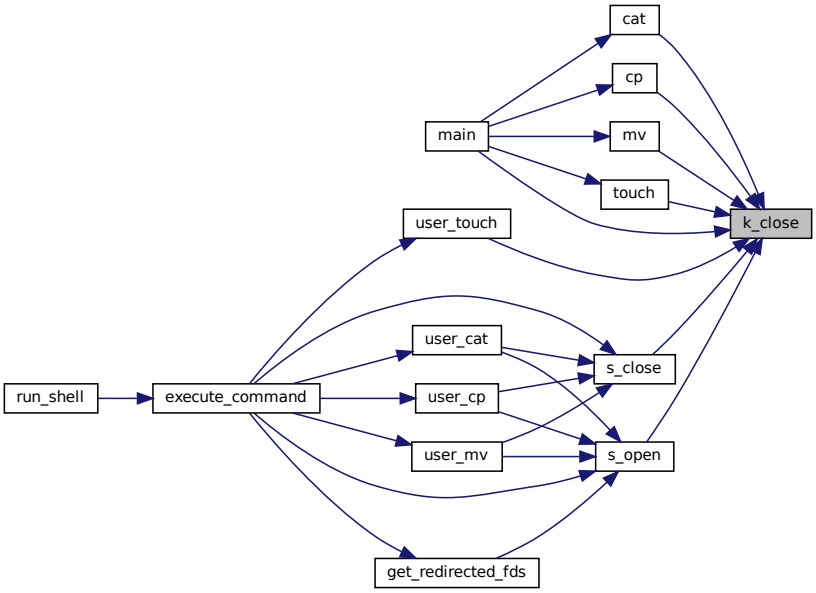
References F_APPEND, F_WRITE, find_file_in_directory(), global_fd_entry_t::first_block, directory_entry_t::firstBlock, global_fd_table, IS_FD_IN_USE, MAX_OPEN_FILES, directory_entry_t::mtime, global_fd_entry_t::refcount, directory_entry_t::size, global_fd_entry_t::size, and update_directory_entry().

Referenced by cat(), cp(), main(), mv(), s_close(), s_open(), touch(), and user_touch().

Here is the call graph for this function:



Here is the caller graph for this function:



4.8.1.7 k_ls() `int k_ls ( const char * filename )`

List directory contents or show file info.

If filename is NULL or empty, lists all files in the directory. Otherwise, displays detailed information about the specific file.

Parameters

filename	Specific file to query, or NULL to list all

Returns

0 on success, -1 on error

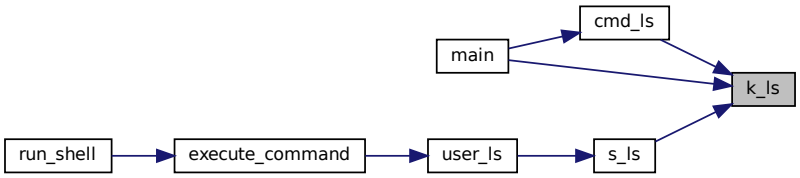
< No filesystem is currently mounted

Definition at line 552 of file k_functions.c.

References _block_size, _current_mount, _fat, _fat_size, _fs_fd, and FS_NOT_MOUNTED.

Referenced by cmd_ls(), main(), and s_ls().

Here is the caller graph for this function:



4.8.1.8 k_lseek()

```
int k_lseek (
        int fd,
        int offset,
        int whence )
```

Reposition the file offset.

Changes the current position in the file for subsequent read/write operations. Supports SEEK_SET, SEEK_CUR, and SEEK_END.

Parameters

fd	File descriptor
offset	Offset in bytes
whence	Reference point: F_SEEK_SET (0), F_SEEK_CUR (1), F_SEEK_END (2)

Returns

New absolute offset, or -1 on error

< Seek from beginning of file

< Seek from current position

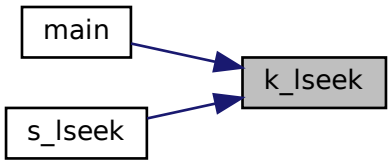
< Seek from end of file

Definition at line 527 of file k_functions.c.

References F_SEEK_CUR, F_SEEK_END, F_SEEK_SET, global_fd_table, IS_FD_IN_USE, MAX_OPEN_FILES, global_fd_entry_t::offset, and global_fd_entry_t::size.

Referenced by main(), and s_lseek().

Here is the caller graph for this function:



4.8.1.9 k_open()

```
int k_open (
        const char * fname,
        int mode )
```

Open a file in the FAT filesystem.

Opens the specified file with the given mode (F_WRITE, F_READ, F_APPEND). Creates the file if it doesn't exist and mode is F_WRITE or F_APPEND. Returns a file descriptor for subsequent operations.

Parameters

fname	Filename to open
mode	Open mode: F_WRITE (0), F_READ (1), or F_APPEND (2)

Returns

File descriptor number, or -1 on failure

< No filesystem is currently mounted

< Write mode - truncate file if exists, create if not

< Read mode - file must exist

< Append mode - create if not exists, seek to end

< Invalid file open mode specified

< Read mode - file must exist

< Requested file does not exist

< Write mode - truncate file if exists, create if not

< Append mode - create if not exists, seek to end

< Write mode - truncate file if exists, create if not

< Append mode - create if not exists, seek to end

< Write mode - truncate file if exists, create if not

< Append mode - create if not exists, seek to end

< File is already open

< No free file descriptors available

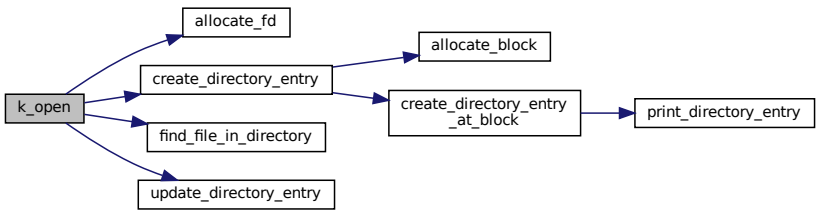
< Append mode - create if not exists, seek to end

Definition at line 233 of file k_functions.c.

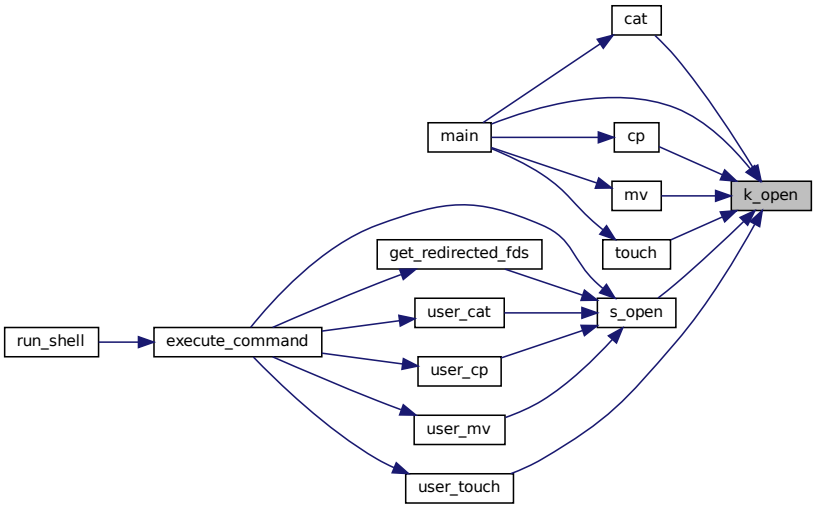
References _current_mount, _fat, allocate_fd(), create_directory_entry(), F_APPEND, F_READ, F_WRITE, FILE_↩ ALREADY_OPEN, FILE_NOT_FOUND, global_fd_entry_t::filename, find_file_in_directory(), global_fd_entry_t::first_↩ block, directory_entry_t::firstBlock, FS_NOT_MOUNTED, global_fd_table, INVALID_MODE, IS_FD_IN_USE, MAX_↩ OPEN_FILES, global_fd_entry_t::mode, directory_entry_t::mtime, NO_FREE_FD, global_fd_entry_t::offset, directory↩ _entry_t::perm, global_fd_entry_t::perm, global_fd_entry_t::refcount, directory_entry_t::size, global_fd_entry_t::size, and update_directory_entry().

Referenced by cat(), cp(), main(), mv(), s_open(), touch(), and user_touch().

Here is the call graph for this function:



Here is the caller graph for this function:



4.8.1.10 k_read()

```
int k_read (
        int fd,
        int n,
        char * buf )
```

Read data from an open file.

Reads up to n bytes from the file associated with fd into buf. Respects file size and current offset.

Parameters

fd	File descriptor
n	Maximum number of bytes to read
buf	Buffer to store read data

Returns

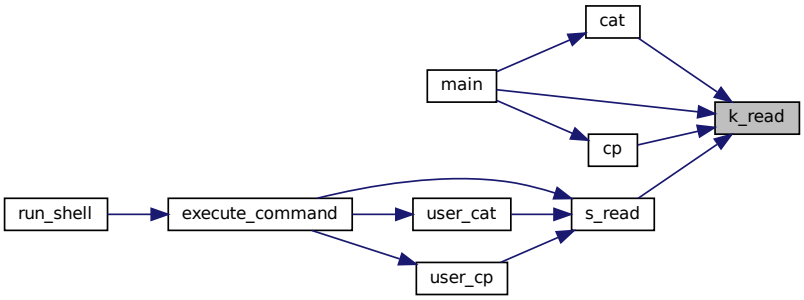
Number of bytes read, or -1 on error

Definition at line 442 of file k_functions.c.

References _block_size, _fat, _fat_size, _fs_fd, global_fd_entry_t::first_block, global_fd_table, IS_FD_IN_USE, MAX↩_OPEN_FILES, global_fd_entry_t::offset, and global_fd_entry_t::size.

Referenced by cat(), cp(), main(), and s_read().

Here is the caller graph for this function:



4.8.1.11 k_unlink() `int k_unlink ( const char * fname )`

Delete a file from the filesystem.

Removes the file's directory entry and frees all blocks in the FAT chain. File must not be currently open.

Parameters

fname	Filename to delete

Returns

0 on success, -1 on error

< No filesystem is currently mounted

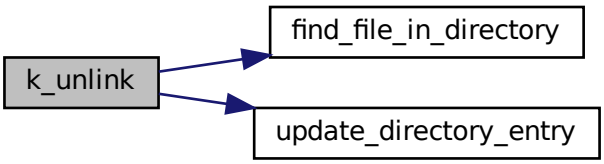
< Requested file does not exist

Definition at line 626 of file k_functions.c.

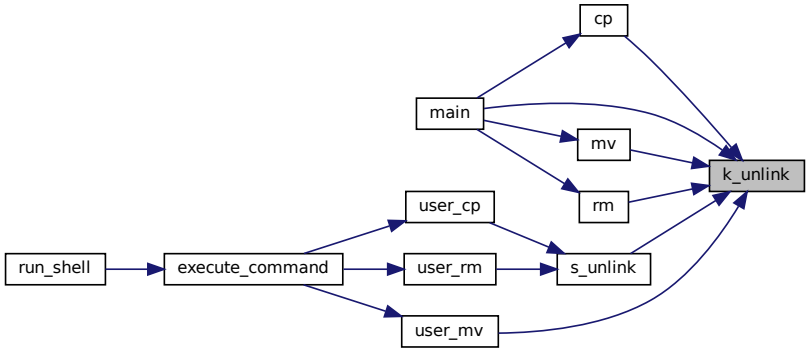
References _current_mount, _fat, FILE_NOT_FOUND, find_file_in_directory(), directory_entry_t::firstBlock, FS_NOT↩ _MOUNTED, global_fd_table, IS_FD_IN_USE, MAX_OPEN_FILES, directory_entry_t::name, and update_directory_↩ entry().

Referenced by cp(), main(), mv(), rm(), s_unlink(), and user_mv().

Here is the call graph for this function:



Here is the caller graph for this function:



4.8.1.12 k_write()

```
int k_write (
        int fd,
        const char * str,
        int n )
```

Write data to an open file.

Writes up to n bytes from str to the file associated with fd. Allocates new blocks as needed. Updates file size in directory entry.

Parameters

fd	File descriptor
str	Buffer containing data to write
n	Number of bytes to write

Returns

Number of bytes written, or -1 on error

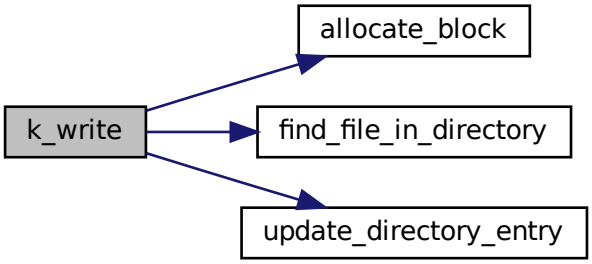
< Read mode - file must exist

Definition at line 355 of file k_functions.c.

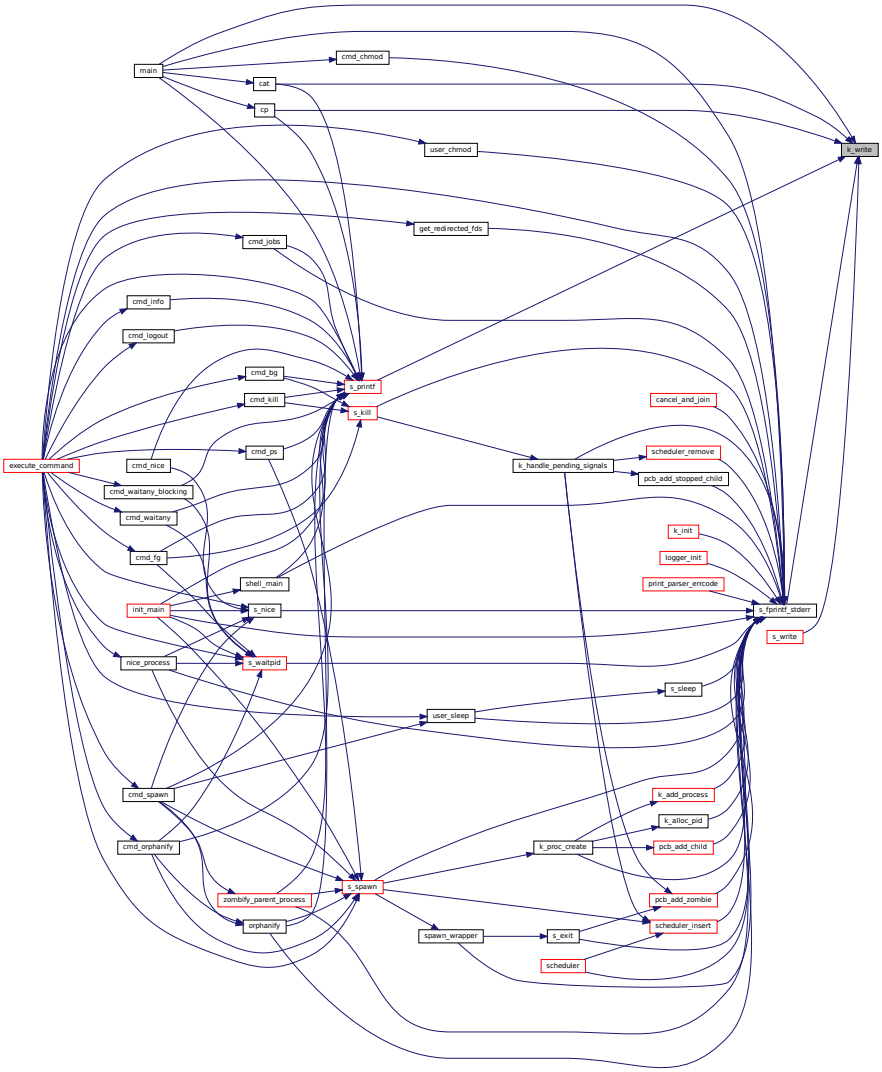
References _block_size, _fat, _fat_size, _fs_fd, allocate_block(), F_READ, find_file_in_directory(), global_fd_entry↩
_t::first_block, directory_entry_t::firstBlock, global_fd_table, IS_FD_IN_USE, MAX_OPEN_FILES, directory_entry_t↩
::mtime, global_fd_entry_t::offset, directory_entry_t::size, global_fd_entry_t::size, and update_directory_entry().

Referenced by cat(), cp(), main(), s_fprintf_stderr(), s_printf(), and s_write().

Here is the call graph for this function:



Here is the caller graph for this function:



4.8.1.13 print_directory_entry() `void print_directory_entry ( directory_entry_t * entry )`

Print a directory entry in human-readable format.

Displays filename, size, permissions, and modification time.

Parameters

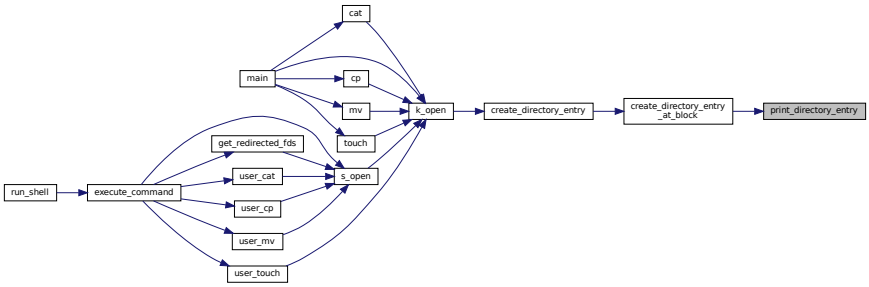
entry	Directory entry to print

Definition at line 36 of file k_functions.c.

References directory_entry_t::firstBlock, directory_entry_t::mtime, directory_entry_t::name, directory_entry_t::perm, directory_entry_t::size, and directory_entry_t::type.

Referenced by create_directory_entry_at_block().

Here is the caller graph for this function:



4.8.1.14 print_fat_table() `void print_fat_table ( )`

Print the entire FAT table for debugging.

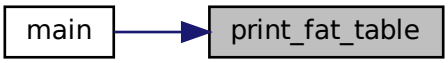
Displays all FAT entries showing block allocation chains.

Definition at line 59 of file k_functions.c.

References _block_size, _fat, and _fat_size.

Referenced by main().

Here is the caller graph for this function:



4.8.1.15 update_directory_entry() `int update_directory_entry ( const char * fname, directory_entry_t * updated_entry )`

Update an existing directory entry.

Writes the updated entry back to the directory on disk. Used to persist changes to file metadata (size, permissions, mtime, etc.).

Parameters

fname	Filename of entry to update
updated_entry	New entry data to write

Returns

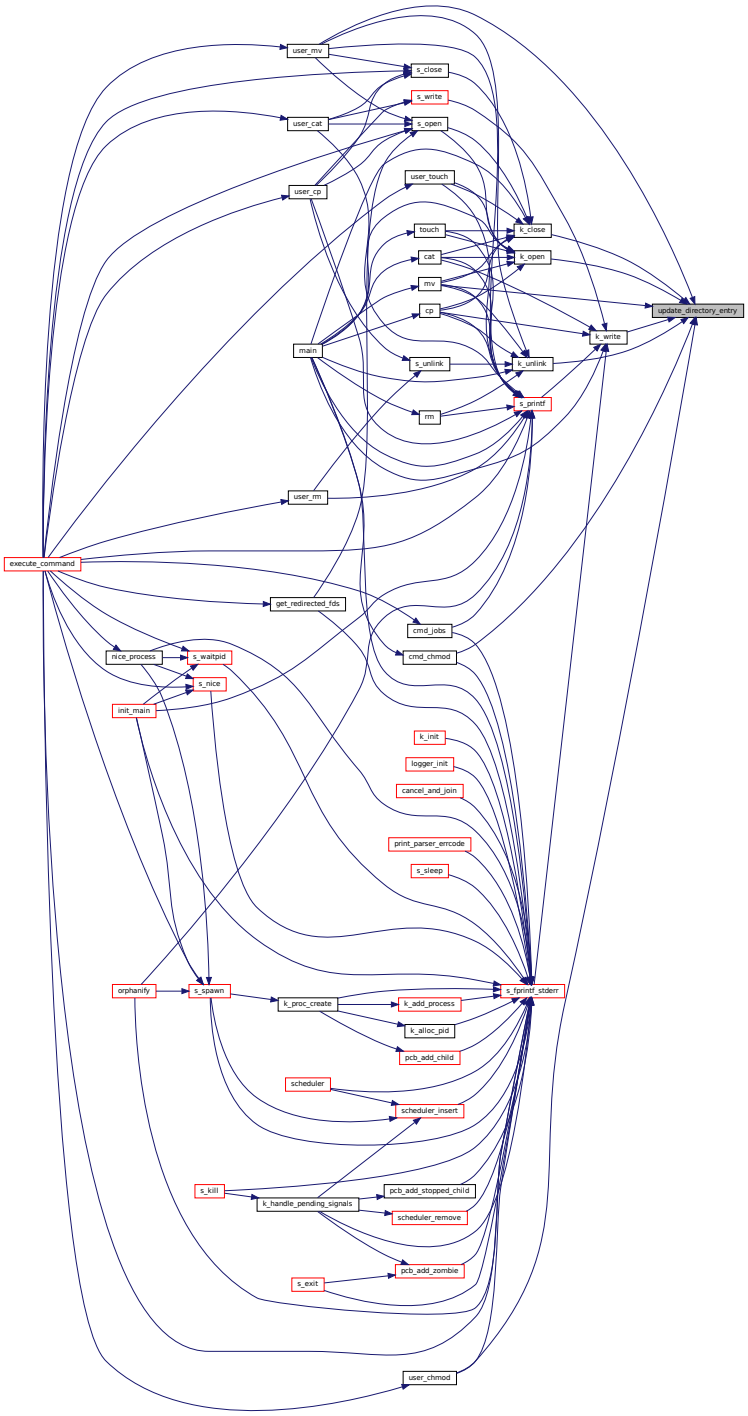
0 on success, -1 on failure

Definition at line 318 of file k_functions.c.

References _block_size, _fat, _fat_size, and _fs_fd.

Referenced by cmd_chmod(), k_close(), k_open(), k_unlink(), k_write(), mv(), user_chmod(), and user_mv().

Here is the caller graph for this function:

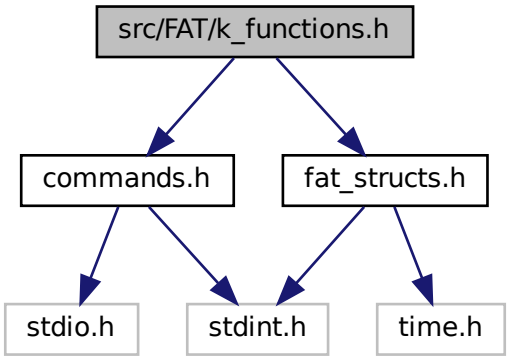


4.9 src/FAT/k_functions.h File Reference

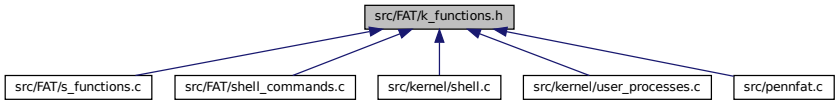
```
#include "commands.h"
```

```
#include "fat_structs.h"
```

Include dependency graph for k_functions.h:



This graph shows which files directly or indirectly include this file:



Functions

- int find_file_in_directory (const char *fname, directory_entry_t *entry)

  Find a file in the directory and retrieve its entry.
- int allocate_fd ()

  Allocate a new file descriptor.
- int create_directory_entry (const char *fname, directory_entry_t *entry)

  Create a new directory entry for a file.
- int update_directory_entry (const char *fname, directory_entry_t *updated_entry)

  Update an existing directory entry.
- int k_open (const char *fname, int mode)

  Open a file in the FAT filesystem.
- int allocate_block ()

  Allocate a free block from the FAT.
- int k_write (int fd, const char *str, int n)

  Write data to an open file.
- int k_read (int fd, int n, char *buf)

Read data from an open file.

- int k_close (int fd)

  Close an open file descriptor.

- int k_lseek (int fd, int offset, int whence)

  Reposition the file offset.

- int k_ls (const char *filename)

  List directory contents or show file info.

- int k_unlink (const char *fname)

  Delete a file from the filesystem.

- void print_directory_entry (directory_entry_t *entry)

  Print a directory entry in human-readable format.

- void print_fat_table ()

  Print the entire FAT table for debugging.

4.9.1 Function Documentation

4.9.1.1 allocate_block() `int allocate_block ( )`

Allocate a free block from the FAT.

Scans the FAT for an available block (value 0) and marks it as end-of-chain (0xFFFF).

Returns

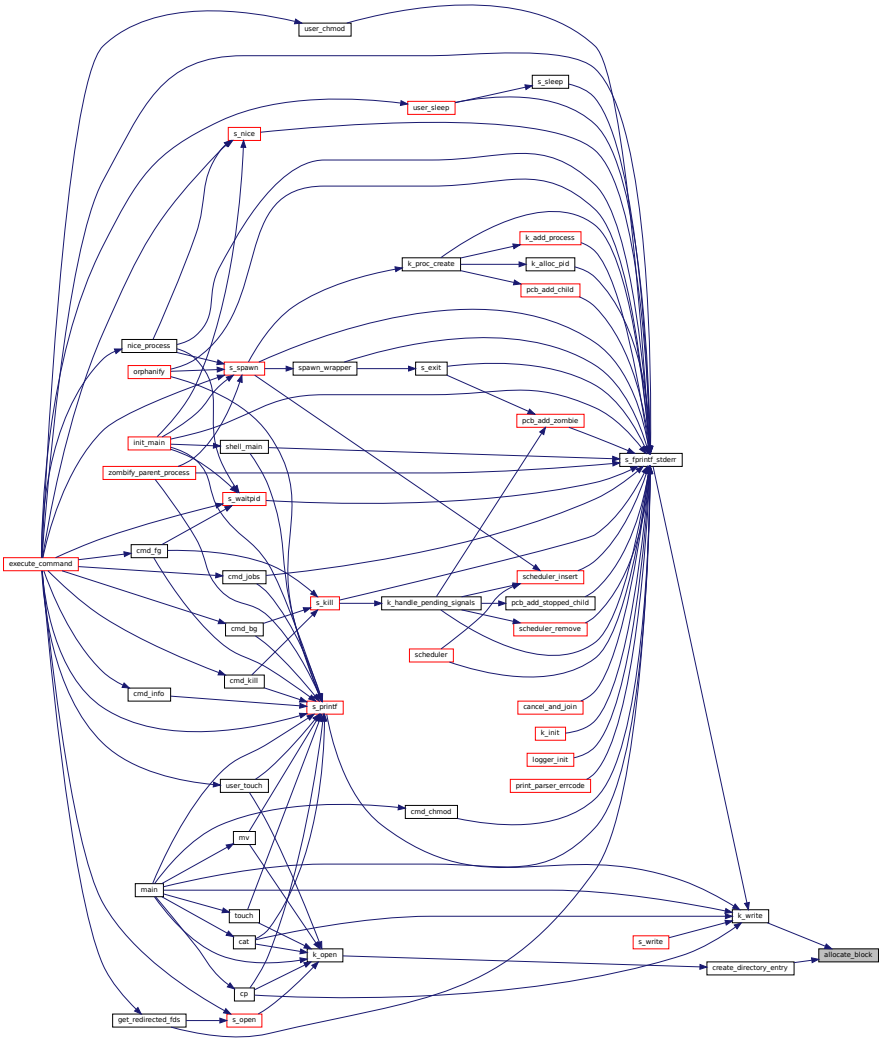
Block number, or -1 if no free blocks available

Definition at line 19 of file k_functions.c.

References _block_size, _fat, _fat_size, and _fs_fd.

Referenced by create_directory_entry(), and k_write().

Here is the caller graph for this function:



4.9.1.2 allocate_fd() `int allocate_fd ( )`

Allocate a new file descriptor.

Finds an available slot in the global file descriptor table and returns its index.

Returns

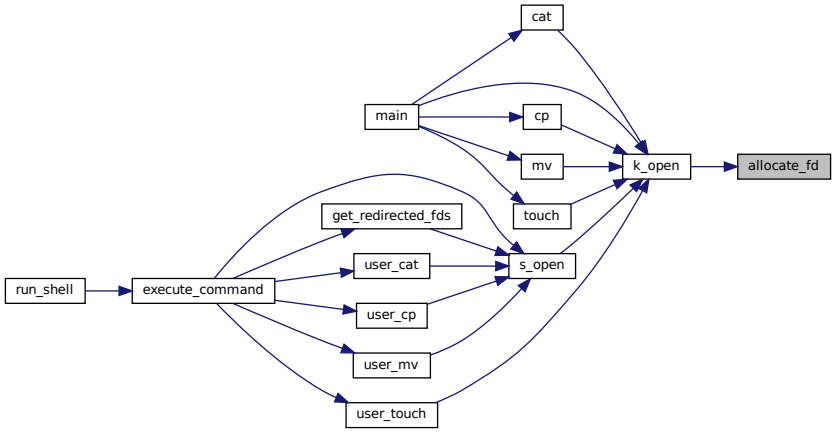
File descriptor number, or -1 if no slots available

Definition at line 141 of file k_functions.c.

References IS_FD_IN_USE, and MAX_OPEN_FILES.

Referenced by k_open().

Here is the caller graph for this function:



4.9.1.3 create_directory_entry()

```
int create_directory_entry (
        const char * fname,
        directory_entry_t * entry )
```

Create a new directory entry for a file.

Allocates space in the root directory and initializes a new entry with the specified filename. Sets initial metadata (size=0, type, permissions, modification time).

Parameters

fname	Filename for the new entry
entry	Output parameter for the created entry

Returns

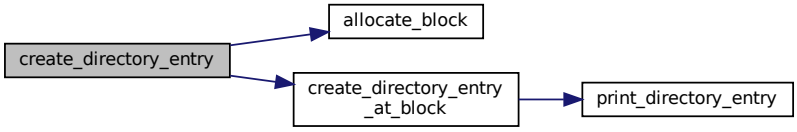
0 on success, -1 on failure

Definition at line 203 of file k_functions.c.

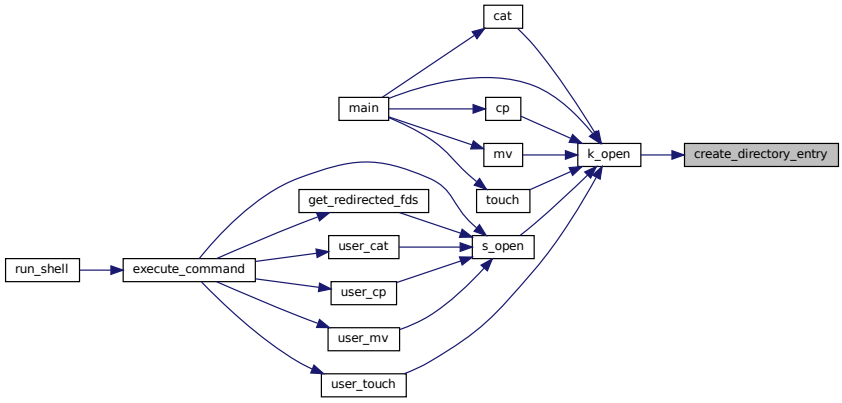
References _fat, allocate_block(), and create_directory_entry_at_block().

Referenced by k_open().

Here is the call graph for this function:



Here is the caller graph for this function:



4.9.1.4 find_file_in_directory()

```
int find_file_in_directory (
        const char * fname,
        directory_entry_t * entry )
```

Find a file in the directory and retrieve its entry.

Searches the root directory for a file with the specified name and populates the entry structure with its metadata if found.

Parameters

fname	Filename to search for
entry	Output parameter for directory entry

Returns

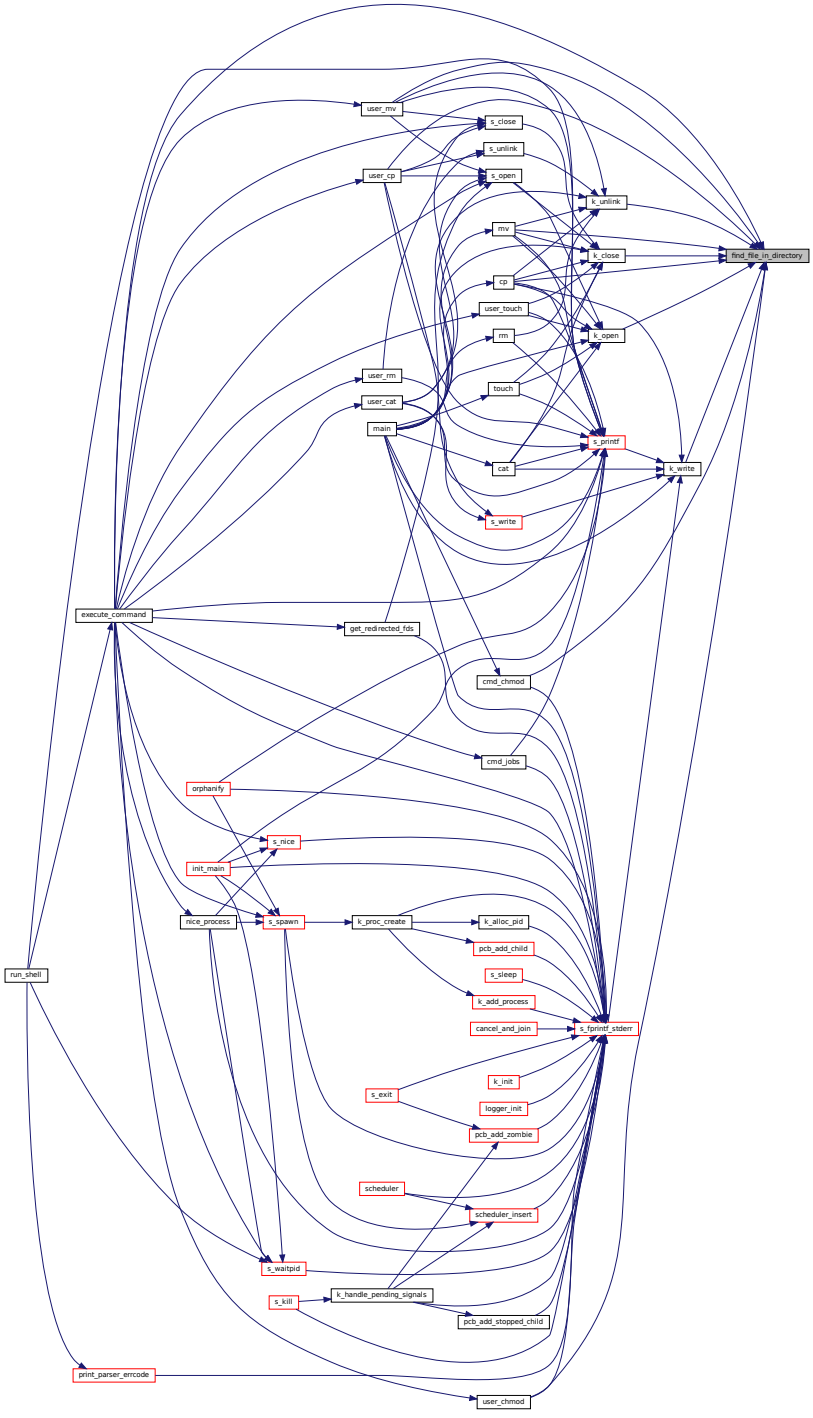
0 on success, -1 if file not found

Definition at line 94 of file k_functions.c.

References _block_size, _fat, _fat_size, and _fs_fd.

Referenced by cmd_chmod(), cp(), execute_command(), k_close(), k_open(), k_unlink(), k_write(), mv(), user_chmod(), user_cp(), and user_mv().

Here is the caller graph for this function:



4.9.1.5 k_close() int k_close (int *fd*)

Close an open file descriptor.

Releases the file descriptor and updates the reference count in the global FD table.

Parameters

fd	File descriptor to close

Returns

0 on success, -1 on error

< Write mode - truncate file if exists, create if not

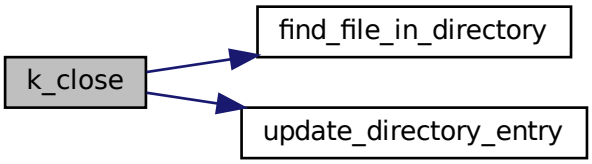
< Append mode - create if not exists, seek to end

Definition at line 502 of file k_functions.c.

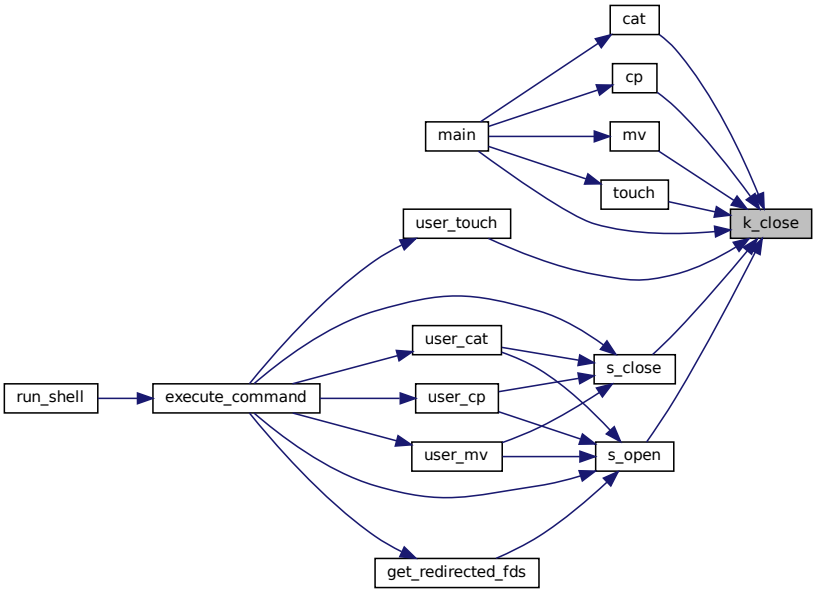
References F_APPEND, F_WRITE, find_file_in_directory(), global_fd_entry_t::first_block, directory_entry_t::firstBlock, global_fd_table, IS_FD_IN_USE, MAX_OPEN_FILES, directory_entry_t::mtime, global_fd_entry_t::refcount, directory_entry_t::size, global_fd_entry_t::size, and update_directory_entry().

Referenced by cat(), cp(), main(), mv(), s_close(), s_open(), touch(), and user_touch().

Here is the call graph for this function:



Here is the caller graph for this function:



4.9.1.6 k_ls() `int k_ls ( const char * filename )`

List directory contents or show file info.

If filename is NULL or empty, lists all files in the directory. Otherwise, displays detailed information about the specific file.

Parameters

filename	Specific file to query, or NULL to list all

Returns

0 on success, -1 on error

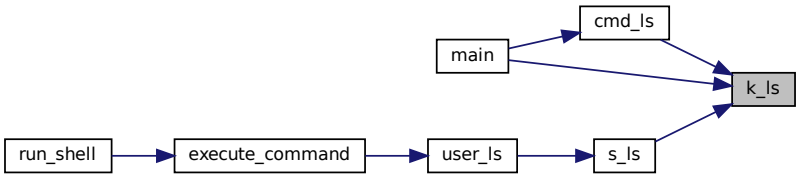
< No filesystem is currently mounted

Definition at line 552 of file k_functions.c.

References _block_size, _current_mount, _fat, _fat_size, _fs_fd, and FS_NOT_MOUNTED.

Referenced by cmd_ls(), main(), and s_ls().

Here is the caller graph for this function:



4.9.1.7 k_lseek()

```
int k_lseek (
        int fd,
        int offset,
        int whence )
```

Reposition the file offset.

Changes the current position in the file for subsequent read/write operations. Supports SEEK_SET, SEEK_CUR, and SEEK_END.

Parameters

fd	File descriptor
offset	Offset in bytes
whence	Reference point: F_SEEK_SET (0), F_SEEK_CUR (1), F_SEEK_END (2)

Returns

New absolute offset, or -1 on error

< Seek from beginning of file

< Seek from current position

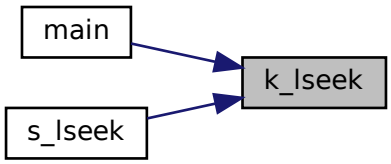
< Seek from end of file

Definition at line 527 of file k_functions.c.

References F_SEEK_CUR, F_SEEK_END, F_SEEK_SET, global_fd_table, IS_FD_IN_USE, MAX_OPEN_FILES, global_fd_entry_t::offset, and global_fd_entry_t::size.

Referenced by main(), and s_lseek().

Here is the caller graph for this function:



4.9.1.8 k_open()

```
int k_open (
        const char * fname,
        int mode )
```

Open a file in the FAT filesystem.

Opens the specified file with the given mode (F_WRITE, F_READ, F_APPEND). Creates the file if it doesn't exist and mode is F_WRITE or F_APPEND. Returns a file descriptor for subsequent operations.

Parameters

fname	Filename to open
mode	Open mode: F_WRITE (0), F_READ (1), or F_APPEND (2)

Returns

File descriptor number, or -1 on failure

< No filesystem is currently mounted

< Write mode - truncate file if exists, create if not

< Read mode - file must exist

< Append mode - create if not exists, seek to end

< Invalid file open mode specified

< Read mode - file must exist

< Requested file does not exist

< Write mode - truncate file if exists, create if not

< Append mode - create if not exists, seek to end

< Write mode - truncate file if exists, create if not

< Append mode - create if not exists, seek to end

< Write mode - truncate file if exists, create if not

< Append mode - create if not exists, seek to end

< File is already open

< No free file descriptors available

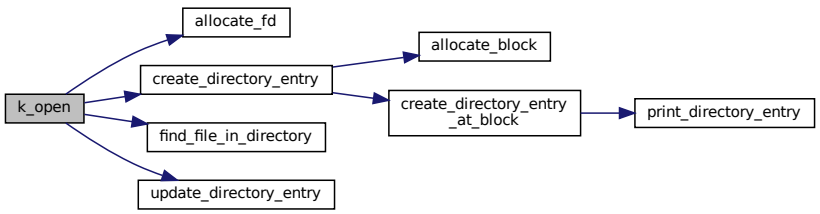
< Append mode - create if not exists, seek to end

Definition at line 233 of file k_functions.c.

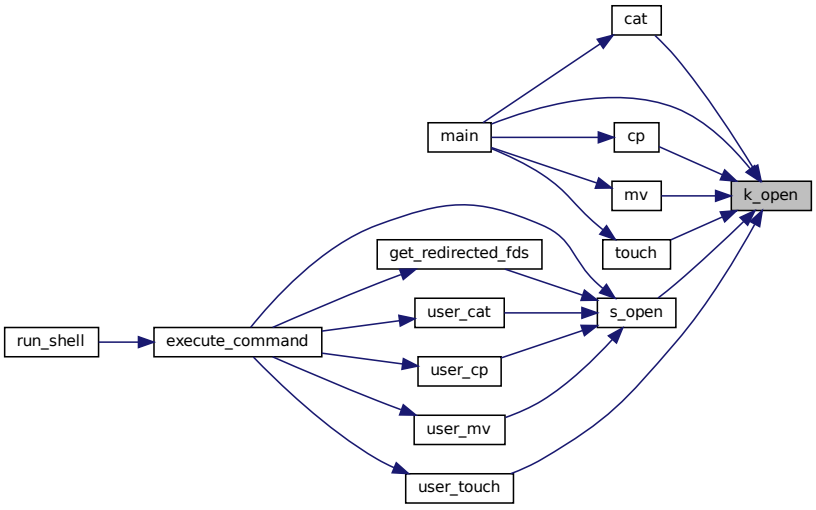
References _current_mount, _fat, allocate_fd(), create_directory_entry(), F_APPEND, F_READ, F_WRITE, FILE_↩ ALREADY_OPEN, FILE_NOT_FOUND, global_fd_entry_t::filename, find_file_in_directory(), global_fd_entry_t::first_↩ block, directory_entry_t::firstBlock, FS_NOT_MOUNTED, global_fd_table, INVALID_MODE, IS_FD_IN_USE, MAX_↩ OPEN_FILES, global_fd_entry_t::mode, directory_entry_t::mtime, NO_FREE_FD, global_fd_entry_t::offset, directory↩ _entry_t::perm, global_fd_entry_t::perm, global_fd_entry_t::refcount, directory_entry_t::size, global_fd_entry_t::size, and update_directory_entry().

Referenced by cat(), cp(), main(), mv(), s_open(), touch(), and user_touch().

Here is the call graph for this function:



Here is the caller graph for this function:



4.9.1.9 k_read()

```
int k_read (
        int fd,
        int n,
        char * buf )
```

Read data from an open file.

Reads up to n bytes from the file associated with fd into buf. Respects file size and current offset.

Parameters

fd	File descriptor
n	Maximum number of bytes to read
buf	Buffer to store read data

Returns

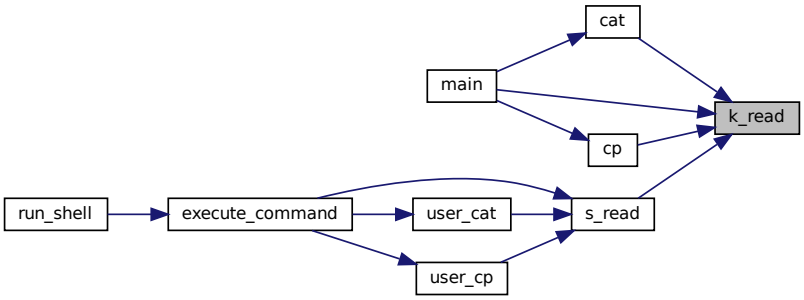
Number of bytes read, or -1 on error

Definition at line 442 of file k_functions.c.

References _block_size, _fat, _fat_size, _fs_fd, global_fd_entry_t::first_block, global_fd_table, IS_FD_IN_USE, MAX↩ _OPEN_FILES, global_fd_entry_t::offset, and global_fd_entry_t::size.

Referenced by cat(), cp(), main(), and s_read().

Here is the caller graph for this function:



4.9.1.10 k_unlink() `int k_unlink ( const char * fname )`

Delete a file from the filesystem.

Removes the file's directory entry and frees all blocks in the FAT chain. File must not be currently open.

Parameters

fname	Filename to delete

Returns

0 on success, -1 on error

< No filesystem is currently mounted

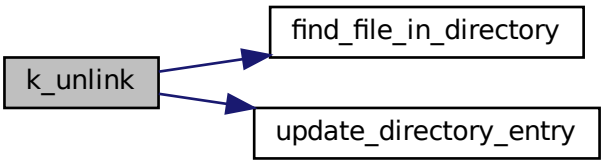
< Requested file does not exist

Definition at line 626 of file k_functions.c.

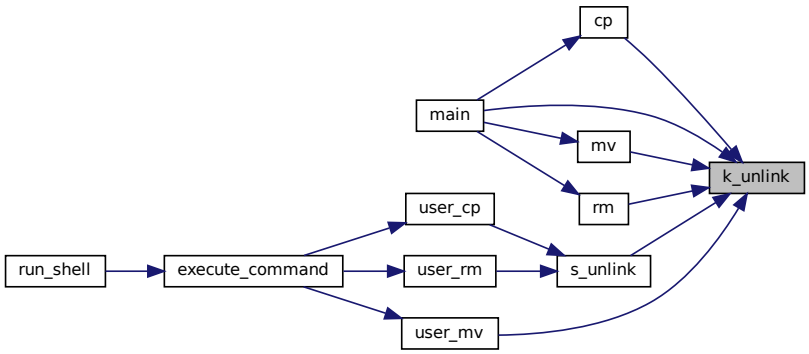
References _current_mount, _fat, FILE_NOT_FOUND, find_file_in_directory(), directory_entry_t::firstBlock, FS_NOT↩ _MOUNTED, global_fd_table, IS_FD_IN_USE, MAX_OPEN_FILES, directory_entry_t::name, and update_directory_↩ entry().

Referenced by cp(), main(), mv(), rm(), s_unlink(), and user_mv().

Here is the call graph for this function:



Here is the caller graph for this function:



4.9.1.11 k_write()

```
int k_write (
        int fd,
        const char * str,
        int n )
```

Write data to an open file.

Writes up to n bytes from str to the file associated with fd. Allocates new blocks as needed. Updates file size in directory entry.

Parameters

fd	File descriptor
str	Buffer containing data to write
n	Number of bytes to write

Returns

Number of bytes written, or -1 on error

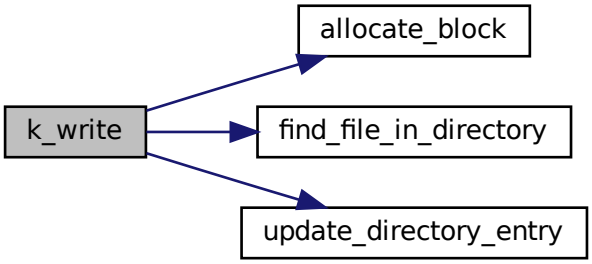
< Read mode - file must exist

Definition at line 355 of file k_functions.c.

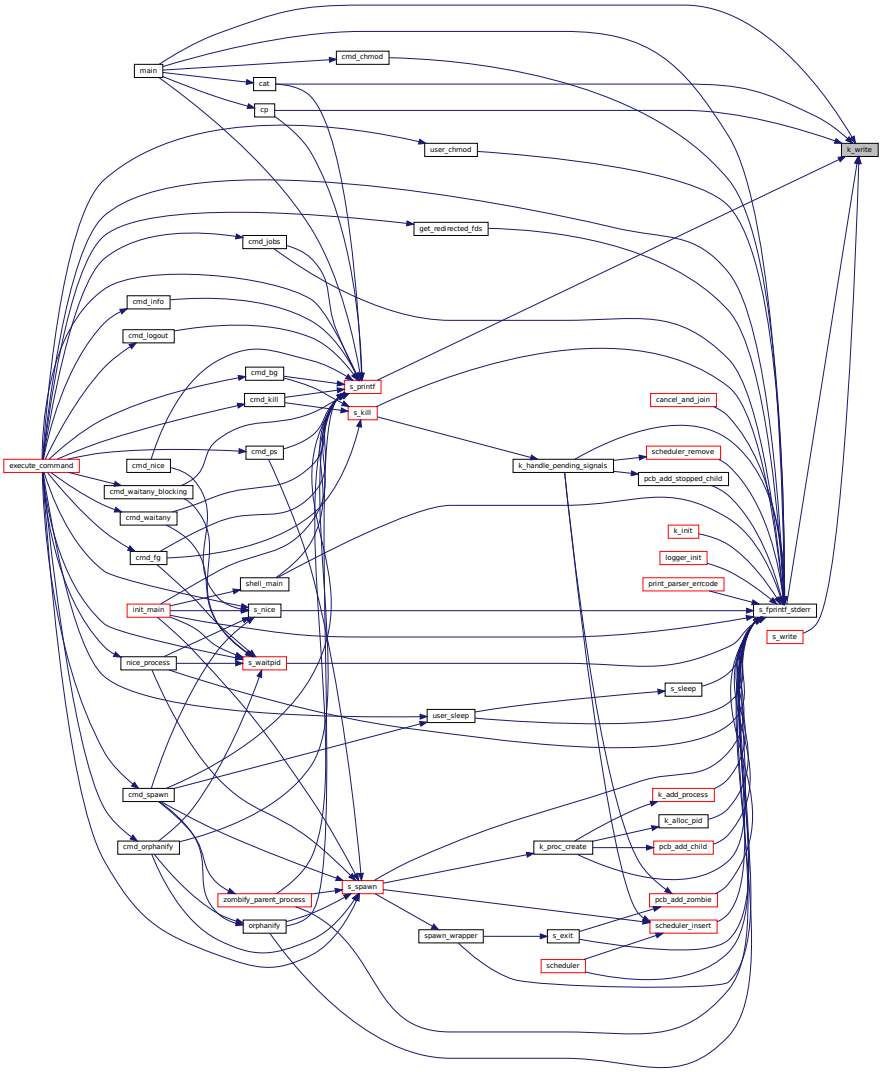
References _block_size, _fat, _fat_size, _fs_fd, allocate_block(), F_READ, find_file_in_directory(), global_fd_entry↩
_t::first_block, directory_entry_t::firstBlock, global_fd_table, IS_FD_IN_USE, MAX_OPEN_FILES, directory_entry_t↩
::mtime, global_fd_entry_t::offset, directory_entry_t::size, global_fd_entry_t::size, and update_directory_entry().

Referenced by cat(), cp(), main(), s_fprintf_stderr(), s_printf(), and s_write().

Here is the call graph for this function:



Here is the caller graph for this function:



4.9.1.12 print_directory_entry() `void print_directory_entry ( directory_entry_t * entry )`

Print a directory entry in human-readable format.

Displays filename, size, permissions, and modification time.

Parameters

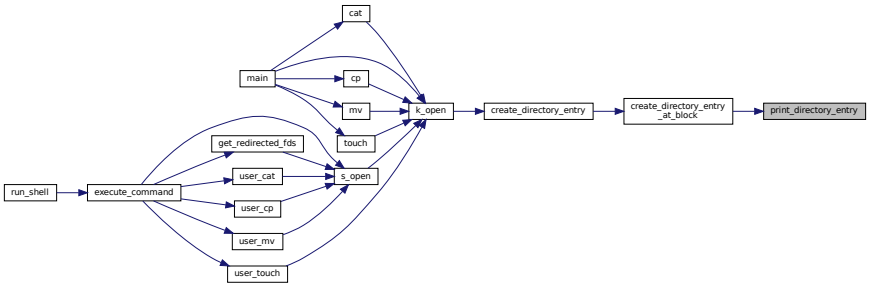
entry	Directory entry to print

Definition at line 36 of file k_functions.c.

References directory_entry_t::firstBlock, directory_entry_t::mtime, directory_entry_t::name, directory_entry_t::perm, directory_entry_t::size, and directory_entry_t::type.

Referenced by create_directory_entry_at_block().

Here is the caller graph for this function:



4.9.1.13 print_fat_table() `void print_fat_table ( )`

Print the entire FAT table for debugging.

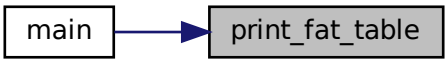
Displays all FAT entries showing block allocation chains.

Definition at line 59 of file k_functions.c.

References _block_size, _fat, and _fat_size.

Referenced by main().

Here is the caller graph for this function:



4.9.1.14 update_directory_entry() `int update_directory_entry (`

```
const char * fname,
directory_entry_t * updated_entry )
```

Update an existing directory entry.

Writes the updated entry back to the directory on disk. Used to persist changes to file metadata (size, permissions, mtime, etc.).

Parameters

fname	Filename of entry to update
updated_entry	New entry data to write

Returns

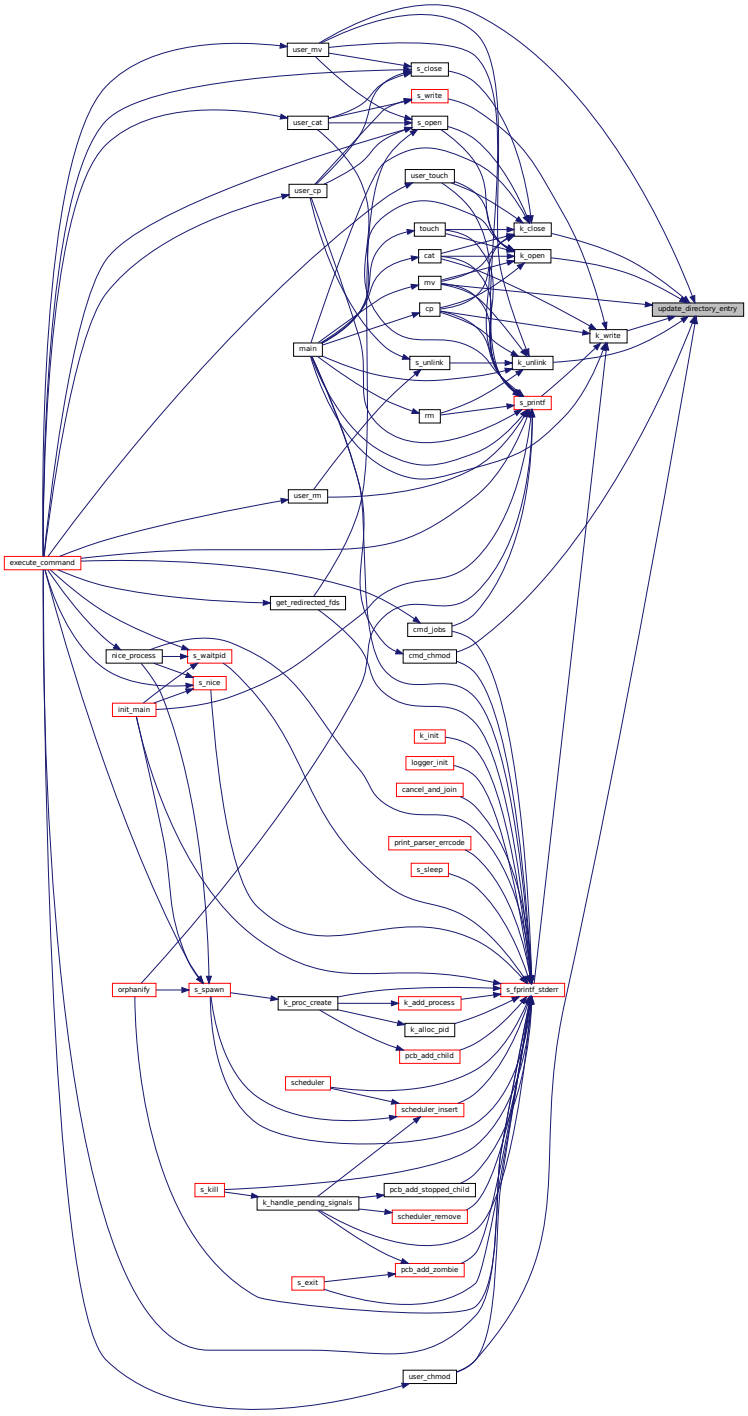
0 on success, -1 on failure

Definition at line 318 of file k_functions.c.

References _block_size, _fat, _fat_size, and _fs_fd.

Referenced by cmd_chmod(), k_close(), k_open(), k_unlink(), k_write(), mv(), user_chmod(), and user_mv().

Here is the caller graph for this function:

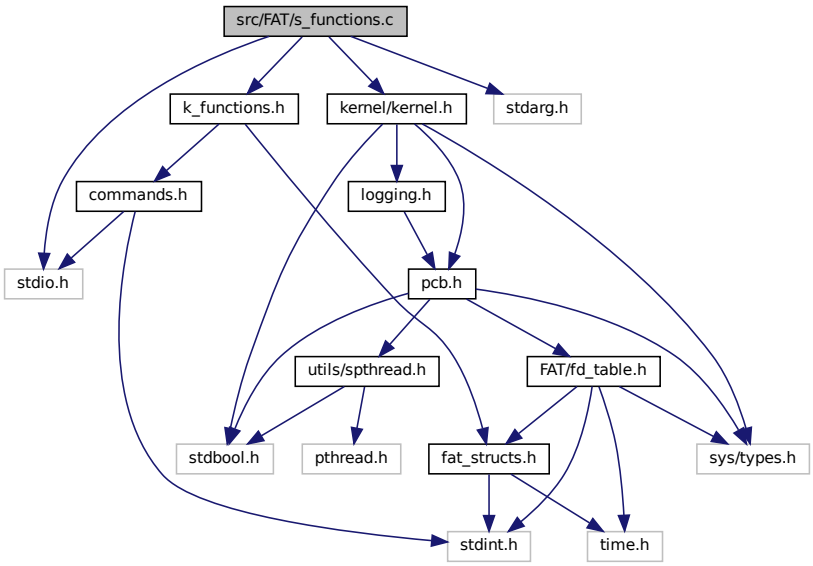


4.10 src/FAT/s_functions.c File Reference

```
#include <stdio.h>
#include <stdarg.h>
```

```
#include "k_functions.h"
#include "kernel/kernel.h"
```

Include dependency graph for s_functions.c:



Functions

- int s_open (const char *fname, int mode)

  User-level file open system call.
- int s_close (int fd)

  User-level file close system call.
- int s_read (int fd, int n, char *buf)

  User-level file read system call.
- int s_write (int fd, const char *str, int n)

  User-level file write system call.
- int s_unlink (const char *fname)

  User-level file deletion system call.
- int s_lseek (int fd, int offset, int whence)

  User-level file seek system call.
- int s_ls (const char *filename)

  User-level directory listing system call.
- int s_printf (const char *format,...)
- int s_fprintf_stderr (const char *format,...)

4.10.1 Function Documentation

4.10.1.1 s_close() `int s_close ( int fd )`

User-level file close system call.

Closes the file descriptor in the current process's FD table. Decrements the reference count in the global FD table.

Parameters

fd	File descriptor to close

Returns

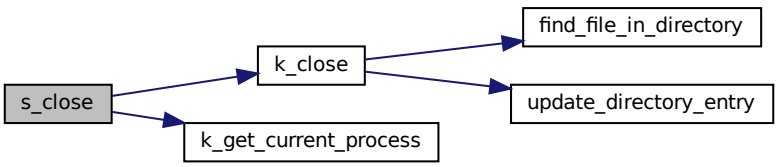
0 on success, -1 on error

Definition at line 32 of file s_functions.c.

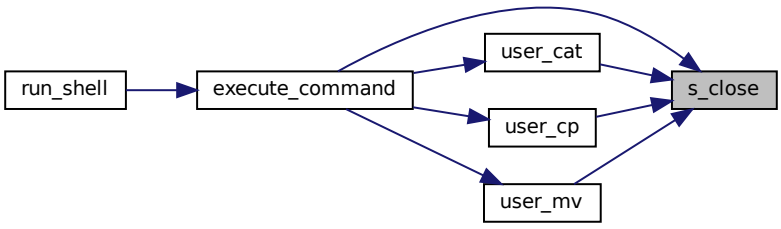
References current_process, pcb::fd_table, k_close(), k_get_current_process(), and MAX_FDS.

Referenced by execute_command(), user_cat(), user_cp(), and user_mv().

Here is the call graph for this function:



Here is the caller graph for this function:



4.10.1.2 s_fprintf_stderr()

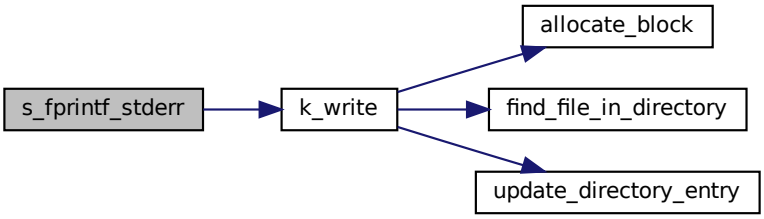
```
int s_fprintf_stderr (
        const char * format,
        ... )
```

Definition at line 116 of file s_functions.c.

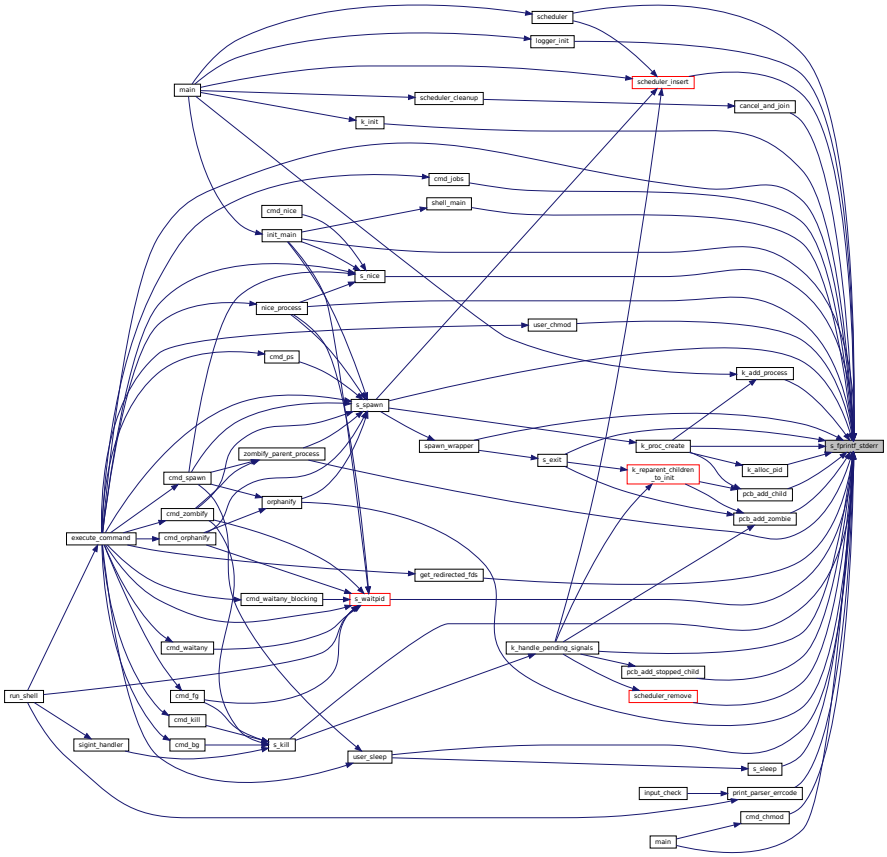
References k_write().

Referenced by cancel_and_join(), cmd_chmod(), cmd_jobs(), execute_command(), get_redirected_fds(), init_main(), k_add_process(), k_alloc_pid(), k_handle_pending_signals(), k_init(), k_proc_create(), logger_init(), main(), nice_process(), orphanify(), pcb_add_child(), pcb_add_stopped_child(), pcb_add_zombie(), print_parser_errcode(), s_exit(), s_kill(), s_nice(), s_sleep(), s_spawn(), s_waitpid(), scheduler(), scheduler_insert(), scheduler_remove(), shell_main(), spawn_wrapper(), user_chmod(), user_sleep(), and zombify_parent_process().

Here is the call graph for this function:



Here is the caller graph for this function:



4.10.1.3 s_ls() `int s_ls ( const char * filename )`

User-level directory listing system call.

Lists directory contents or shows information about a specific file. If filename is NULL or empty, lists all files.

Parameters

filename	Specific file to query, or NULL to list all

Returns

0 on success, -1 on error

Definition at line 91 of file s_functions.c.

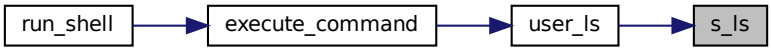
References k_ls().

Referenced by user_ls().

Here is the call graph for this function:



Here is the caller graph for this function:



4.10.1.4 s_lseek()

```
int s_lseek (
        int fd,
        int offset,
        int whence )
```

User-level file seek system call.

Repositions the file offset for subsequent read/write operations. Uses the current process's file descriptor table.

Parameters

fd	File descriptor
offset	Offset in bytes
whence	Reference point: F_SEEK_SET (0), F_SEEK_CUR (1), F_SEEK_END (2)

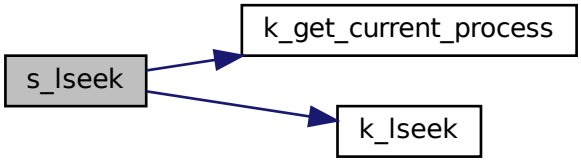
Returns

New absolute offset, or -1 on error

Definition at line 83 of file s_functions.c.

References current_process, pcb::fd_table, k_get_current_process(), k_lseek(), and MAX_FDS.

Here is the call graph for this function:



4.10.1.5 s_open()

```
int s_open (
        const char * fname,
        int mode )
```

User-level file open system call.

Opens a file in the FAT filesystem for the current process. Uses the process's file descriptor table. Inherits file descriptors across fork.

Parameters

fname	Filename to open
mode	Open mode: F_WRITE (0), F_READ (1), or F_APPEND (2)

Returns

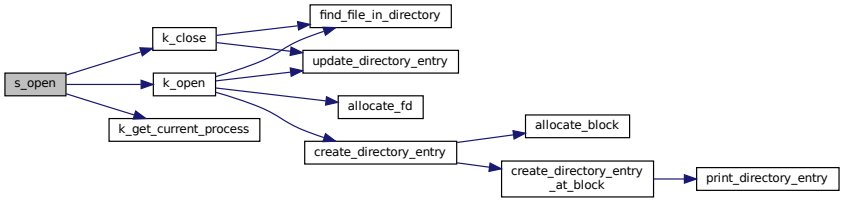
File descriptor number, or -1 on error

Definition at line 6 of file s_functions.c.

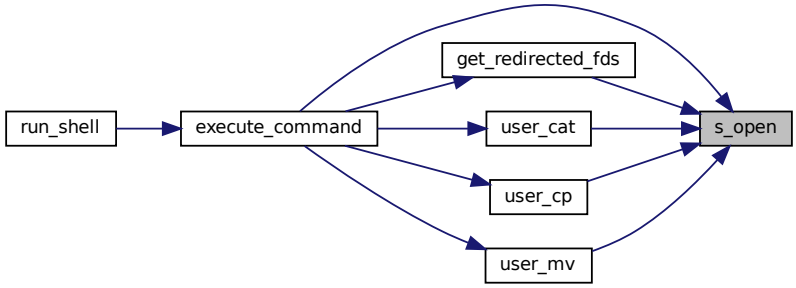
References current_process, pcb::fd_table, k_close(), k_get_current_process(), k_open(), and MAX_FDS.

Referenced by execute_command(), get_redirected_fds(), user_cat(), user_cp(), and user_mv().

Here is the call graph for this function:



Here is the caller graph for this function:



4.10.1.6 s_printf()

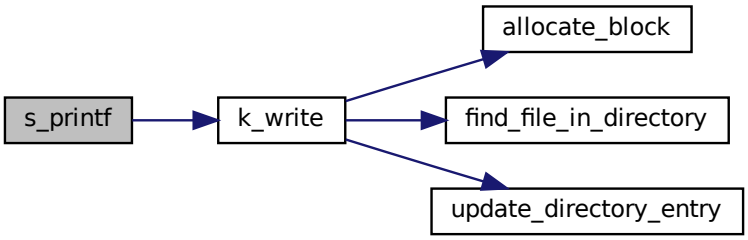
```
int s_printf (
        const char * format,
         ...  )
```

Definition at line 95 of file s_functions.c.

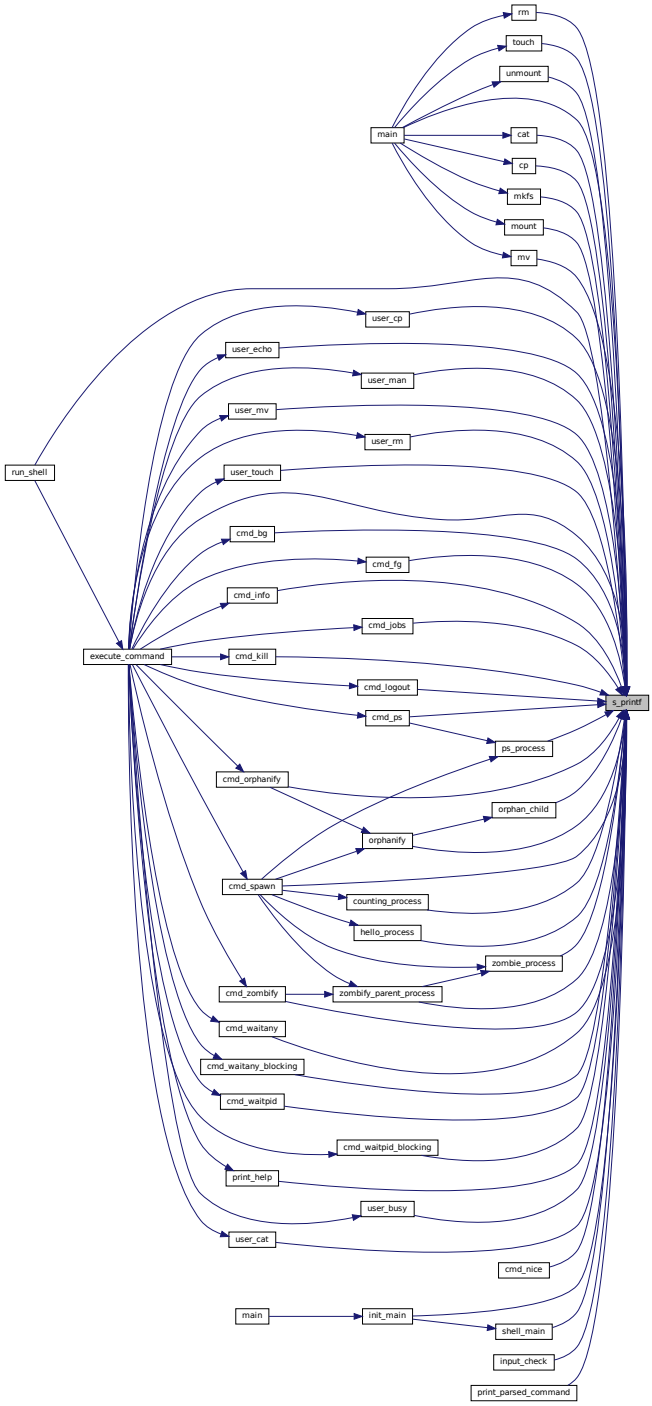
References k_write().

Referenced by cat(), cmd_bg(), cmd_fg(), cmd_info(), cmd_jobs(), cmd_kill(), cmd_logout(), cmd_nice(), cmd_orphanify(), cmd_ps(), cmd_spawn(), cmd_waitany(), cmd_waitany_blocking(), cmd_waitpid(), cmd_waitpid_blocking(), cmd_zombify(), counting_process(), cp(), execute_command(), hello_process(), init_main(), input_check(), main(), mkfs(), mount(), mv(), orphan_child(), orphanify(), print_help(), print_parsed_command(), ps_process(), rm(), run_shell(), shell_main(), touch(), unmount(), user_busy(), user_cat(), user_cp(), user_echo(), user_man(), user_mv(), user_rm(), user_touch(), zombie_process(), and zombify_parent_process().

Here is the call graph for this function:



Here is the caller graph for this function:



4.10.1.7 s_read() int s_read (

int *fd*,

```
int n,
char * buf )
```

User-level file read system call.

Reads up to n bytes from the file into buf. Uses the current process's file descriptor table.

Parameters

fd	File descriptor
n	Maximum number of bytes to read
buf	Buffer to store read data

Returns

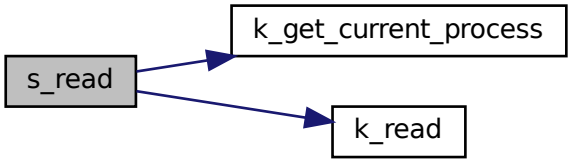
Number of bytes read, or -1 on error

Definition at line 43 of file s_functions.c.

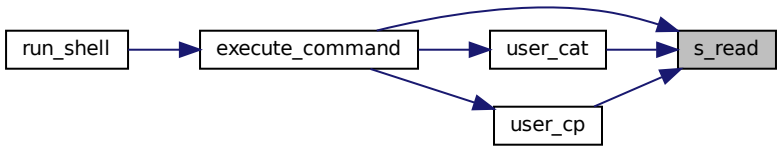
References current_process, pcb::fd_table, k_get_current_process(), k_read(), and MAX_FDS.

Referenced by execute_command(), user_cat(), and user_cp().

Here is the call graph for this function:



Here is the caller graph for this function:



4.10.1.8 s_unlink() `int s_unlink ( const char * fname )`

User-level file deletion system call.

Deletes the specified file from the FAT filesystem. File must not be currently open by any process.

Parameters

fname	Filename to delete

Returns

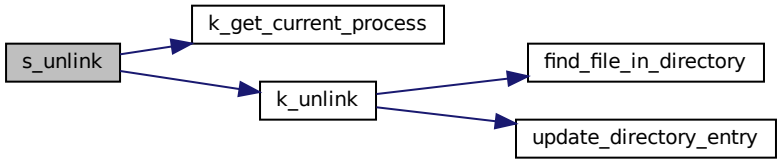
0 on success, -1 on error

Definition at line 73 of file s_functions.c.

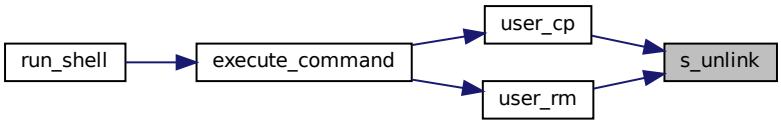
References current_process, pcb::fd_table, k_get_current_process(), and k_unlink().

Referenced by user_cp(), and user_rm().

Here is the call graph for this function:



Here is the caller graph for this function:



4.10.1.9 s_write()

```
int s_write (
    int fd,
    const char * str,
    int n )
```

User-level file write system call.

Writes up to n bytes from str to the file. Uses the current process's file descriptor table. Special handling for stdout (fd=1) and stderr (fd=2).

Parameters

fd	File descriptor
str	Buffer containing data to write
n	Number of bytes to write

Returns

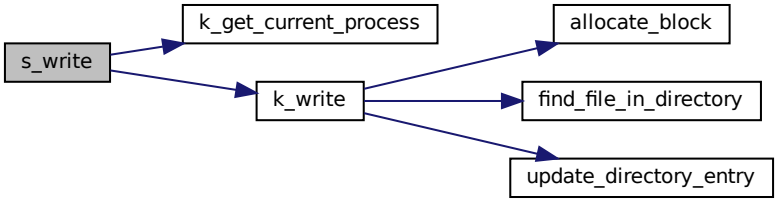
Number of bytes written, or -1 on error

Definition at line 53 of file s_functions.c.

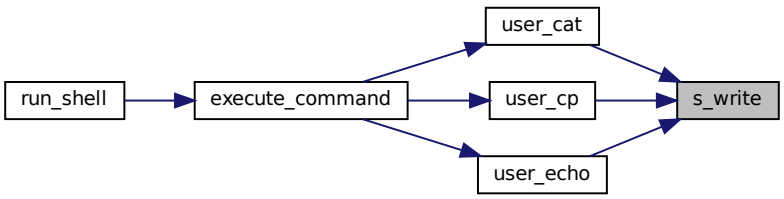
References current_process, pcb::fd_table, k_get_current_process(), k_write(), and MAX_FDS.

Referenced by user_cat(), user_cp(), and user_echo().

Here is the call graph for this function:



Here is the caller graph for this function:



4.11 src/FAT/s_functions.h File Reference

This graph shows which files directly or indirectly include this file:



Functions

- int s_open (const char *fname, int mode)

  User-level file open system call.
- int s_read (int fd, int n, char *buf)

  User-level file read system call.
- int s_write (int fd, const char *str, int n)

  User-level file write system call.
- int s_close (int fd)

  User-level file close system call.
- int s_unlink (const char *fname)

  User-level file deletion system call.
- int s_lseek (int fd, int offset, int whence)

  User-level file seek system call.
- int s_ls (const char *filename)

  User-level directory listing system call.
- int s_printf (const char *format,...) __attribute__((format(printf

  User-level printf to stdout.
- int int s_fprintf_stderr (const char *format,...) __attribute__((format(printf

  User-level fprintf to stderr.

4.11.1 Function Documentation

4.11.1.1 s_close() `int s_close ( int fd )`

User-level file close system call.

Closes the file descriptor in the current process's FD table. Decrements the reference count in the global FD table.

Parameters

fd	File descriptor to close

Returns

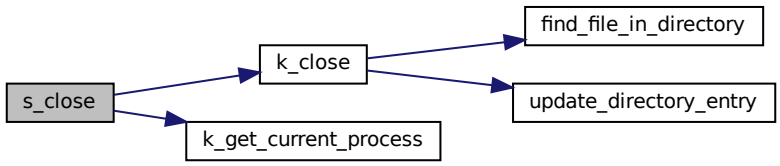
0 on success, -1 on error

Definition at line 32 of file s_functions.c.

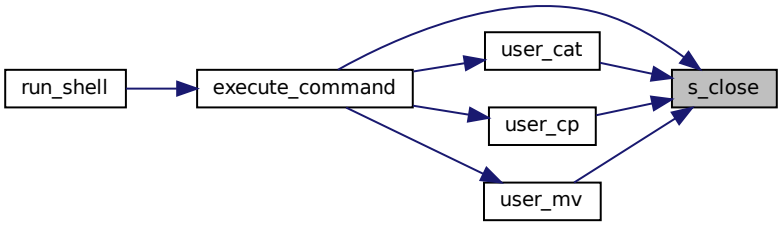
References current_process, pcb::fd_table, k_close(), k_get_current_process(), and MAX_FDS.

Referenced by execute_command(), user_cat(), user_cp(), and user_mv().

Here is the call graph for this function:



Here is the caller graph for this function:



4.11.1.2 s_fprintf_stderr() `int int s_fprintf_stderr (`

```
const char * format,
 ...  )
```

User-level fprintf to stderr.

Formatted output function that writes to the current process's stderr (fd=2) using k_write. Supports standard printf format specifiers.

Parameters

format	Printf-style format string
...	Variable arguments for format string

Returns

Number of characters written, or -1 on error

4.11.1.3 s_ls() `int s_ls ( const char * filename )`

User-level directory listing system call.

Lists directory contents or shows information about a specific file. If filename is NULL or empty, lists all files.

Parameters

filename	Specific file to query, or NULL to list all

Returns

0 on success, -1 on error

Definition at line 91 of file s_functions.c.

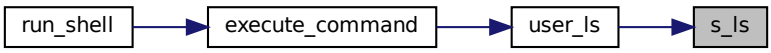
References k_ls().

Referenced by user_ls().

Here is the call graph for this function:



Here is the caller graph for this function:



4.11.1.4 s_lseek() `int s_lseek (`

```
int fd,
int offset,
int whence )
```

User-level file seek system call.

Repositions the file offset for subsequent read/write operations. Uses the current process's file descriptor table.

Parameters

fd	File descriptor
offset	Offset in bytes
whence	Reference point: F_SEEK_SET (0), F_SEEK_CUR (1), F_SEEK_END (2)

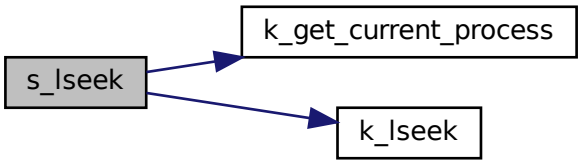
Returns

New absolute offset, or -1 on error

Definition at line 83 of file s_functions.c.

References current_process, pcb::fd_table, k_get_current_process(), k_lseek(), and MAX_FDS.

Here is the call graph for this function:



4.11.1.5 s_open()

```
int s_open (
        const char * fname,
        int mode )
```

User-level file open system call.

Opens a file in the FAT filesystem for the current process. Uses the process's file descriptor table. Inherits file descriptors across fork.

Parameters

fname	Filename to open
mode	Open mode: F_WRITE (0), F_READ (1), or F_APPEND (2)

Returns

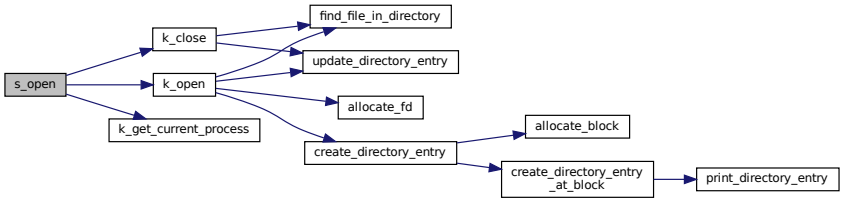
File descriptor number, or -1 on error

Definition at line 6 of file s_functions.c.

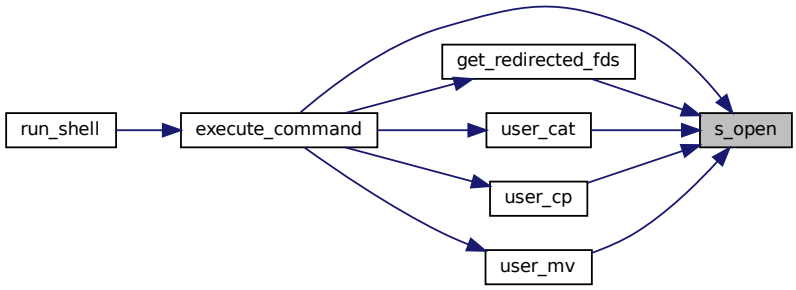
References current_process, pcb::fd_table, k_close(), k_get_current_process(), k_open(), and MAX_FDS.

Referenced by execute_command(), get_redirected_fds(), user_cat(), user_cp(), and user_mv().

Here is the call graph for this function:



Here is the caller graph for this function:



4.11.1.6 s_printf()

```
int s_printf (
    const char * format,
    ... )
```

User-level printf to stdout.

Formatted output function that writes to the current process's stdout (fd=1) using k_write. Supports standard printf format specifiers.

Parameters

format	Printf-style format string
...	Variable arguments for format string

Returns

Number of characters written, or -1 on error

4.11.1.7 s_read()

```
int s_read (
    int fd,
    int n,
    char * buf )
```

User-level file read system call.

Reads up to n bytes from the file into buf. Uses the current process's file descriptor table.

Parameters

fd	File descriptor
n	Maximum number of bytes to read
buf	Buffer to store read data

Returns

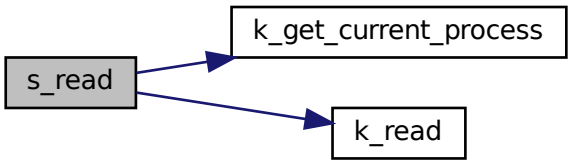
Number of bytes read, or -1 on error

Definition at line 43 of file s_functions.c.

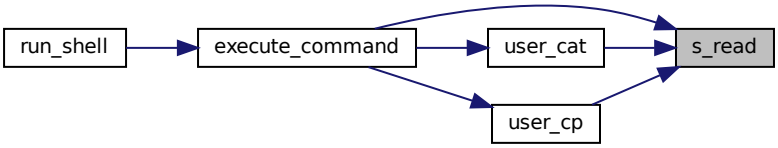
References current_process, pcb::fd_table, k_get_current_process(), k_read(), and MAX_FDS.

Referenced by execute_command(), user_cat(), and user_cp().

Here is the call graph for this function:



Here is the caller graph for this function:



4.11.1.8 s_unlink() `int s_unlink ( const char * fname )`

User-level file deletion system call.

Deletes the specified file from the FAT filesystem. File must not be currently open by any process.

Parameters

fname	Filename to delete

Returns

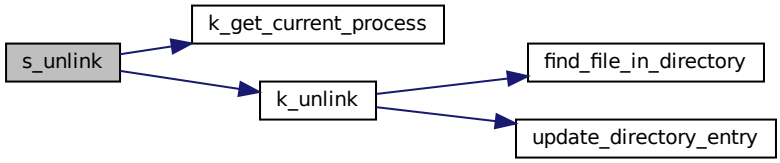
0 on success, -1 on error

Definition at line 73 of file s_functions.c.

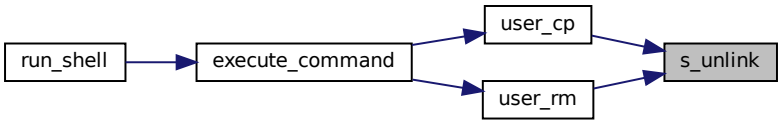
References current_process, pcb::fd_table, k_get_current_process(), and k_unlink().

Referenced by user_cp(), and user_rm().

Here is the call graph for this function:



Here is the caller graph for this function:



4.11.1.9 s_write()

```
int s_write (
        int fd,
        const char * str,
        int n )
```

User-level file write system call.

Writes up to n bytes from str to the file. Uses the current process's file descriptor table. Special handling for stdout (fd=1) and stderr (fd=2).

Parameters

fd	File descriptor
str	Buffer containing data to write
n	Number of bytes to write

Returns

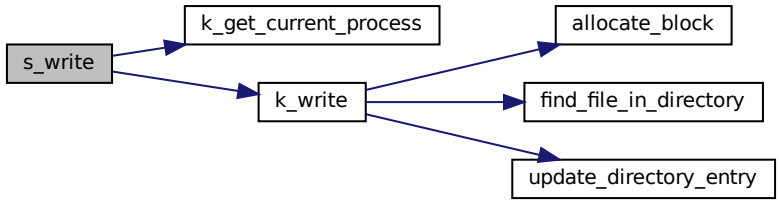
Number of bytes written, or -1 on error

Definition at line 53 of file s_functions.c.

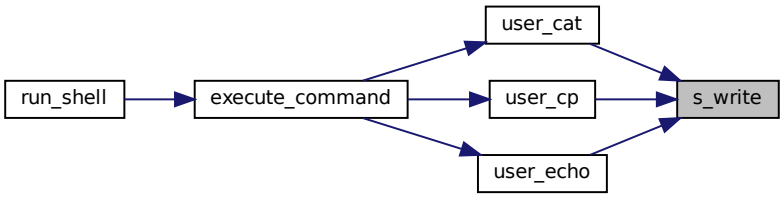
References current_process, pcb::fd_table, k_get_current_process(), k_write(), and MAX_FDS.

Referenced by user_cat(), user_cp(), and user_echo().

Here is the call graph for this function:



Here is the caller graph for this function:

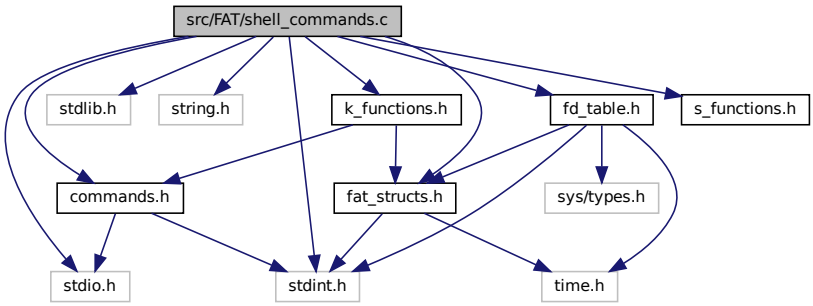


4.12 src/FAT/shell_commands.c File Reference

```
#include <stdint.h>
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include "commands.h"
#include "fat_structs.h"
#include "fd_table.h"
#include "k_functions.h"
```

```
#include "s_functions.h"
```

Include dependency graph for shell_commands.c:



Functions

- void cmd_ls (char **args)

  List directory contents.
- void cmd_chmod (char **args)

  Change file permissions.
- void rm (char **args)

  Remove (delete) a file.
- void touch (char **args)

  Create an empty file.
- void mv (char **args)

  Move/rename a file.
- void cat (char **args)

  Concatenate and display file contents.
- void cp (char **args)

  Copy a file.

4.12.1 Function Documentation

4.12.1.1 cat() `void cat ( char ** args )`

Concatenate and display file contents.

Reads and displays the contents of one or more files to stdout. If multiple files are specified, concatenates them in order.

Usage: cat <filename> [filename2 ...]

Parameters

args	Null-terminated argument array (args[0] = "cat", args[1..n] = filenames)

< Read mode - file must exist

< No filesystem is currently mounted

< Write mode - truncate file if exists, create if not

< Append mode - create if not exists, seek to end

< Requested file does not exist

< Write mode - truncate file if exists, create if not

< Append mode - create if not exists, seek to end

< Read mode - file must exist

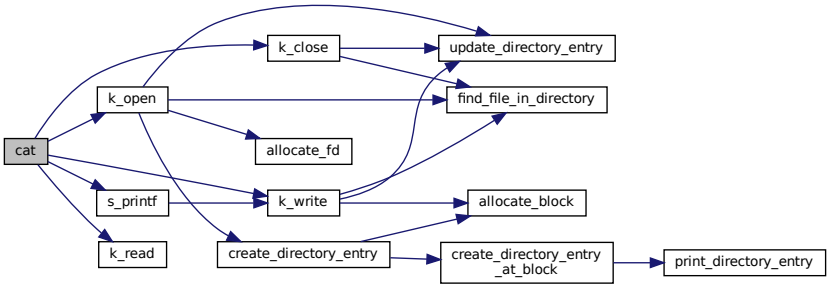
< Requested file does not exist

Definition at line 187 of file shell_commands.c.

References F_APPEND, F_READ, F_WRITE, FILE_NOT_FOUND, FS_NOT_MOUNTED, k_close(), k_open(), k_↩ read(), k_write(), and s_printf().

Referenced by main().

Here is the call graph for this function:



Here is the caller graph for this function:



4.12.1.2 cmd_chmod() `void cmd_chmod ( char ** args )`

Change file permissions.

Modifies the permission bits of a file. Permissions are specified as octal values (0-7) corresponding to read(4), write(2), execute(1).

Usage: chmod <permissions> <filename>

Parameters

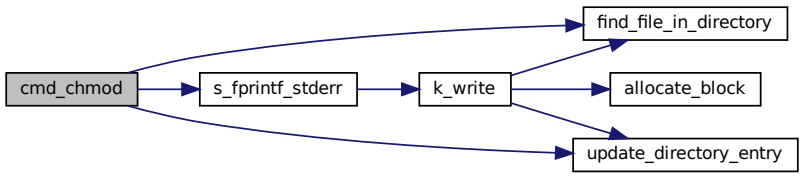
args	Null-terminated argument array (args[0] = "chmod", args[1] = perms, args[2] = filename)

Definition at line 19 of file shell_commands.c.

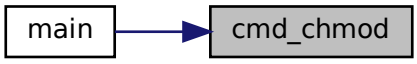
References find_file_in_directory(), directory_entry_t::perm, s_fprintf_stderr(), and update_directory_entry().

Referenced by main().

Here is the call graph for this function:



Here is the caller graph for this function:



4.12.1.3 cmd_ls() `void cmd_ls ( char ** args )`

List directory contents.

Displays files in the filesystem with their metadata (size, permissions, modification time). If args[1] is provided, shows detailed info for that specific file. Otherwise, lists all files.

Usage: ls [filename]

Parameters

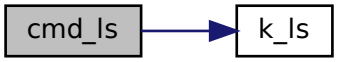
args	Null-terminated argument array (args[0] = "ls", args[1] = optional filename)

Definition at line 11 of file shell_commands.c.

References k_ls().

Referenced by main().

Here is the call graph for this function:



Here is the caller graph for this function:



4.12.1.4 cp()

```
void cp (
        char ** args )
```

Copy a file.

Creates a duplicate of a file. Reads the entire source file and writes it to the destination.

Usage: cp <source> <destination>

Parameters

args	Null-terminated argument array (args[0] = "cp", args[1] = source, args[2] = dest)

< Read mode - file must exist

< No filesystem is currently mounted

< Write mode - truncate file if exists, create if not

< Read mode - file must exist

< Requested file does not exist

< Read mode - file must exist

< Requested file does not exist

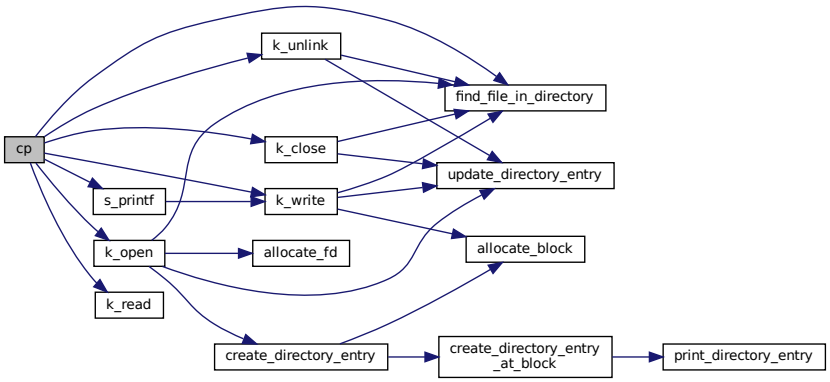
< Write mode - truncate file if exists, create if not

Definition at line 317 of file shell_commands.c.

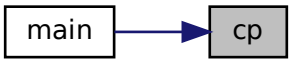
References F_READ, F_WRITE, FILE_NOT_FOUND, find_file_in_directory(), FS_NOT_MOUNTED, k_close(), k_↩ open(), k_read(), k_unlink(), k_write(), and s_printf().

Referenced by main().

Here is the call graph for this function:



Here is the caller graph for this function:



4.12.1.5 mv() `void mv ( char ** args )`

Move/rename a file.

Renames a file from source to destination. Implemented by reading the entire source file, creating destination, and deleting source.

Usage: mv <source> <destination>

Parameters

args	Null-terminated argument array (args[0] = "mv", args[1] = source, args[2] = dest)

< Read mode - file must exist

< No filesystem is currently mounted

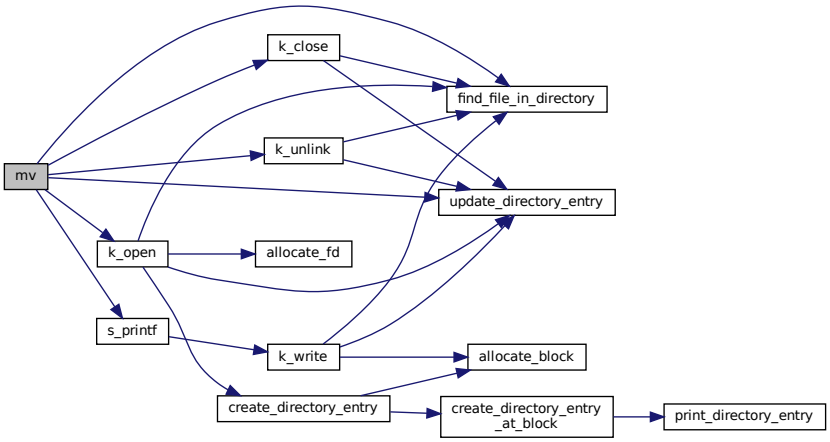
< Requested file does not exist

Definition at line 128 of file shell_commands.c.

References F_READ, FILE_NOT_FOUND, find_file_in_directory(), FS_NOT_MOUNTED, k_close(), k_open(), k_↩ unlink(), directory_entry_t::name, s_printf(), and update_directory_entry().

Referenced by main().

Here is the call graph for this function:



Here is the caller graph for this function:



4.12.1.6 rm()

```
void rm (
            char ** args )
```

Remove (delete) a file.

Deletes the specified file from the filesystem. File must not be currently open by any process.

Usage: rm <filename>

Parameters

args	Null-terminated argument array (args[0] = "rm", args[1] = filename)

< No filesystem is currently mounted

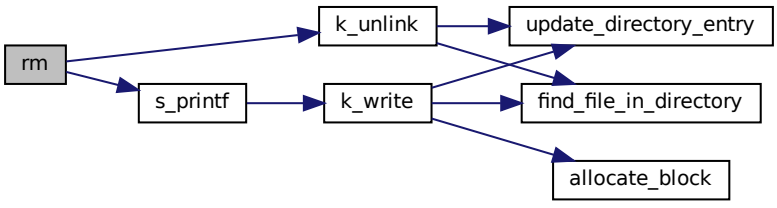
< Requested file does not exist

Definition at line 91 of file shell_commands.c.

References FILE_NOT_FOUND, FS_NOT_MOUNTED, k_unlink(), and s_printf().

Referenced by main().

Here is the call graph for this function:



Here is the caller graph for this function:



4.12.1.7 touch() `void touch ( char ** args )`

Create an empty file.

Creates a new empty file with default permissions (PERM_READ_WRITE). If file already exists, updates its modification time.

Usage: touch <filename>

Parameters

args	Null-terminated argument array (args[0] = "touch", args[1] = filename)

< Append mode - create if not exists, seek to end

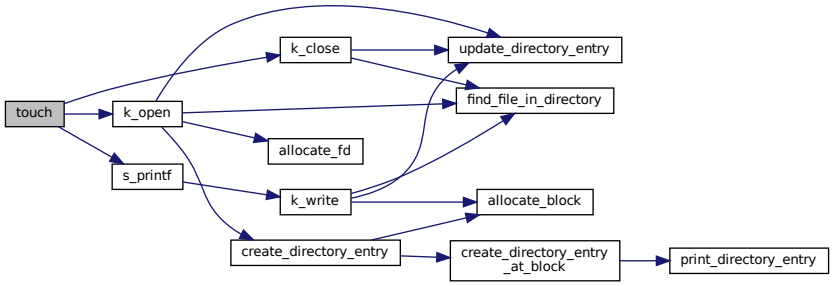
< No filesystem is currently mounted

Definition at line 108 of file shell_commands.c.

References F_APPEND, FS_NOT_MOUNTED, k_close(), k_open(), and s_printf().

Referenced by main().

Here is the call graph for this function:



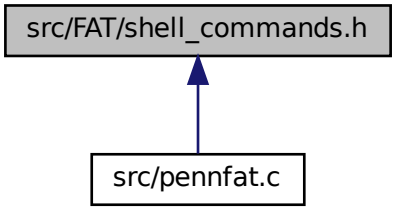
Here is the caller graph for this function:



4.13 src/FAT/shell_commands.h File Reference

PennFAT shell commands.

This graph shows which files directly or indirectly include this file:



Functions

- void cmd_ls (char **args)
  List directory contents.
- void cmd_chmod (char **args)
  Change file permissions.
- void touch (char **args)
  Create an empty file.
- void mv (char **args)
  Move/rename a file.
- void rm (char **args)
  Remove (delete) a file.
- void cat (char **args)
  Concatenate and display file contents.
- void cp (char **args)
  Copy a file.

4.13.1 Detailed Description

PennFAT shell commands.

Provides Unix-like file manipulation commands for the PennFAT filesystem. These commands are used by both the interactive shell and as user processes.

4.13.2 Function Documentation

4.13.2.1 cat() `void cat ( char ** args )`

Concatenate and display file contents.

Reads and displays the contents of one or more files to stdout. If multiple files are specified, concatenates them in order.

Usage: cat <filename> [filename2 ...]

Parameters

args	Null-terminated argument array (args[0] = "cat", args[1..n] = filenames)

< Read mode - file must exist

< No filesystem is currently mounted

< Write mode - truncate file if exists, create if not

< Append mode - create if not exists, seek to end

< Requested file does not exist

< Write mode - truncate file if exists, create if not

< Append mode - create if not exists, seek to end

< Read mode - file must exist

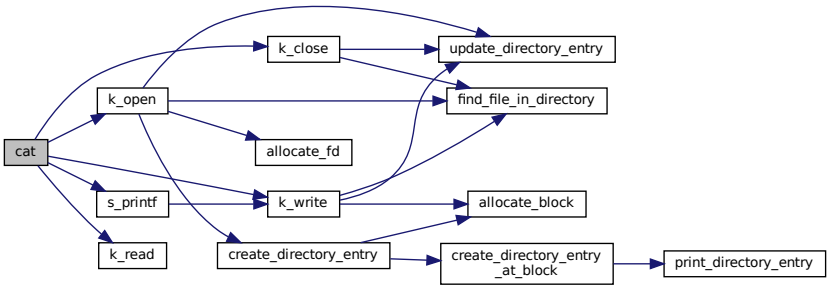
< Requested file does not exist

Definition at line 187 of file shell_commands.c.

References F_APPEND, F_READ, F_WRITE, FILE_NOT_FOUND, FS_NOT_MOUNTED, k_close(), k_open(), k_↩ read(), k_write(), and s_printf().

Referenced by main().

Here is the call graph for this function:



Here is the caller graph for this function:



4.13.2.2 cmd_chmod() `void cmd_chmod ( char ** args )`

Change file permissions.

Modifies the permission bits of a file. Permissions are specified as octal values (0-7) corresponding to read(4), write(2), execute(1).

Usage: chmod <permissions> <filename>

Parameters

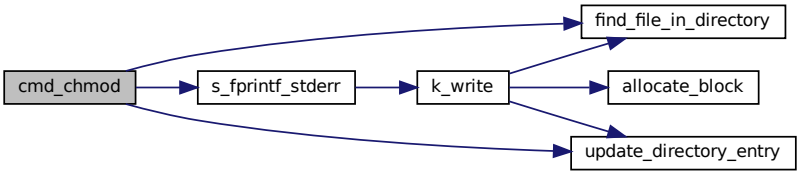
args	Null-terminated argument array (args[0] = "chmod", args[1] = perms, args[2] = filename)

Definition at line 19 of file shell_commands.c.

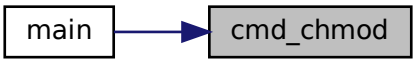
References find_file_in_directory(), directory_entry_t::perm, s_fprintf_stderr(), and update_directory_entry().

Referenced by main().

Here is the call graph for this function:



Here is the caller graph for this function:



4.13.2.3 cmd_ls() `void cmd_ls ( char ** args )`

List directory contents.

Displays files in the filesystem with their metadata (size, permissions, modification time). If args[1] is provided, shows detailed info for that specific file. Otherwise, lists all files.

Usage: ls [filename]

Parameters

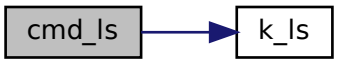
args	Null-terminated argument array (args[0] = "ls", args[1] = optional filename)

Definition at line 11 of file shell_commands.c.

References k_ls().

Referenced by main().

Here is the call graph for this function:



Here is the caller graph for this function:



4.13.2.4 cp() `void cp ( char ** args )`

Copy a file.

Creates a duplicate of a file. Reads the entire source file and writes it to the destination.

Usage: cp <source> <destination>

Parameters

args	Null-terminated argument array (args[0] = "cp", args[1] = source, args[2] = dest)

< Read mode - file must exist

< No filesystem is currently mounted

< Write mode - truncate file if exists, create if not

< Read mode - file must exist

< Requested file does not exist

< Read mode - file must exist

< Requested file does not exist

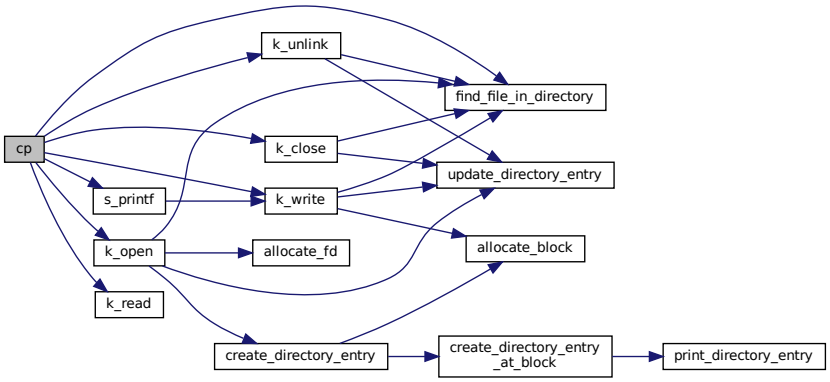
< Write mode - truncate file if exists, create if not

Definition at line 317 of file shell_commands.c.

References F_READ, F_WRITE, FILE_NOT_FOUND, find_file_in_directory(), FS_NOT_MOUNTED, k_close(), k_open(), k_read(), k_unlink(), k_write(), and s_printf().

Referenced by main().

Here is the call graph for this function:



Here is the caller graph for this function:



4.13.2.5 mv() `void mv ( char ** args )`

Move/rename a file.

Renames a file from source to destination. Implemented by reading the entire source file, creating destination, and deleting source.

Usage: mv <source> <destination>

Parameters

args	Null-terminated argument array (args[0] = "mv", args[1] = source, args[2] = dest)

< Read mode - file must exist

< No filesystem is currently mounted

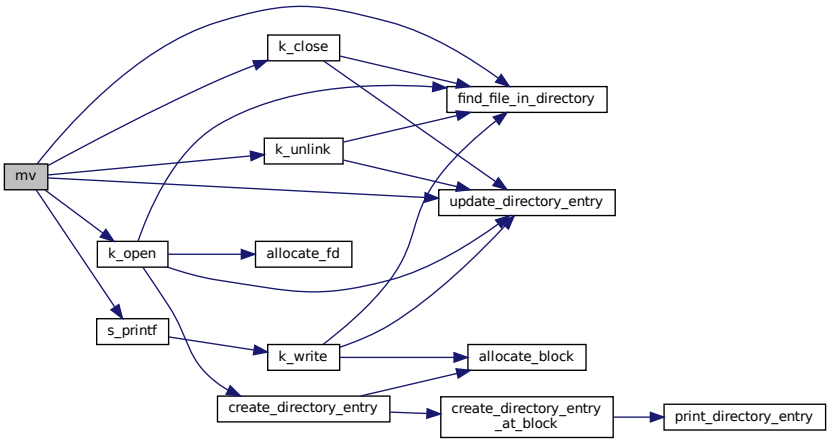
< Requested file does not exist

Definition at line 128 of file shell_commands.c.

References F_READ, FILE_NOT_FOUND, find_file_in_directory(), FS_NOT_MOUNTED, k_close(), k_open(), k_↩ unlink(), directory_entry_t::name, s_printf(), and update_directory_entry().

Referenced by main().

Here is the call graph for this function:



Here is the caller graph for this function:



4.13.2.6 rm() `void rm ( char ** args )`

Remove (delete) a file.

Deletes the specified file from the filesystem. File must not be currently open by any process.

Usage: rm <filename>

Parameters

args	Null-terminated argument array (args[0] = "rm", args[1] = filename)

< No filesystem is currently mounted

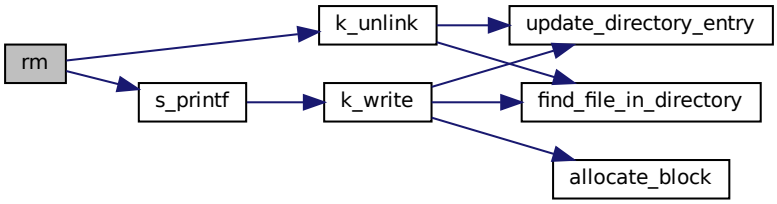
< Requested file does not exist

Definition at line 91 of file shell_commands.c.

References FILE_NOT_FOUND, FS_NOT_MOUNTED, k_unlink(), and s_printf().

Referenced by main().

Here is the call graph for this function:



Here is the caller graph for this function:



4.13.2.7 touch() `void touch ( char ** args )`

Create an empty file.

Creates a new empty file with default permissions (PERM_READ_WRITE). If file already exists, updates its modification time.

Usage: touch <filename>

Parameters

args	Null-terminated argument array (args[0] = "touch", args[1] = filename)

< Append mode - create if not exists, seek to end

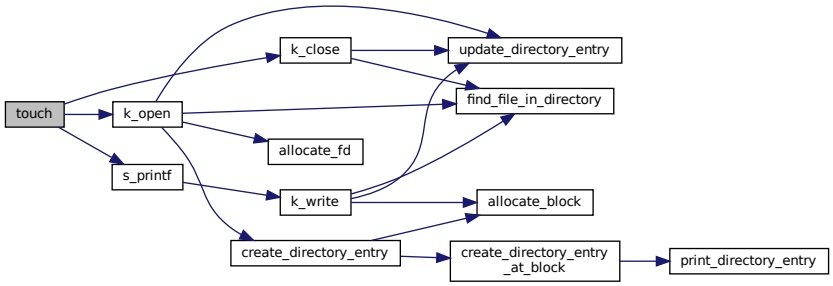
< No filesystem is currently mounted

Definition at line 108 of file shell_commands.c.

References F_APPEND, FS_NOT_MOUNTED, k_close(), k_open(), and s_printf().

Referenced by main().

Here is the call graph for this function:



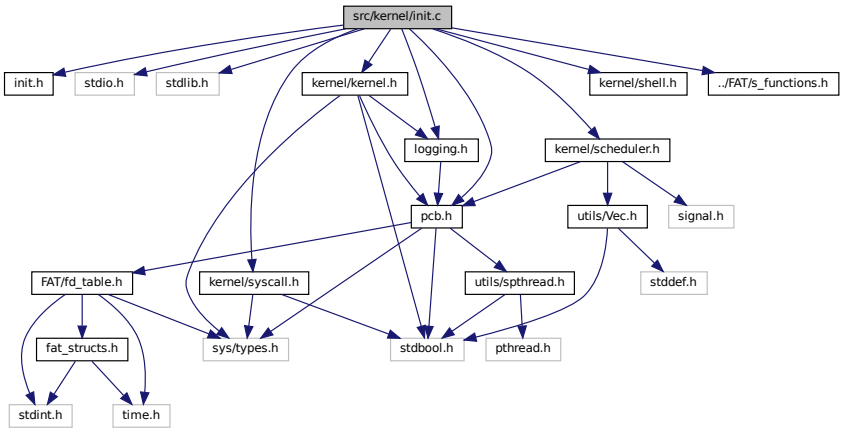
Here is the caller graph for this function:



4.14 src/kernel/init.c File Reference

```
#include "init.h"
#include <stdio.h>
#include <stdlib.h>
#include "kernel/kernel.h"
#include "kernel/logging.h"
#include "kernel/pcb.h"
#include "kernel/scheduler.h"
#include "kernel/shell.h"
#include "kernel/syscall.h"
#include "../FAT/s_functions.h"
```

Include dependency graph for init.c:



Functions

- void init_shutdown (void)

  Signal the INIT process to begin shutdown.
- void * init_main (void *arg)

  Main function for the INIT process (PID 1)

Variables

- static volatile bool system_running = true

4.14.1 Function Documentation

4.14.1.1 init_main()

```
void* init_main (
        void * arg )
```

Main function for the INIT process (PID 1)

The INIT process is the first user-space process in PennOS. It is responsible for:

- Running the interactive shell
- Reaping orphaned zombie processes
- Managing system shutdown

This function runs in an infinite loop until init_shutdown() is called. It serves as the ancestor of all user processes and adopts orphaned children when their parent processes exit.

Parameters

arg	Unused argument (required for thread function signature)

Returns

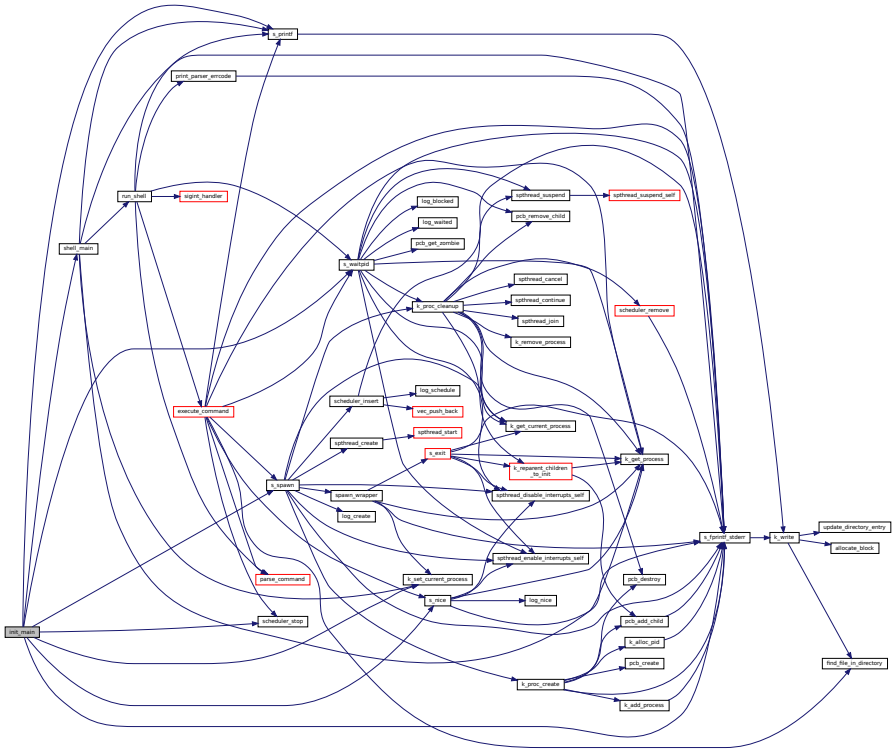
NULL when the INIT process exits

Definition at line 18 of file init.c.

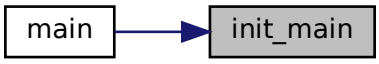
References init_process, k_set_current_process(), P_EXITSTATUS, P_WIFEXITED, P_WIFSIGNALED, s_fprintf_↩ stderr(), s_nice(), s_printf(), s_spawn(), s_waitpid(), scheduler_stop(), shell_main(), and system_running.

Referenced by main().

Here is the call graph for this function:



Here is the caller graph for this function:



4.14.1.2 init_shutdown() `void init_shutdown ( void )`

Signal the INIT process to begin shutdown.

Sets the shutdown flag that causes the INIT process main loop to exit.

Definition at line 14 of file init.c.

References system_running.

Referenced by cmd_logout().

Here is the caller graph for this function:



4.14.2 Variable Documentation

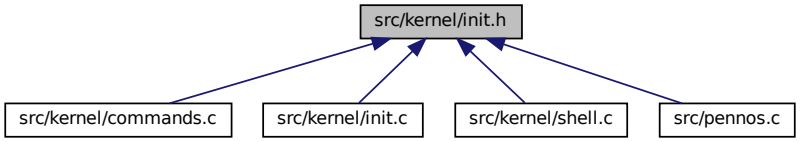
4.14.2.1 system_running `volatile bool system_running = true [static]`

Definition at line 12 of file init.c.

Referenced by init_main(), and init_shutdown().

4.15 src/kernel/init.h File Reference

This graph shows which files directly or indirectly include this file:



Functions

- void init_shutdown (void)

  Signal the INIT process to begin shutdown.
- void * init_main (void *arg)

  Main function for the INIT process (PID 1)

4.15.1 Function Documentation

4.15.1.1 init_main() `void* init_main ( void * arg )`

Main function for the INIT process (PID 1)

The INIT process is the first user-space process in PennOS. It is responsible for:

- Running the interactive shell
- Reaping orphaned zombie processes
- Managing system shutdown

This function runs in an infinite loop until init_shutdown() is called. It serves as the ancestor of all user processes and adopts orphaned children when their parent processes exit.

Parameters

arg	Unused argument (required for thread function signature)

Returns

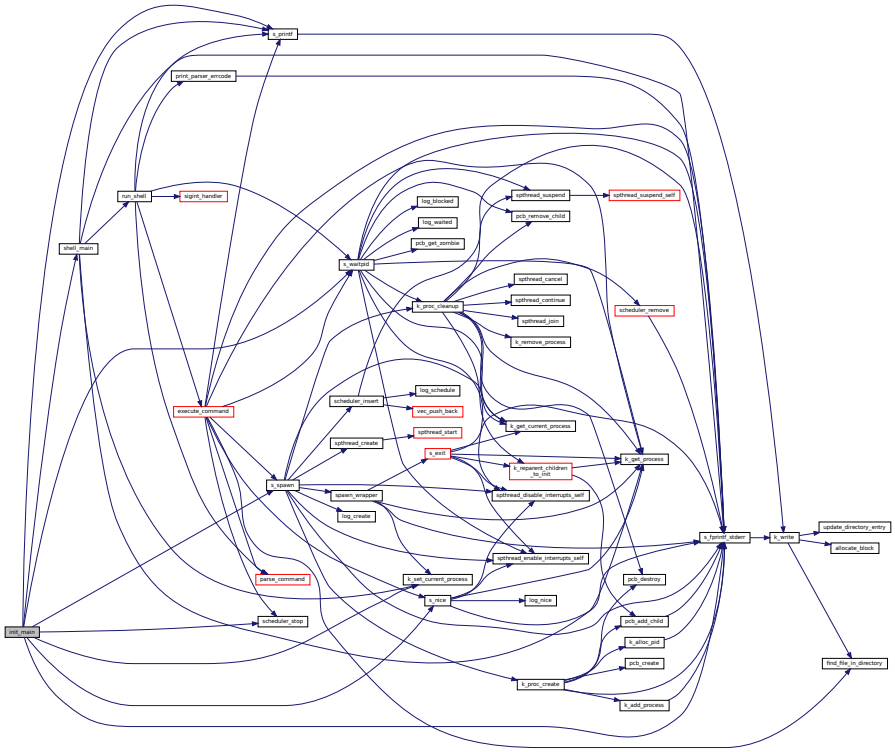
NULL when the INIT process exits

Definition at line 18 of file init.c.

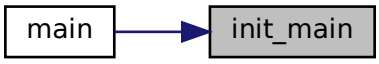
References init_process, k_set_current_process(), P_EXITSTATUS, P_WIFEXITED, P_WIFSIGNALED, s_fprintf_↩ stderr(), s_nice(), s_printf(), s_spawn(), s_waitpid(), scheduler_stop(), shell_main(), and system_running.

Referenced by main().

Here is the call graph for this function:



Here is the caller graph for this function:



4.15.1.2 init_shutdown() `void init_shutdown ( void )`

Signal the INIT process to begin shutdown.

Sets the shutdown flag that causes the INIT process main loop to exit.

Definition at line 14 of file init.c.

References system_running.

Referenced by cmd_logout().

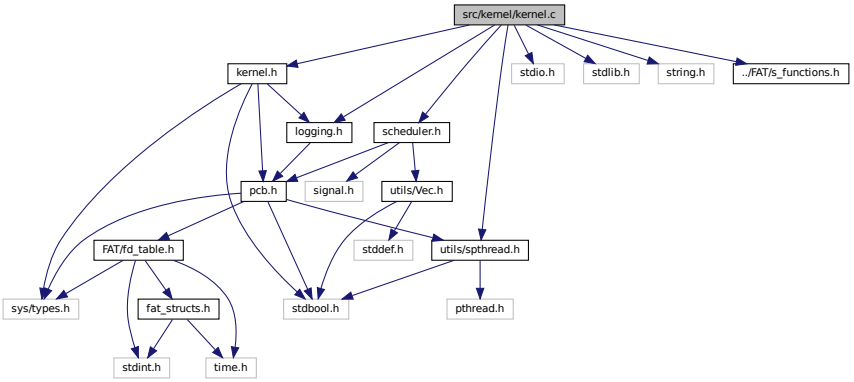
Here is the caller graph for this function:



4.16 src/kernel/kernel.c File Reference

```
#include "kernel.h"
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include "logging.h"
#include "scheduler.h"
#include "utils/spthread.h"
#include "../FAT/s_functions.h"
```

Include dependency graph for kernel.c:



Functions

- void k_init (void)

  Initialize the kernel and process table.
- void k_shutdown (void)

  Shutdown the kernel and clean up all resources.
- pid_t k_alloc_pid (void)

  Allocate a new unique PID.

- int k_add_process (pcb_t *proc)

  Add a process to the process table.
- void k_remove_process (pid_t pid)

  Remove a process from the process table.
- pcb_t * k_get_process (pid_t pid)

  Get a process by PID.
- pcb_t * k_get_current_process (void)

  Get the currently running process.
- void k_set_current_process (pcb_t *proc)

  Set the currently running process.
- pcb_t * k_get_init_process (void)

  Get the INIT process.
- pcb_t * k_proc_create (pcb_t *parent)

  Create a new child process, inheriting applicable properties from the parent.
- void k_reparent_children_to_init (pid_t parent_pid)

  Reparent children to INIT process.
- void k_notify_parent (pid_t child_pid, pcb_t *parent)

  Notify parent that a child has changed state.
- void k_proc_cleanup (pcb_t *proc)

  Clean up a terminated/finished process's resources.
- void k_handle_pending_signals (pcb_t *proc)

  Handle pending signals for a process.
- int k_get_process_count (void)

  Get total number of active processes.
- pcb_t ** k_get_process_table (void)

  To give scheduler access to process table.

Variables

- static pcb_t * process_table [256]
- static __thread pcb_t * current_process = NULL
- static pcb_t * init_process = NULL
- static pid_t next_pid = 0
- static bool kernel_initialized = false

4.16.1 Function Documentation

4.16.1.1 k_add_process() `int k_add_process ( pcb_t * proc )`

Add a process to the process table.

Parameters

proc	Process to add

Returns

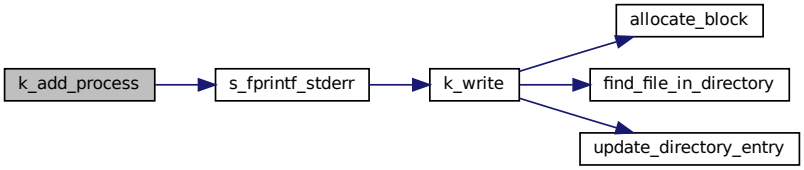
0 on success, -1 on failure

Definition at line 105 of file kernel.c.

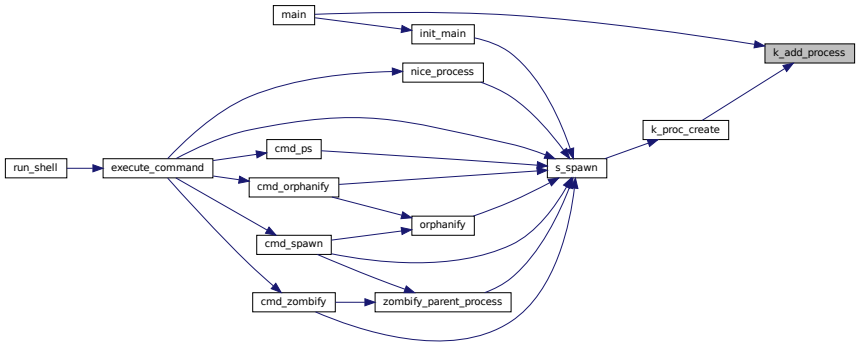
References INIT_PID, init_process, kernel_initialized, MAX_PROCESSES, pcb::pid, process_table, and s_fprintf_↩ stderr().

Referenced by k_proc_create(), and main().

Here is the call graph for this function:



Here is the caller graph for this function:



4.16.1.2 k_alloc_pid() `pid_t k_alloc_pid ( void )`

Allocate a new unique PID.

Returns

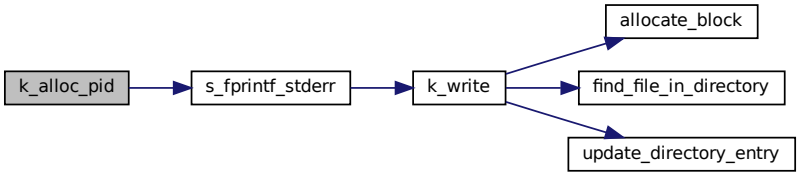
New PID, or -1 if no PIDs available

Definition at line 76 of file kernel.c.

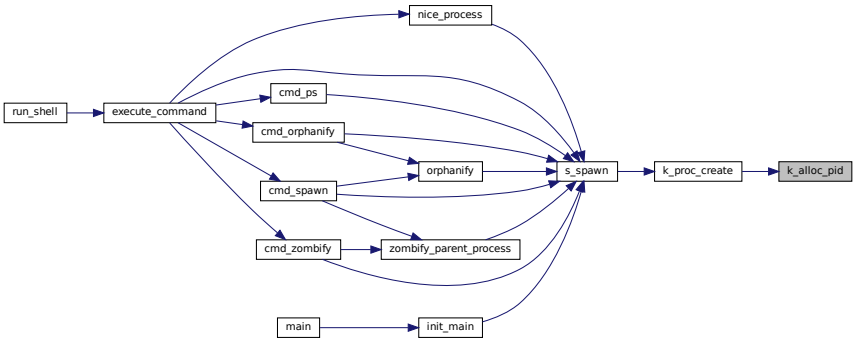
References kernel_initialized, MAX_PROCESSES, next_pid, process_table, and s_fprintf_stderr().

Referenced by k_proc_create().

Here is the call graph for this function:



Here is the caller graph for this function:



4.16.1.3 k_get_current_process() `pcb_t* k_get_current_process ( void )`

Get the currently running process.

Returns

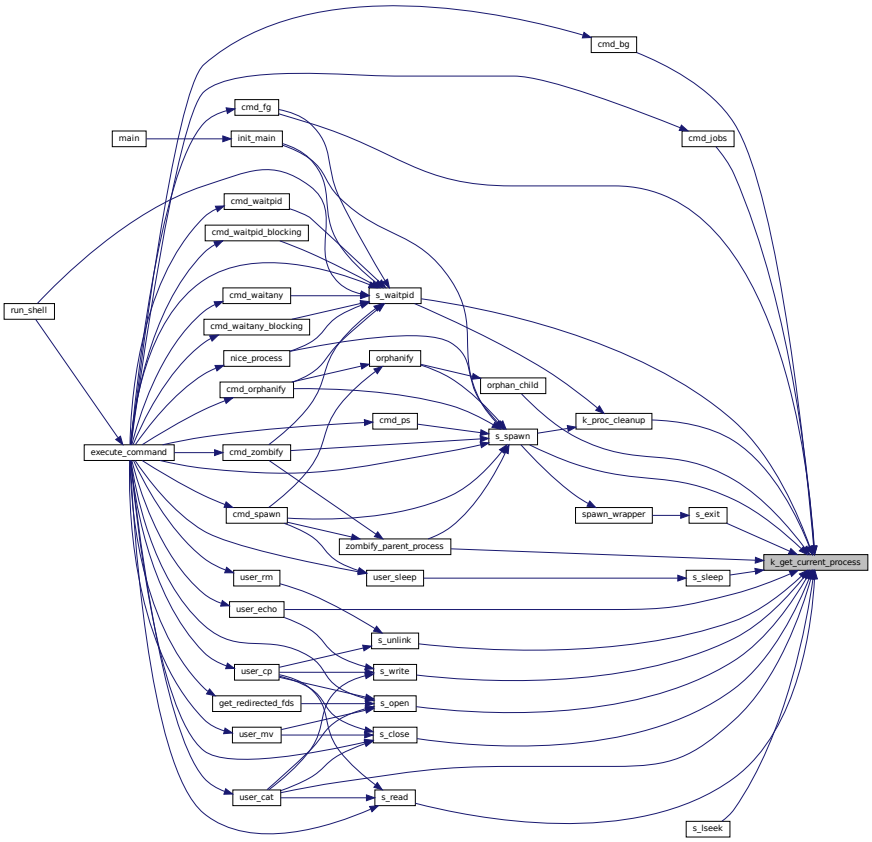
Pointer to the current process's PCB

Definition at line 176 of file kernel.c.

References current_process.

Referenced by cmd_bg(), cmd_fg(), cmd_jobs(), k_proc_cleanup(), orphan_child(), s_close(), s_exit(), s_lseek(), s_↩ open(), s_read(), s_sleep(), s_spawn(), s_unlink(), s_waitpid(), s_write(), user_cat(), user_echo(), and zombify_parent↩ _process().

Here is the caller graph for this function:



4.16.1.4 k_get_init_process() pcb_t* k_get_init_process (void)

Get the INIT process.

Returns

Pointer to INIT process's PCB

Definition at line 194 of file kernel.c.

References init_process.

4.16.1.5 k_get_process() pcb_t* k_get_process (pid_t *pid*)

Get a process by PID.

Parameters

pid	Process ID to look up

Returns

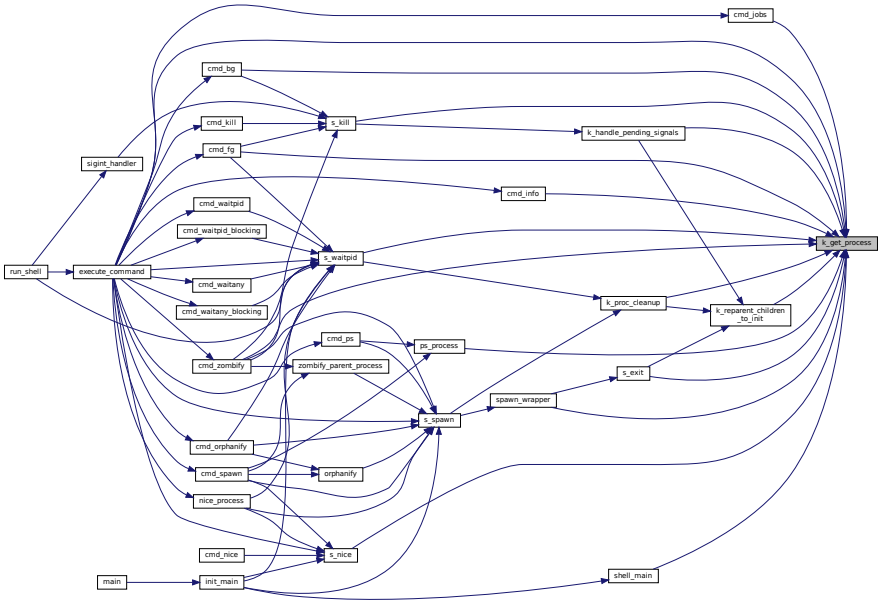
Pointer to PCB, or NULL if not found

Definition at line 159 of file kernel.c.

References kernel_initialized, MAX_PROCESSES, and process_table.

Referenced by cmd_bg(), cmd_fg(), cmd_info(), cmd_jobs(), cmd_zombify(), execute_command(), k_handle_pending_signals(), k_proc_cleanup(), k_reparent_children_to_init(), ps_process(), s_exit(), s_kill(), s_nice(), s_waitpid(), shell_main(), and spawn_wrapper().

Here is the caller graph for this function:



4.16.1.6 k_get_process_count() `int k_get_process_count ( void )`

Get total number of active processes.

Returns

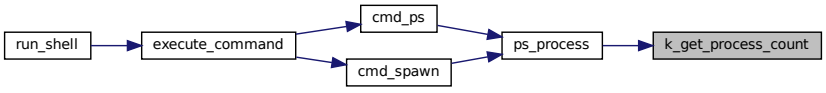
Number of processes in the system (excluding zombies)

Definition at line 488 of file kernel.c.

References kernel_initialized, MAX_PROCESSES, process_table, and ZOMBIE.

Referenced by ps_process().

Here is the caller graph for this function:



4.16.1.7 k_get_process_table() pcb_t** k_get_process_table (void)

To give scheduler access to process table.

Returns

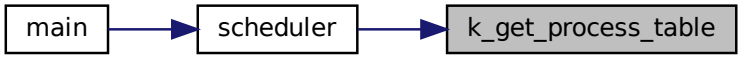
pcb_t** ptr to process table

Definition at line 508 of file kernel.c.

References kernel_initialized, and process_table.

Referenced by scheduler().

Here is the caller graph for this function:



4.16.1.8 k_handle_pending_signals() void k_handle_pending_signals (pcb_t * *proc*)

Handle pending signals for a process.

Should be called by the scheduler before context switching to check if the process has any pending signals that need to be handled.

Parameters

proc	Process to check

< Stop (pause) a process

< Continue a stopped process

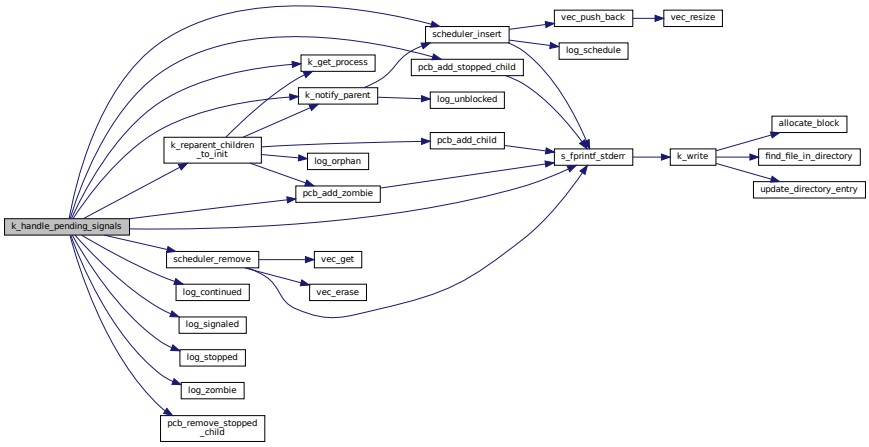
< Terminate a process

Definition at line 408 of file kernel.c.

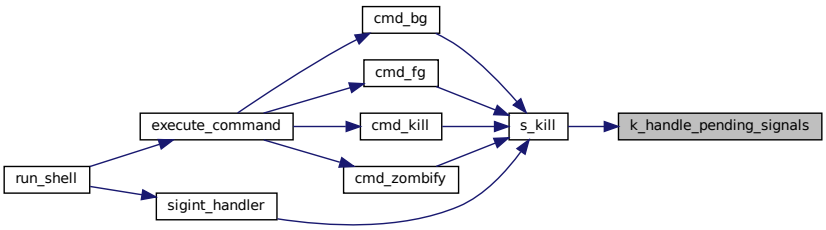
References pcb::exit_status, INIT_PID, k_get_process(), k_notify_parent(), k_reparent_children_to_init(), log_↩ continued(), log_signaled(), log_stopped(), log_zombie(), pcb::name, pcb::num_children, pcb::num_zombies, P_↩ SIGCONT, P_SIGSTOP, P_SIGTERM, pcb_add_stopped_child(), pcb_add_zombie(), pcb_remove_stopped_child(), pcb::pending_signal, pcb::pid, pcb::ppid, pcb::priority, RUNNING, s_fprintf_stderr(), scheduler_insert(), scheduler_↩ remove(), pcb::state, STOPPED, and ZOMBIE.

Referenced by s_kill().

Here is the call graph for this function:



Here is the caller graph for this function:



4.16.1.9 k_init() `void k_init ( void )`

Initialize the kernel and process table.

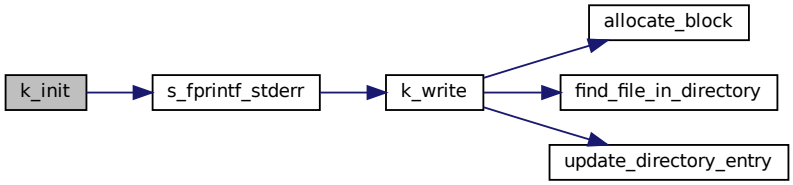
This should be called once at boot time before any other kernel functions.

Definition at line 29 of file kernel.c.

References current_process, init_process, kernel_initialized, MAX_PROCESSES, next_pid, process_table, and s_↩ fprintf_stderr().

Referenced by main().

Here is the call graph for this function:



Here is the caller graph for this function:



4.16.1.10 k_notify_parent()

```
void k_notify_parent (
    pid_t child_pid,
    pcb_t * parent )
```

Notify parent that a child has changed state.

This is called when a child process:

- Exits (becomes zombie)
- Stops (P_SIGSTOP)
- Continues (P_SIGCONT)

Parameters

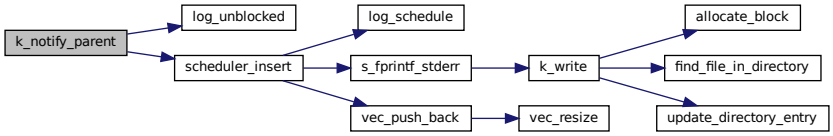
child_pid	Child process ID
parent	Parent PCB

Definition at line 319 of file kernel.c.

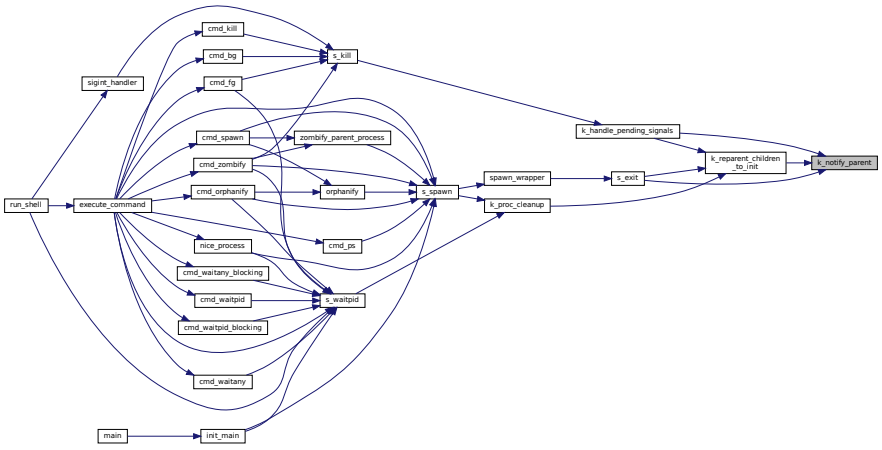
References BLOCKED, pcb::is_waiting, log_unblocked(), pcb::name, pcb::pid, pcb::priority, RUNNING, scheduler_↩ insert(), pcb::state, and pcb::waiting_for_pid.

Referenced by k_handle_pending_signals(), k_reparent_children_to_init(), and s_exit().

Here is the call graph for this function:



Here is the caller graph for this function:



4.16.1.11 k_proc_cleanup() void k_proc_cleanup (pcb_t * proc)

Clean up a terminated/finished process's resources.

This handles:

- Reparenting children to INIT
- Notifying parent of termination
- Removing from parent's children list
- Removing from process table
- Freeing the PCB

Parameters

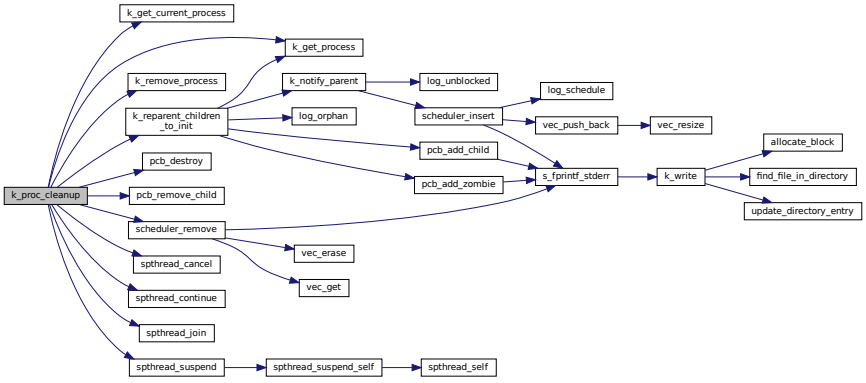
proc	Process to clean up

Definition at line 355 of file kernel.c.

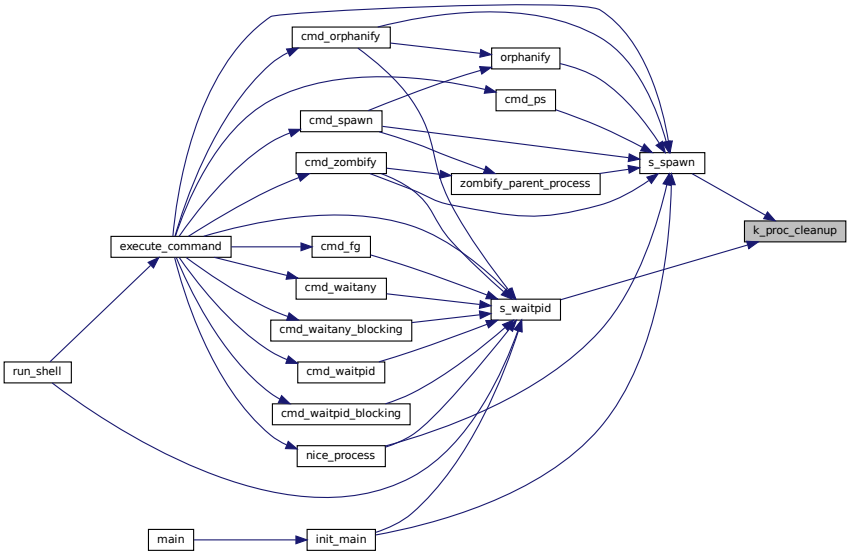
References k_get_current_process(), k_get_process(), k_remove_process(), k_reparent_children_to_init(), kernel_initialized, pcb::num_children, pcb::num_zombies, pcb_destroy(), pcb_remove_child(), pcb::pid, pcb::ppid, scheduler_remove(), spthread_cancel(), spthread_continue(), spthread_join(), spthread_suspend(), pcb::thread, and pcb::thread_created.

Referenced by s_spawn(), and s_waitpid().

Here is the call graph for this function:



Here is the caller graph for this function:



4.16.1.12 k_proc_create() `pcb_t* k_proc_create ( pcb_t * parent )`

Create a new child process, inheriting applicable properties from the parent.

Parameters

parent	Parent PCB (can be NULL for INIT process)

Returns

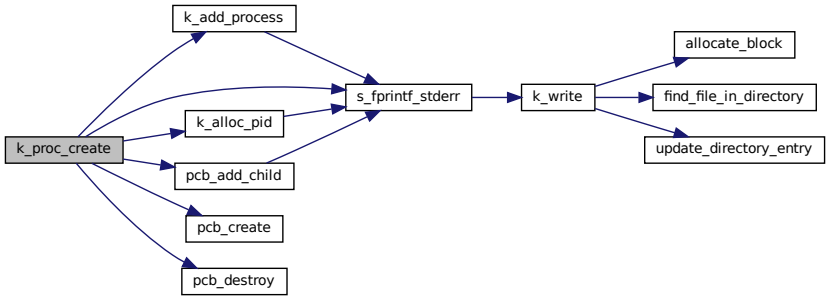
Pointer to the child PCB, or NULL on failure

Definition at line 205 of file kernel.c.

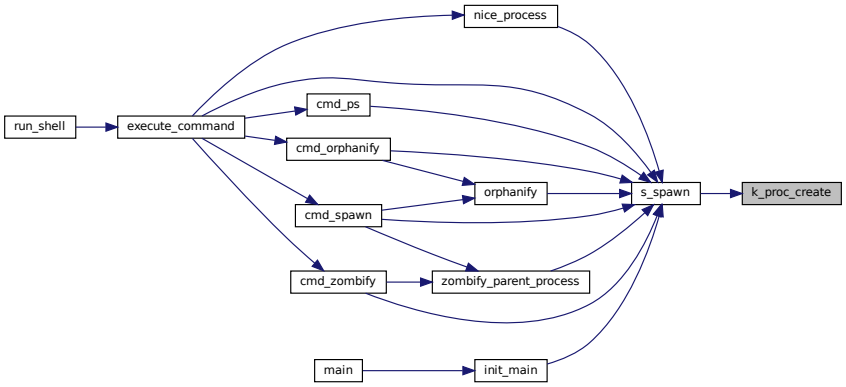
References pcb::fd_table, k_add_process(), k_alloc_pid(), kernel_initialized, MAX_FDS, pcb_add_child(), pcb_create(), pcb_destroy(), pcb::pid, and s_fprintf_stderr().

Referenced by s_spawn().

Here is the call graph for this function:



Here is the caller graph for this function:



4.16.1.13 k_remove_process() `void k_remove_process ( pid_t pid )`

Remove a process from the process table.

Parameters

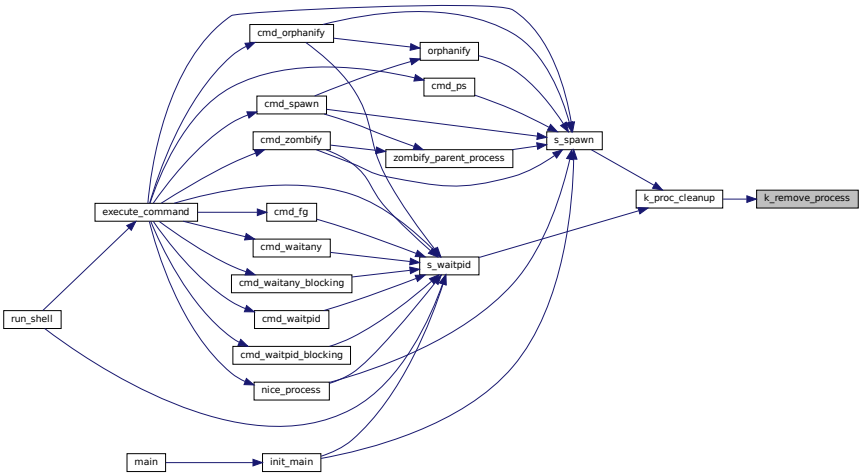
pid	Process ID to remove

Definition at line 141 of file kernel.c.

References kernel_initialized, MAX_PROCESSES, and process_table.

Referenced by k_proc_cleanup().

Here is the caller graph for this function:



4.16.1.14 k_reparent_children_to_init() `void k_reparent_children_to_init ( pid_t parent_pid )`

Reparent children to INIT process.

Called when a parent process exits with living children.

Parameters

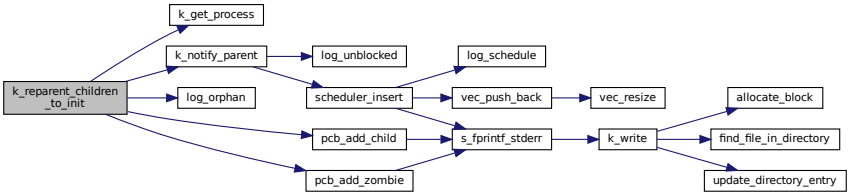
parent_pid	PID of the parent that is exiting

Definition at line 256 of file kernel.c.

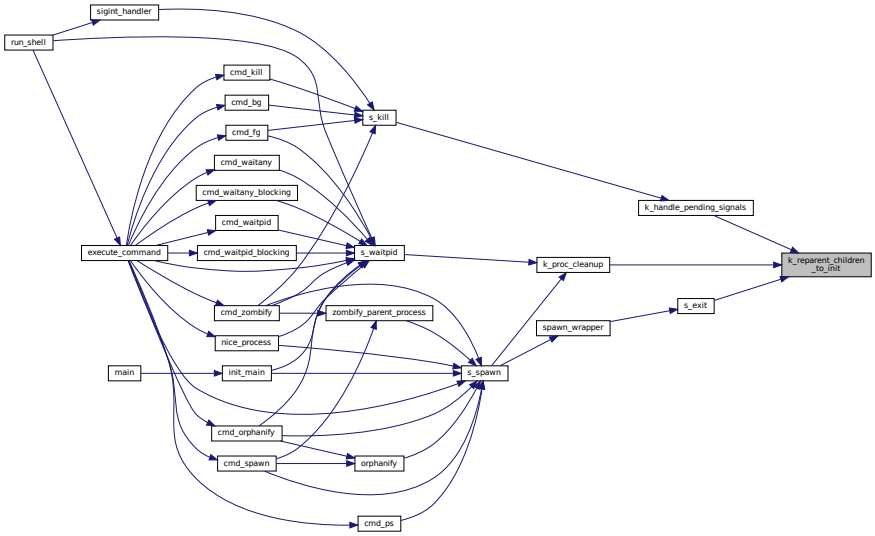
References pcb::children, pcb::exit_statuses, INIT_PID, init_process, k_get_process(), k_notify_parent(), kernel↩ _initialized, log_orphan(), pcb::name, pcb::num_children, pcb::num_zombies, pcb_add_child(), pcb_add_zombie(), pcb::pid, pcb::ppid, pcb::priority, and pcb::zombie_children.

Referenced by k_handle_pending_signals(), k_proc_cleanup(), and s_exit().

Here is the call graph for this function:



Here is the caller graph for this function:



4.16.1.15 k_set_current_process() `void k_set_current_process (`

`pcb_t * proc )`

Set the currently running process.

Parameters

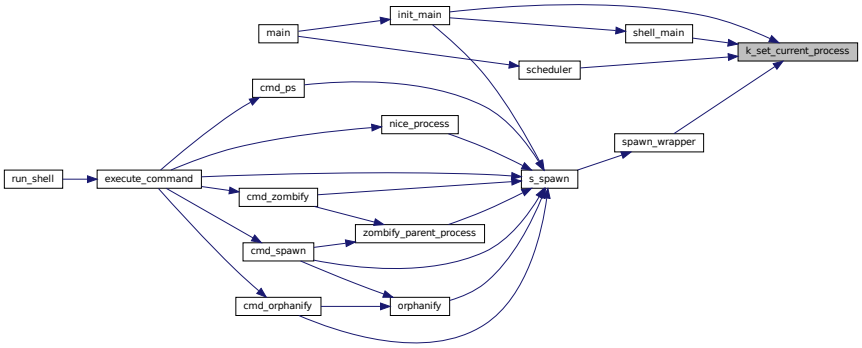
proc	Process to set as current

Definition at line 185 of file kernel.c.

References current_process.

Referenced by init_main(), scheduler(), shell_main(), and spawn_wrapper().

Here is the caller graph for this function:



4.16.1.16 k_shutdown() `void k_shutdown ( void )`

Shutdown the kernel and clean up all resources.

Definition at line 53 of file kernel.c.

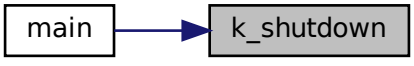
References current_process, init_process, kernel_initialized, MAX_PROCESSES, pcb_destroy(), and process_table.

Referenced by main().

Here is the call graph for this function:



Here is the caller graph for this function:



4.16.2 Variable Documentation

4.16.2.1 current_process

```
__thread pcb_t* current_process = NULL  [static]
```

Definition at line 15 of file kernel.c.

Referenced by k_get_current_process(), k_init(), k_set_current_process(), k_shutdown(), s_close(), s_lseek(), s_open(), s_read(), s_unlink(), and s_write().

4.16.2.2 init_process

```
pcb_t* init_process = NULL  [static]
```

Definition at line 18 of file kernel.c.

Referenced by init_main(), k_add_process(), k_get_init_process(), k_init(), k_reparent_children_to_init(), and k_shutdown().

4.16.2.3 kernel_initialized

```
bool kernel_initialized = false  [static]
```

Definition at line 24 of file kernel.c.

Referenced by k_add_process(), k_alloc_pid(), k_get_process(), k_get_process_count(), k_get_process_table(), k_init(), k_proc_cleanup(), k_proc_create(), k_remove_process(), k_reparent_children_to_init(), and k_shutdown().

4.16.2.4 next_pid

```
pid_t next_pid = 0  [static]
```

Definition at line 21 of file kernel.c.

Referenced by k_alloc_pid(), and k_init().

4.16.2.5 process_table

```
pcb_t* process_table[256]  [static]
```

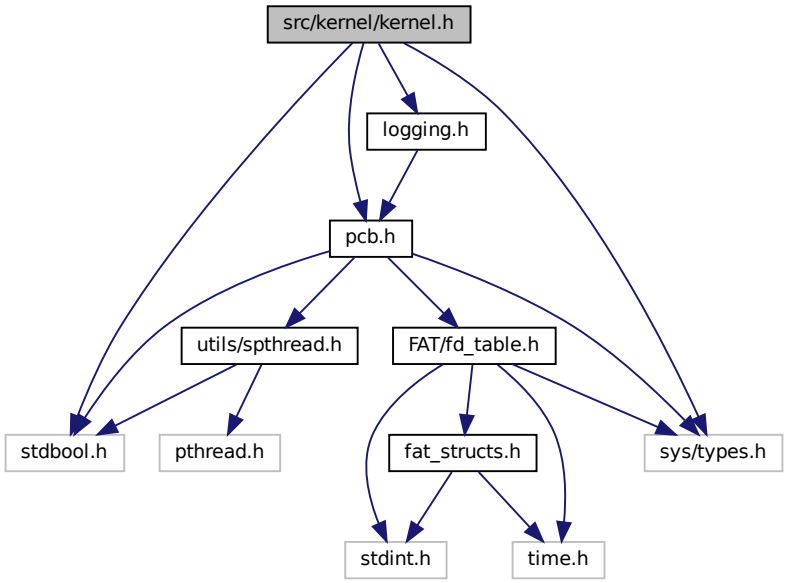
Definition at line 11 of file kernel.c.

Referenced by k_add_process(), k_alloc_pid(), k_get_process(), k_get_process_count(), k_get_process_table(), k_init(), k_remove_process(), k_shutdown(), and scheduler().

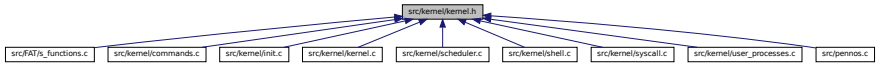
4.17 src/kernel/kernel.h File Reference

```
#include <stdbool.h>
#include <sys/types.h>
#include "logging.h"
#include "pcb.h"
```

Include dependency graph for kernel.h:



This graph shows which files directly or indirectly include this file:



Macros

- #define MAX_PROCESSES 256
- #define INIT_PID 1

Functions

- void k_init (void)

  Initialize the kernel and process table.

- void k_shutdown (void)

  Shutdown the kernel and clean up all resources.
- pcb_t * k_proc_create (pcb_t *parent)

  Create a new child process, inheriting applicable properties from the parent.
- void k_proc_cleanup (pcb_t *proc)

  Clean up a terminated/finished process's resources.
- pcb_t * k_get_process (pid_t pid)

  Get a process by PID.
- pcb_t * k_get_current_process (void)

  Get the currently running process.
- void k_set_current_process (pcb_t *proc)

  Set the currently running process.
- pcb_t * k_get_init_process (void)

  Get the INIT process.
- pid_t k_alloc_pid (void)

  Allocate a new unique PID.
- int k_add_process (pcb_t *proc)

  Add a process to the process table.
- void k_remove_process (pid_t pid)

  Remove a process from the process table.
- void k_notify_parent (pid_t child_pid, pcb_t *parent)

  Notify parent that a child has changed state.
- void k_reparent_children_to_init (pid_t parent_pid)

  Reparent children to INIT process.
- void k_handle_pending_signals (pcb_t *proc)

  Handle pending signals for a process.
- int k_get_process_count (void)

  Get total number of active processes.
- pcb_t ** k_get_process_table (void)

  To give scheduler access to process table.

4.17.1 Macro Definition Documentation

4.17.1.1 INIT_PID `#define INIT_PID 1`

Definition at line 12 of file kernel.h.

4.17.1.2 MAX_PROCESSES `#define MAX_PROCESSES 256`

Definition at line 9 of file kernel.h.

4.17.2 Function Documentation

4.17.2.1 k_add_process() `int k_add_process ( pcb_t * proc )`

Add a process to the process table.

Parameters

proc	Process to add

Returns

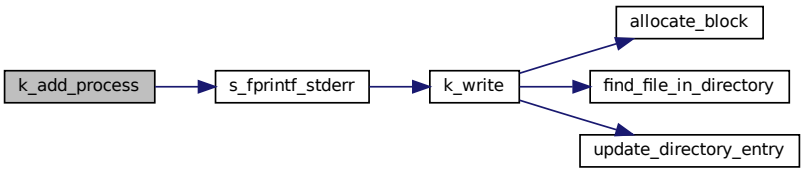
0 on success, -1 on failure

Definition at line 105 of file kernel.c.

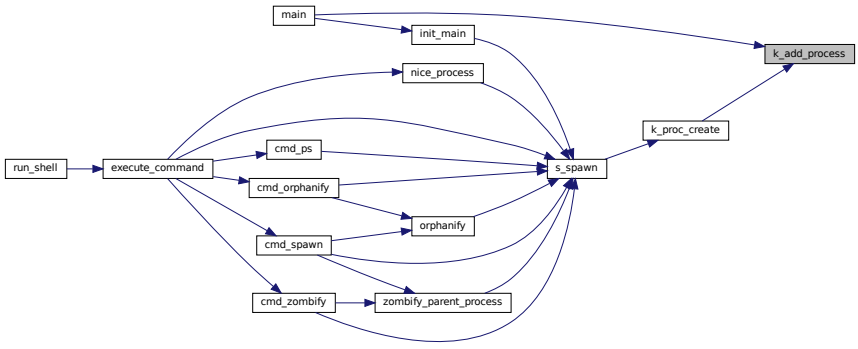
References INIT_PID, init_process, kernel_initialized, MAX_PROCESSES, pcb::pid, process_table, and s_fprintf_↩ stderr().

Referenced by k_proc_create(), and main().

Here is the call graph for this function:



Here is the caller graph for this function:



4.17.2.2 k_alloc_pid() `pid_t k_alloc_pid ( void )`

Allocate a new unique PID.

Returns

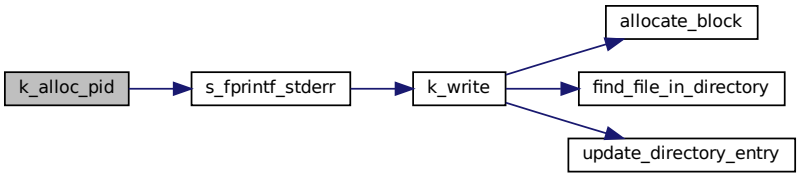
New PID, or -1 if no PIDs available

Definition at line 76 of file kernel.c.

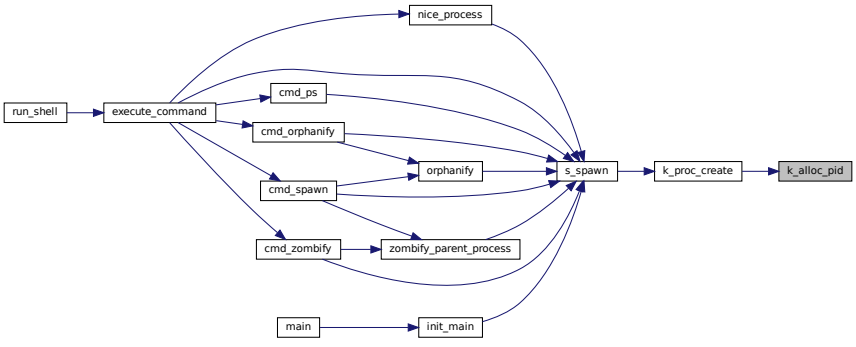
References kernel_initialized, MAX_PROCESSES, next_pid, process_table, and s_fprintf_stderr().

Referenced by k_proc_create().

Here is the call graph for this function:



Here is the caller graph for this function:



4.17.2.3 k_get_current_process() `pcb_t* k_get_current_process ( void )`

Get the currently running process.

Returns

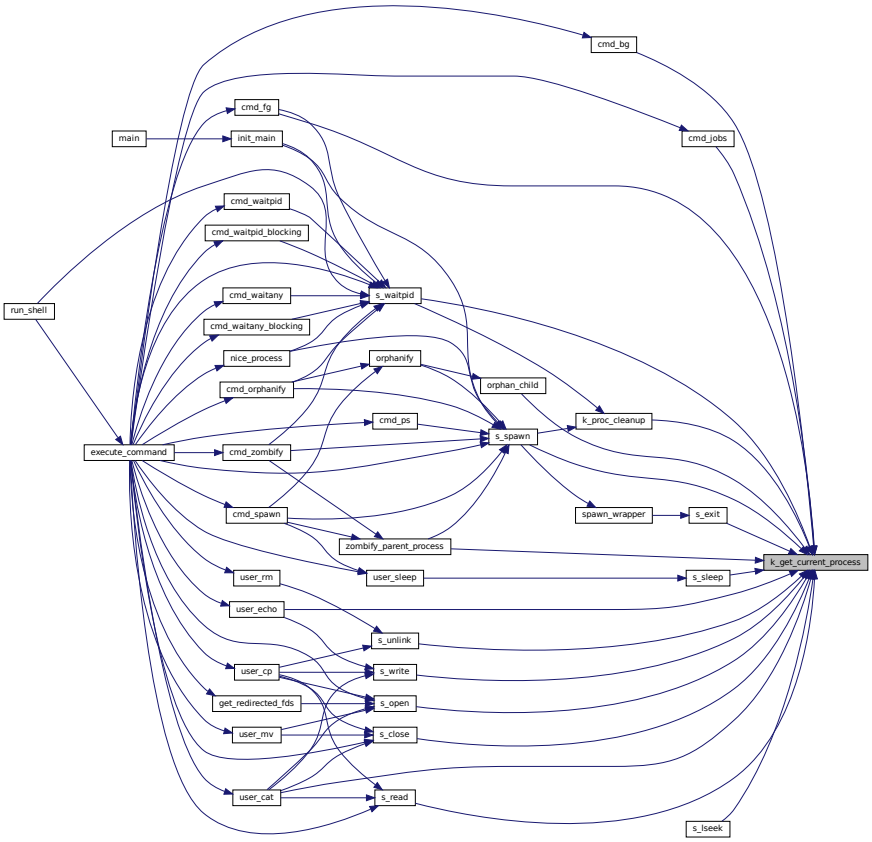
Pointer to the current process's PCB

Definition at line 176 of file kernel.c.

References current_process.

Referenced by cmd_bg(), cmd_fg(), cmd_jobs(), k_proc_cleanup(), orphan_child(), s_close(), s_exit(), s_lseek(), s_↩ open(), s_read(), s_sleep(), s_spawn(), s_unlink(), s_waitpid(), s_write(), user_cat(), user_echo(), and zombify_parent↩ _process().

Here is the caller graph for this function:



4.17.2.4 k_get_init_process() pcb_t* k_get_init_process (void)

Get the INIT process.

Returns

Pointer to INIT process's PCB

Definition at line 194 of file kernel.c.

References init_process.

4.17.2.5 k_get_process() pcb_t* k_get_process (pid_t *pid*)

Get a process by PID.

Parameters

pid	Process ID to look up

Returns

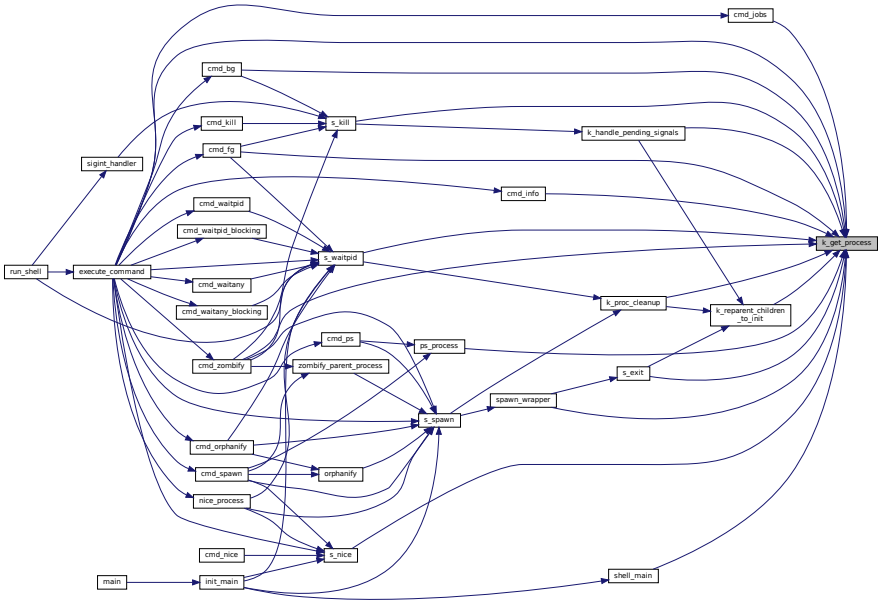
Pointer to PCB, or NULL if not found

Definition at line 159 of file kernel.c.

References kernel_initialized, MAX_PROCESSES, and process_table.

Referenced by cmd_bg(), cmd_fg(), cmd_info(), cmd_jobs(), cmd_zombify(), execute_command(), k_handle_pending_signals(), k_proc_cleanup(), k_reparent_children_to_init(), ps_process(), s_exit(), s_kill(), s_nice(), s_waitpid(), shell_main(), and spawn_wrapper().

Here is the caller graph for this function:



4.17.2.6 k_get_process_count() `int k_get_process_count ( void )`

Get total number of active processes.

Returns

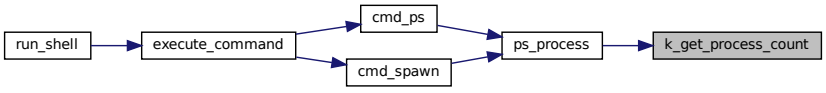
Number of processes in the system (excluding zombies)

Definition at line 488 of file kernel.c.

References kernel_initialized, MAX_PROCESSES, process_table, and ZOMBIE.

Referenced by ps_process().

Here is the caller graph for this function:



4.17.2.7 k_get_process_table() pcb_t** k_get_process_table (void)

To give scheduler access to process table.

Returns

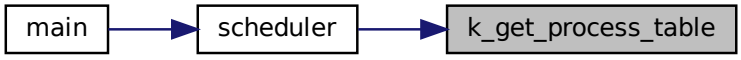
pcb_t** ptr to process table

Definition at line 508 of file kernel.c.

References kernel_initialized, and process_table.

Referenced by scheduler().

Here is the caller graph for this function:



4.17.2.8 k_handle_pending_signals() void k_handle_pending_signals (pcb_t * *proc*)

Handle pending signals for a process.

Should be called by the scheduler before context switching to check if the process has any pending signals that need to be handled.

Parameters

proc	Process to check

< Stop (pause) a process

< Continue a stopped process

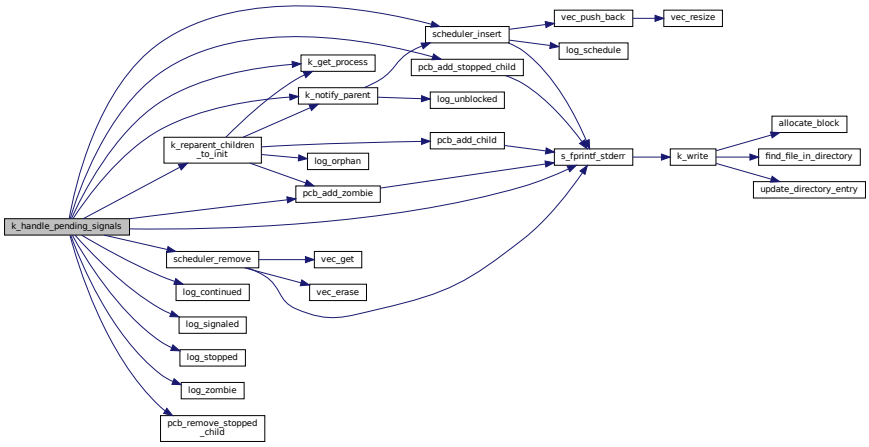
< Terminate a process

Definition at line 408 of file kernel.c.

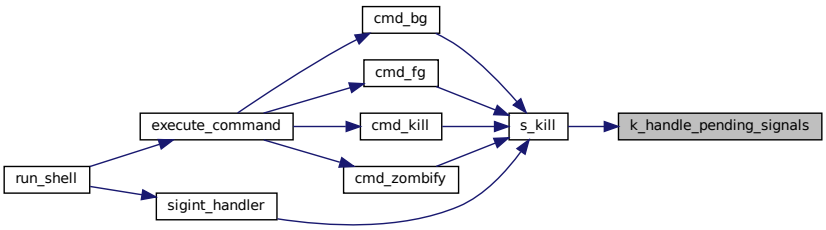
References pcb::exit_status, INIT_PID, k_get_process(), k_notify_parent(), k_reparent_children_to_init(), log_↩ continued(), log_signaled(), log_stopped(), log_zombie(), pcb::name, pcb::num_children, pcb::num_zombies, P_↩ SIGCONT, P_SIGSTOP, P_SIGTERM, pcb_add_stopped_child(), pcb_add_zombie(), pcb_remove_stopped_child(), pcb::pending_signal, pcb::pid, pcb::ppid, pcb::priority, RUNNING, s_fprintf_stderr(), scheduler_insert(), scheduler_↩ remove(), pcb::state, STOPPED, and ZOMBIE.

Referenced by s_kill().

Here is the call graph for this function:



Here is the caller graph for this function:



4.17.2.9 k_init() `void k_init ( void )`

Initialize the kernel and process table.

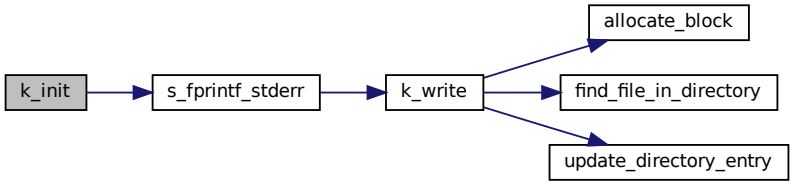
This should be called once at boot time before any other kernel functions.

Definition at line 29 of file kernel.c.

References current_process, init_process, kernel_initialized, MAX_PROCESSES, next_pid, process_table, and s_↩ fprintf_stderr().

Referenced by main().

Here is the call graph for this function:



Here is the caller graph for this function:



4.17.2.10 k_notify_parent()

```
void k_notify_parent (
    pid_t child_pid,
    pcb_t * parent )
```

Notify parent that a child has changed state.

This is called when a child process:

- Exits (becomes zombie)
- Stops (P_SIGSTOP)
- Continues (P_SIGCONT)

Parameters

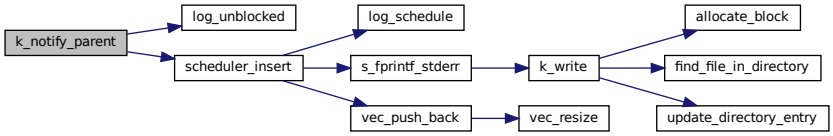
child_pid	Child process ID
parent	Parent PCB

Definition at line 319 of file kernel.c.

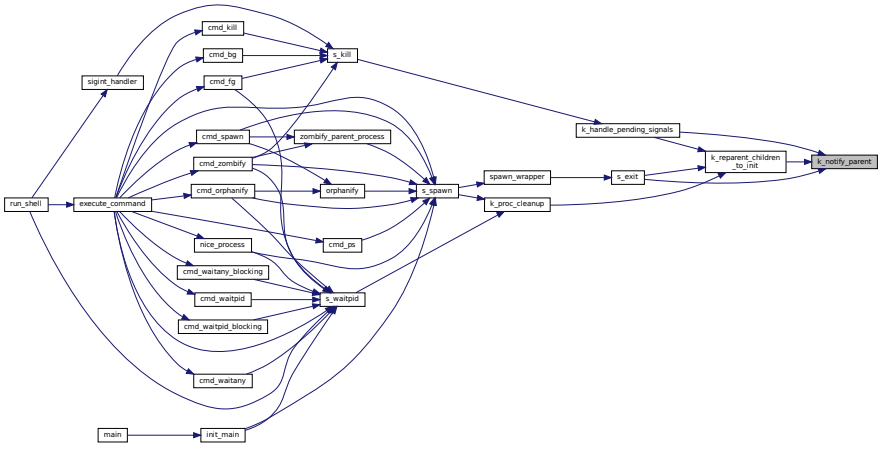
References BLOCKED, pcb::is_waiting, log_unblocked(), pcb::name, pcb::pid, pcb::priority, RUNNING, scheduler_↩ insert(), pcb::state, and pcb::waiting_for_pid.

Referenced by k_handle_pending_signals(), k_reparent_children_to_init(), and s_exit().

Here is the call graph for this function:



Here is the caller graph for this function:



4.17.2.11 k_proc_cleanup() void k_proc_cleanup (pcb_t * proc)

Clean up a terminated/finished process's resources.

This handles:

- Removing the process from the process table
- Handling zombie children (reparenting to INIT)
- Notifying parent of termination
- Freeing the PCB

Parameters

proc	Process to clean up

This handles:

- Reparenting children to INIT
- Notifying parent of termination
- Removing from parent's children list
- Removing from process table
- Freeing the PCB

Parameters

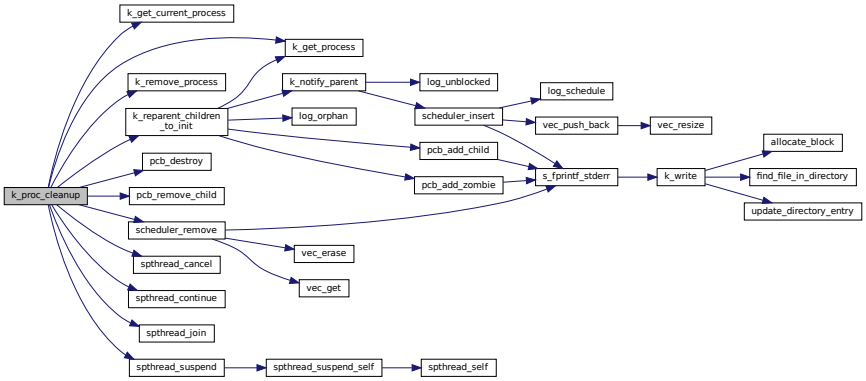
proc	Process to clean up

Definition at line 355 of file kernel.c.

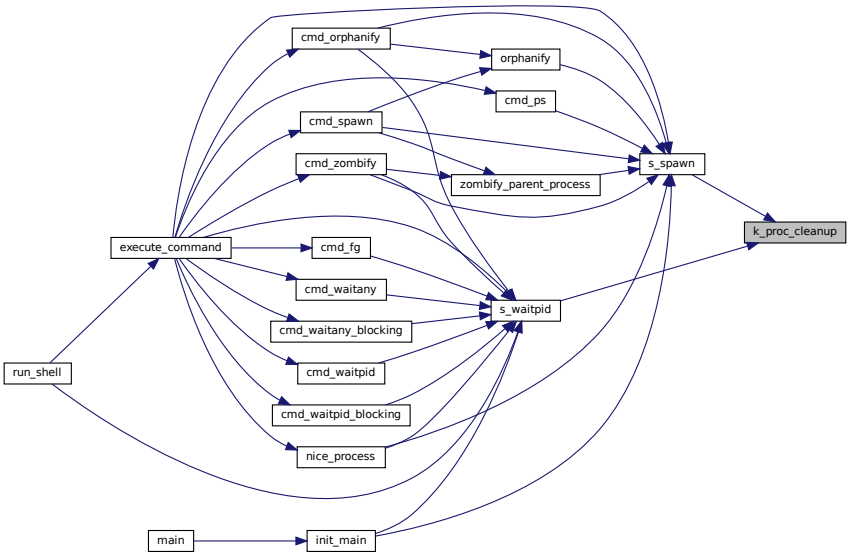
References k_get_current_process(), k_get_process(), k_remove_process(), k_reparent_children_to_init(), kernel_initialized, pcb::num_children, pcb::num_zombies, pcb_destroy(), pcb_remove_child(), pcb::pid, pcb::ppid, scheduler_remove(), spthread_cancel(), spthread_continue(), spthread_join(), spthread_suspend(), pcb::thread, and pcb::thread_created.

Referenced by s_spawn(), and s_waitpid().

Here is the call graph for this function:



Here is the caller graph for this function:



4.17.2.12 k_proc_create() pcb_t* k_proc_create (pcb_t * *parent*)

Create a new child process, inheriting applicable properties from the parent.

Parameters

parent	Parent PCB (can be NULL for INIT process)

Returns

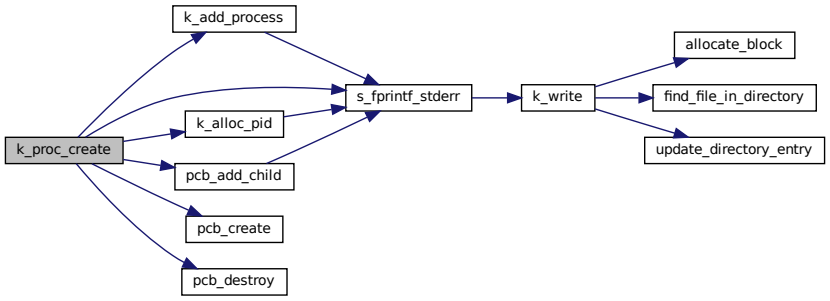
Pointer to the child PCB, or NULL on failure

Definition at line 205 of file kernel.c.

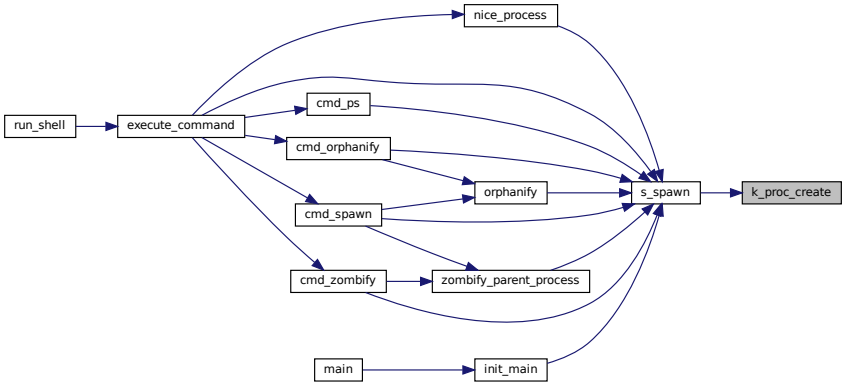
References pcb::fd_table, k_add_process(), k_alloc_pid(), kernel_initialized, MAX_FDS, pcb_add_child(), pcb_create(), pcb_destroy(), pcb::pid, and s_fprintf_stderr().

Referenced by s_spawn().

Here is the call graph for this function:



Here is the caller graph for this function:



4.17.2.13 k_remove_process() void k_remove_process (pid_t *pid*)

Remove a process from the process table.

Parameters

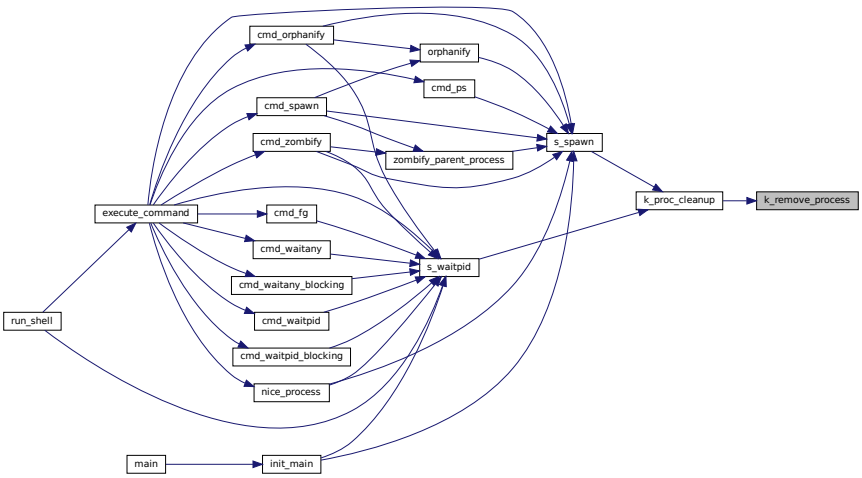
pid	Process ID to remove

Definition at line 141 of file kernel.c.

References kernel_initialized, MAX_PROCESSES, and process_table.

Referenced by k_proc_cleanup().

Here is the caller graph for this function:



4.17.2.14 k_reparent_children_to_init() `void k_reparent_children_to_init ( pid_t parent_pid )`

Reparent children to INIT process.

Called when a parent process exits with living children.

Parameters

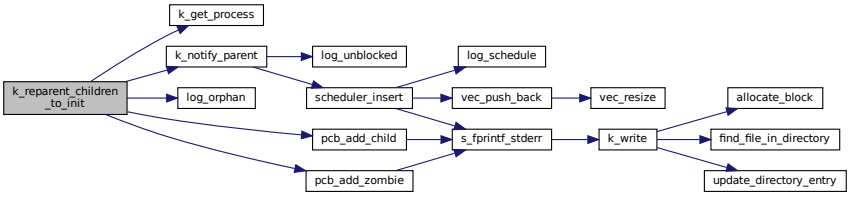
parent_pid	PID of the parent that is exiting

Definition at line 256 of file kernel.c.

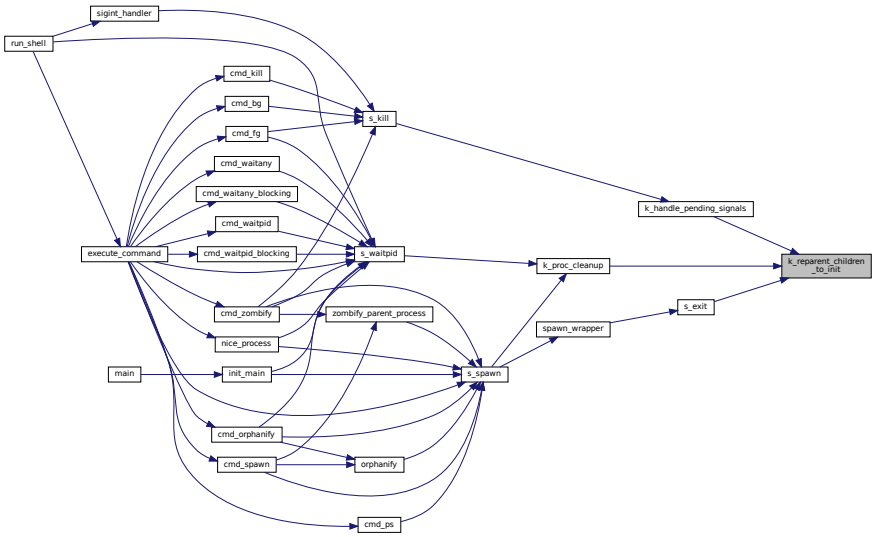
References pcb::children, pcb::exit_statuses, INIT_PID, init_process, k_get_process(), k_notify_parent(), kernel↩ _initialized, log_orphan(), pcb::name, pcb::num_children, pcb::num_zombies, pcb_add_child(), pcb_add_zombie(), pcb::pid, pcb::ppid, pcb::priority, and pcb::zombie_children.

Referenced by k_handle_pending_signals(), k_proc_cleanup(), and s_exit().

Here is the call graph for this function:



Here is the caller graph for this function:



4.17.2.15 k_set_current_process() `void k_set_current_process ( pcb_t * proc )`

Set the currently running process.

Parameters

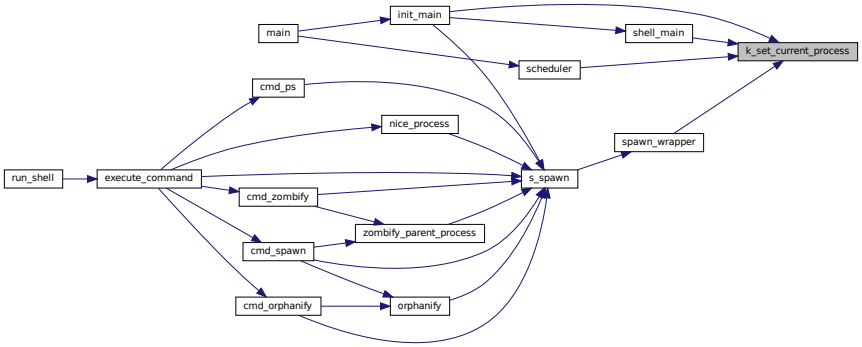
proc	Process to set as current

Definition at line 185 of file kernel.c.

References current_process.

Referenced by init_main(), scheduler(), shell_main(), and spawn_wrapper().

Here is the caller graph for this function:



4.17.2.16 k_shutdown() `void k_shutdown ( void )`

Shutdown the kernel and clean up all resources.

Definition at line 53 of file kernel.c.

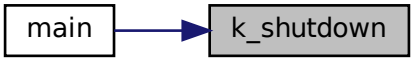
References current_process, init_process, kernel_initialized, MAX_PROCESSES, pcb_destroy(), and process_table.

Referenced by main().

Here is the call graph for this function:



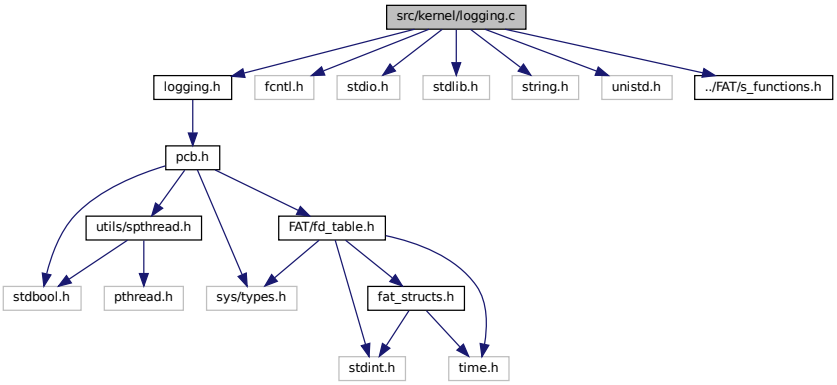
Here is the caller graph for this function:



4.18 src/kernel/logging.c File Reference

```
#include "logging.h"
#include <fcntl.h>
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include "../FAT/s_functions.h"
```

Include dependency graph for logging.c:



Functions

- void logger_init (const char *filename)

  Initialize the logger and open the log file.
- void logger_close (void)

  Close the log file and clean up logger resources.
- int logger_get_ticks (void)

  Get the current tick count for logging.
- void logger_set_ticks (int ticks)

  Set the current tick count.
- void log_schedule (int pid, int priority_queue, const char *process_name)

  Log a SCHEDULE event when a process is scheduled to run.
- void log_create (int pid, int nice_value, const char *process_name)

  Log a CREATE event when a new process is created.
- void log_signaled (int pid, int nice_value, const char *process_name)

  Log a SIGNALED event when a process is terminated by signal.
- void log_exited (int pid, int nice_value, const char *process_name)

  Log an EXITED event when a process exits normally.
- void log_zombie (int pid, int nice_value, const char *process_name)

  Log a ZOMBIE event when a process becomes a zombie.
- void log_orphan (int pid, int nice_value, const char *process_name)

Log an ORPHAN event when a process become orphan and is reparented to INIT.

- void log_waited (int pid, int nice_value, const char ∗process_name)

  Log a WAITED event when a zombie process is reaped by parent.
- void log_nice (int pid, int old_nice_value, int new_nice_value, const char ∗process_name)

  Log a NICE event when a process priority is changed.
- void log_blocked (int pid, int nice_value, const char ∗process_name)

  Log a BLOCKED event when a process enters blocked state.
- void log_unblocked (int pid, int nice_value, const char ∗process_name)

  Log an UNBLOCKED event when a process exits blocked state.
- void log_stopped (int pid, int nice_value, const char ∗process_name)

  Log a STOPPED event when a process stops.
- void log_continued (int pid, int nice_value, const char ∗process_name)

  Log a CONTINUED event when a stopped process continues.

Variables

- static int log_fd = -1
- static int current_ticks

4.18.1 Function Documentation

4.18.1.1 log_blocked()

```
void log_blocked (
        int pid,
        int nice_value,
        const char * process_name )
```

Log a BLOCKED event when a process enters blocked state.

Parameters

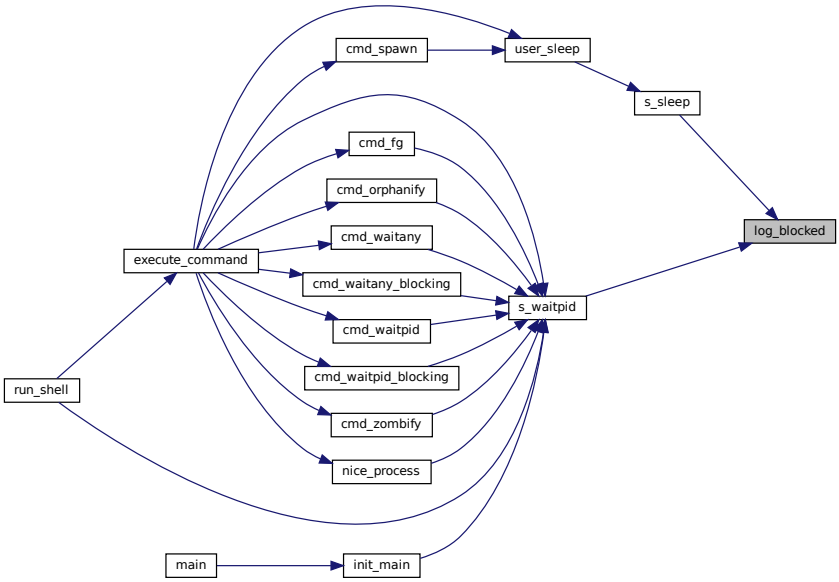
pid	Process ID
nice_value	Process priority/nice value
process_name	Name of the process

Definition at line 115 of file logging.c.

References current_ticks, and log_fd.

Referenced by s_sleep(), and s_waitpid().

Here is the caller graph for this function:



4.18.1.2 log_continued()

```
void log_continued (
        int pid,
        int nice_value,
        const char * process_name )
```

Log a CONTINUED event when a stopped process continues.

Parameters

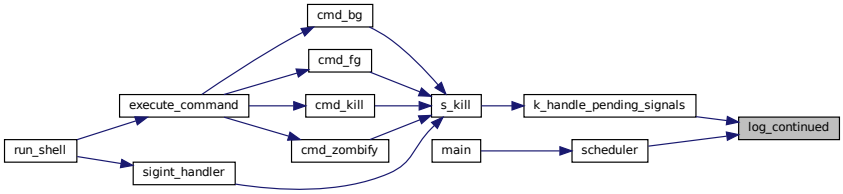
pid	Process ID
nice_value	Process priority/nice value
process_name	Name of the process

Definition at line 143 of file logging.c.

References current_ticks, and log_fd.

Referenced by k_handle_pending_signals(), and scheduler().

Here is the caller graph for this function:



4.18.1.3 log_create()

```
void log_create (
        int pid,
        int nice_value,
        const char * process_name )
```

Log a CREATE event when a new process is created.

Parameters

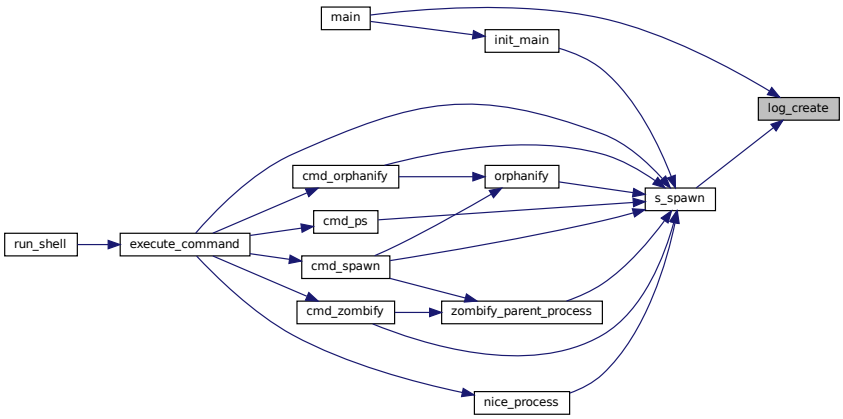
pid	Process ID
nice_value	Process priority/nice value
process_name	Name of the process

Definition at line 47 of file logging.c.

References current_ticks, and log_fd.

Referenced by main(), and s_spawn().

Here is the caller graph for this function:



4.18.1.4 log_exited() `void log_exited (`

```
int pid,
int nice_value,
const char * process_name )
```

Log an EXITED event when a process exits normally.

Parameters

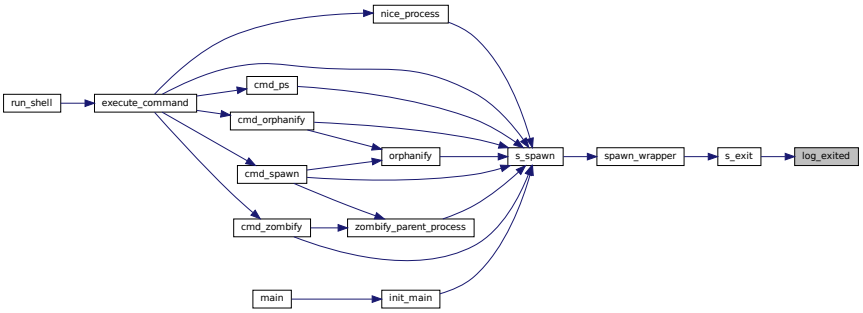
pid	Process ID
nice_value	Process priority/nice value
process_name	Name of the process

Definition at line 65 of file logging.c.

References current_ticks, and log_fd.

Referenced by s_exit().

Here is the caller graph for this function:



4.18.1.5 log_nice() `void log_nice (`

```
int pid,
int old_nice_value,
int new_nice_value,
const char * process_name )
```

Log a NICE event when a process priority is changed.

Parameters

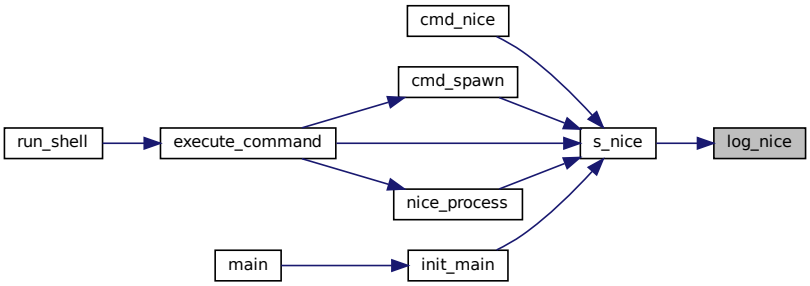
pid	Process ID
old_nice_value	Previous priority value
new_nice_value	New priority value
process_name	Name of the process

Definition at line 102 of file logging.c.

References current_ticks, and log_fd.

Referenced by s_nice().

Here is the caller graph for this function:



4.18.1.6 log_orphan()

```
void log_orphan (
        int pid,
        int nice_value,
        const char * process_name )
```

Log an ORPHAN event when a process become orphan and is reparented to INIT.

Parameters

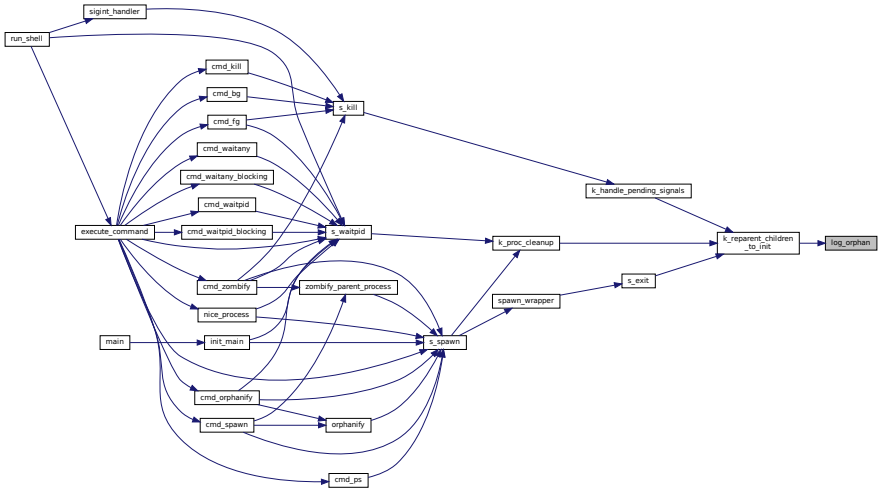
pid	Process ID
nice_value	Process priority/nice value
process_name	Name of the process

Definition at line 83 of file logging.c.

References current_ticks, and log_fd.

Referenced by k_reparent_children_to_init().

Here is the caller graph for this function:



4.18.1.7 log_schedule()

```
void log_schedule (
        int pid,
        int priority,
        const char * process_name )
```

Log a SCHEDULE event when a process is scheduled to run.

Parameters

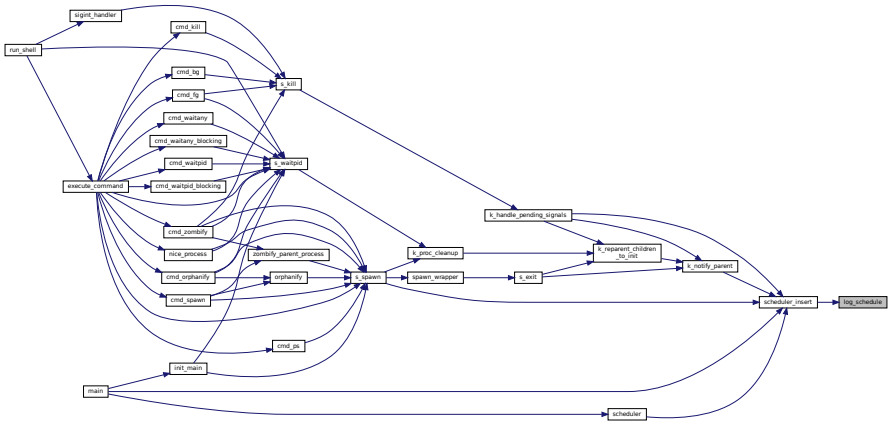
pid	Process ID
priority	Process priority level (0, 1, or 2)
process_name	Name of the process

Definition at line 37 of file logging.c.

References current_ticks, and log_fd.

Referenced by scheduler_insert().

Here is the caller graph for this function:



4.18.1.8 log_signaled()

```
void log_signaled (
        int pid,
        int nice_value,
        const char * process_name )
```

Log a SIGNALED event when a process is terminated by signal.

Parameters

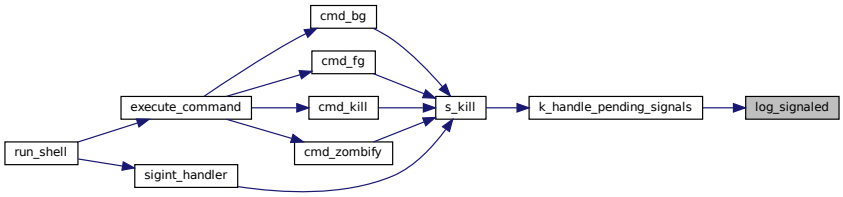
pid	Process ID
nice_value	Process priority/nice value
process_name	Name of the process

Definition at line 56 of file logging.c.

References current_ticks, and log_fd.

Referenced by k_handle_pending_signals().

Here is the caller graph for this function:



4.18.1.9 log_stopped() `void log_stopped (`

```
int pid,
int nice_value,
const char * process_name )
```

Log a STOPPED event when a process stops.

Parameters

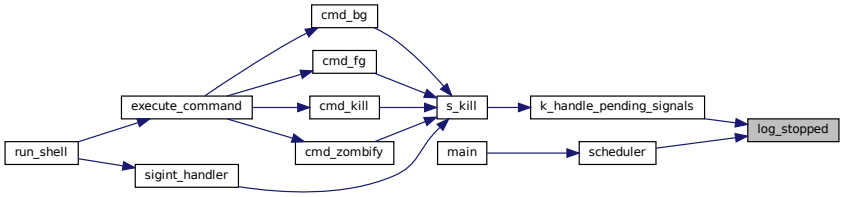
pid	Process ID
nice_value	Process priority/nice value
process_name	Name of the process

Definition at line 134 of file logging.c.

References current_ticks, and log_fd.

Referenced by k_handle_pending_signals(), and scheduler().

Here is the caller graph for this function:



4.18.1.10 log_unblocked() `void log_unblocked (`

```
int pid,
int nice_value,
const char * process_name )
```

Log an UNBLOCKED event when a process exits blocked state.

Parameters

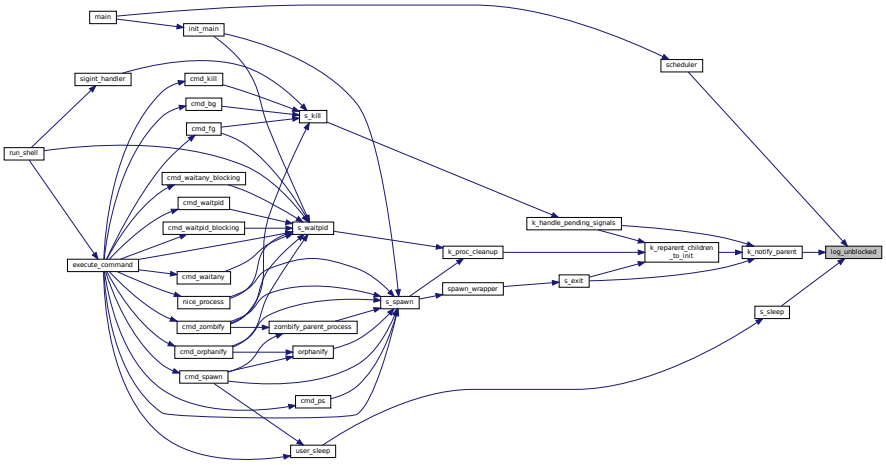
pid	Process ID
nice_value	Process priority/nice value
process_name	Name of the process

Definition at line 124 of file logging.c.

References current_ticks, and log_fd.

Referenced by k_notify_parent(), s_sleep(), and scheduler().

Here is the caller graph for this function:



4.18.1.11 log_waited()

```
void log_waited (
    int pid,
    int nice_value,
    const char * process_name )
```

Log a WAITED event when a zombie process is reaped by parent.

Parameters

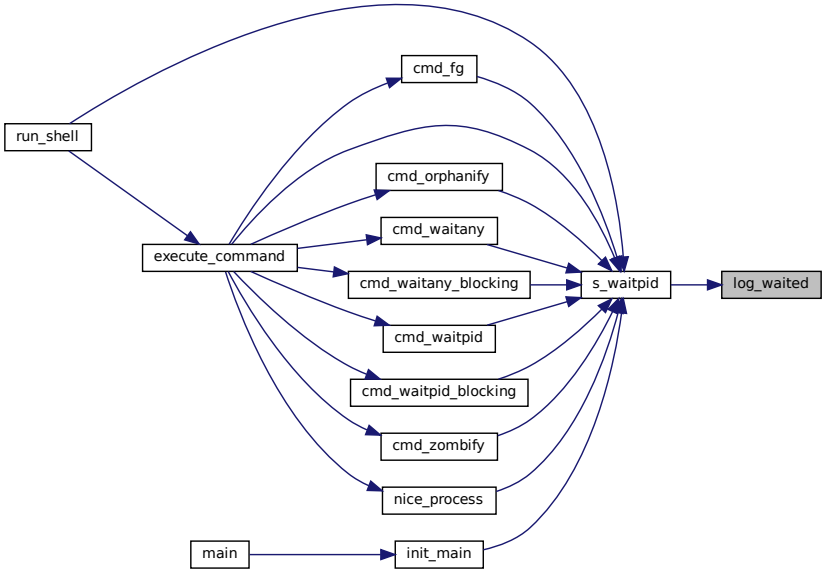
pid	Process ID
nice_value	Process priority/nice value
process_name	Name of the process

Definition at line 92 of file logging.c.

References current_ticks, and log_fd.

Referenced by s_waitpid().

Here is the caller graph for this function:



4.18.1.12 log_zombie()

```
void log_zombie (
    int pid,
    int nice_value,
    const char * process_name )
```

Log a ZOMBIE event when a process becomes a zombie.

Parameters

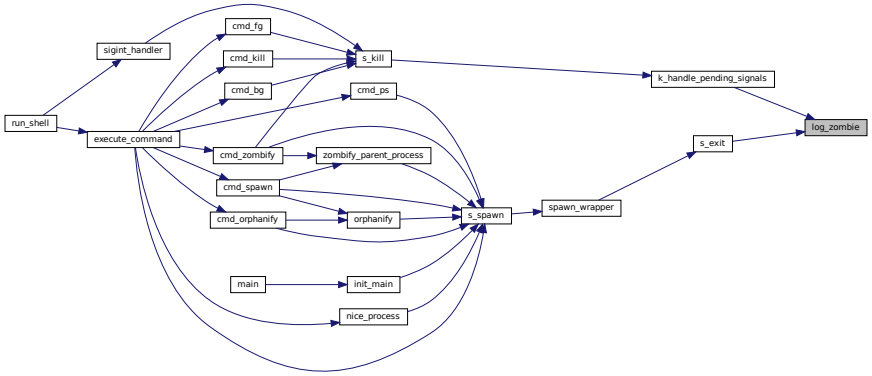
pid	Process ID
nice_value	Process priority/nice value
process_name	Name of the process

Definition at line 74 of file logging.c.

References current_ticks, and log_fd.

Referenced by k_handle_pending_signals(), and s_exit().

Here is the caller graph for this function:



4.18.1.13 logger_close() `void logger_close ( void )`

Close the log file and clean up logger resources.

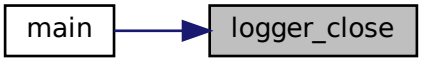
Closes the log file handle and resets the logger state.

Definition at line 21 of file logging.c.

References log_fd.

Referenced by main().

Here is the caller graph for this function:



4.18.1.14 logger_get_ticks() `int logger_get_ticks ( void )`

Get the current tick count for logging.

Returns the current system tick counter used for timestamping log entries. The tick is incremented by the scheduler every quantum (100ms).

Returns

Current tick count

Definition at line 28 of file logging.c.

References current_ticks.

4.18.1.15 logger_init() `void logger_init ( const char * log_filename )`

Initialize the logger and open the log file.

Opens the specified log file for writing and initializes the tick counter to 0. Must be called once during kernel initialization before any logging functions are used.

Parameters

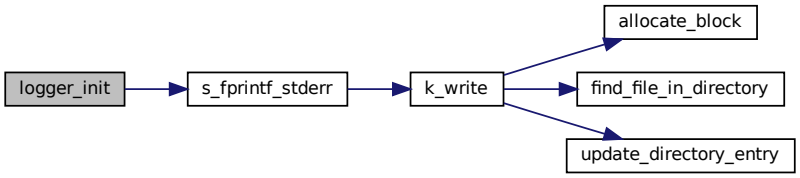
log_filename	Path to the log file to create/open

Definition at line 12 of file logging.c.

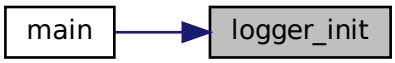
References current_ticks, log_fd, and s_fprintf_stderr().

Referenced by main().

Here is the call graph for this function:



Here is the caller graph for this function:



4.18.1.16 logger_set_ticks() `void logger_set_ticks ( int ticks )`

Set the current tick count.

Updates the system tick counter. Called by the scheduler at the start of each quantum to maintain the tick count for timestamping log entries.

Parameters

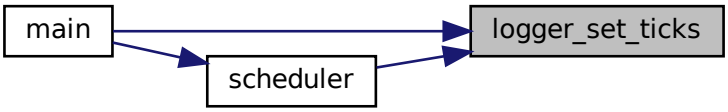
ticks	New tick count value

Definition at line 32 of file logging.c.

References current_ticks.

Referenced by main(), and scheduler().

Here is the caller graph for this function:



4.18.2 Variable Documentation

4.18.2.1 current_ticks `int current_ticks [static]`

Definition at line 10 of file logging.c.

Referenced by log_blocked(), log_continued(), log_create(), log_exited(), log_nice(), log_orphan(), log_schedule(), log_signaled(), log_stopped(), log_unblocked(), log_waited(), log_zombie(), logger_get_ticks(), logger_init(), and logger_set_ticks().

4.18.2.2 log_fd `int log_fd = -1 [static]`

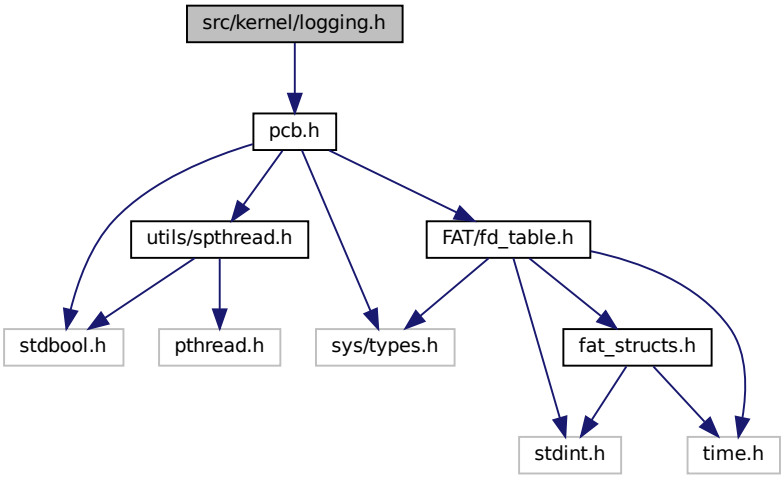
Definition at line 9 of file logging.c.

Referenced by log_blocked(), log_continued(), log_create(), log_exited(), log_nice(), log_orphan(), log_schedule(), log_signaled(), log_stopped(), log_unblocked(), log_waited(), log_zombie(), logger_close(), and logger_init().

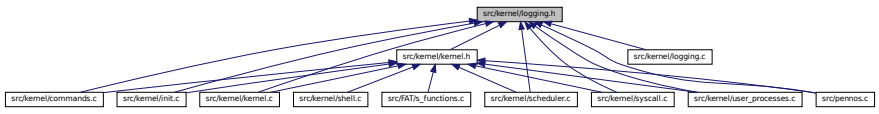
4.19 src/kernel/logging.h File Reference

```
#include "pcb.h"
```

Include dependency graph for logging.h:



This graph shows which files directly or indirectly include this file:



Functions

- void logger_init (const char *log_filename)

  Initialize the logger and open the log file.
- void logger_close (void)

  Close the log file and clean up logger resources.
- int logger_get_ticks (void)

  Get the current tick count for logging.
- void logger_set_ticks (int ticks)

  Set the current tick count.
- void log_schedule (int pid, int priority, const char *process_name)

  Log a SCHEDULE event when a process is scheduled to run.
- void log_create (int pid, int nice_value, const char *process_name)

  Log a CREATE event when a new process is created.
- void log_signaled (int pid, int nice_value, const char *process_name)

  Log a SIGNALED event when a process is terminated by signal.
- void log_exited (int pid, int nice_value, const char *process_name)

  Log an EXITED event when a process exits normally.
- void log_zombie (int pid, int nice_value, const char *process_name)

  Log a ZOMBIE event when a process becomes a zombie.
- void log_orphan (int pid, int nice_value, const char *process_name)

  Log an ORPHAN event when a process become orphan and is reparented to INIT.
- void log_waited (int pid, int nice_value, const char *process_name)

  Log a WAITED event when a zombie process is reaped by parent.
- void log_nice (int pid, int old_nice_value, int new_nice_value, const char *process_name)

  Log a NICE event when a process priority is changed.
- void log_blocked (int pid, int nice_value, const char *process_name)

  Log a BLOCKED event when a process enters blocked state.
- void log_unblocked (int pid, int nice_value, const char *process_name)

  Log an UNBLOCKED event when a process exits blocked state.
- void log_stopped (int pid, int nice_value, const char *process_name)

  Log a STOPPED event when a process stops.
- void log_continued (int pid, int nice_value, const char *process_name)

  Log a CONTINUED event when a stopped process continues.

4.19.1 Function Documentation

4.19.1.1 log_blocked()

```
void log_blocked (
    int pid,
    int nice_value,
    const char * process_name )
```

Log a BLOCKED event when a process enters blocked state.

Parameters

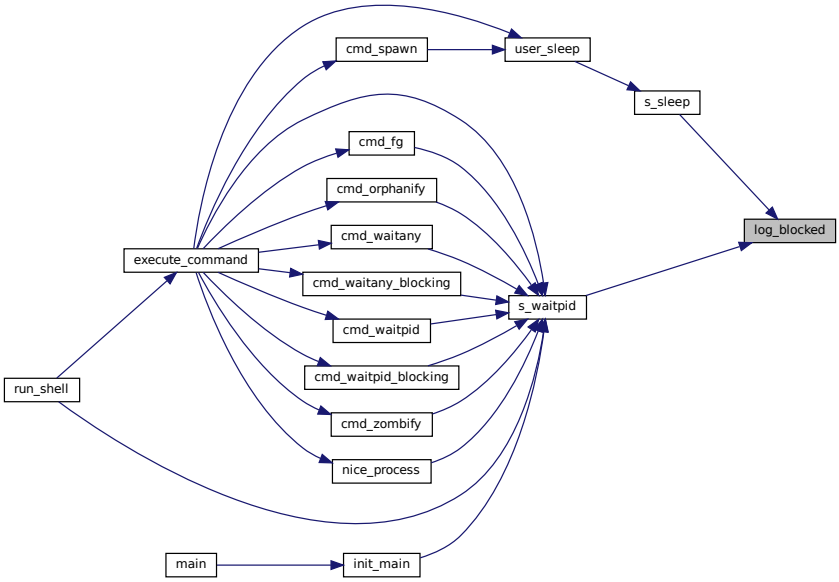
pid	Process ID
nice_value	Process priority/nice value
process_name	Name of the process

Definition at line 115 of file logging.c.

References current_ticks, and log_fd.

Referenced by s_sleep(), and s_waitpid().

Here is the caller graph for this function:



4.19.1.2 log_continued()

```
void log_continued (
        int pid,
        int nice_value,
        const char * process_name )
```

Log a CONTINUED event when a stopped process continues.

Parameters

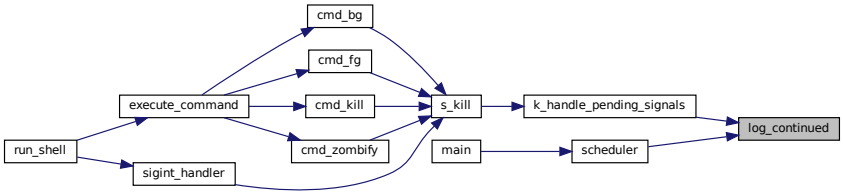
pid	Process ID
nice_value	Process priority/nice value
process_name	Name of the process

Definition at line 143 of file logging.c.

References current_ticks, and log_fd.

Referenced by k_handle_pending_signals(), and scheduler().

Here is the caller graph for this function:



4.19.1.3 log_create()

```
void log_create (
        int pid,
        int nice_value,
        const char * process_name )
```

Log a CREATE event when a new process is created.

Parameters

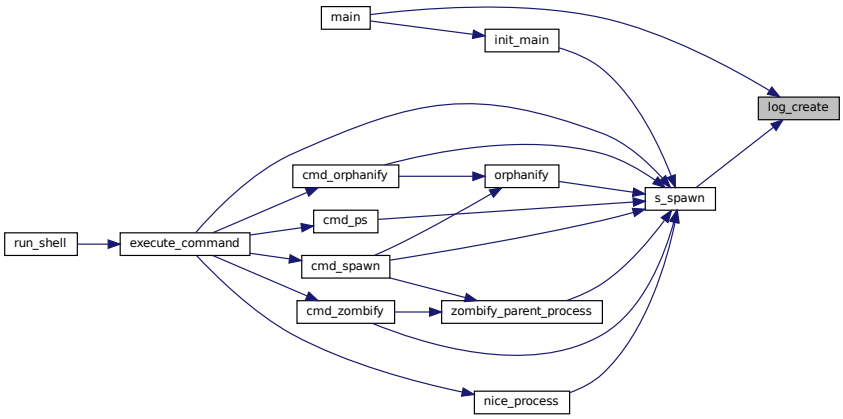
pid	Process ID
nice_value	Process priority/nice value
process_name	Name of the process

Definition at line 47 of file logging.c.

References current_ticks, and log_fd.

Referenced by main(), and s_spawn().

Here is the caller graph for this function:



4.19.1.4 log_exited()

```
void log_exited (
        int pid,
        int nice_value,
        const char * process_name )
```

Log an EXITED event when a process exits normally.

Parameters

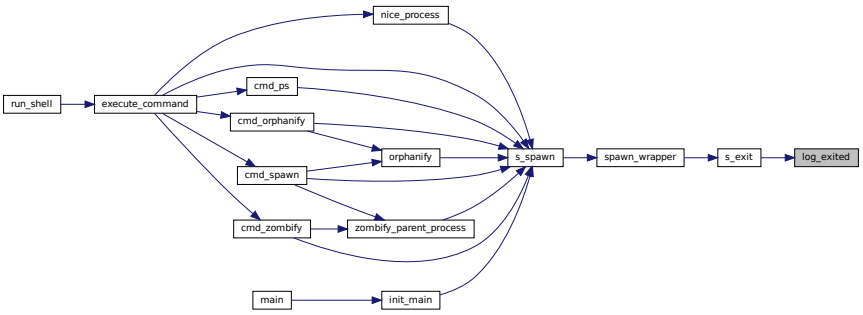
pid	Process ID
nice_value	Process priority/nice value
process_name	Name of the process

Definition at line 65 of file logging.c.

References current_ticks, and log_fd.

Referenced by s_exit().

Here is the caller graph for this function:



4.19.1.5 log_nice()

```
void log_nice (
        int pid,
        int old_nice_value,
        int new_nice_value,
        const char * process_name )
```

Log a NICE event when a process priority is changed.

Parameters

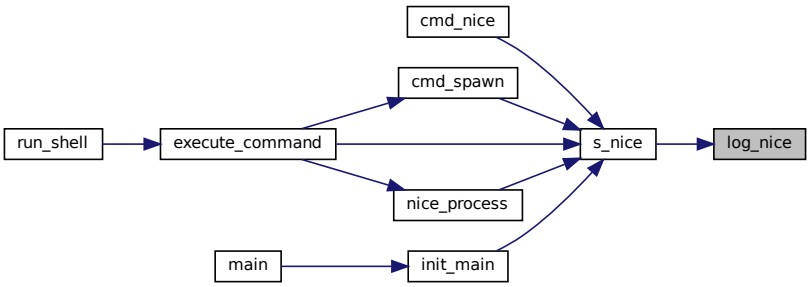
pid	Process ID
old_nice_value	Previous priority value
new_nice_value	New priority value
process_name	Name of the process

Definition at line 102 of file logging.c.

References current_ticks, and log_fd.

Referenced by s_nice().

Here is the caller graph for this function:



4.19.1.6 log_orphan()

```
void log_orphan (
        int pid,
        int nice_value,
        const char * process_name )
```

Log an ORPHAN event when a process become orphan and is reparented to INIT.

Parameters

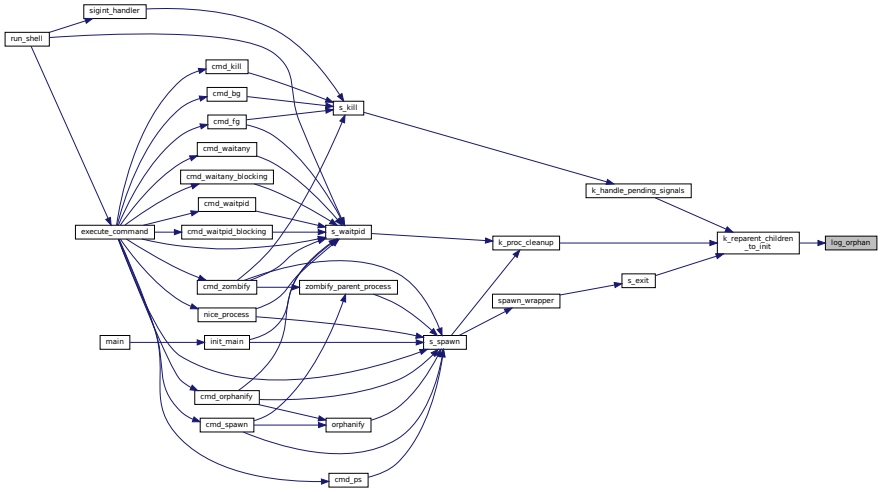
pid	Process ID
nice_value	Process priority/nice value
process_name	Name of the process

Definition at line 83 of file logging.c.

References current_ticks, and log_fd.

Referenced by k_reparent_children_to_init().

Here is the caller graph for this function:



4.19.1.7 log_schedule()

```
void log_schedule (
        int pid,
        int priority,
        const char * process_name )
```

Log a SCHEDULE event when a process is scheduled to run.

Parameters

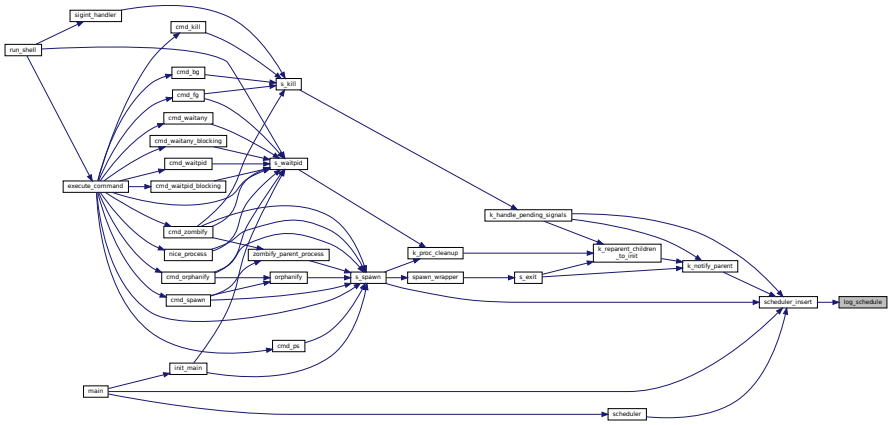
pid	Process ID
priority	Process priority level (0, 1, or 2)
process_name	Name of the process

Definition at line 37 of file logging.c.

References current_ticks, and log_fd.

Referenced by scheduler_insert().

Here is the caller graph for this function:



4.19.1.8 log_signaled()

```
void log_signaled (
        int pid,
        int nice_value,
        const char * process_name )
```

Log a SIGNALED event when a process is terminated by signal.

Parameters

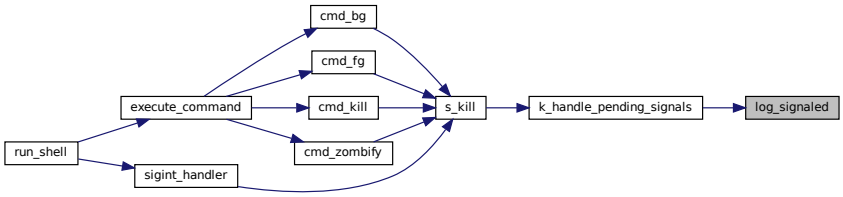
pid	Process ID
nice_value	Process priority/nice value
process_name	Name of the process

Definition at line 56 of file logging.c.

References current_ticks, and log_fd.

Referenced by k_handle_pending_signals().

Here is the caller graph for this function:



4.19.1.9 log_stopped()

```
void log_stopped (
        int pid,
        int nice_value,
        const char * process_name )
```

Log a STOPPED event when a process stops.

Parameters

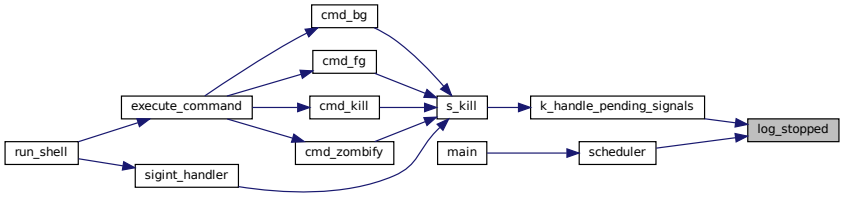
pid	Process ID
nice_value	Process priority/nice value
process_name	Name of the process

Definition at line 134 of file logging.c.

References current_ticks, and log_fd.

Referenced by k_handle_pending_signals(), and scheduler().

Here is the caller graph for this function:



4.19.1.10 log_unblocked()

```
void log_unblocked (
        int pid,
        int nice_value,
        const char * process_name )
```

Log an UNBLOCKED event when a process exits blocked state.

Parameters

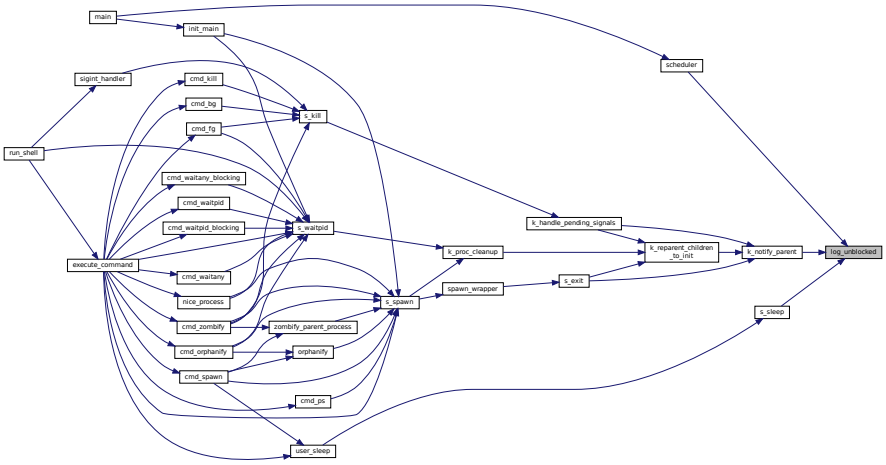
pid	Process ID
nice_value	Process priority/nice value
process_name	Name of the process

Definition at line 124 of file logging.c.

References current_ticks, and log_fd.

Referenced by k_notify_parent(), s_sleep(), and scheduler().

Here is the caller graph for this function:



4.19.1.11 log_waited()

```
void log_waited (
    int pid,
    int nice_value,
    const char * process_name )
```

Log a WAITED event when a zombie process is reaped by parent.

Parameters

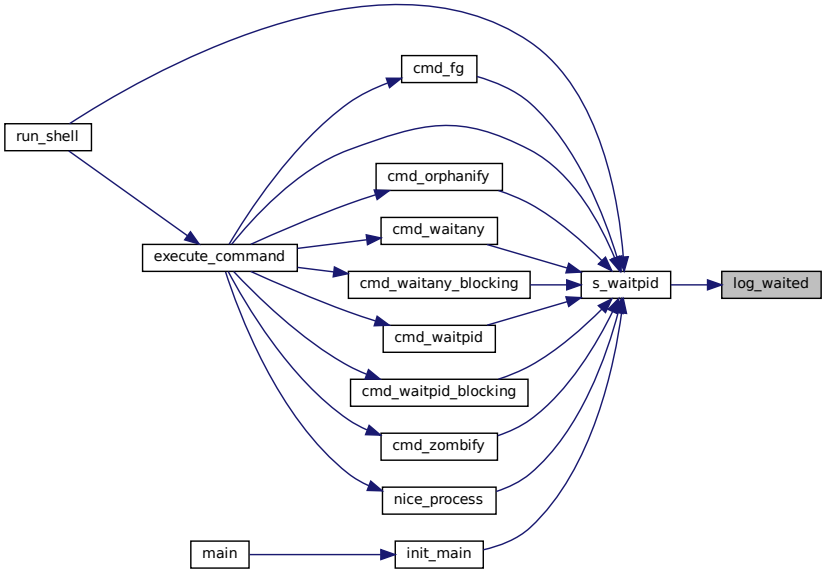
pid	Process ID
nice_value	Process priority/nice value
process_name	Name of the process

Definition at line 92 of file logging.c.

References current_ticks, and log_fd.

Referenced by s_waitpid().

Here is the caller graph for this function:



4.19.1.12 log_zombie()

```
void log_zombie (
    int pid,
    int nice_value,
    const char * process_name )
```

Log a ZOMBIE event when a process becomes a zombie.

Parameters

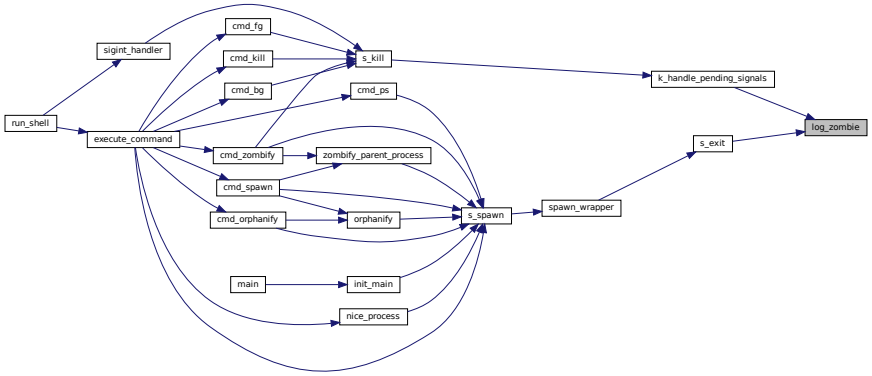
pid	Process ID
nice_value	Process priority/nice value
process_name	Name of the process

Definition at line 74 of file logging.c.

References current_ticks, and log_fd.

Referenced by k_handle_pending_signals(), and s_exit().

Here is the caller graph for this function:



4.19.1.13 logger_close() `void logger_close ( void )`

Close the log file and clean up logger resources.

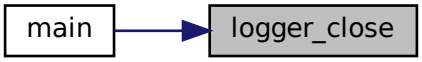
Closes the log file handle and resets the logger state.

Definition at line 21 of file logging.c.

References log_fd.

Referenced by main().

Here is the caller graph for this function:



4.19.1.14 logger_get_ticks()

```
int logger_get_ticks (
        void  )
```

Get the current tick count for logging.

Returns the current system tick counter used for timestamping log entries. The tick is incremented by the scheduler every quantum (100ms).

Returns

Current tick count

Definition at line 28 of file logging.c.

References current_ticks.

4.19.1.15 logger_init()

```
void logger_init (
        const char * log_filename )
```

Initialize the logger and open the log file.

Opens the specified log file for writing and initializes the tick counter to 0. Must be called once during kernel initialization before any logging functions are used.

Parameters

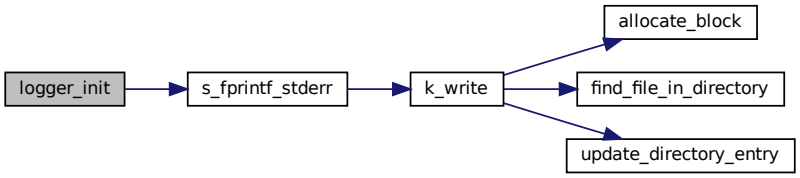
log_filename	Path to the log file to create/open

Definition at line 12 of file logging.c.

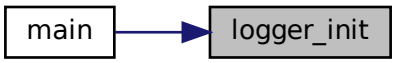
References current_ticks, log_fd, and s_fprintf_stderr().

Referenced by main().

Here is the call graph for this function:



Here is the caller graph for this function:



4.19.1.16 logger_set_ticks() `void logger_set_ticks ( int ticks )`

Set the current tick count.

Updates the system tick counter. Called by the scheduler at the start of each quantum to maintain the tick count for timestamping log entries.

Parameters

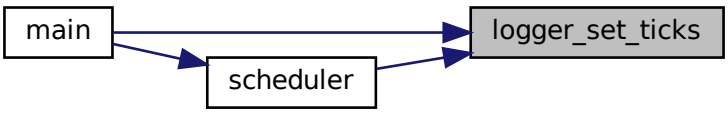
ticks	New tick count value

Definition at line 32 of file logging.c.

References current_ticks.

Referenced by main(), and scheduler().

Here is the caller graph for this function:

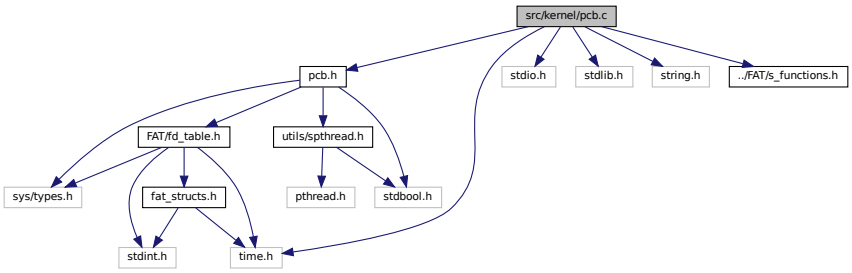


4.20 src/kernel/pcb.c File Reference

```
#include "pcb.h"
#include <stdio.h>
```

```
#include <stdlib.h>
#include <string.h>
#include <time.h>
#include "../FAT/s_functions.h"
```

Include dependency graph for pcb.c:



Functions

- pcb_t * pcb_create (pid_t pid, pid_t ppid, const char *name, int priority)

  Create a new PCB.
- void pcb_destroy (pcb_t *pcb)

  Destroy a PCB and free its resources.
- void pcb_add_child (pcb_t *parent, pid_t child_pid)

  Add a child PID to the parent's children list.
- void pcb_remove_child (pcb_t *parent, pid_t child_pid)

  Remove a child PID from the parent's children list.
- void pcb_add_zombie (pcb_t *parent, pid_t child_pid, int exit_status)

  Add a zombie child to the parent's zombie queue.
- pid_t pcb_get_zombie (pcb_t *parent, pid_t pid, int *exit_status)

  Get and remove a zombie child from the parent's zombie queue.
- void pcb_add_stopped_child (pcb_t *parent, pid_t child_pid)

  Add a stopped child to the parent's stopped children list.
- void pcb_remove_stopped_child (pcb_t *parent, pid_t child_pid)

  Remove a stopped child from the parent's stopped children list.

4.20.1 Function Documentation

4.20.1.1 pcb_add_child() `void pcb_add_child (`

```
    pcb_t * parent,
    pid_t child_pid )
```

Add a child PID to the parent's children list.

Add a child to the parent's children list.

Parameters

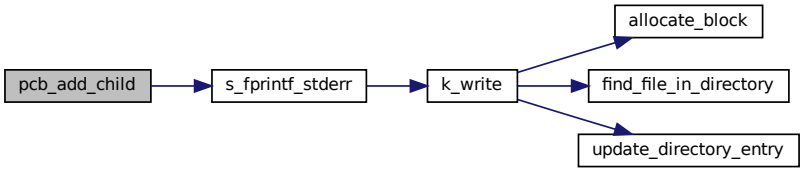
parent	Parent PCB
child_pid	Child process ID

Definition at line 80 of file pcb.c.

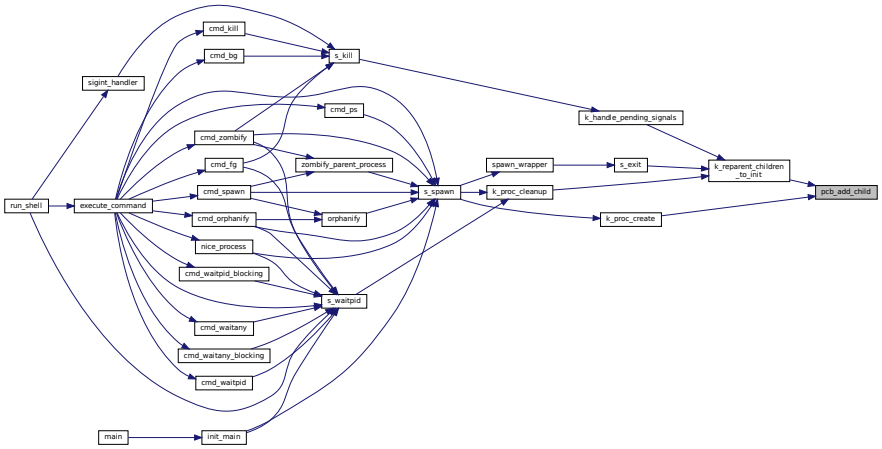
References pcb::children, MAX_CHILDREN, pcb::num_children, pcb::pid, and s_fprintf_stderr().

Referenced by k_proc_create(), and k_reparent_children_to_init().

Here is the call graph for this function:



Here is the caller graph for this function:



4.20.1.2 pcb_add_stopped_child() `void pcb_add_stopped_child (`

```
pcb_t * parent,
pid_t child_pid )
```

Add a stopped child to the parent's stopped children list.

Add a child to the parent's stopped children list.

Parameters

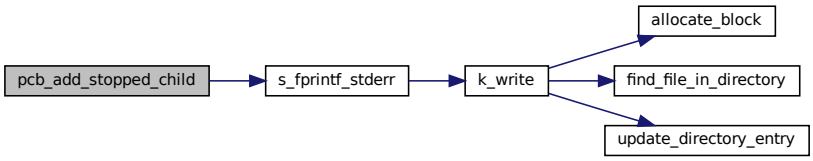
parent	Parent PCB
child_pid	Child process ID

Definition at line 200 of file pcb.c.

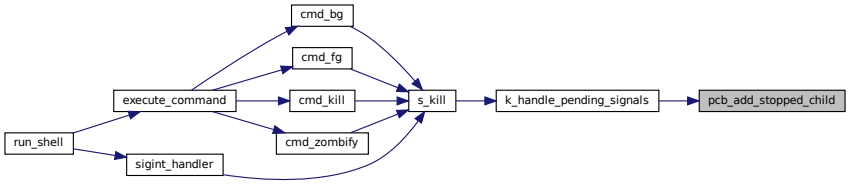
References MAX_CHILDREN, pcb::num_stopped, pcb::pid, s_fprintf_stderr(), and pcb::stopped_children.

Referenced by k_handle_pending_signals().

Here is the call graph for this function:



Here is the caller graph for this function:



4.20.1.3 pcb_add_zombie()

```
void pcb_add_zombie (
        pcb_t * parent,
        pid_t child_pid,
        int exit_status )
```

Add a zombie child to the parent's zombie queue.

Parameters

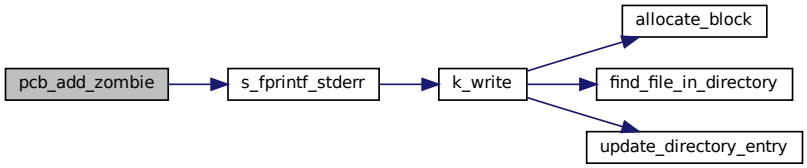
parent	Parent PCB
child_pid	Child process ID
exit_status	Exit status of the child

Definition at line 125 of file pcb.c.

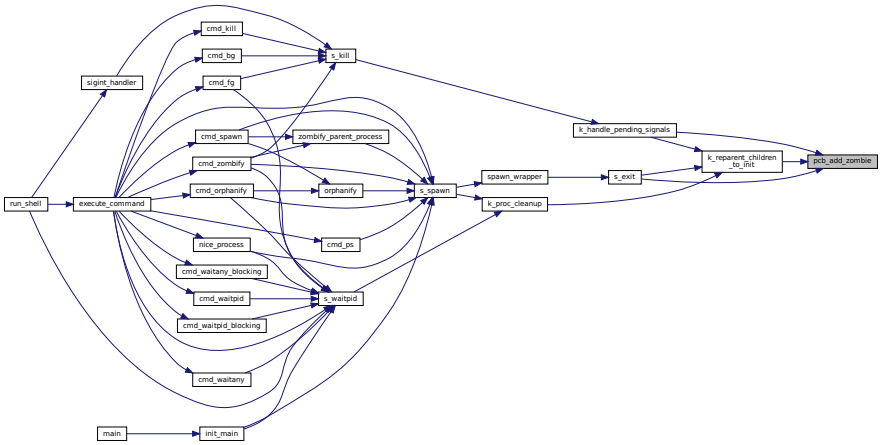
References pcb::exit_statuses, MAX_CHILDREN, pcb::num_zombies, pcb::pid, s_fprintf_stderr(), and pcb::zombie_↩ children.

Referenced by k_handle_pending_signals(), k_reparent_children_to_init(), and s_exit().

Here is the call graph for this function:



Here is the caller graph for this function:



4.20.1.4 pcb_create()

```
pcb_t* pcb_create (
        pid_t pid,
        pid_t ppid,
        const char * name,
        int priority )
```

Create a new PCB.

Create a new process control block.

Parameters

pid	Process ID
ppid	Parent process ID
name	Process name
priority	Priority level (0, 1, or 2)

Returns

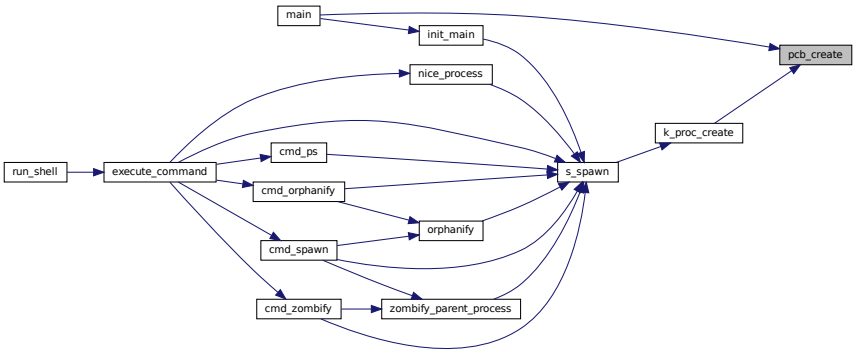
Pointer to newly created PCB, or NULL on failure

Definition at line 17 of file pcb.c.

References pcb::exit_status, pcb::fd_table, pcb::is_background, pcb::is_waiting, MAX_FDS, pcb::name, pcb::num↩ _children, pcb::num_stopped, pcb::num_zombies, pcb::pending_signal, pcb::pid, pcb::ppid, pcb::priority, RUNNING, pcb::sleep_ticks_remaining, pcb::state, and pcb::waiting_for_pid.

Referenced by k_proc_create(), and main().

Here is the caller graph for this function:



4.20.1.5 pcb_destroy() `void pcb_destroy (`

`pcb_t * pcb )`

Destroy a PCB and free its resources.

Destroy a process control block and free its memory.

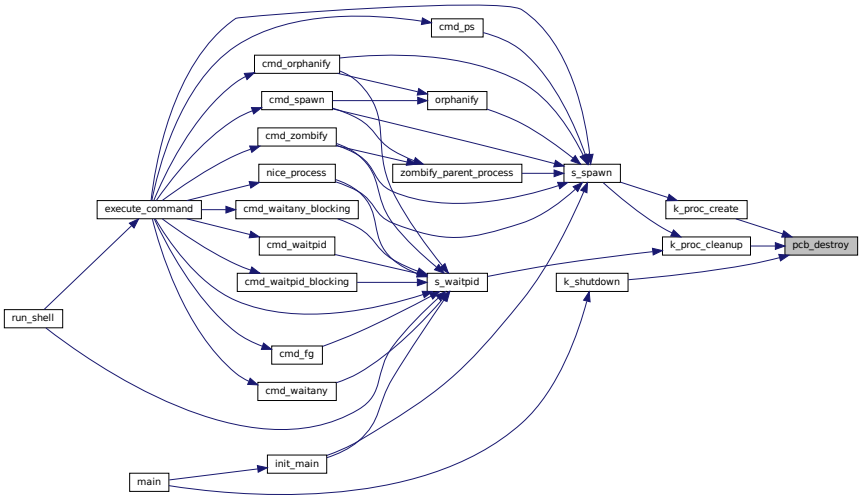
Parameters

pcb	Pointer to PCB to destroy

Definition at line 68 of file pcb.c.

Referenced by k_proc_cleanup(), k_proc_create(), and k_shutdown().

Here is the caller graph for this function:



4.20.1.6 pcb_get_zombie() `pid_t pcb_get_zombie (`

```
pcb_t * parent,
pid_t pid,
int * exit_status )
```

Get and remove a zombie child from the parent's zombie queue.

Parameters

parent	Parent PCB
pid	Specific child PID to get, or -1 for any zombie child
exit_status	Pointer to store the exit status

Returns

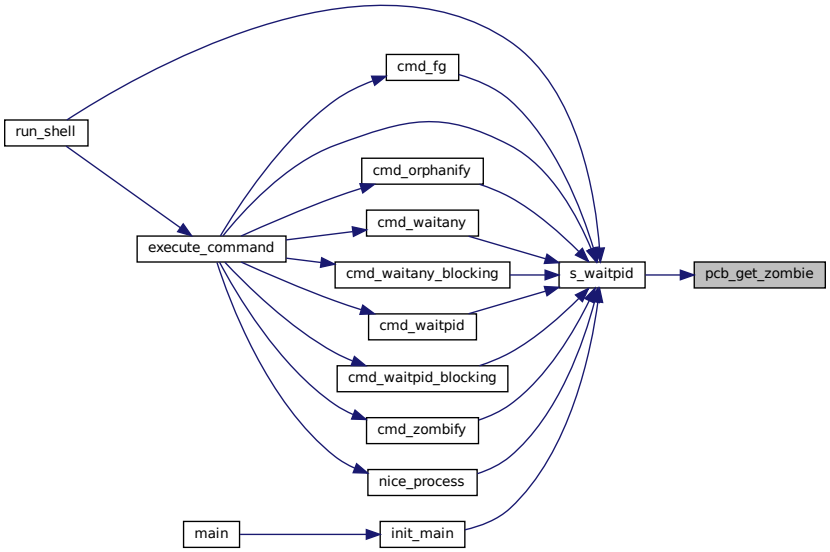
PID of the zombie child, or -1 if none found

Definition at line 149 of file pcb.c.

References pcb::exit_statuses, pcb::num_zombies, and pcb::zombie_children.

Referenced by s_waitpid().

Here is the caller graph for this function:



4.20.1.7 pcb_remove_child()

```
void pcb_remove_child (
        pcb_t * parent,
        pid_t child_pid )
```

Remove a child PID from the parent's children list.

Remove a child from the parent's children list.

Parameters

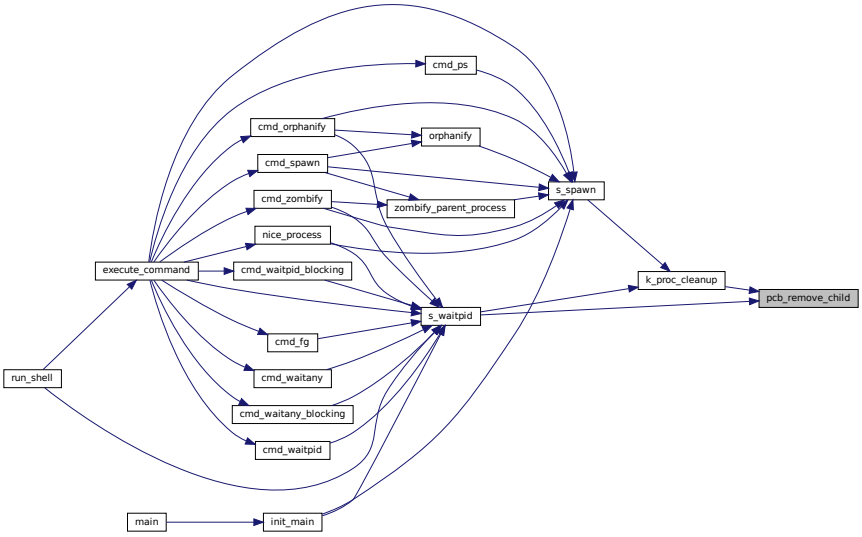
parent	Parent PCB
child_pid	Child process ID to remove

Definition at line 101 of file pcb.c.

References pcb::children, and pcb::num_children.

Referenced by k_proc_cleanup(), and s_waitpid().

Here is the caller graph for this function:



4.20.1.8 pcb_remove_stopped_child() `void pcb_remove_stopped_child (`

```
pcb_t * parent,
pid_t child_pid )
```

Remove a stopped child from the parent's stopped children list.

Remove a child from the parent's stopped children list.

Parameters

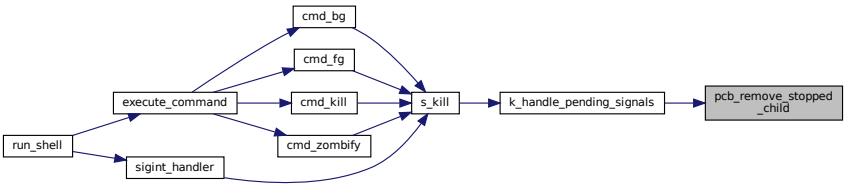
parent	Parent PCB
child_pid	Child process ID to remove

Definition at line 228 of file pcb.c.

References pcb::num_stopped, and pcb::stopped_children.

Referenced by k_handle_pending_signals().

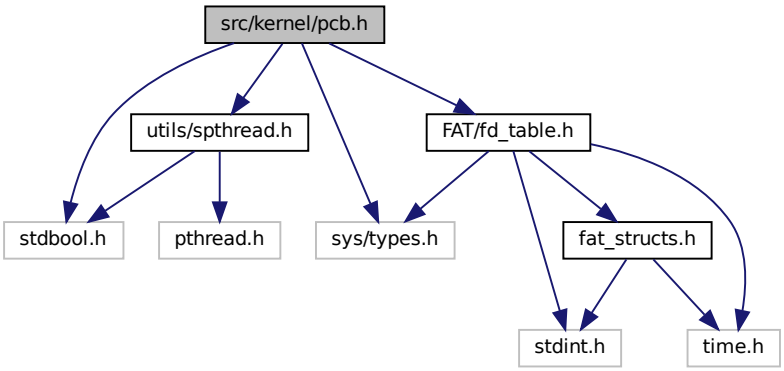
Here is the caller graph for this function:



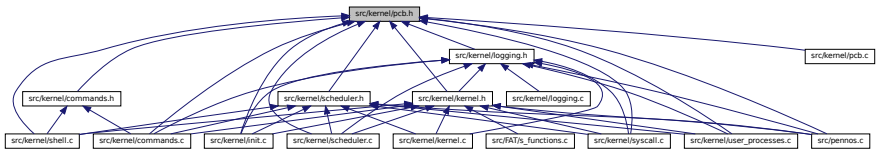
4.21 src/kernel/pcb.h File Reference

```
#include <stdbool.h>
#include <sys/types.h>
#include "FAT/fd_table.h"
#include "utils/spthread.h"
```

Include dependency graph for pcb.h:



This graph shows which files directly or indirectly include this file:



Data Structures

- struct pcb

  Process Control Block (PCB)

Macros

- #define P_SIGSTOP 0

  Signal definitions for PennOS.
- #define P_SIGCONT 1
- #define P_SIGTERM 2
- #define P_WIFEXITED(status) ((status & 0x7F) == 0)

  Status macros for s_waitpid - check how a process terminated.
- #define P_WIFSTOPPED(status) ((status & 0xFF) == 0x7F)
- #define P_WIFSIGNALED(status) (((status & 0x7F) + 1) >> 1 > 0)
- #define P_EXITSTATUS(status) ((status >> 8) & 0xFF)

  Extract exit code from status returned by s_waitpid.
- #define MAX_CHILDREN 128

  Maximum number of children a process can have.

Typedefs

- typedef struct pcb pcb_t

  Process Control Block (PCB)

Enumerations

- enum process_state_t { RUNNING , BLOCKED , STOPPED , ZOMBIE }

  Process states in PennOS.

Functions

- pcb_t * pcb_create (pid_t pid, pid_t ppid, const char *name, int priority)

  Create a new process control block.
- void pcb_destroy (pcb_t *pcb)

  Destroy a process control block and free its memory.
- void pcb_add_child (pcb_t *parent, pid_t child_pid)

  Add a child to the parent's children list.
- void pcb_remove_child (pcb_t *parent, pid_t child_pid)

  Remove a child from the parent's children list.
- void pcb_add_zombie (pcb_t *parent, pid_t child_pid, int exit_status)

  Add a zombie child to the parent's zombie queue.
- pid_t pcb_get_zombie (pcb_t *parent, pid_t pid, int *exit_status)

  Get and remove a zombie child from the parent's zombie queue.
- void pcb_add_stopped_child (pcb_t *parent, pid_t child_pid)

  Add a child to the parent's stopped children list.
- void pcb_remove_stopped_child (pcb_t *parent, pid_t child_pid)

  Remove a child from the parent's stopped children list.

4.21.1 Macro Definition Documentation

4.21.1.1 MAX_CHILDREN `#define MAX_CHILDREN 128`

Maximum number of children a process can have.

Definition at line 41 of file pcb.h.

4.21.1.2 P_EXITSTATUS `#define P_EXITSTATUS( status ) ((status >> 8) & 0xFF)`

Extract exit code from status returned by s_waitpid.

Definition at line 36 of file pcb.h.

4.21.1.3 P_SIGCONT `#define P_SIGCONT 1`

Continue a stopped process

Definition at line 23 of file pcb.h.

4.21.1.4 P_SIGSTOP `#define P_SIGSTOP 0`

Signal definitions for PennOS.

Stop (pause) a process

Definition at line 22 of file pcb.h.

4.21.1.5 P_SIGTERM `#define P_SIGTERM 2`

Terminate a process

Definition at line 24 of file pcb.h.

4.21.1.6 P_WIFEXITED `#define P_WIFEXITED(`

```
status ) ((status & 0x7F) == 0)
```

Status macros for s_waitpid - check how a process terminated.

True if process exited normally

Definition at line 29 of file pcb.h.

4.21.1.7 P_WIFSIGNALED `#define P_WIFSIGNALED(`

```
status ) (((status & 0x7F) + 1) >> 1 > 0)
```

True if process was terminated by signal

Definition at line 31 of file pcb.h.

4.21.1.8 P_WIFSTOPPED `#define P_WIFSTOPPED(`

```
status ) ((status & 0xFF) == 0x7F)
```

True if process was stopped

Definition at line 30 of file pcb.h.

4.21.2 Typedef Documentation

4.21.2.1 pcb_t `typedef struct pcb pcb_t`

Process Control Block (PCB)

Contains all state information for a single process in PennOS. Each process has a unique PID, parent PID, priority level, and tracks its children, zombies, and stopped children for proper process lifecycle management.

4.21.3 Enumeration Type Documentation

4.21.3.1 process_state_t `enum process_state_t`

Process states in PennOS.

- RUNNING: Process is active and schedulable
- BLOCKED: Process is waiting for I/O or child state change
- STOPPED: Process is paused by P_SIGSTOP signal
- ZOMBIE: Process has terminated but not yet been reaped by parent

Enumerator

RUNNING	
BLOCKED	
STOPPED	
ZOMBIE	

Definition at line 17 of file pcb.h.

4.21.4 Function Documentation

4.21.4.1 pcb_add_child()

```
void pcb_add_child (
        pcb_t * parent,
        pid_t child_pid )
```

Add a child to the parent's children list.

Adds the child PID to the parent's living children array.

Parameters

parent	Parent PCB
child_pid	PID of child to add

Add a child to the parent's children list.

Parameters

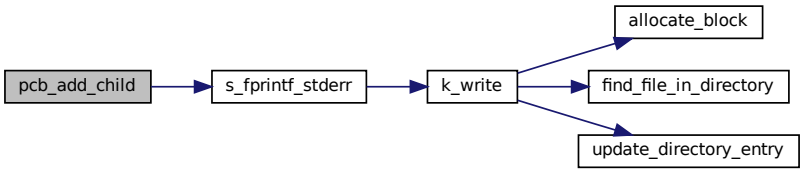
parent	Parent PCB
child_pid	Child process ID

Definition at line 80 of file pcb.c.

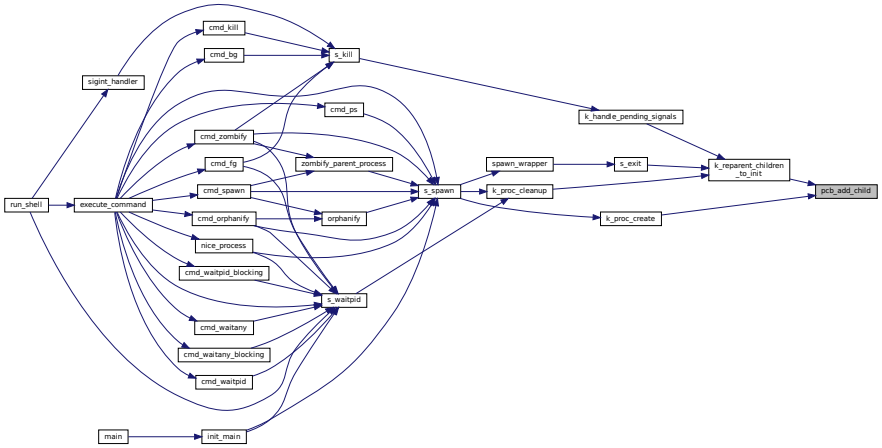
References pcb::children, MAX_CHILDREN, pcb::num_children, pcb::pid, and s_fprintf_stderr().

Referenced by k_proc_create(), and k_reparent_children_to_init().

Here is the call graph for this function:



Here is the caller graph for this function:



4.21.4.2 pcb_add_stopped_child()

```
void pcb_add_stopped_child (
        pcb_t * parent,
        pid_t child_pid )
```

Add a child to the parent's stopped children list.

Tracks that a child has been stopped (P_SIGSTOP)

Parameters

parent	Parent PCB
child_pid	PID of child that was stopped

Add a child to the parent's stopped children list.

Parameters

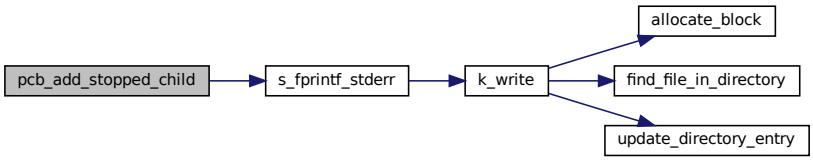
parent	Parent PCB
child_pid	Child process ID

Definition at line 200 of file pcb.c.

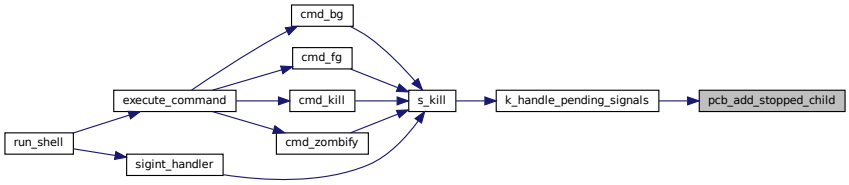
References MAX_CHILDREN, pcb::num_stopped, pcb::pid, s_fprintf_stderr(), and pcb::stopped_children.

Referenced by k_handle_pending_signals().

Here is the call graph for this function:



Here is the caller graph for this function:



4.21.4.3 pcb_add_zombie()

```
void pcb_add_zombie (
        pcb_t * parent,
        pid_t child_pid,
        int exit_status )
```

Add a zombie child to the parent's zombie queue.

Moves a child from the living children list to the zombie queue with its exit status. Used when a child process exits.

Parameters

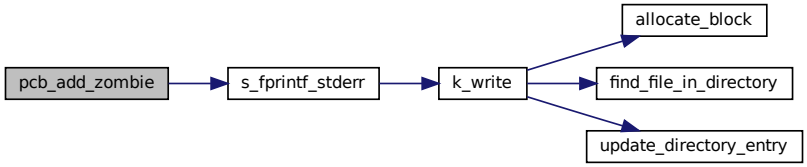
parent	Parent PCB
child_pid	PID of child that became a zombie
exit_status	Exit status of the zombie child
parent	Parent PCB
child_pid	Child process ID
exit_status	Exit status of the child

Definition at line 125 of file pcb.c.

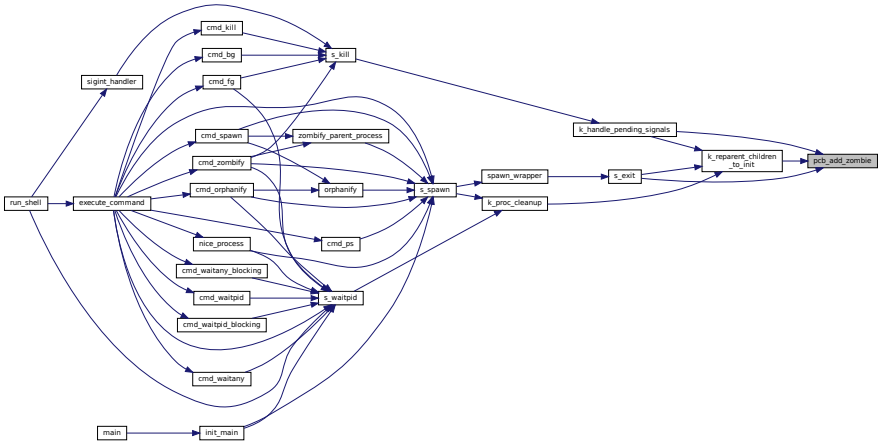
References pcb::exit_statuses, MAX_CHILDREN, pcb::num_zombies, pcb::pid, s_fprintf_stderr(), and pcb::zombie_↩ children.

Referenced by k_handle_pending_signals(), k_reparent_children_to_init(), and s_exit().

Here is the call graph for this function:



Here is the caller graph for this function:



4.21.4.4 pcb_create()

```
pcb_t* pcb_create (
        pid_t pid,
        pid_t ppid,
        const char * name,
        int priority )
```

Create a new process control block.

Allocates and initializes a new PCB with the specified parameters.

Parameters

pid	Process ID for the new process
ppid	Parent process ID
name	Process name
priority	Priority level (0, 1, or 2)

Returns

Pointer to newly allocated PCB, or NULL on failure

Create a new process control block.

Parameters

pid	Process ID
ppid	Parent process ID
name	Process name
priority	Priority level (0, 1, or 2)

Returns

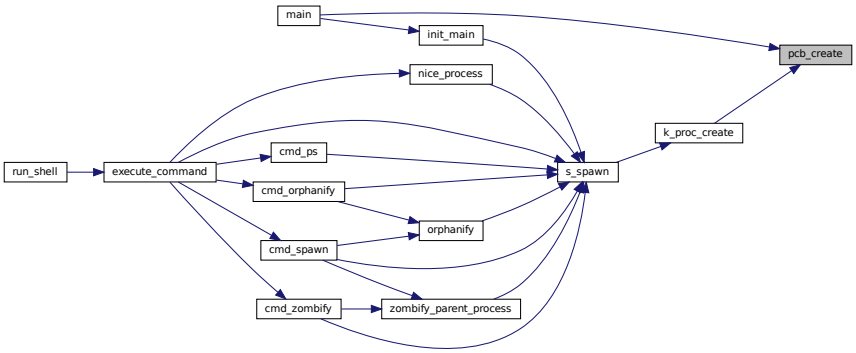
Pointer to newly created PCB, or NULL on failure

Definition at line 17 of file pcb.c.

References pcb::exit_status, pcb::fd_table, pcb::is_background, pcb::is_waiting, MAX_FDS, pcb::name, pcb::num↩_children, pcb::num_stopped, pcb::num_zombies, pcb::pending_signal, pcb::pid, pcb::ppid, pcb::priority, RUNNING, pcb::sleep_ticks_remaining, pcb::state, and pcb::waiting_for_pid.

Referenced by k_proc_create(), and main().

Here is the caller graph for this function:



4.21.4.5 pcb_destroy() `void pcb_destroy ( pcb_t * pcb )`

Destroy a process control block and free its memory.

Frees all resources associated with the PCB.

Parameters

pcb	PCB to destroy

Destroy a process control block and free its memory.

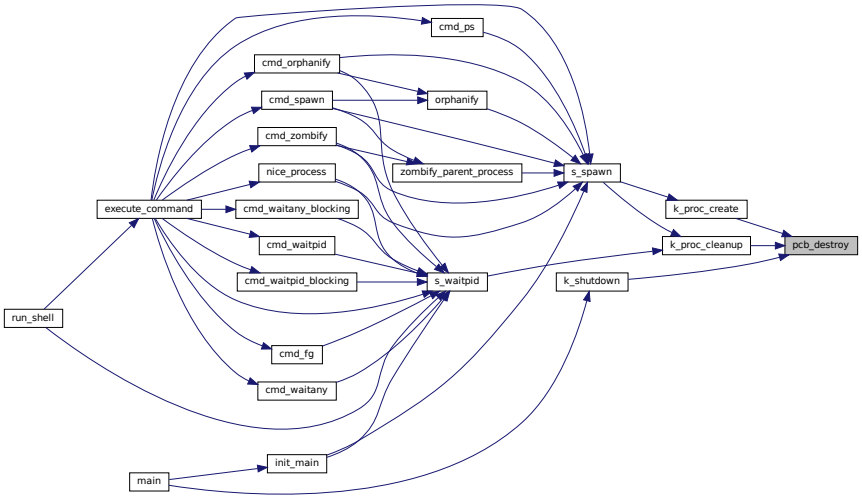
Parameters

pcb	Pointer to PCB to destroy

Definition at line 68 of file pcb.c.

Referenced by k_proc_cleanup(), k_proc_create(), and k_shutdown().

Here is the caller graph for this function:



4.21.4.6 pcb_get_zombie() pid_t pcb_get_zombie (

```
    pcb_t * parent,
    pid_t pid,
    int * exit_status )
```

Get and remove a zombie child from the parent's zombie queue.

Searches for a zombie child matching the given PID (or any zombie if pid == -1) and removes it from the queue. Returns the zombie's PID and exit status.

Parameters

parent	Parent PCB
pid	PID to search for, or -1 for any zombie
exit_status	Output parameter for zombie's exit status

Returns

PID of zombie found, or -1 if no matching zombie

Parameters

parent	Parent PCB
pid	Specific child PID to get, or -1 for any zombie child
exit_status	Pointer to store the exit status

Returns

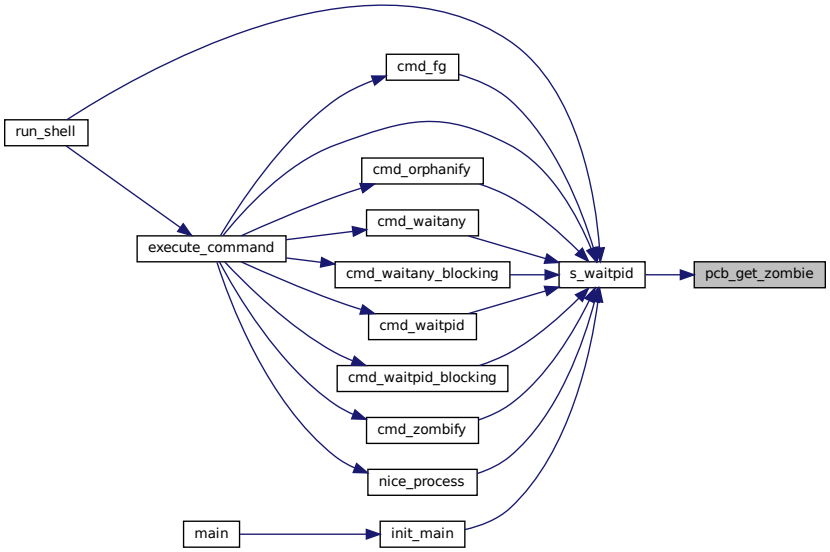
PID of the zombie child, or -1 if none found

Definition at line 149 of file pcb.c.

References pcb::exit_statuses, pcb::num_zombies, and pcb::zombie_children.

Referenced by s_waitpid().

Here is the caller graph for this function:



4.21.4.7 pcb_remove_child()

```
void pcb_remove_child (
        pcb_t * parent,
        pid_t child_pid )
```

Remove a child from the parent's children list.

Removes the child PID from the parent's living children array.

Parameters

parent	Parent PCB
child_pid	PID of child to remove

Remove a child from the parent's children list.

Parameters

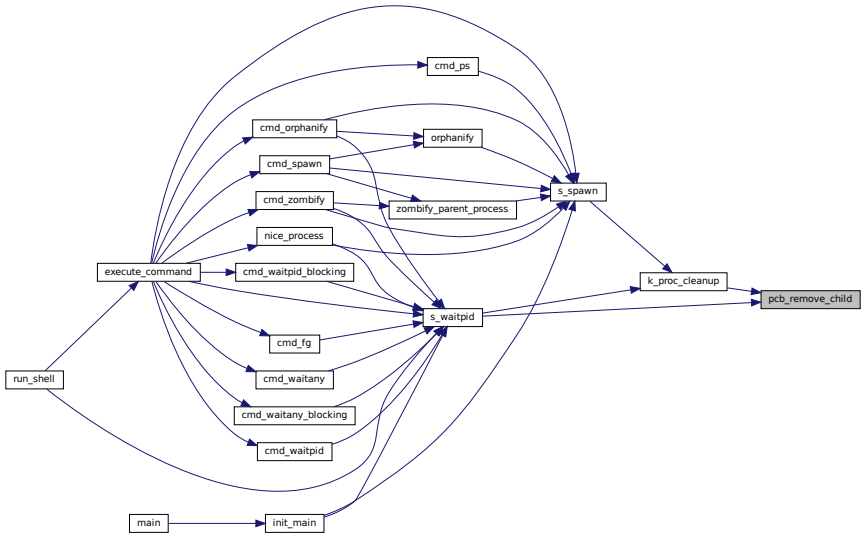
parent	Parent PCB
child_pid	Child process ID to remove

Definition at line 101 of file pcb.c.

References pcb::children, and pcb::num_children.

Referenced by k_proc_cleanup(), and s_waitpid().

Here is the caller graph for this function:



4.21.4.8 pcb_remove_stopped_child() `void pcb_remove_stopped_child (`

```
pcb_t * parent,
pid_t child_pid )
```

Remove a child from the parent's stopped children list.

Used when a stopped child is continued (P_SIGCONT) or terminated.

Parameters

parent	Parent PCB
child_pid	PID of child to remove from stopped list

Remove a child from the parent's stopped children list.

Parameters

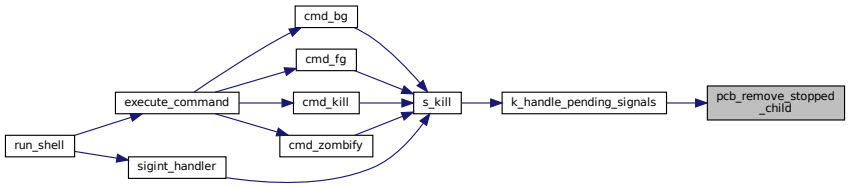
parent	Parent PCB
child_pid	Child process ID to remove

Definition at line 228 of file pcb.c.

References pcb::num_stopped, and pcb::stopped_children.

Referenced by k_handle_pending_signals().

Here is the caller graph for this function:

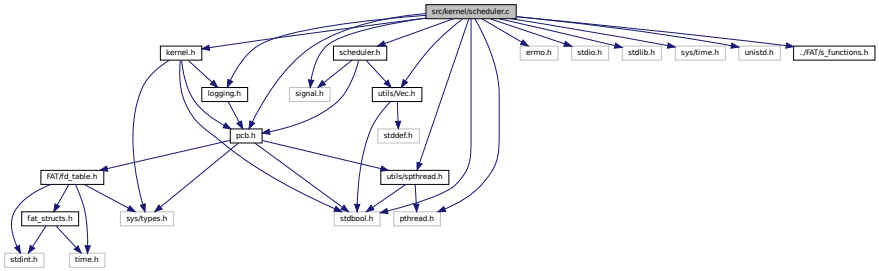


4.22 src/kernel/scheduler.c File Reference

```
#include "scheduler.h"
#include <errno.h>
#include <pthread.h>
#include <signal.h>
#include <stdbool.h>
#include <stdio.h>
#include <stdlib.h>
#include <sys/time.h>
#include <unistd.h>
#include "kernel.h"
#include "logging.h"
#include "pcb.h"
#include "utils/Vec.h"
#include "utils/spthread.h"
```

```
#include "../FAT/s_functions.h"
```

Include dependency graph for scheduler.c:



Macros

- #define _DEFAULT_SOURCE
- #define _GNU_SOURCE
- #define _POSIX_C_SOURCE 200809L

Functions

- static void alarm_handler (int signum)
- void scheduler_init ()

  Initialize the scheduler and its priority queues.
- static void cancel_and_join (spthread_t thread)
- void scheduler_cleanup ()

  Clean up scheduler resources and terminate all threads.
- bool scheduler_insert (pcb_t *pcb)

  Insert a process into the appropriate priority queue.
- bool scheduler_remove (pcb_t *pcb)

  Remove a process from its priority queue.
- pcb_t * scheduler_pop (int priority)

  Pop the first process from a specific priority queue.
- void scheduler_stop (void)

  Signal the scheduler to stop.
- pcb_t * get_next_proc ()

  Get the next process to schedule based on priority ratios.
- void scheduler (void)

  Main scheduler loop - implements preemptive multitasking.

Variables

- static Vec processes_queue [3]
- static const int quantum = 100000
- static const int priority_max_count [3] = {9, 6, 4}
- volatile sig_atomic_t priority_counter [3] = {0}
- volatile sig_atomic_t tick_count = 0
- static bool done = false

4.22.1 Macro Definition Documentation

4.22.1.1 _DEFAULT_SOURCE `#define _DEFAULT_SOURCE`

Definition at line 1 of file scheduler.c.

4.22.1.2 _GNU_SOURCE `#define _GNU_SOURCE`

Definition at line 2 of file scheduler.c.

4.22.1.3 _POSIX_C_SOURCE `#define _POSIX_C_SOURCE 200809L`

Definition at line 3 of file scheduler.c.

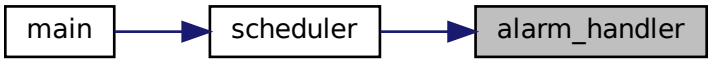
4.22.2 Function Documentation

4.22.2.1 alarm_handler() `static void alarm_handler ( int signum ) [static]`

Definition at line 29 of file scheduler.c.

Referenced by scheduler().

Here is the caller graph for this function:



4.22.2.2 cancel_and_join() `static void cancel_and_join (`

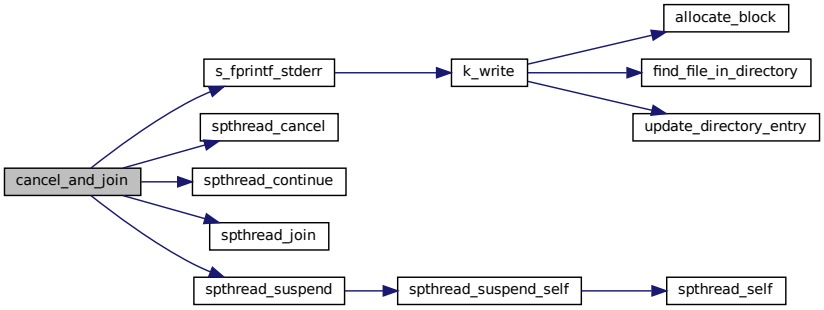
`spthread_t thread ) [static]`

Definition at line 38 of file scheduler.c.

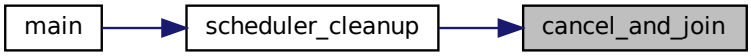
References s_fprintf_stderr(), spthread_cancel(), spthread_continue(), spthread_join(), and spthread_suspend().

Referenced by scheduler_cleanup().

Here is the call graph for this function:



Here is the caller graph for this function:



4.22.2.3 get_next_proc() `pcb_t* get_next_proc (`

`void )`

Get the next process to schedule based on priority ratios.

Implements weighted round-robin scheduling with ratios 9:6:4 for priority levels 0:1:2. This means:

- Priority 0 runs 1.5x more than priority 1
- Priority 1 runs 1.5x more than priority 2
- Priority 0 runs 2.25x more than priority 2

The algorithm uses counters to track how many times each priority has been scheduled, resetting when all reach their max counts.

Returns

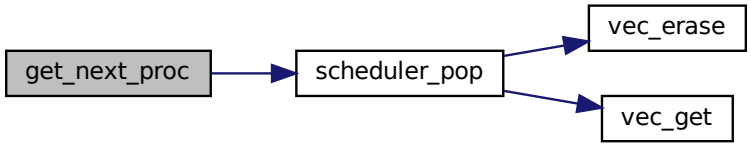
Pointer to next process to run, or NULL if all queues empty

Definition at line 142 of file scheduler.c.

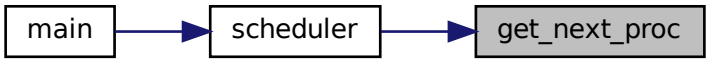
References NUM_PRIORITY, priority_counter, priority_max_count, and scheduler_pop().

Referenced by scheduler().

Here is the call graph for this function:



Here is the caller graph for this function:



4.22.2.4 scheduler() `void scheduler ( void )`

Main scheduler loop - implements preemptive multitasking.

1. Sets up signal handling for SIGALRM (quantum timer)
2. Configures the interval timer for 100ms quantums
3. Continuously schedules processes in round-robin with priority weighting
4. Handles context switching between processes

5. Manages sleeping processes (decrements sleep ticks each quantum)
6. Logs all scheduling events (CONTINUED, STOPPED, SCHEDULE, UNBLOCKED)

The scheduler runs until scheduler_stop() is called. Each iteration:

- Gets next process based on priority ratios (9:6:4)
- Sets process state to RUNNING
- Increments tick counter and sets logger tick
- Handles sleeping process wake-ups (sets them to RUNNING state)
- Continues the selected process thread
- Waits for SIGALRM (quantum expiration)
- Suspends the process thread
- Re-inserts running processes back into queue

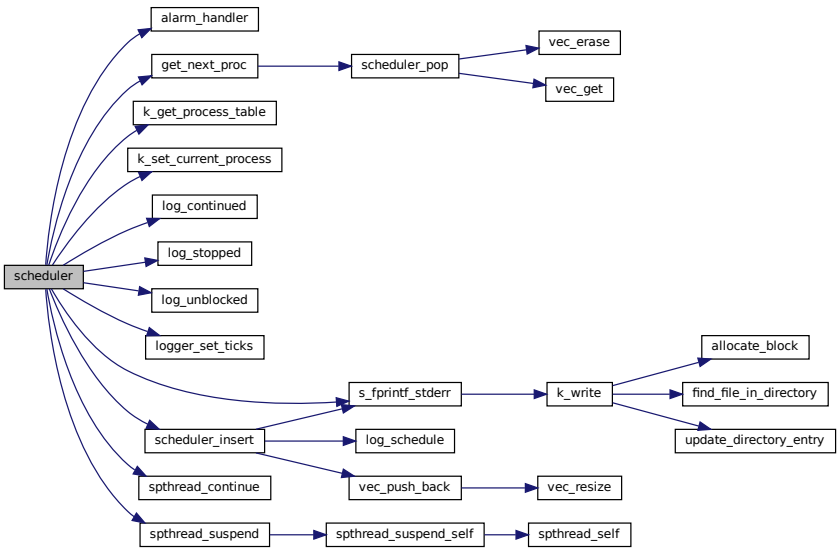
If all processes are blocked, the scheduler suspends itself until a signal arrives (e.g., from I/O completion or process wake-up).

Definition at line 177 of file scheduler.c.

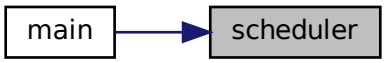
References alarm_handler(), BLOCKED, done, get_next_proc(), k_get_process_table(), k_set_current_process(), log_continued(), log_stopped(), log_unblocked(), logger_set_ticks(), MAX_PROCESSES, pcb::name, pcb::pid, pcb::priority, process_table, quantum, RUNNING, s_fprintf_stderr(), scheduler_insert(), pcb::sleep_ticks_remaining, spthread_continue(), spthread_suspend(), pcb::state, STOPPED, pcb::thread, tick_count, and ZOMBIE.

Referenced by main().

Here is the call graph for this function:



Here is the caller graph for this function:



4.22.2.5 scheduler_cleanup() `void scheduler_cleanup ( void )`

Clean up scheduler resources and terminate all threads.

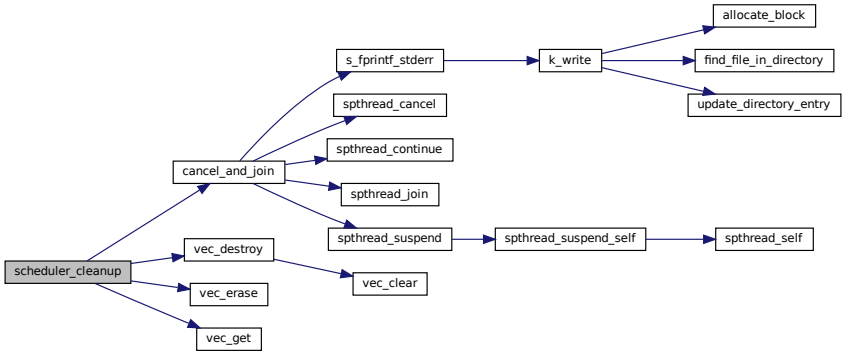
Iterates through all priority queues, cancels and joins all remaining threads, and destroys the queue data structures. Should be called during kernel shutdown.

Definition at line 63 of file scheduler.c.

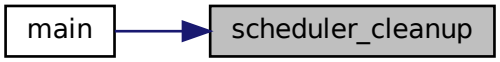
References cancel_and_join(), NUM_PRIORITY, processes_queue, pcb::thread, vec_destroy(), vec_erase(), vec_get(), and vec_is_empty.

Referenced by main().

Here is the call graph for this function:



Here is the caller graph for this function:



4.22.2.6 scheduler_init() `void scheduler_init ( void )`

Initialize the scheduler and its priority queues.

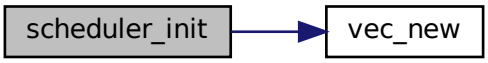
Creates three priority queues (one for each priority level 0, 1, 2). Must be called once during kernel initialization before any other scheduler functions are used.

Definition at line 32 of file scheduler.c.

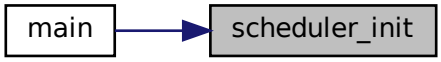
References NUM_PRIORITY, processes_queue, and vec_new().

Referenced by main().

Here is the call graph for this function:



Here is the caller graph for this function:



4.22.2.7 scheduler_insert() `bool scheduler_insert (pcb_t * pcb )`

Insert a process into the appropriate priority queue.

Adds the process to the scheduler queue based on its priority level. Only inserts if the process is in RUNNING state (not BLOCKED, STOPPED, or ZOMBIE). Validates priority is in range [0, 2]. Logs a SCHEDULE event upon successful insertion.

Parameters

pcb	Process control block to insert

Returns

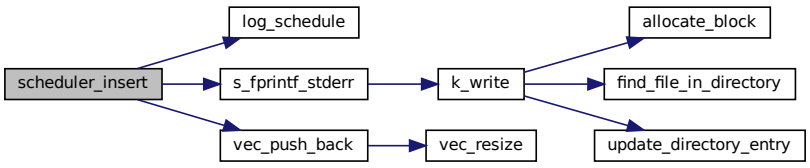
true if successfully inserted, false otherwise

Definition at line 76 of file scheduler.c.

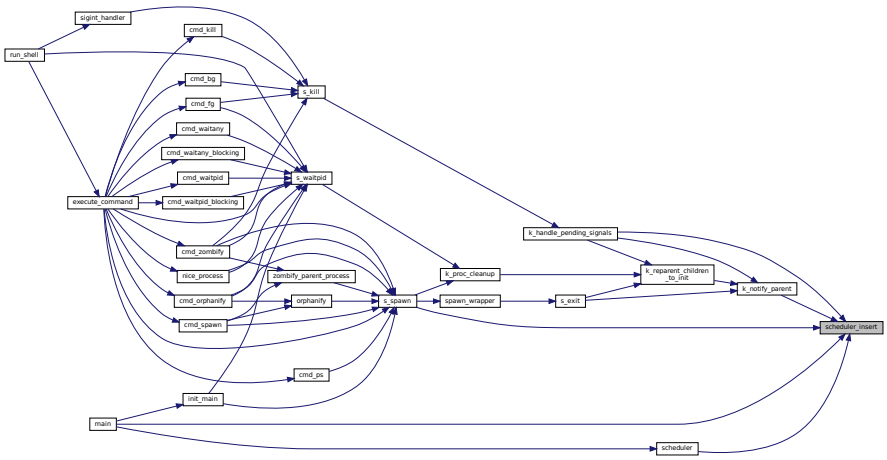
References BLOCKED, log_schedule(), pcb::name, NUM_PRIORITY, pcb::pid, pcb::priority, processes_queue, s_↩ fprintf_stderr(), pcb::state, STOPPED, vec_push_back(), and ZOMBIE.

Referenced by k_handle_pending_signals(), k_notify_parent(), main(), s_spawn(), and scheduler().

Here is the call graph for this function:



Here is the caller graph for this function:



4.22.2.8 scheduler_pop() `pcb_t* scheduler_pop ( int priority )`

Pop the first process from a specific priority queue.

Removes and returns the first process from the queue at the given priority level. Uses FIFO ordering within each priority level.

Parameters

priority	Priority level (0, 1, or 2)

Returns

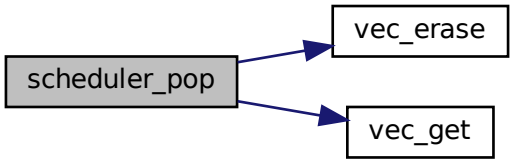
Pointer to the process, or NULL if queue is empty or invalid priority

Definition at line 123 of file scheduler.c.

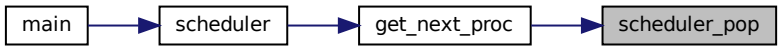
References NUM_PRIORITY, processes_queue, vec_erase(), vec_get(), and vec_is_empty.

Referenced by get_next_proc().

Here is the call graph for this function:



Here is the caller graph for this function:



4.22.2.9 scheduler_remove() `bool scheduler_remove (pcb_t * pcb )`

Remove a process from its priority queue.

Searches for the process in its priority queue by PID and removes it. Used when a process needs to be taken out of scheduling (e.g., when it becomes blocked, stopped, or terminated). Validates priority is in range [0, 2].

Parameters

pcb	Process control block to remove

Returns

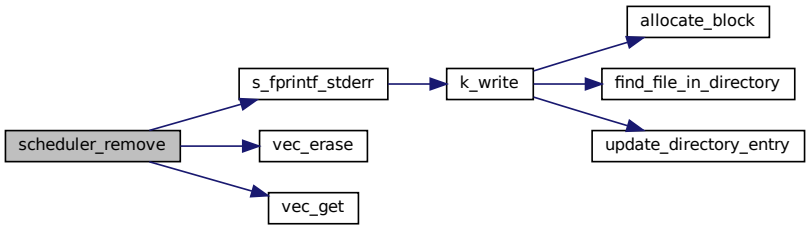
true if successfully removed, false if not found or invalid

Definition at line 98 of file scheduler.c.

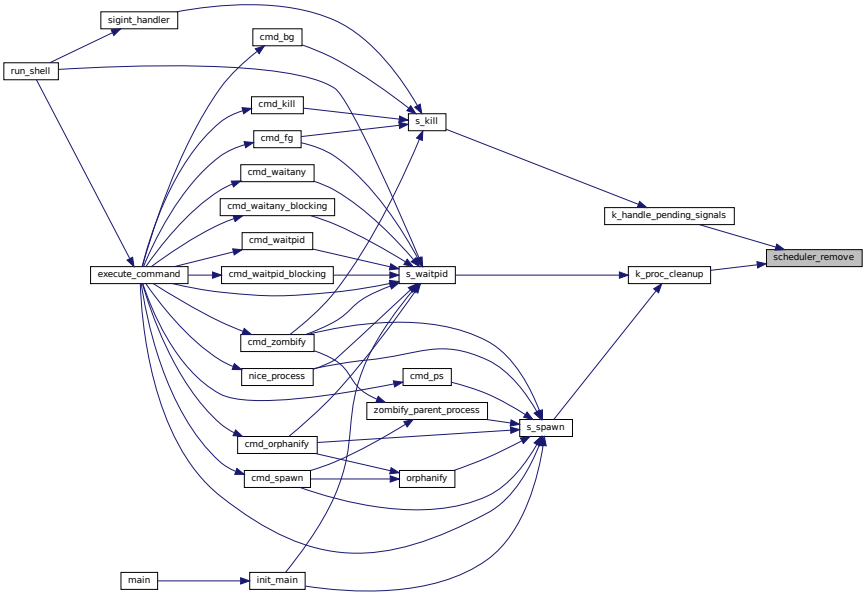
References NUM_PRIORITY, pcb::pid, pcb::priority, processes_queue, s_fprintf_stderr(), vec_erase(), vec_get(), and vec_len.

Referenced by k_handle_pending_signals(), and k_proc_cleanup().

Here is the call graph for this function:



Here is the caller graph for this function:



4.22.2.10 scheduler_stop() `void scheduler_stop ( void )`

Signal the scheduler to stop.

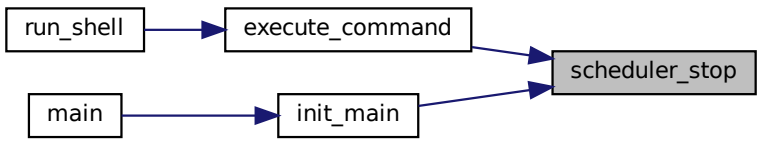
Sets the internal done flag to true, causing the scheduler main loop to exit after the current quantum completes. Does not immediately interrupt the scheduler.

Definition at line 137 of file scheduler.c.

References done.

Referenced by execute_command(), and init_main().

Here is the caller graph for this function:



4.22.3 Variable Documentation

4.22.3.1 done `bool done = false [static]`

Definition at line 26 of file scheduler.c.

Referenced by scheduler(), and scheduler_stop().

4.22.3.2 priority_counter `volatile sig_atomic_t priority_counter[3] = {0}`

Definition at line 23 of file scheduler.c.

Referenced by get_next_proc().

4.22.3.3 priority_max_count `const int priority_max_count[3] = {9, 6, 4} [static]`

Definition at line 22 of file scheduler.c.

Referenced by get_next_proc().

4.22.3.4 processes_queue `Vec processes_queue[3] [static]`

Definition at line 20 of file scheduler.c.

Referenced by scheduler_cleanup(), scheduler_init(), scheduler_insert(), scheduler_pop(), and scheduler_remove().

4.22.3.5 quantum `const int quantum = 100000 [static]`

Definition at line 21 of file scheduler.c.

Referenced by scheduler().

4.22.3.6 tick_count `volatile sig_atomic_t tick_count = 0`

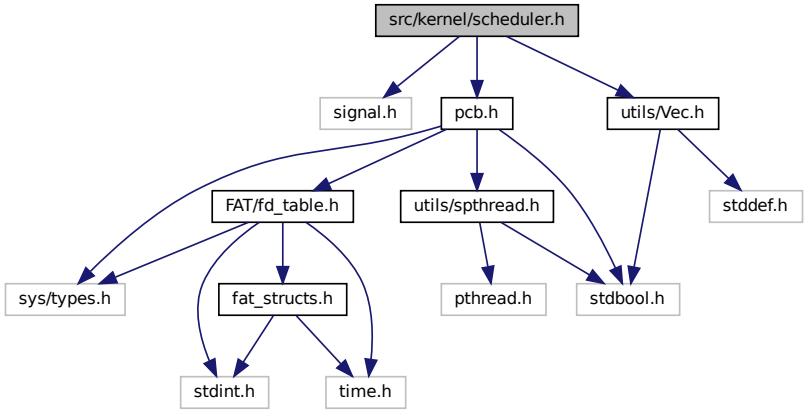
Definition at line 24 of file scheduler.c.

Referenced by scheduler().

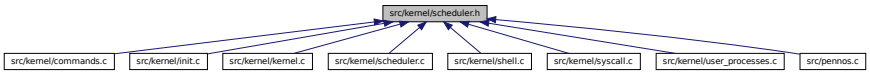
4.23 src/kernel/scheduler.h File Reference

```
#include <signal.h>
#include "pcb.h"
#include "utils/Vec.h"
```

Include dependency graph for scheduler.h:



This graph shows which files directly or indirectly include this file:



Macros

- #define NUM_PRIORITY 3

Functions

- void scheduler_init (void)

  Initialize the scheduler and its priority queues.
- void scheduler_cleanup (void)

  Clean up scheduler resources and terminate all threads.
- bool scheduler_insert (pcb_t *pcb)

  Insert a process into the appropriate priority queue.
- bool scheduler_remove (pcb_t *pcb)

  Remove a process from its priority queue.
- pcb_t * scheduler_pop (int priority)

  Pop the first process from a specific priority queue.
- pcb_t * get_next_proc (void)

  Get the next process to schedule based on priority ratios.
- void scheduler (void)

  Main scheduler loop - implements preemptive multitasking.
- void scheduler_stop (void)

  Signal the scheduler to stop.

4.23.1 Macro Definition Documentation

4.23.1.1 NUM_PRIORITY `#define NUM_PRIORITY 3`

Definition at line 8 of file scheduler.h.

4.23.2 Function Documentation

4.23.2.1 get_next_proc() `pcb_t* get_next_proc ( void )`

Get the next process to schedule based on priority ratios.

Implements weighted round-robin scheduling with ratios 9:6:4 for priority levels 0:1:2. This means:

- Priority 0 runs 1.5x more than priority 1
- Priority 1 runs 1.5x more than priority 2
- Priority 0 runs 2.25x more than priority 2

The algorithm uses counters to track how many times each priority has been scheduled, resetting when all reach their max counts.

Returns

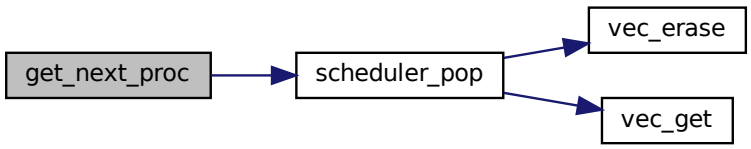
Pointer to next process to run, or NULL if all queues empty

Definition at line 142 of file scheduler.c.

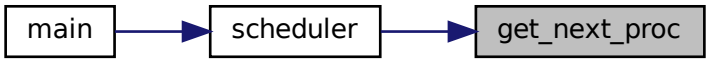
References NUM_PRIORITY, priority_counter, priority_max_count, and scheduler_pop().

Referenced by scheduler().

Here is the call graph for this function:



Here is the caller graph for this function:



4.23.2.2 scheduler() `void scheduler ( void )`

Main scheduler loop - implements preemptive multitasking.

1. Sets up signal handling for SIGALRM (quantum timer)
2. Configures the interval timer for 100ms quantums
3. Continuously schedules processes in round-robin with priority weighting
4. Handles context switching between processes

5. Manages sleeping processes (decrements sleep ticks each quantum)
6. Logs all scheduling events (CONTINUED, STOPPED, SCHEDULE, UNBLOCKED)

The scheduler runs until scheduler_stop() is called. Each iteration:

- Gets next process based on priority ratios (9:6:4)
- Sets process state to RUNNING
- Increments tick counter and sets logger tick
- Handles sleeping process wake-ups (sets them to RUNNING state)
- Continues the selected process thread
- Waits for SIGALRM (quantum expiration)
- Suspends the process thread
- Re-inserts running processes back into queue

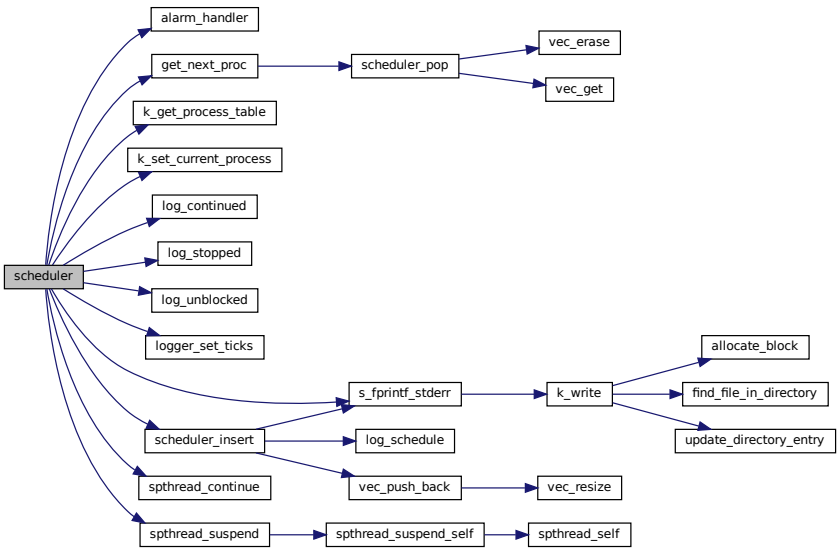
If all processes are blocked, the scheduler suspends itself until a signal arrives (e.g., from I/O completion or process wake-up).

Definition at line 177 of file scheduler.c.

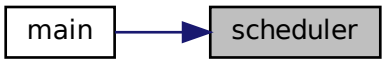
References alarm_handler(), BLOCKED, done, get_next_proc(), k_get_process_table(), k_set_current_process(), log_continued(), log_stopped(), log_unblocked(), logger_set_ticks(), MAX_PROCESSES, pcb::name, pcb::pid, pcb↩ ::priority, process_table, quantum, RUNNING, s_fprintf_stderr(), scheduler_insert(), pcb::sleep_ticks_remaining, spthread_continue(), spthread_suspend(), pcb::state, STOPPED, pcb::thread, tick_count, and ZOMBIE.

Referenced by main().

Here is the call graph for this function:



Here is the caller graph for this function:



4.23.2.3 scheduler_cleanup() `void scheduler_cleanup ( void )`

Clean up scheduler resources and terminate all threads.

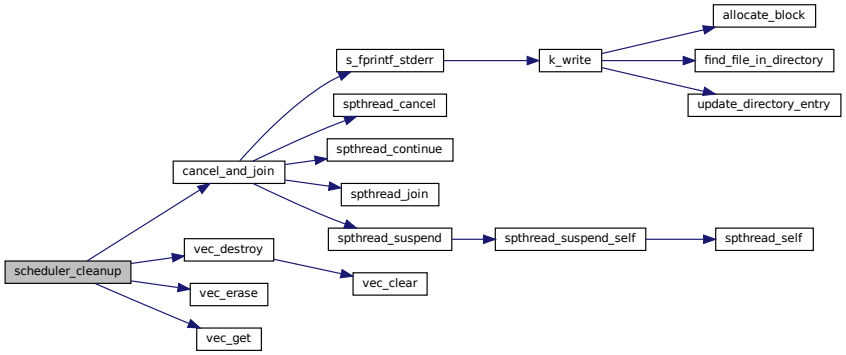
Iterates through all priority queues, cancels and joins all remaining threads, and destroys the queue data structures. Should be called during kernel shutdown.

Definition at line 63 of file scheduler.c.

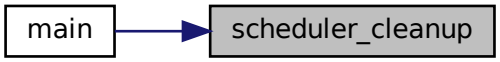
References cancel_and_join(), NUM_PRIORITY, processes_queue, pcb::thread, vec_destroy(), vec_erase(), vec_get(), and vec_is_empty.

Referenced by main().

Here is the call graph for this function:



Here is the caller graph for this function:



4.23.2.4 scheduler_init() `void scheduler_init ( void )`

Initialize the scheduler and its priority queues.

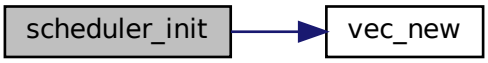
Creates three priority queues (one for each priority level 0, 1, 2). Must be called once during kernel initialization before any other scheduler functions are used.

Definition at line 32 of file scheduler.c.

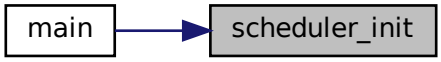
References NUM_PRIORITY, processes_queue, and vec_new().

Referenced by main().

Here is the call graph for this function:



Here is the caller graph for this function:



4.23.2.5 scheduler_insert() `bool scheduler_insert (pcb_t * pcb )`

Insert a process into the appropriate priority queue.

Adds the process to the scheduler queue based on its priority level. Only inserts if the process is in RUNNING state (not BLOCKED, STOPPED, or ZOMBIE). Validates priority is in range [0, 2]. Logs a SCHEDULE event upon successful insertion.

Parameters

pcb	Process control block to insert

Returns

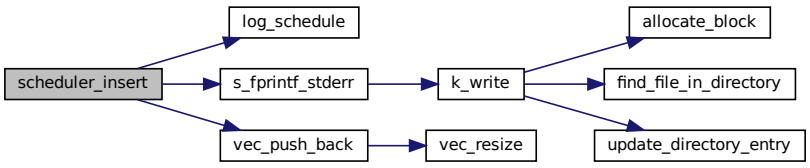
true if successfully inserted, false otherwise

Definition at line 76 of file scheduler.c.

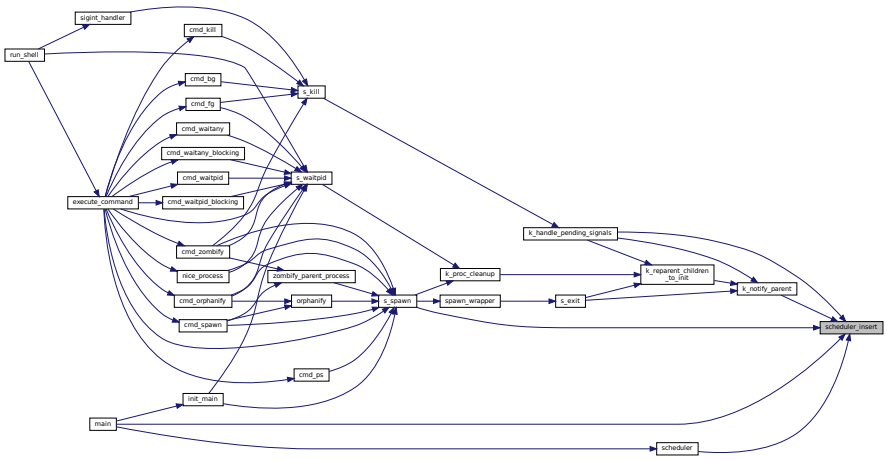
References BLOCKED, log_schedule(), pcb::name, NUM_PRIORITY, pcb::pid, pcb::priority, processes_queue, s_↩ fprintf_stderr(), pcb::state, STOPPED, vec_push_back(), and ZOMBIE.

Referenced by k_handle_pending_signals(), k_notify_parent(), main(), s_spawn(), and scheduler().

Here is the call graph for this function:



Here is the caller graph for this function:



4.23.2.6 scheduler_pop() `pcb_t* scheduler_pop ( int priority )`

Pop the first process from a specific priority queue.

Removes and returns the first process from the queue at the given priority level. Uses FIFO ordering within each priority level.

Parameters

priority	Priority level (0, 1, or 2)

Returns

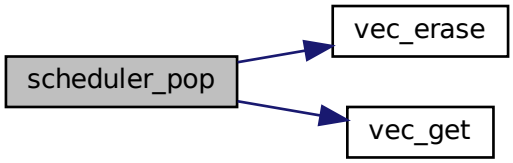
Pointer to the process, or NULL if queue is empty or invalid priority

Definition at line 123 of file scheduler.c.

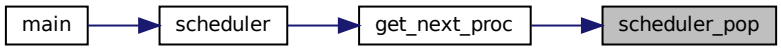
References NUM_PRIORITY, processes_queue, vec_erase(), vec_get(), and vec_is_empty.

Referenced by get_next_proc().

Here is the call graph for this function:



Here is the caller graph for this function:



4.23.2.7 scheduler_remove() `bool scheduler_remove (`

`pcb_t * pcb )`

Remove a process from its priority queue.

Searches for the process in its priority queue by PID and removes it. Used when a process needs to be taken out of scheduling (e.g., when it becomes blocked, stopped, or terminated). Validates priority is in range [0, 2].

Parameters

pcb	Process control block to remove

Returns

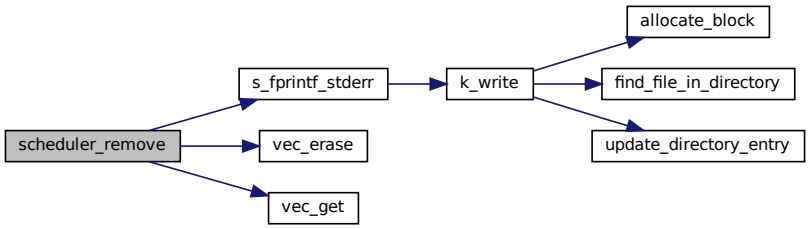
true if successfully removed, false if not found or invalid

Definition at line 98 of file scheduler.c.

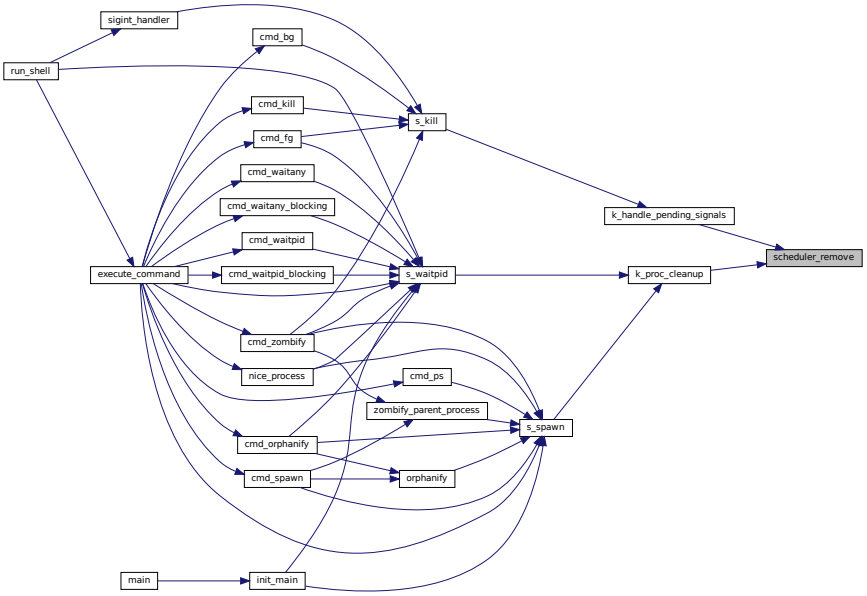
References NUM_PRIORITY, pcb::pid, pcb::priority, processes_queue, s_fprintf_stderr(), vec_erase(), vec_get(), and vec_len.

Referenced by k_handle_pending_signals(), and k_proc_cleanup().

Here is the call graph for this function:



Here is the caller graph for this function:



4.23.2.8 scheduler_stop() `void scheduler_stop ( void )`

Signal the scheduler to stop.

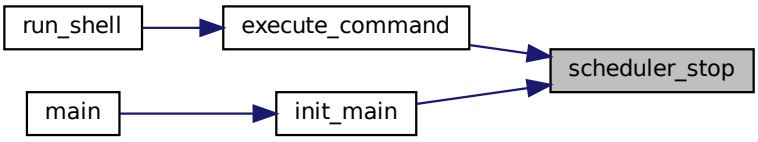
Sets the internal done flag to true, causing the scheduler main loop to exit after the current quantum completes. Does not immediately interrupt the scheduler.

Definition at line 137 of file scheduler.c.

References done.

Referenced by execute_command(), and init_main().

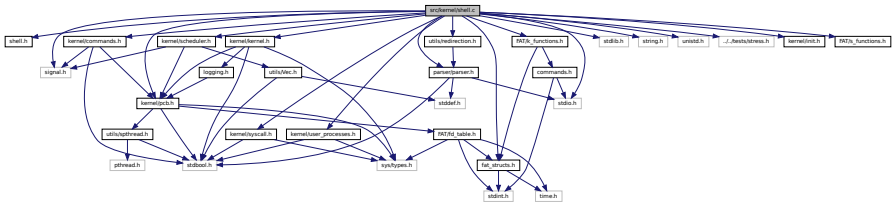
Here is the caller graph for this function:



4.24 src/kernel/shell.c File Reference

```
#include "shell.h"
#include <signal.h>
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include "../../tests/stress.h"
#include "kernel/commands.h"
#include "kernel/init.h"
#include "kernel/kernel.h"
#include "kernel/pcb.h"
#include "kernel/scheduler.h"
#include "kernel/syscall.h"
#include "kernel/user_processes.h"
#include "parser/parser.h"
#include "utils/redirection.h"
#include "FAT/fat_structs.h"
#include "FAT/k_functions.h"
#include "FAT/s_functions.h"
```

Include dependency graph for shell.c:



Functions

- static bool execute_command (struct parsed_command *pcmd)
- void run_shell (void)

  Run the interactive shell in the current process context.
- void * shell_main (void *arg)

  Thread wrapper function for running the shell.

4.24.1 Function Documentation

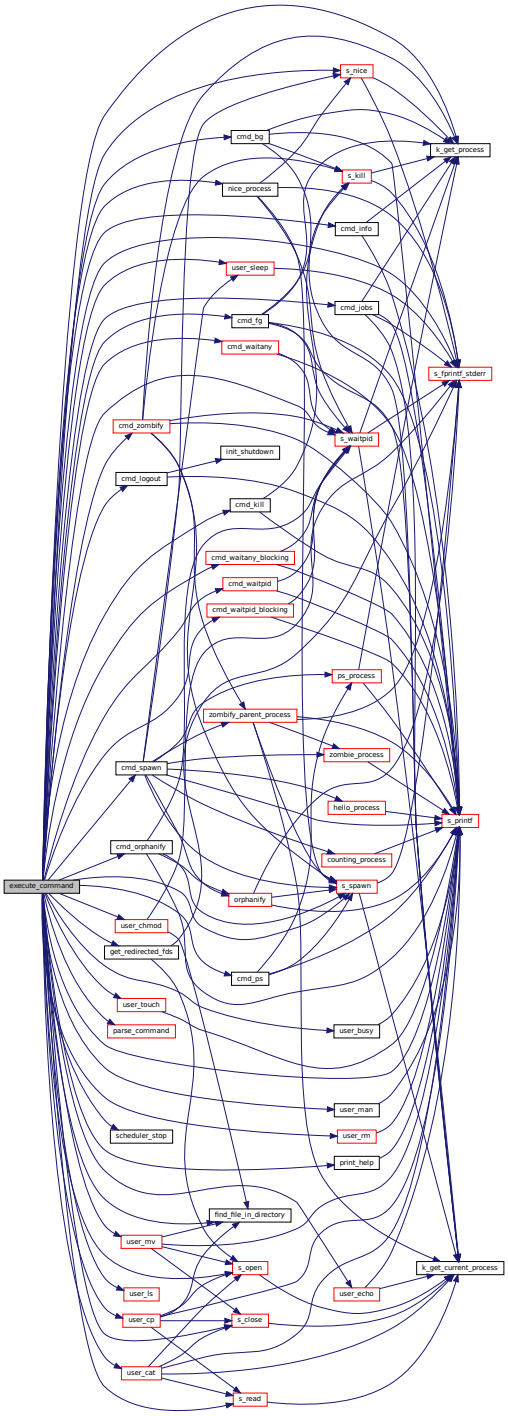
4.24.1.1 execute_command() static bool execute_command (struct parsed_command * pcmd) [static]

Definition at line 25 of file shell.c.

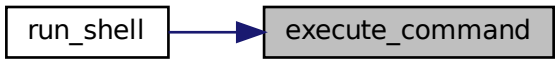
References cmd_bg(), cmd_fg(), cmd_info(), cmd_jobs(), cmd_kill(), cmd_logout(), cmd_orphanify(), cmd_ps(), cmd_spawn(), cmd_waitany(), cmd_waitany_blocking(), cmd_waitpid(), cmd_waitpid_blocking(), cmd_zombify(), parsed_command::commands, F_READ, find_file_in_directory(), foreground_pid, get_redirected_fds(), pcb::is_background, parsed_command::is_background, parsed_command::is_file_append, k_get_process(), nice_process(), P_SIGCONT, P_SIGSTOP, P_SIGTERM, parse_command(), directory_entry_t::perm, PERM_ALL, PERM_READ_EXEC, print_help(), s_close(), s_fprintf_stderr(), s_nice(), s_open(), s_printf(), s_read(), s_spawn(), s_waitpid(), scheduler_stop(), directory_entry_t::size, parsed_command::stdout_file, user_busy(), user_cat(), user_chmod(), user_cp(), user_echo(), user_ls(), user_man(), user_mv(), user_rm(), user_sleep(), and user_touch().

Referenced by run_shell().

Here is the call graph for this function:



Here is the caller graph for this function:



4.24.1.2 run_shell() `void run_shell ( void )`

Run the interactive shell in the current process context.

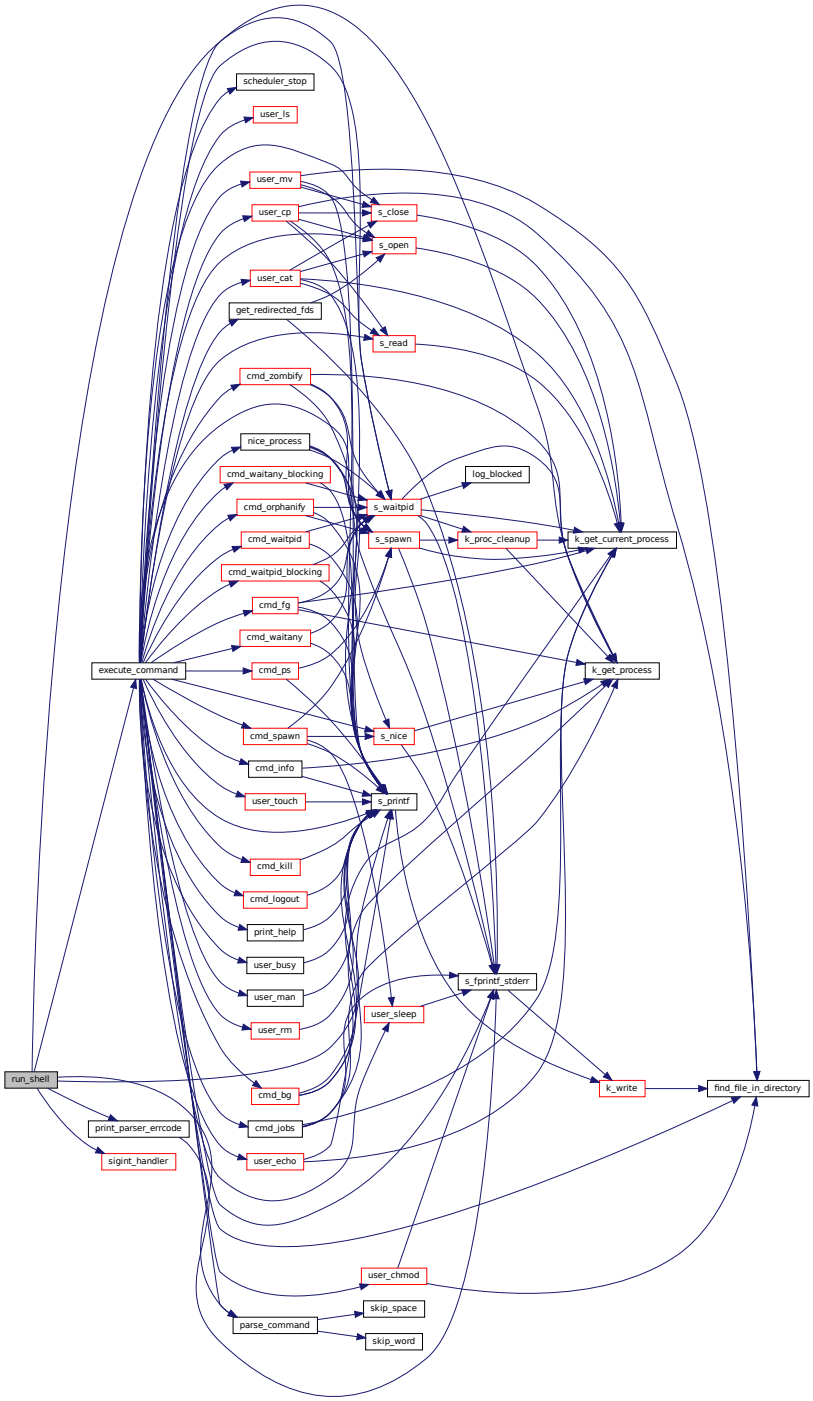
Main shell loop - reads commands, parses them, and executes them.

Definition at line 751 of file shell.c.

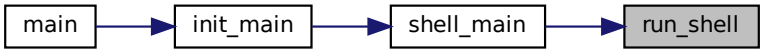
References execute_command(), P_WIFSTOPPED, parse_command(), print_parser_errcode(), s_printf(), s_waitpid(), and sigint_handler().

Referenced by shell_main().

Here is the call graph for this function:



Here is the caller graph for this function:



4.24.1.3 shell_main() `void* shell_main ( void * arg )`

Thread wrapper function for running the shell.

Parameters

arg	Unused argument (required for thread function signature)

Returns

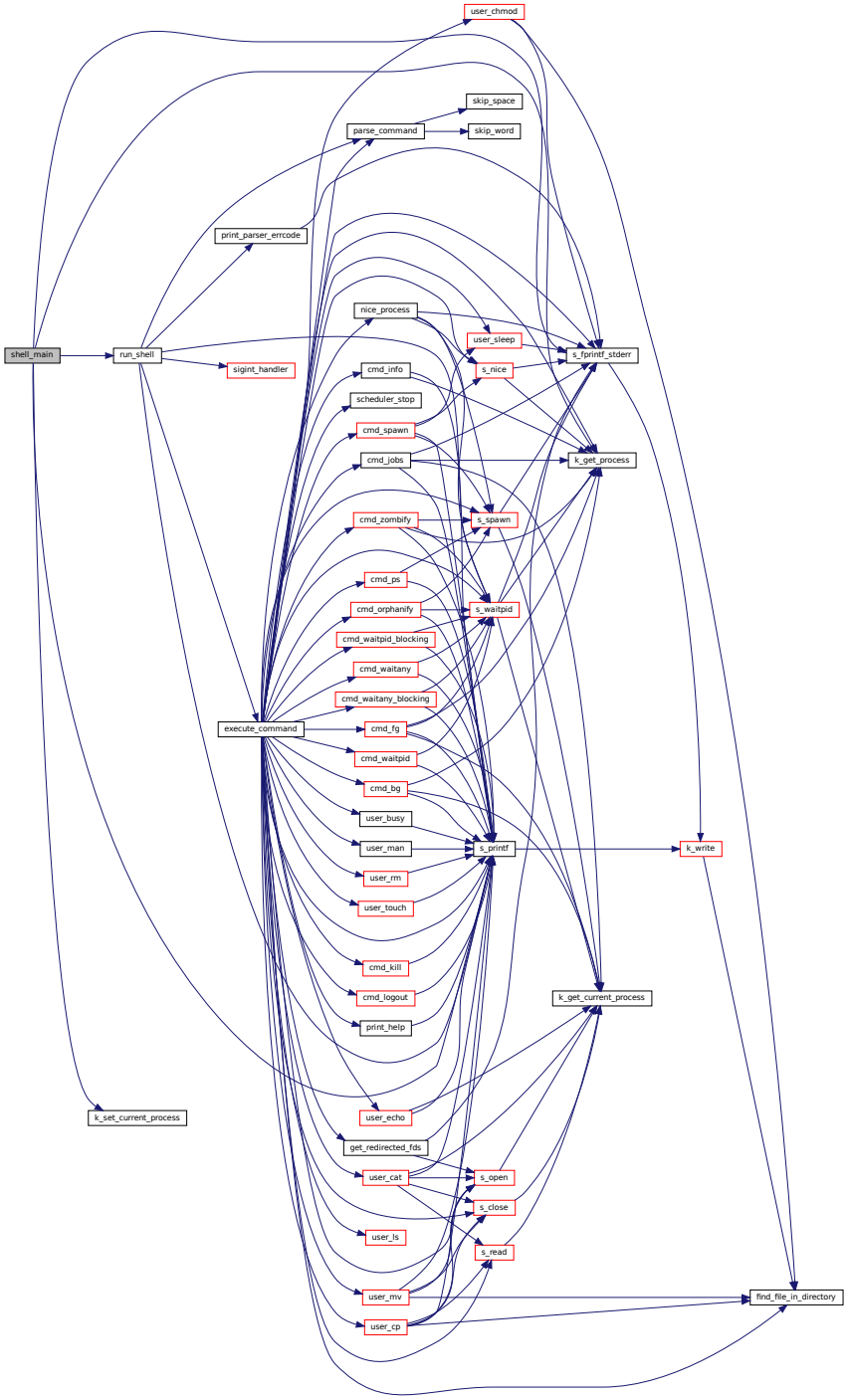
NULL when shell exits

Definition at line 840 of file shell.c.

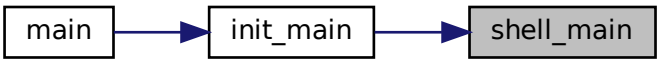
References pcb::children, k_get_process(), k_set_current_process(), pcb::num_children, run_shell(), s_fprintf_stderr(), and s_printf().

Referenced by init_main().

Here is the call graph for this function:

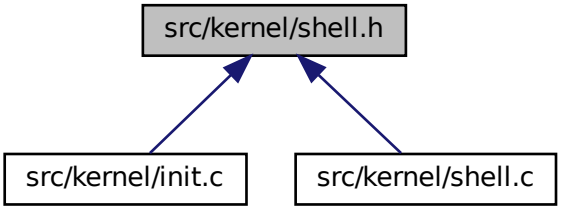


Here is the caller graph for this function:



4.25 src/kernel/shell.h File Reference

This graph shows which files directly or indirectly include this file:



Functions

- void run_shell (void)

  Run the interactive shell in the current process context.
- void ∗ shell_main (void ∗arg)

  Thread wrapper function for running the shell.

4.25.1 Function Documentation

4.25.1.1 run_shell() `void run_shell ( void )`

Run the interactive shell in the current process context.

Executes the main shell loop, reading commands from stdin and executing them. This function handles:

- Reading user input
- Parsing commands
- Executing built-in commands (cat, sleep, busy, ls, touch, etc.)
- Spawning processes for user programs
- Managing job control (foreground/background processes)

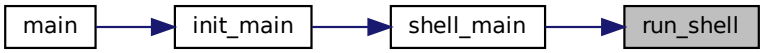
This function runs until the user exits (Ctrl+D or exit command).

Main shell loop - reads commands, parses them, and executes them.

Definition at line 751 of file shell.c.

Referenced by shell_main().

Here is the caller graph for this function:



4.25.1.2 shell_main() `void* shell_main ( void * arg )`

Thread wrapper function for running the shell.

Thin wrapper around run_shell() that provides the proper signature for use as a thread function. Used when spawning the shell as a separate thread in the INIT process.

Parameters

arg	Unused argument (required for thread function signature)

Returns

NULL when the shell exits

Parameters

arg	Unused argument (required for thread function signature)

Returns

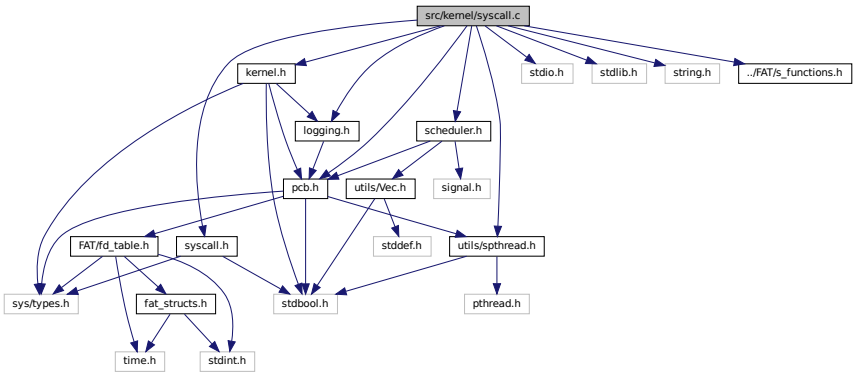
NULL when shell exits

Definition at line 840 of file shell.c.

4.26 src/kernel/syscall.c File Reference

```
#include "syscall.h"
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include "kernel.h"
#include "logging.h"
#include "pcb.h"
#include "scheduler.h"
#include "utils/spthread.h"
#include "../FAT/s_functions.h"
```

Include dependency graph for syscall.c:



Data Structures

- struct spawn_args_t

  Wrapper structure for passing arguments to spawned thread.

Functions

- static void * spawn_wrapper (void *arg)

  Wrapper function for spawned threads Sets the current process and executes the user function.
- pid_t s_spawn (void *(*func)(void *), char *argv[], int fd0, int fd1)

  Create a child process that executes the function `func`. The child will retain some attributes of the parent.
- pid_t s_waitpid (pid_t pid, int *wstatus, bool nohang)

  Wait on a child of the calling process, until it changes state. If `nohang` is true, this will not block the calling process and return immediately.
- int s_kill (pid_t pid, int signal)

  Send a signal to a particular process.
- void s_exit (void)

  Unconditionally exit the calling process.
- int s_nice (pid_t pid, int priority)

  Set the priority of the specified thread.
- void s_sleep (unsigned int ticks)

  Suspends execution of the calling process for a specified number of clock ticks.

4.26.1 Function Documentation

4.26.1.1 s_exit() `void s_exit ( void )`

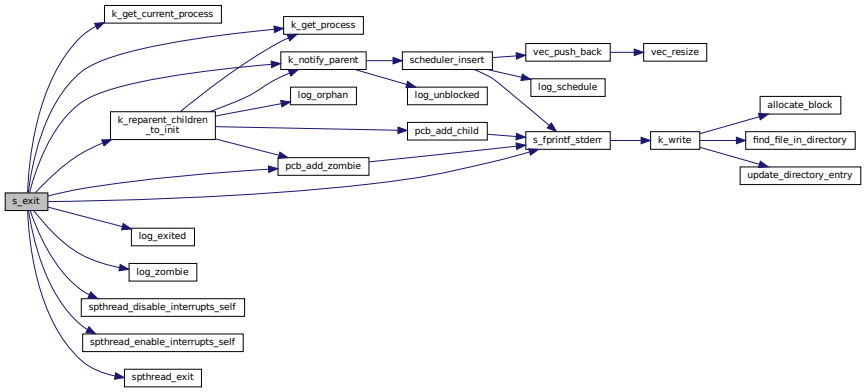
Unconditionally exit the calling process.

Definition at line 320 of file syscall.c.

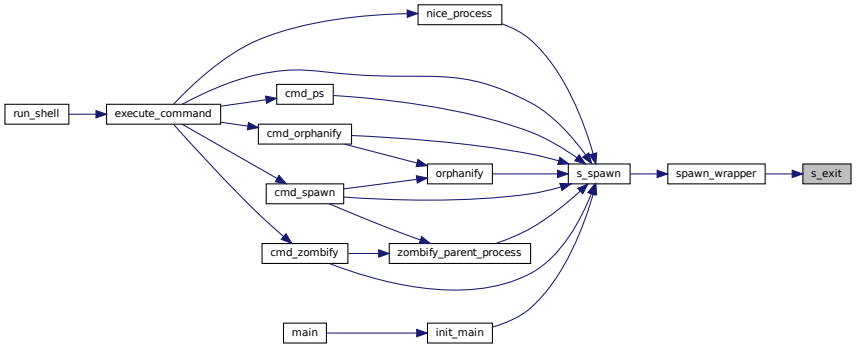
References pcb::exit_status, INIT_PID, k_get_current_process(), k_get_process(), k_notify_parent(), k_reparent_children_to_init(), log_exited(), log_zombie(), pcb::name, pcb::num_children, pcb::num_zombies, pcb_add_zombie(), pcb::pid, pcb::ppid, pcb::priority, s_fprintf_stderr(), spthread_disable_interrupts_self(), spthread_enable_interrupts_self(), spthread_exit(), pcb::state, and ZOMBIE.

Referenced by spawn_wrapper().

Here is the call graph for this function:



Here is the caller graph for this function:



4.26.1.2 s_kill() `int s_kill (`

```
    pid_t pid,
    int signal )
```

Send a signal to a particular process.

Parameters

pid	Process ID of the target process.
signal	Signal number to be sent (P_SIGSTOP, P_SIGCONT, P_SIGTERM).

Returns

0 on success, -1 on error.

< Stop (pause) a process

< Continue a stopped process

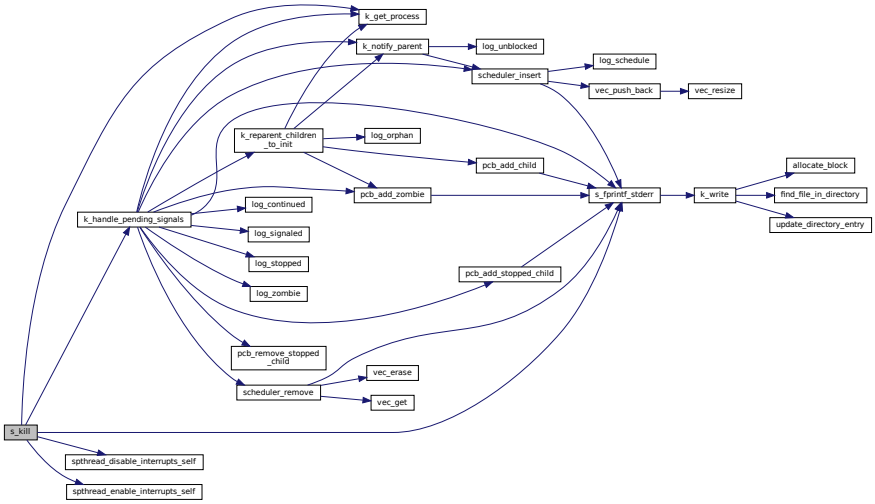
< Terminate a process

Definition at line 288 of file syscall.c.

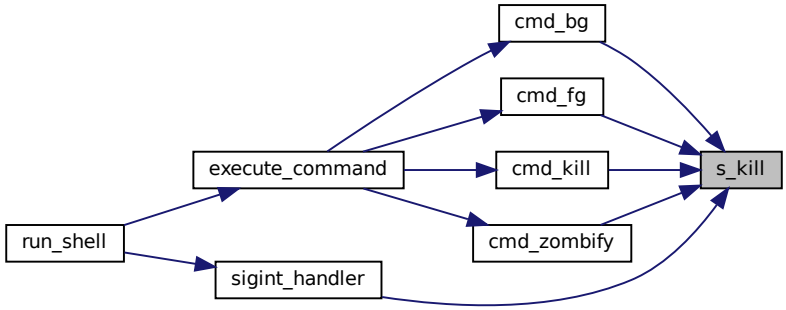
References k_get_process(), k_handle_pending_signals(), P_SIGCONT, P_SIGSTOP, P_SIGTERM, pcb::pending_↩ signal, s_fprintf_stderr(), spthread_disable_interrupts_self(), and spthread_enable_interrupts_self().

Referenced by cmd_bg(), cmd_fg(), cmd_kill(), cmd_zombify(), and sigint_handler().

Here is the call graph for this function:



Here is the caller graph for this function:



4.26.1.3 s_nice() `int s_nice (`

```
    pid_t pid,
    int priority )
```

Set the priority of the specified thread.

Parameters

pid	Process ID of the target thread.
priority	The new priority value of the thread (0, 1, or 2).

Returns

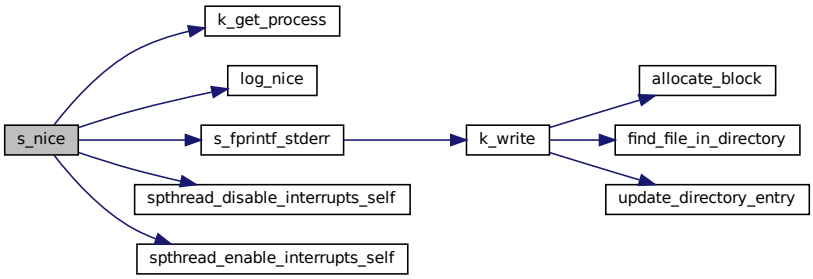
0 on success, -1 on failure.

Definition at line 376 of file syscall.c.

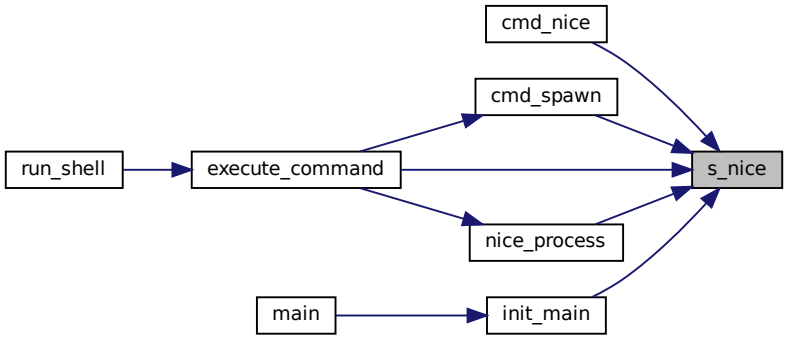
References k_get_process(), log_nice(), pcb::name, pcb::pid, pcb::priority, s_fprintf_stderr(), spthread_disable_↩ interrupts_self(), and spthread_enable_interrupts_self().

Referenced by cmd_nice(), cmd_spawn(), execute_command(), init_main(), and nice_process().

Here is the call graph for this function:



Here is the caller graph for this function:



4.26.1.4 s_sleep() `void s_sleep ( unsigned int ticks )`

Suspends execution of the calling process for a specified number of clock ticks.

This function is analogous to `sleep(3)` in Linux, with the behavior that the system clock continues to tick even if the call is interrupted. The sleep can be interrupted by a P_SIGTERM signal, after which the function will return prematurely.

Parameters

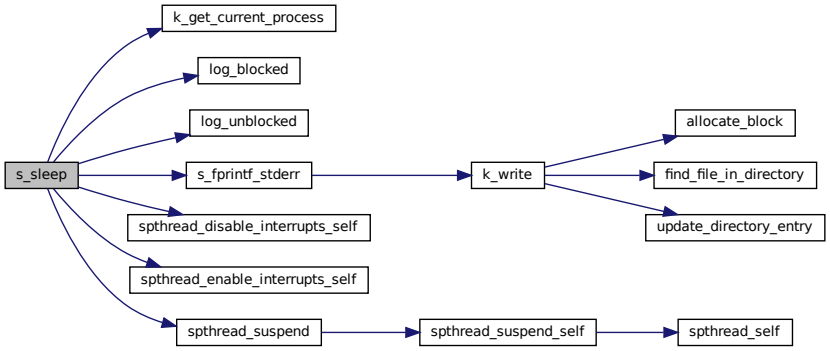
ticks	Duration of the sleep in system clock ticks. Must be greater than 0.

Definition at line 424 of file syscall.c.

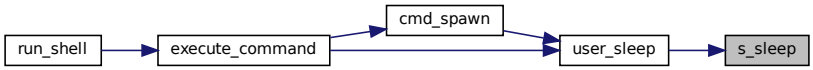
References BLOCKED, k_get_current_process(), log_blocked(), log_unblocked(), pcb::name, pcb::pid, pcb::priority, RUNNING, s_fprintf_stderr(), pcb::sleep_ticks_remaining, spthread_disable_interrupts_self(), spthread_enable_↩ interrupts_self(), spthread_suspend(), pcb::state, and pcb::thread.

Referenced by user_sleep().

Here is the call graph for this function:



Here is the caller graph for this function:



4.26.1.5 s_spawn()

```
pid_t s_spawn (
        void *(*)(void *) func,
        char * argv[],
        int fd0,
        int fd1 )
```

Create a child process that executes the function `func`. The child will retain some attributes of the parent.

Parameters

func	Function to be executed by the child process.
argv	Null-terminated array of args, including the command name as argv[0].
fd0	Input file descriptor.
fd1	Output file descriptor.

Returns

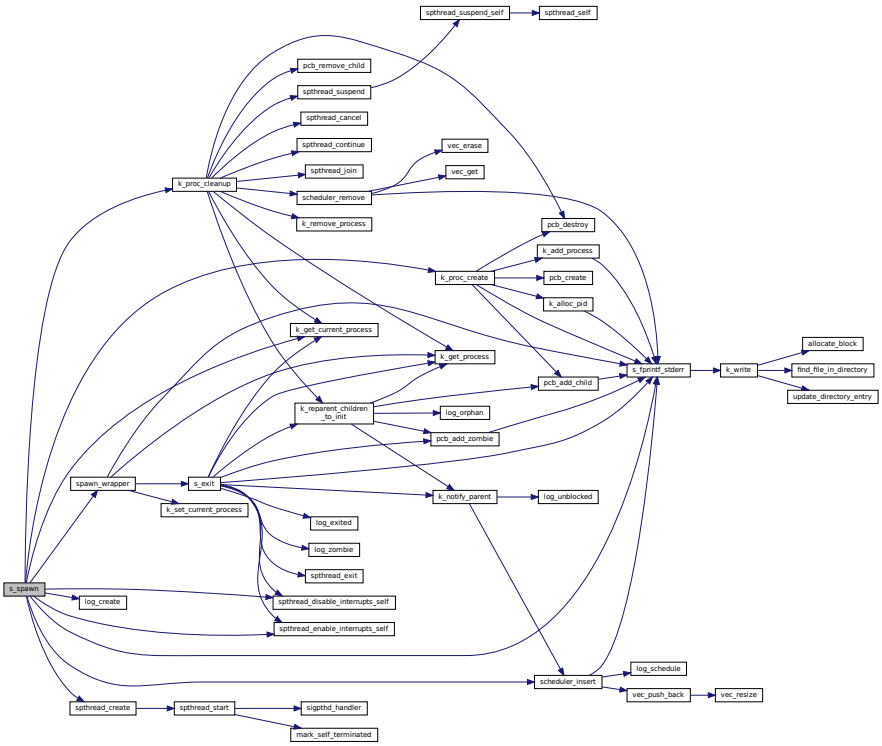
pid_t The process ID of the created child process, or -1 on error.

Definition at line 62 of file syscall.c.

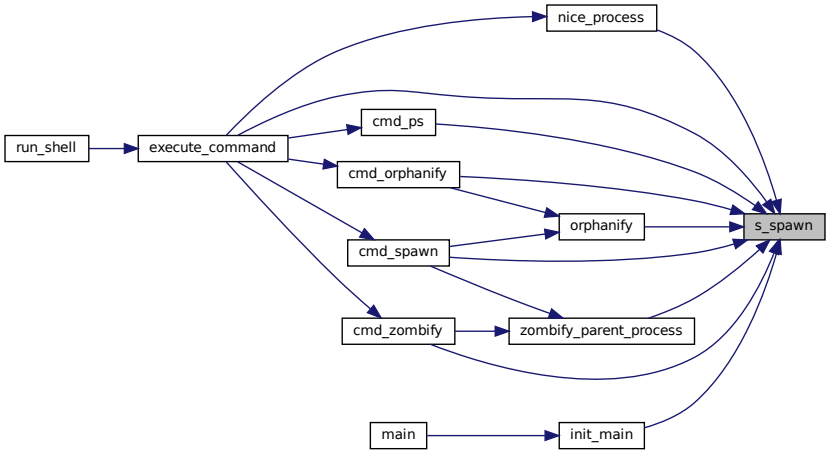
References spawn_args_t::argv, pcb::fd_table, spawn_args_t::func, k_get_current_process(), k_proc_cleanup(), k_↩ proc_create(), log_create(), pcb::name, pcb::pid, spawn_args_t::pid, pcb::priority, s_fprintf_stderr(), scheduler_insert(), spawn_wrapper(), spthread_create(), spthread_disable_interrupts_self(), spthread_enable_interrupts_self(), and pcb↩ ::thread.

Referenced by cmd_orphanify(), cmd_ps(), cmd_spawn(), cmd_zombify(), execute_command(), init_main(), nice_↩ process(), orphanify(), and zombify_parent_process().

Here is the call graph for this function:



Here is the caller graph for this function:



4.26.1.6 s_waitpid()

```
pid_t s_waitpid (
        pid_t pid,
        int * wstatus,
        bool nohang )
```

Wait on a child of the calling process, until it changes state. If `nohang` is true, this will not block the calling process and return immediately.

Parameters

pid	Process ID of the child to wait for (-1 for any child).
wstatus	Pointer to an integer variable where the status will be stored.
nohang	If true, return immediately if no child has exited.

Returns

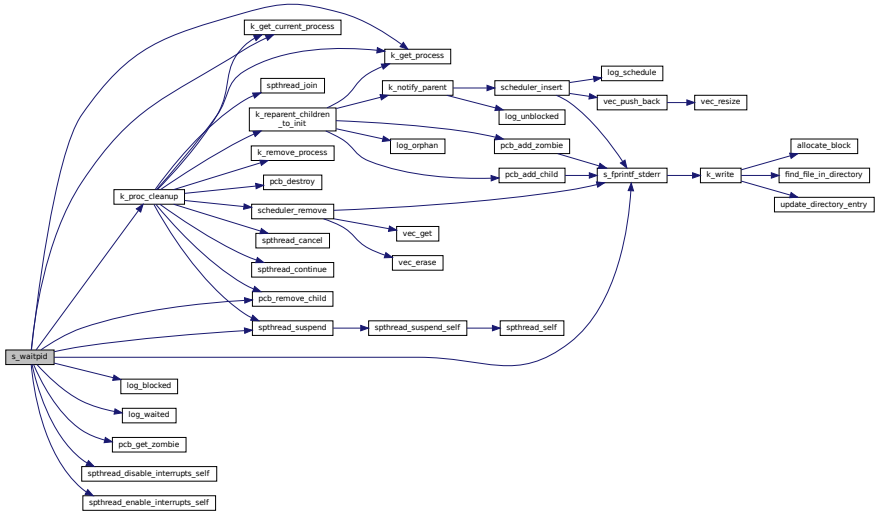
pid_t The process ID of the child which has changed state on success, -1 on error.

Definition at line 164 of file syscall.c.

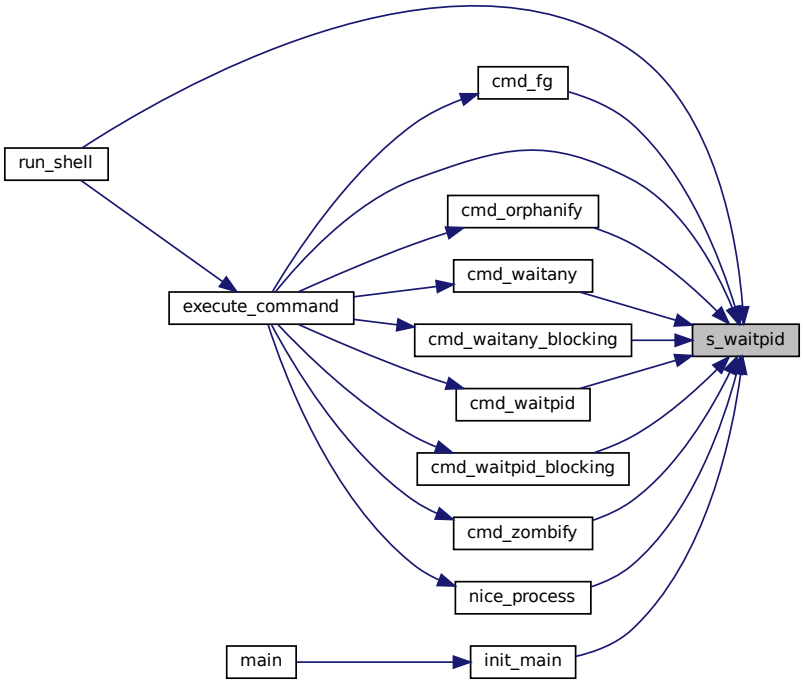
References BLOCKED, pcb::children, pcb::is_waiting, k_get_current_process(), k_get_process(), k_proc_cleanup(), log_blocked(), log_waited(), pcb::name, pcb::num_children, pcb::num_stopped, pcb::num_zombies, pcb_get_zombie(), pcb_remove_child(), pcb::pid, pcb::priority, s_fprintf_stderr(), spthread_disable_interrupts_self(), spthread_enable_↩ interrupts_self(), spthread_suspend(), pcb::state, pcb::stopped_children, pcb::thread, pcb::waiting_for_pid, and pcb↩ ::zombie_children.

Referenced by cmd_fg(), cmd_orphanify(), cmd_waitany(), cmd_waitany_blocking(), cmd_waitpid(), cmd_waitpid_↩ blocking(), cmd_zombify(), execute_command(), init_main(), nice_process(), and run_shell().

Here is the call graph for this function:



Here is the caller graph for this function:



4.26.1.7 spawn_wrapper() `static void* spawn_wrapper ( void * arg ) [static]`

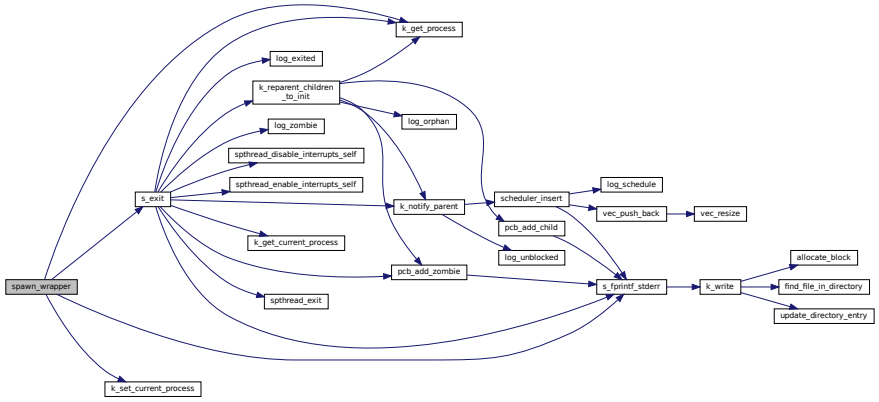
Wrapper function for spawned threads Sets the current process and executes the user function.

Definition at line 24 of file syscall.c.

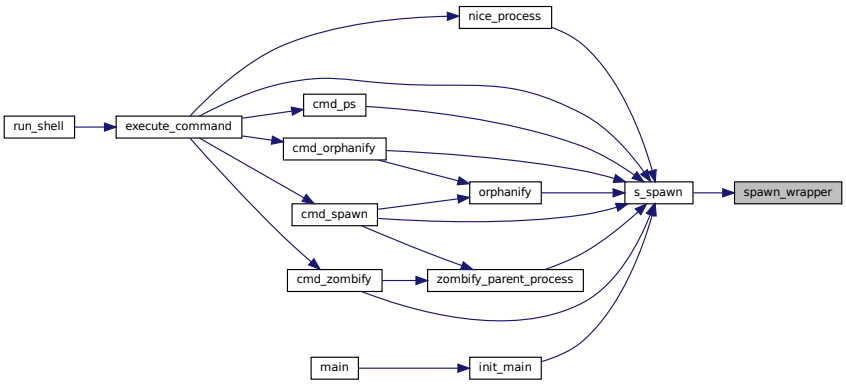
References spawn_args_t::argv, spawn_args_t::func, k_get_process(), k_set_current_process(), spawn_args_t::pid, s_exit(), and s_fprintf_stderr().

Referenced by s_spawn().

Here is the call graph for this function:



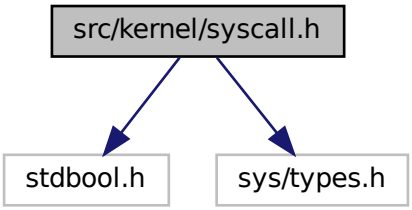
Here is the caller graph for this function:



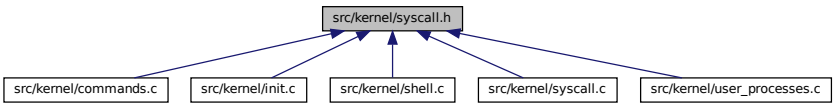
4.27 src/kernel/syscall.h File Reference

```
#include <stdbool.h>
#include <sys/types.h>
```

Include dependency graph for syscall.h:



This graph shows which files directly or indirectly include this file:



Functions

- pid_t s_spawn (void ∗(∗func)(void ∗), char ∗argv[ ], int fd0, int fd1)

  Create a child process that executes the function `func`. The child will retain some attributes of the parent.
- pid_t s_waitpid (pid_t pid, int ∗wstatus, bool nohang)

  Wait on a child of the calling process, until it changes state. If `nohang` is true, this will not block the calling process and return immediately.
- int s_kill (pid_t pid, int signal)

  Send a signal to a particular process.
- void s_exit (void)

  Unconditionally exit the calling process.
- int s_nice (pid_t pid, int priority)

  Set the priority of the specified thread.
- void s_sleep (unsigned int ticks)

  Suspends execution of the calling process for a specified number of clock ticks.

4.27.1 Function Documentation

4.27.1.1 s_exit() `void s_exit ( void )`

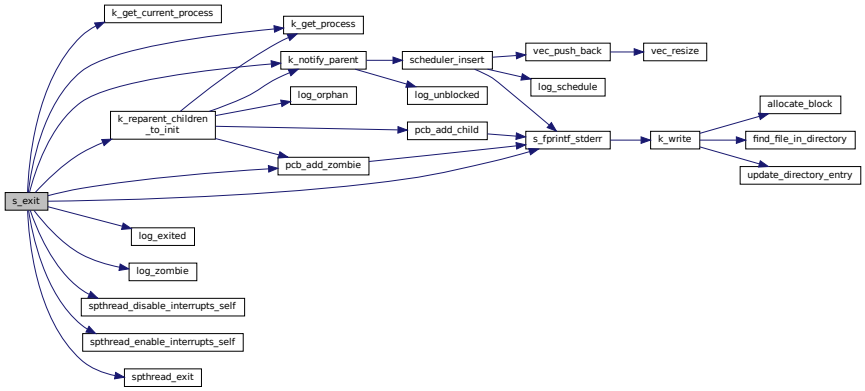
Unconditionally exit the calling process.

Definition at line 320 of file syscall.c.

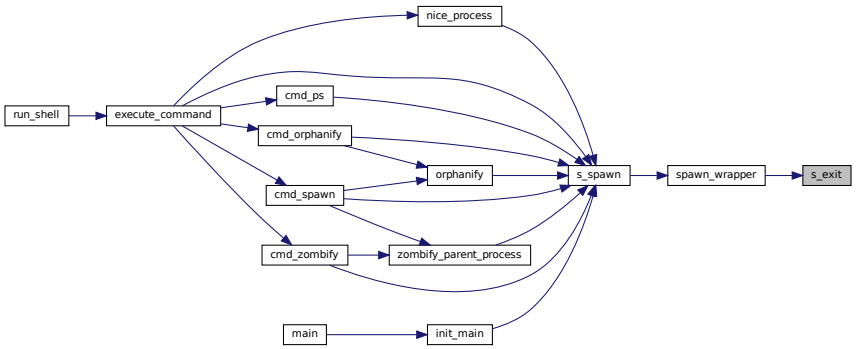
References pcb::exit_status, INIT_PID, k_get_current_process(), k_get_process(), k_notify_parent(), k_reparent_children_to_init(), log_exited(), log_zombie(), pcb::name, pcb::num_children, pcb::num_zombies, pcb_add_zombie(), pcb::pid, pcb::ppid, pcb::priority, s_fprintf_stderr(), spthread_disable_interrupts_self(), spthread_enable_interrupts_self(), spthread_exit(), pcb::state, and ZOMBIE.

Referenced by spawn_wrapper().

Here is the call graph for this function:



Here is the caller graph for this function:



4.27.1.2 s_kill() `int s_kill ( pid_t pid, int signal )`

Send a signal to a particular process.

Parameters

pid	Process ID of the target process.
signal	Signal number to be sent (P_SIGSTOP, P_SIGCONT, P_SIGTERM).

Returns

0 on success, -1 on error.

< Stop (pause) a process

< Continue a stopped process

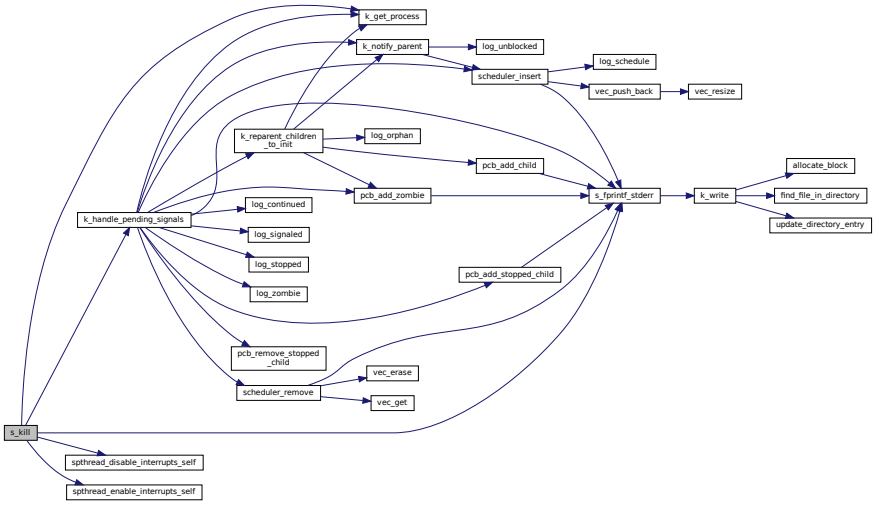
< Terminate a process

Definition at line 288 of file syscall.c.

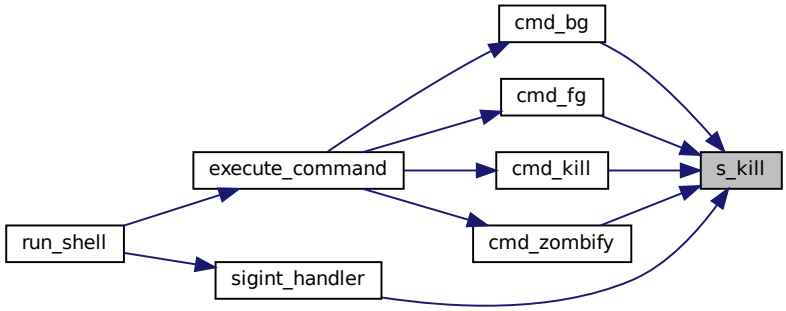
References k_get_process(), k_handle_pending_signals(), P_SIGCONT, P_SIGSTOP, P_SIGTERM, pcb::pending_↩ signal, s_fprintf_stderr(), spthread_disable_interrupts_self(), and spthread_enable_interrupts_self().

Referenced by cmd_bg(), cmd_fg(), cmd_kill(), cmd_zombify(), and sigint_handler().

Here is the call graph for this function:



Here is the caller graph for this function:



4.27.1.3 s_nice()

```
int s_nice (
        pid_t pid,
        int priority )
```

Set the priority of the specified thread.

Parameters

pid	Process ID of the target thread.
priority	The new priority value of the thread (0, 1, or 2).

Returns

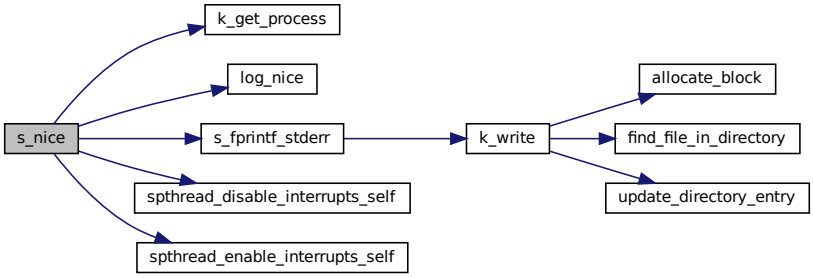
0 on success, -1 on failure.

Definition at line 376 of file syscall.c.

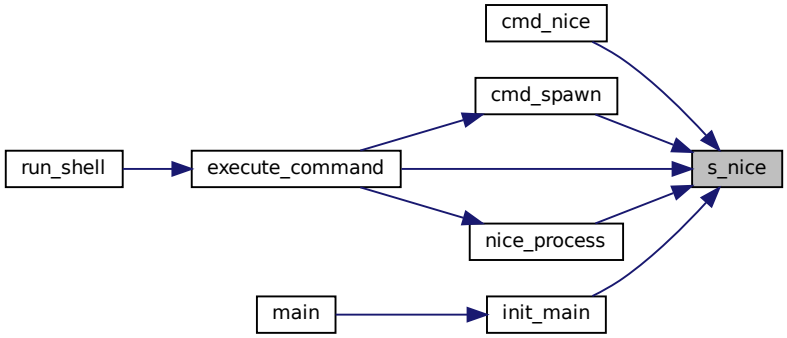
References k_get_process(), log_nice(), pcb::name, pcb::pid, pcb::priority, s_fprintf_stderr(), spthread_disable_↩ interrupts_self(), and spthread_enable_interrupts_self().

Referenced by cmd_nice(), cmd_spawn(), execute_command(), init_main(), and nice_process().

Here is the call graph for this function:



Here is the caller graph for this function:



4.27.1.4 s_sleep() `void s_sleep ( unsigned int ticks )`

Suspends execution of the calling process for a specified number of clock ticks.

This function is analogous to `sleep(3)` in Linux, with the behavior that the system clock continues to tick even if the call is interrupted. The sleep can be interrupted by a P_SIGTERM signal, after which the function will return prematurely.

Parameters

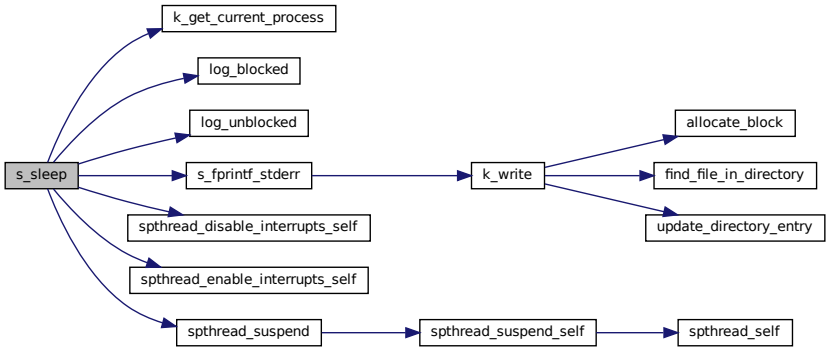
ticks	Duration of the sleep in system clock ticks. Must be greater than 0.

Definition at line 424 of file syscall.c.

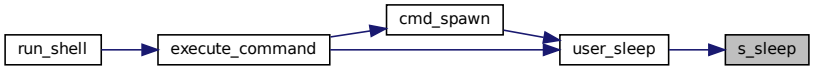
References BLOCKED, k_get_current_process(), log_blocked(), log_unblocked(), pcb::name, pcb::pid, pcb::priority, RUNNING, s_fprintf_stderr(), pcb::sleep_ticks_remaining, spthread_disable_interrupts_self(), spthread_enable_↩ interrupts_self(), spthread_suspend(), pcb::state, and pcb::thread.

Referenced by user_sleep().

Here is the call graph for this function:



Here is the caller graph for this function:



4.27.1.5 s_spawn()

```
pid_t s_spawn (
    void *(*)(void *) func,
    char * argv[],
    int fd0,
    int fd1 )
```

Create a child process that executes the function `func`. The child will retain some attributes of the parent.

Parameters

func	Function to be executed by the child process.
argv	Null-terminated array of args, including the command name as argv[0].
fd0	Input file descriptor.
fd1	Output file descriptor.

Returns

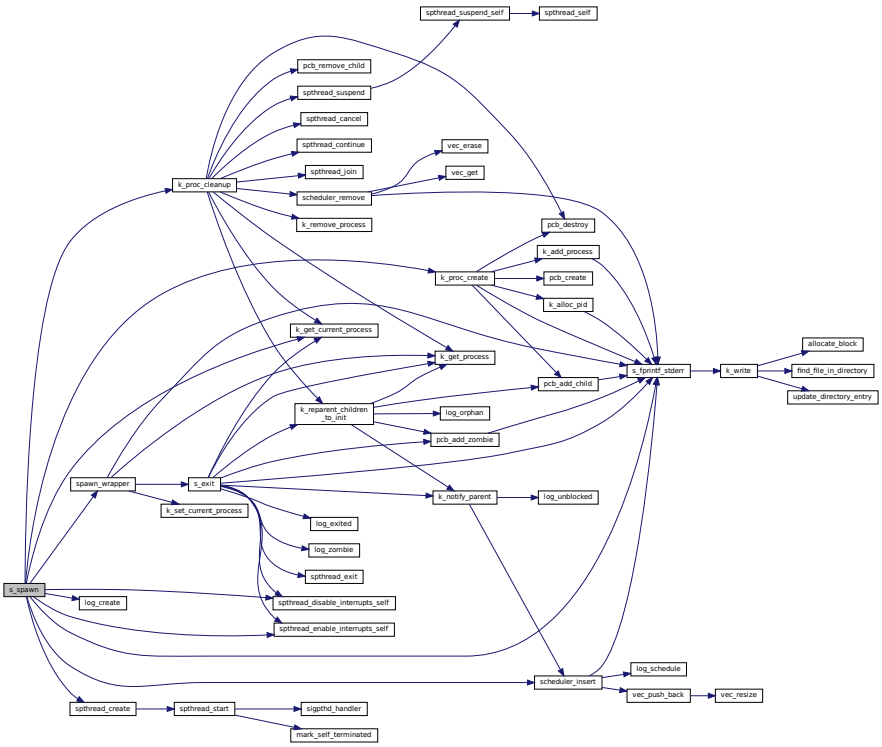
pid_t The process ID of the created child process, or -1 on error.

Definition at line 62 of file syscall.c.

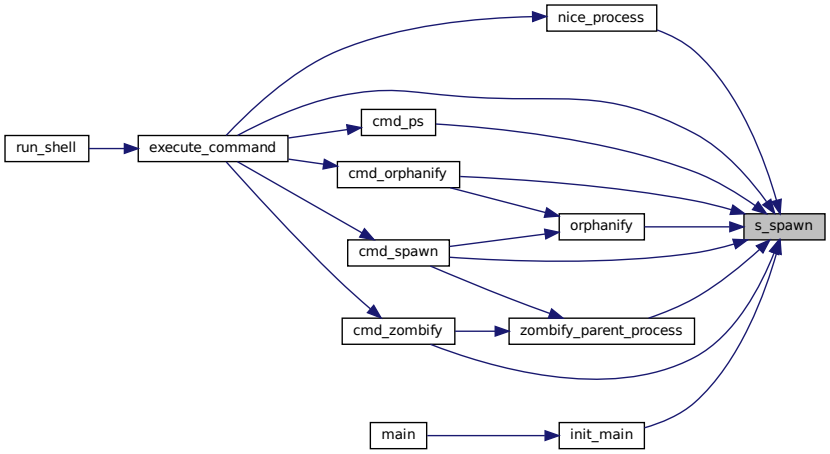
References spawn_args_t::argv, pcb::fd_table, spawn_args_t::func, k_get_current_process(), k_proc_cleanup(), k_↩ proc_create(), log_create(), pcb::name, pcb::pid, spawn_args_t::pid, pcb::priority, s_fprintf_stderr(), scheduler_insert(), spawn_wrapper(), spthread_create(), spthread_disable_interrupts_self(), spthread_enable_interrupts_self(), and pcb↩ ::thread.

Referenced by cmd_orphanify(), cmd_ps(), cmd_spawn(), cmd_zombify(), execute_command(), init_main(), nice_↩ process(), orphanify(), and zombify_parent_process().

Here is the call graph for this function:



Here is the caller graph for this function:



4.27.1.6 s_waitpid()

```
pid_t s_waitpid (
        pid_t pid,
        int * wstatus,
        bool nohang )
```

Wait on a child of the calling process, until it changes state. If `nohang` is true, this will not block the calling process and return immediately.

Parameters

pid	Process ID of the child to wait for (-1 for any child).
wstatus	Pointer to an integer variable where the status will be stored.
nohang	If true, return immediately if no child has exited.

Returns

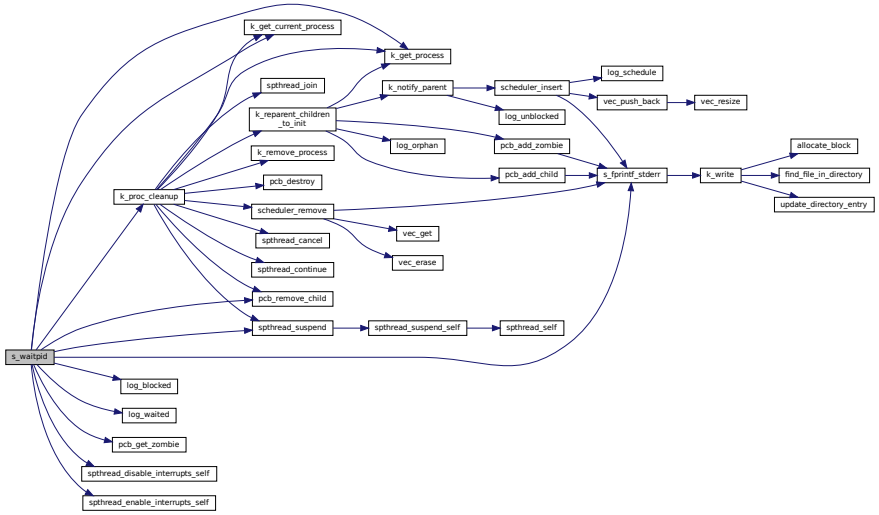
pid_t The process ID of the child which has changed state on success, -1 on error.

Definition at line 164 of file syscall.c.

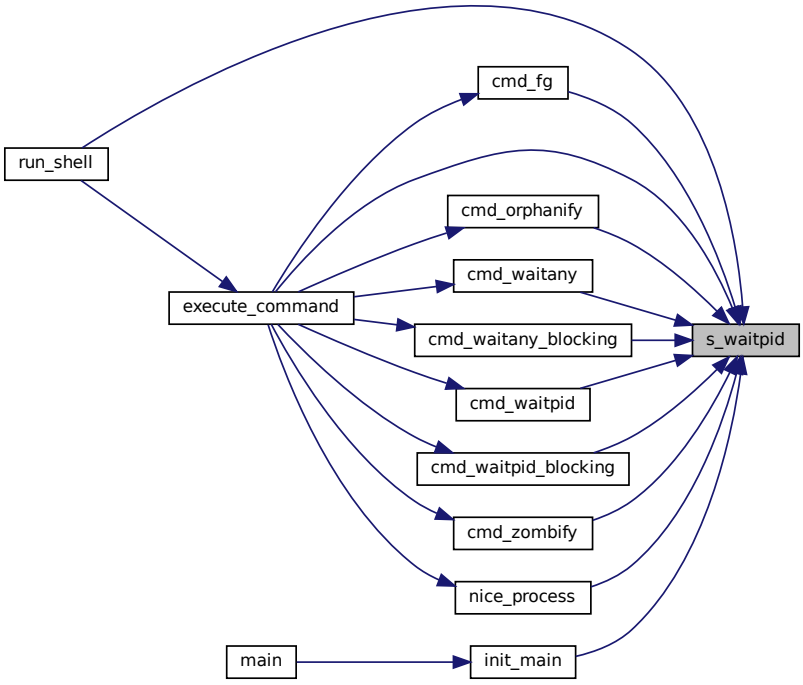
References BLOCKED, pcb::children, pcb::is_waiting, k_get_current_process(), k_get_process(), k_proc_cleanup(), log_blocked(), log_waited(), pcb::name, pcb::num_children, pcb::num_stopped, pcb::num_zombies, pcb_get_zombie(), pcb_remove_child(), pcb::pid, pcb::priority, s_fprintf_stderr(), spthread_disable_interrupts_self(), spthread_enable_↩ interrupts_self(), spthread_suspend(), pcb::state, pcb::stopped_children, pcb::thread, pcb::waiting_for_pid, and pcb↩ ::zombie_children.

Referenced by cmd_fg(), cmd_orphanify(), cmd_waitany(), cmd_waitany_blocking(), cmd_waitpid(), cmd_waitpid_↩ blocking(), cmd_zombify(), execute_command(), init_main(), nice_process(), and run_shell().

Here is the call graph for this function:



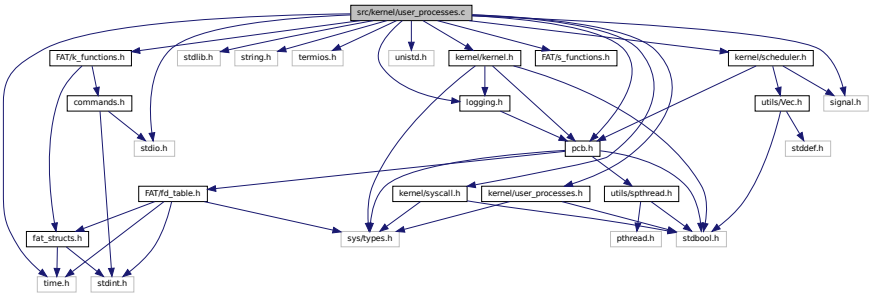
Here is the caller graph for this function:



4.28 src/kernel/user_processes.c File Reference

```
#include "kernel/user_processes.h"
#include <signal.h>
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <termios.h>
#include <time.h>
#include <unistd.h>
#include "FAT/k_functions.h"
#include "FAT/s_functions.h"
#include "kernel/kernel.h"
#include "kernel/logging.h"
#include "kernel/pcb.h"
#include "kernel/scheduler.h"
#include "kernel/syscall.h"
```

Include dependency graph for user_processes.c:



Functions

- void * hello_process (void *arg)

  Simple test process that prints "Hello World".
- void * counting_process (void *arg)

  Process that counts from 1 to n with delays.
- void * user_sleep (void *arg)

  Process that sleeps for a specified number of ticks.
- void * zombie_process (void *arg)

  Test process that immediately exits to become a zombie.
- void * ps_process (void *arg)

  Process that displays all running processes (ps command)
- void * orphan_child (void *arg)

  Helper for orphanify - orphan child that runs forever after being reparented.
- void * orphanify (void *arg)

  Used to test orphanifying functionality of your kernel. Spawns a child process then immediately exits, orphaning the child. The child should be reparented to INIT (PID 1) and will run forever.
- void * zombify_parent_process (void *arg)

Process that creates a zombie child then exits (orphaning the zombie)

- pid_t nice_process (void ∗(∗func)(void ∗), char ∗argv[ ], int fd0, int fd1, int priority, bool bg)

  Spawn a process with a specific priority level.
- void ∗ user_busy (void ∗arg)

  Busy-wait process for CPU stress testing.
- void ∗ user_cat (void ∗arg)

  User-space cat command - concatenate and display files.
- void ∗ user_echo (void ∗arg)

  User-space echo command - display text.
- void ∗ user_ls (void ∗arg)

  User-space ls command - list directory contents.
- void ∗ user_touch (void ∗arg)

  User-space touch command - create empty file.
- void ∗ user_mv (void ∗arg)

  User-space mv command - move/rename file.
- void ∗ user_cp (void ∗arg)

  User-space cp command - copy file.
- void ∗ user_rm (void ∗arg)

  User-space rm command - remove file.
- void ∗ user_man (void ∗arg)

  User-space man command - display command manual.
- void user_chmod (char ∗∗args)

  User-space chmod command - change file permissions.

4.28.1 Function Documentation

4.28.1.1 counting_process() `void* counting_process ( void * arg )`

Process that counts from 1 to n with delays.

Parameters

arg	Command-line arguments (expects count as argv[1])

Returns

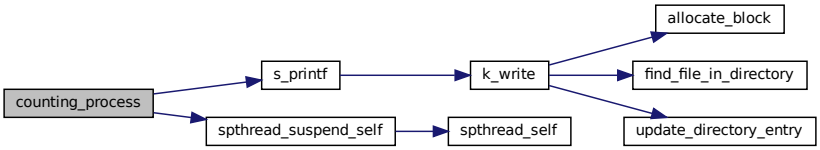
NULL

Definition at line 25 of file user_processes.c.

References s_printf(), and spthread_suspend_self().

Referenced by cmd_spawn().

Here is the call graph for this function:



Here is the caller graph for this function:



4.28.1.2 hello_process() `void* hello_process ( void * arg )`

Simple test process that prints "Hello World".

Parameters

arg	Command-line arguments (unused)

Returns

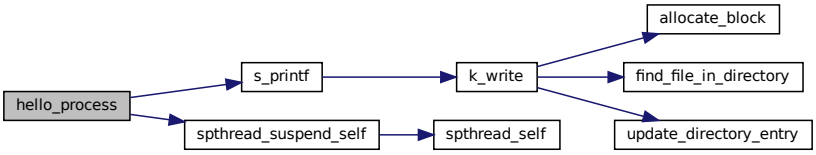
NULL

Definition at line 18 of file user_processes.c.

References s_printf(), and spthread_suspend_self().

Referenced by cmd_spawn().

Here is the call graph for this function:



Here is the caller graph for this function:



4.28.1.3 nice_process()

```
pid_t nice_process (
        void *(*)(void *) func,
        char * argv[],
        int fd0,
        int fd1,
        int priority,
        bool bg )
```

Spawn a process with a specific priority level.

Wrapper around s_spawn() that sets the process priority after creation using s_nice(). Used by the shell to implement priority specification via the 'nice' prefix.

Parameters

func	Function to execute in the new process
argv	Null-terminated argument array
fd0	Input file descriptor
fd1	Output file descriptor
priority	Priority level (0=high, 1=medium, 2=low)
bg	Whether to run in background

Returns

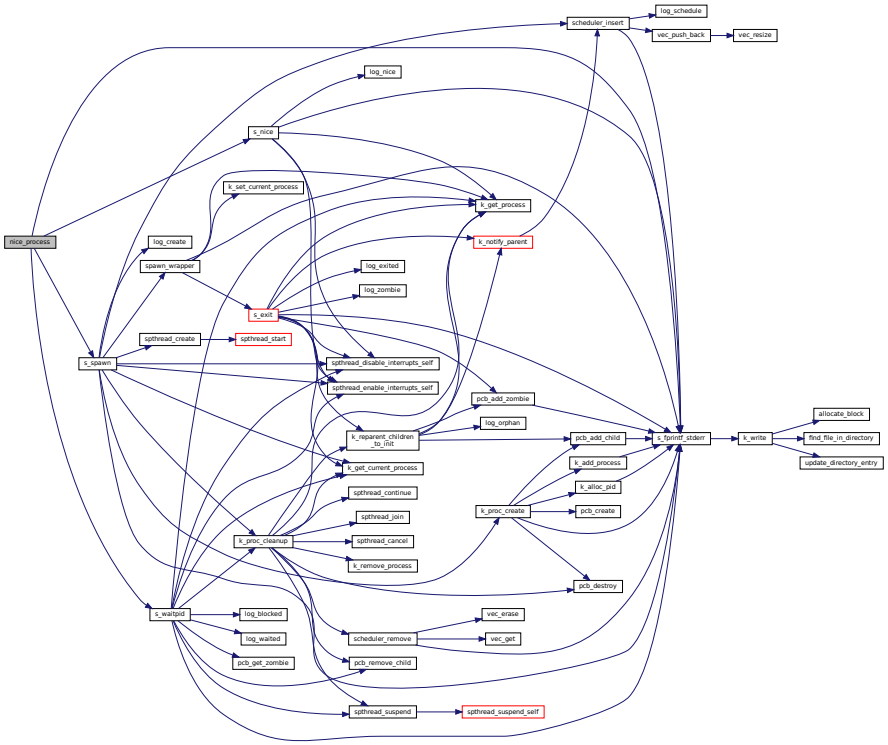
PID of spawned process, or -1 on error

Definition at line 211 of file user_processes.c.

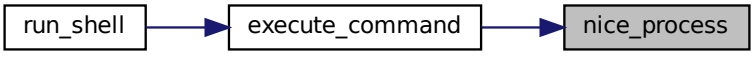
References s_fprintf_stderr(), s_nice(), s_spawn(), and s_waitpid().

Referenced by execute_command().

Here is the call graph for this function:



Here is the caller graph for this function:



4.28.1.4 orphan_child() `void* orphan_child ( void * arg )`

Helper for orphanify - orphan child that runs forever after being reparented.

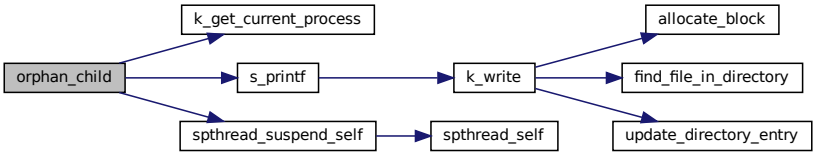
Child process for orphan testing.

Definition at line 103 of file user_processes.c.

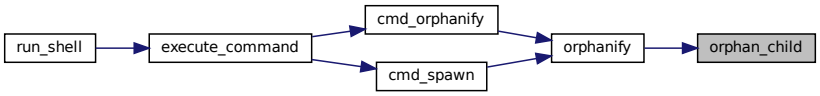
References k_get_current_process(), s_printf(), and spthread_suspend_self().

Referenced by orphanify().

Here is the call graph for this function:



Here is the caller graph for this function:



4.28.1.5 orphanify() `void* orphanify ( void * arg )`

Used to test orphanifying functionality of your kernel. Spawns a child process then immediately exits, orphaning the child. The child should be reparented to INIT (PID 1) and will run forever.

Parent process that creates a child then immediately exits.

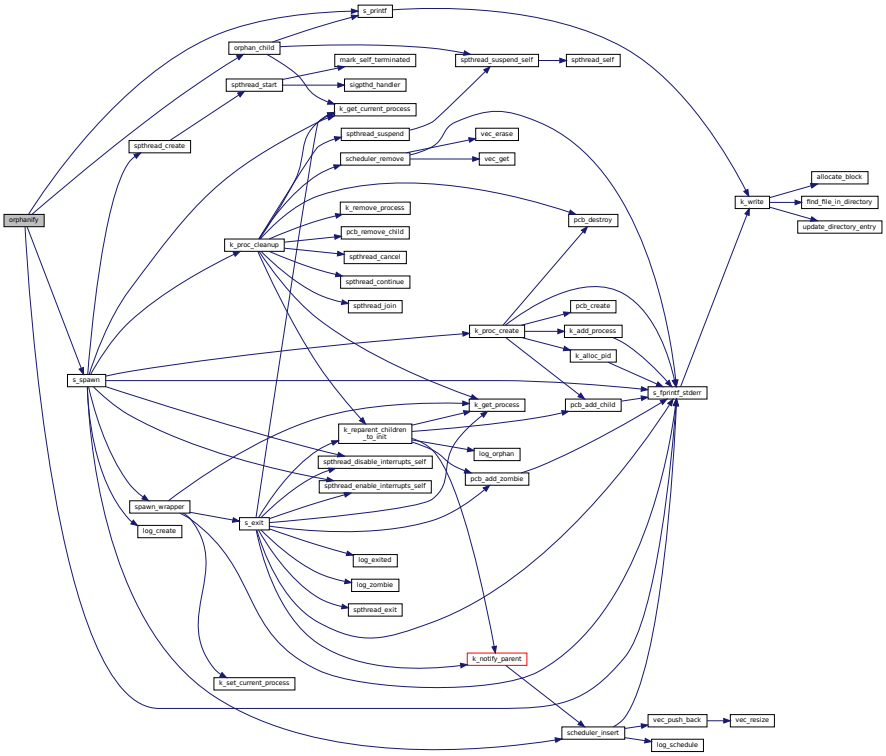
Example Usage: orphanify

Definition at line 144 of file user_processes.c.

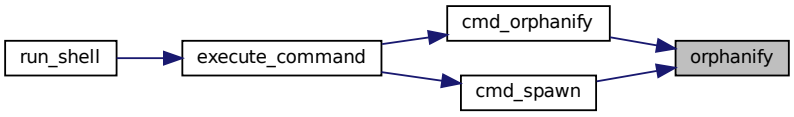
References orphan_child(), s_fprintf_stderr(), s_printf(), and s_spawn().

Referenced by cmd_orphanify(), and cmd_spawn().

Here is the call graph for this function:



Here is the caller graph for this function:



4.28.1.6 ps_process() `void* ps_process ( void * arg )`

Process that displays all running processes (ps command)

Prints a formatted table showing PID, PPID, priority, and state for all processes in the system.

Parameters

arg	Command-line arguments (unused)

Returns

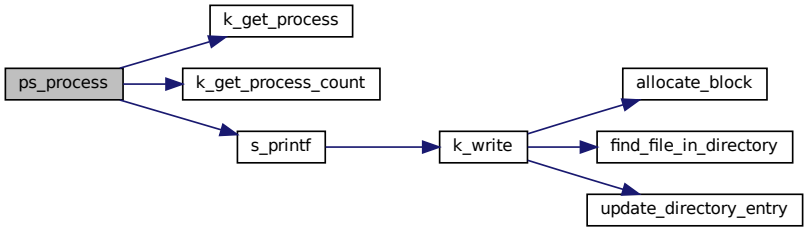
NULL

Definition at line 63 of file user_processes.c.

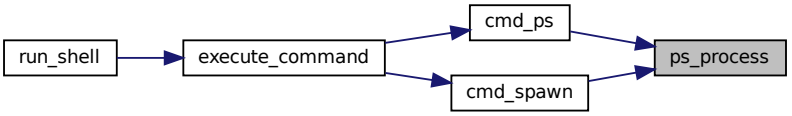
References BLOCKED, k_get_process(), k_get_process_count(), MAX_PROCESSES, pcb::name, pcb::pid, pcb::ppid, pcb::priority, RUNNING, s_printf(), pcb::state, STOPPED, and ZOMBIE.

Referenced by cmd_ps(), and cmd_spawn().

Here is the call graph for this function:



Here is the caller graph for this function:



4.28.1.7 user_busy() `void* user_busy ( void * arg )`

Busy-wait process for CPU stress testing.

Executes an infinite loop performing meaningless computation to consume CPU time. Used for testing scheduler behavior under load.

Parameters

arg	busy_args_t* containing process name and background flag

Returns

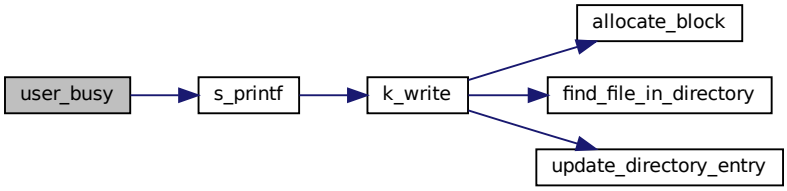
NULL (never returns normally)

Definition at line 247 of file user_processes.c.

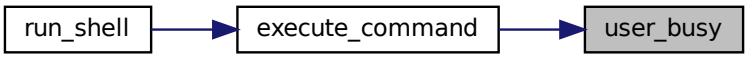
References s_printf().

Referenced by execute_command().

Here is the call graph for this function:



Here is the caller graph for this function:



4.28.1.8 user_cat() `void* user_cat ( void * arg )`

User-space cat command - concatenate and display files.

Parameters

arg	Command-line arguments (filenames)

Returns

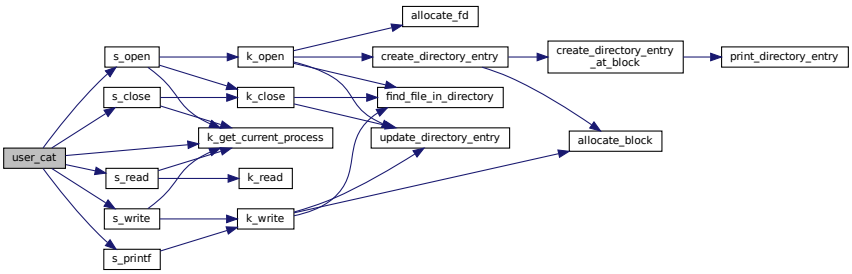
NULL

Definition at line 255 of file user_processes.c.

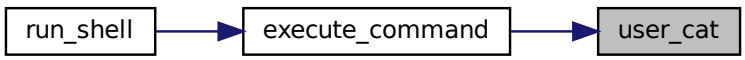
References F_APPEND, F_READ, F_WRITE, FILE_NOT_FOUND, FS_NOT_MOUNTED, k_get_current_process(), s_close(), s_open(), s_printf(), s_read(), and s_write().

Referenced by execute_command().

Here is the call graph for this function:



Here is the caller graph for this function:



4.28.1.9 user_chmod() `void user_chmod ( char ** arg )`

User-space chmod command - change file permissions.

Parameters

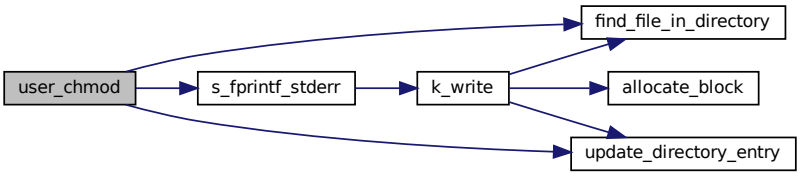
arg	Command-line arguments (permissions, filename)

Definition at line 818 of file user_processes.c.

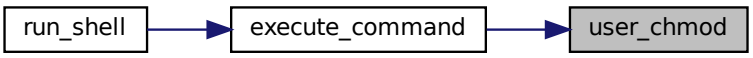
References find_file_in_directory(), directory_entry_t::perm, s_fprintf_stderr(), and update_directory_entry().

Referenced by execute_command().

Here is the call graph for this function:



Here is the caller graph for this function:



4.28.1.10 user_cp() `void* user_cp ( void * arg )`

User-space cp command - copy file.

Parameters

arg	Command-line arguments (source, destination)

Returns

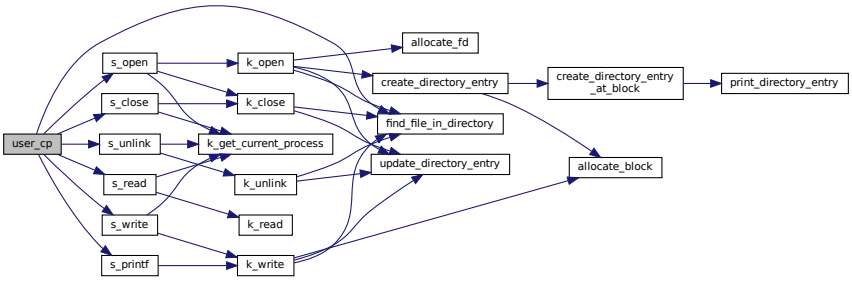
NULL

Definition at line 609 of file user_processes.c.

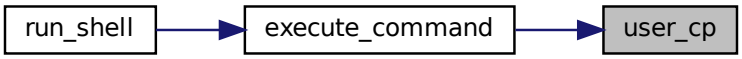
References F_READ, F_WRITE, FILE_NOT_FOUND, find_file_in_directory(), FS_NOT_MOUNTED, s_close(), s_↩ open(), s_printf(), s_read(), s_unlink(), and s_write().

Referenced by execute_command().

Here is the call graph for this function:



Here is the caller graph for this function:



4.28.1.11 user_echo() `void* user_echo ( void * arg )`

User-space echo command - display text.

Parameters

arg	Command-line arguments (text to echo)

Returns

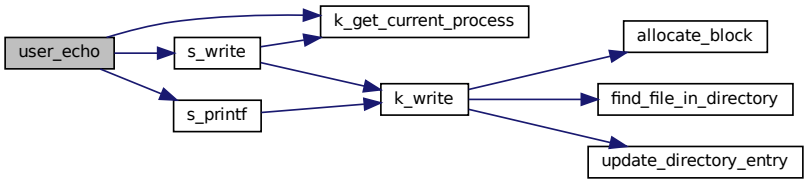
NULL

Definition at line 486 of file user_processes.c.

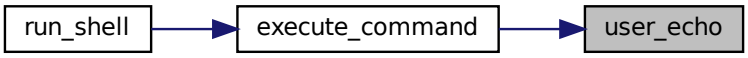
References pcb::fd_table, k_get_current_process(), s_printf(), and s_write().

Referenced by execute_command().

Here is the call graph for this function:



Here is the caller graph for this function:



4.28.1.12 user_ls() `void* user_ls ( void * arg )`

User-space ls command - list directory contents.

Parameters

arg	Command-line arguments (directory path, optional)

Returns

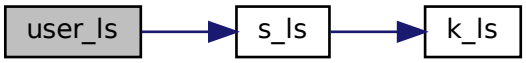
NULL

Definition at line 517 of file user_processes.c.

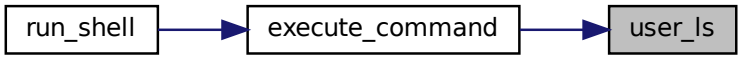
References s_ls().

Referenced by execute_command().

Here is the call graph for this function:



Here is the caller graph for this function:



4.28.1.13 user_man() `void* user_man ( void * arg )`

User-space man command - display command manual.

Parameters

arg	Command-line arguments (command name)

Returns

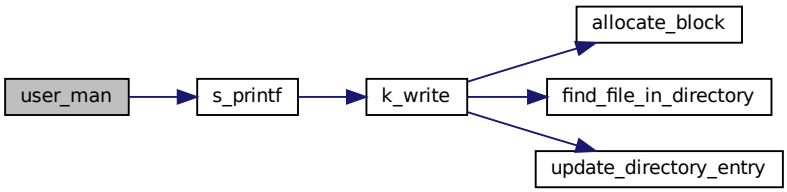
NULL

Definition at line 774 of file user_processes.c.

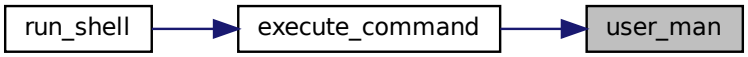
References s_printf().

Referenced by execute_command().

Here is the call graph for this function:



Here is the caller graph for this function:



4.28.1.14 user_mv() `void* user_mv ( void * arg )`

User-space mv command - move/rename file.

Parameters

arg	Command-line arguments (source, destination)

Returns

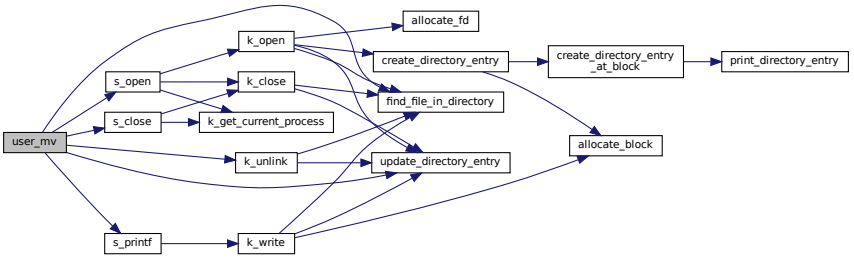
NULL

Definition at line 549 of file user_processes.c.

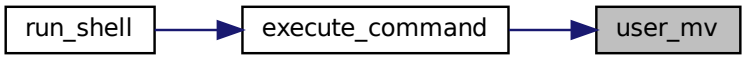
References F_READ, FILE_NOT_FOUND, find_file_in_directory(), FS_NOT_MOUNTED, k_unlink(), directory_entry↩_t::name, s_close(), s_open(), s_printf(), and update_directory_entry().

Referenced by execute_command().

Here is the call graph for this function:



Here is the caller graph for this function:



4.28.1.15 user_rm() `void* user_rm ( void * arg )`

User-space rm command - remove file.

Parameters

arg	Command-line arguments (filename)

Returns

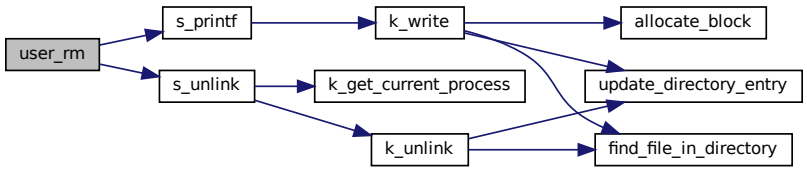
NULL

Definition at line 755 of file user_processes.c.

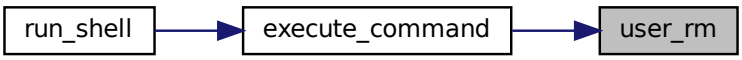
References FILE_NOT_FOUND, FS_NOT_MOUNTED, s_printf(), and s_unlink().

Referenced by execute_command().

Here is the call graph for this function:



Here is the caller graph for this function:



4.28.1.16 user_sleep() `void* user_sleep ( void * arg )`

Process that sleeps for a specified number of ticks.

Calls s_sleep() with the tick count specified in argv[1].

Parameters

arg	Command-line arguments (expects tick count as argv[1])

Returns

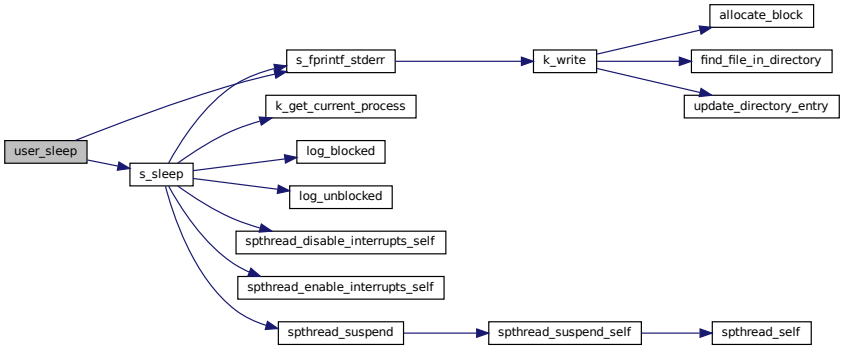
NULL

Definition at line 36 of file user_processes.c.

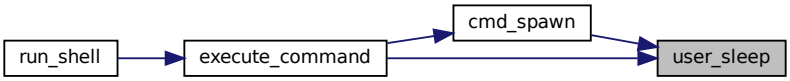
References s_fprintf_stderr(), and s_sleep().

Referenced by cmd_spawn(), and execute_command().

Here is the call graph for this function:



Here is the caller graph for this function:



4.28.1.17 user_touch() `void* user_touch ( void * arg )`

User-space touch command - create empty file.

Parameters

arg	Command-line arguments (filename)

Returns

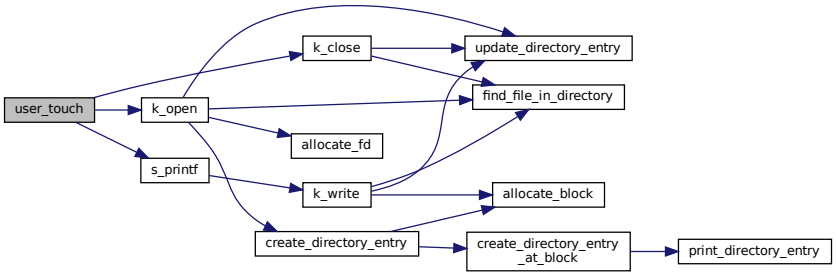
NULL

Definition at line 527 of file user_processes.c.

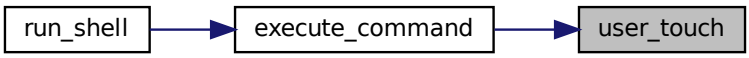
References F_APPEND, FS_NOT_MOUNTED, k_close(), k_open(), and s_printf().

Referenced by execute_command().

Here is the call graph for this function:



Here is the caller graph for this function:



4.28.1.18 zombie_process() `void* zombie_process ( void * arg )`

Test process that immediately exits to become a zombie.

Used for testing zombie process handling and s_waitpid().

Parameters

arg	Command-line arguments (unused)

Returns

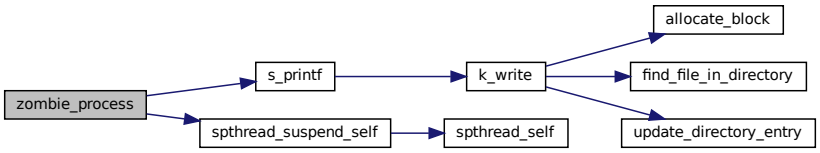
NULL

Definition at line 56 of file user_processes.c.

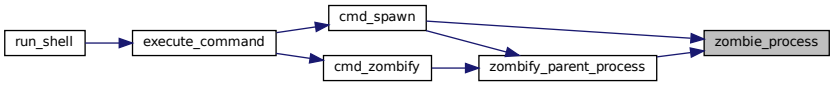
References s_printf(), and spthread_suspend_self().

Referenced by cmd_spawn(), and zombify_parent_process().

Here is the call graph for this function:



Here is the caller graph for this function:



4.28.1.19 zombify_parent_process() `void* zombify_parent_process ( void * arg )`

Process that creates a zombie child then exits (orphaning the zombie)

Used for testing orphaned zombie handling and INIT reparenting.

Parameters

arg	Command-line arguments (unused)

Returns

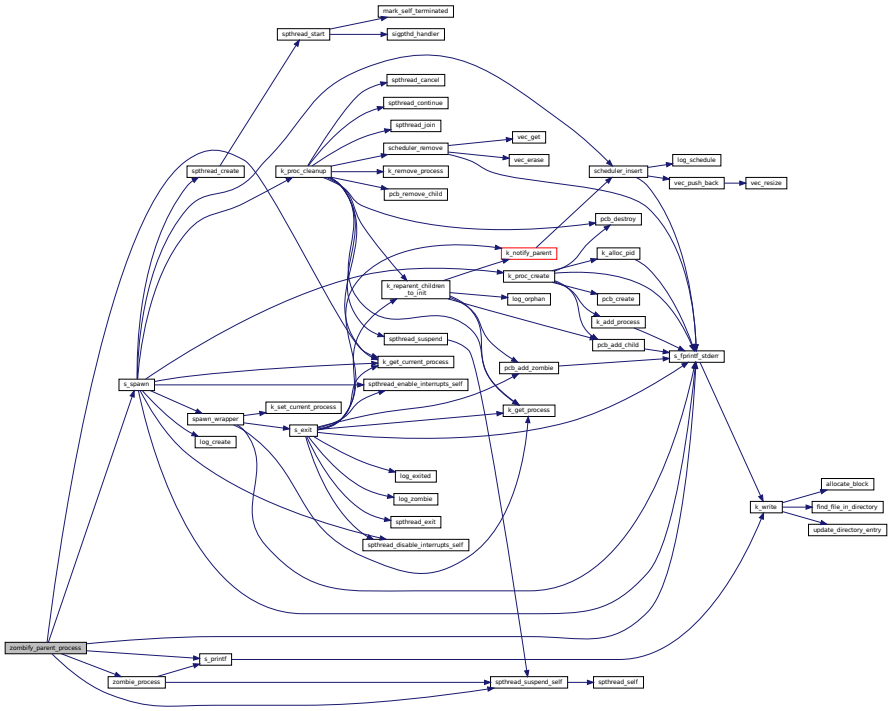
NULL

Definition at line 169 of file user_processes.c.

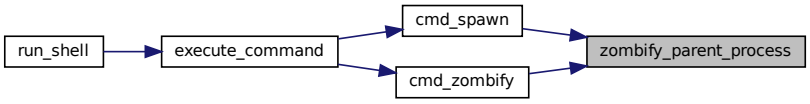
References k_get_current_process(), s_fprintf_stderr(), s_printf(), s_spawn(), spthread_suspend_self(), ZOMBIE, and zombie_process().

Referenced by cmd_spawn(), and cmd_zombify().

Here is the call graph for this function:



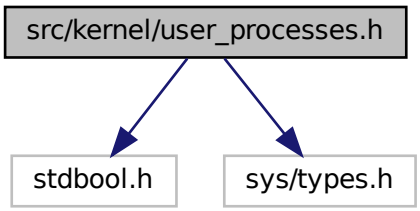
Here is the caller graph for this function:



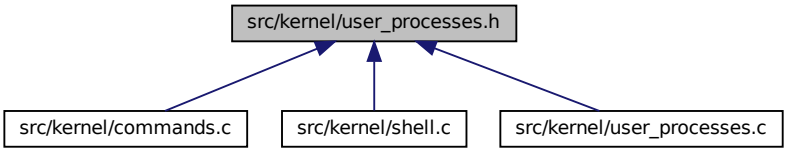
4.29 src/kernel/user_processes.h File Reference

```
#include <stdbool.h>
#include <sys/types.h>
```

Include dependency graph for user_processes.h:



This graph shows which files directly or indirectly include this file:



Data Structures

- struct busy_args_t

  Argument structure for busy process.

Functions

- void * hello_process (void *arg)

  Simple test process that prints "Hello World".
- void * counting_process (void *arg)

  Process that counts from 1 to n with delays.
- void * user_sleep (void *arg)

  Process that sleeps for a specified number of ticks.
- void * zombie_process (void *arg)

  Test process that immediately exits to become a zombie.
- void * zombify_parent_process (void *arg)

  Process that creates a zombie child then exits (orphaning the zombie)
- void * ps_process (void *arg)

Process that displays all running processes (ps command)

- void * orphan_child (void *arg)

  Child process for orphan testing.
- void * orphanify (void *arg)

  Parent process that creates a child then immediately exits.
- pid_t nice_process (void *(*func)(void *), char *argv[], int fd0, int fd1, int priority, bool bg)

  Spawn a process with a specific priority level.
- void * user_busy (void *arg)

  Busy-wait process for CPU stress testing.
- void * user_cat (void *arg)

  User-space cat command - concatenate and display files.
- void * user_echo (void *arg)

  User-space echo command - display text.
- void * user_ls (void *arg)

  User-space ls command - list directory contents.
- void * user_touch (void *arg)

  User-space touch command - create empty file.
- void * user_mv (void *arg)

  User-space mv command - move/rename file.
- void * user_cp (void *arg)

  User-space cp command - copy file.
- void * user_rm (void *arg)

  User-space rm command - remove file.
- void * user_man (void *arg)

  User-space man command - display command manual.
- void user_chmod (char **arg)

  User-space chmod command - change file permissions.

4.29.1 Function Documentation

4.29.1.1 counting_process() `void* counting_process ( void * arg )`

Process that counts from 1 to n with delays.

Parameters

arg	Command-line arguments (expects count as argv[1])

Returns

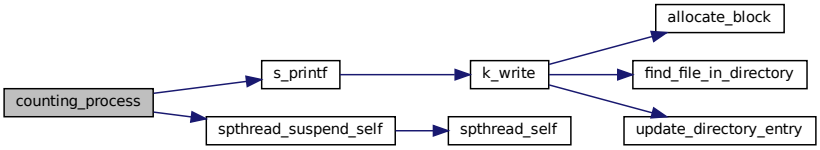
NULL

Definition at line 25 of file user_processes.c.

References s_printf(), and spthread_suspend_self().

Referenced by cmd_spawn().

Here is the call graph for this function:



Here is the caller graph for this function:



4.29.1.2 hello_process() `void* hello_process ( void * arg )`

Simple test process that prints "Hello World".

Parameters

arg	Command-line arguments (unused)

Returns

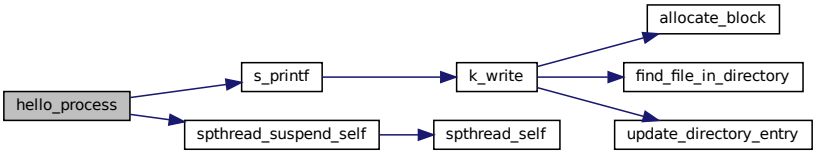
NULL

Definition at line 18 of file user_processes.c.

References s_printf(), and spthread_suspend_self().

Referenced by cmd_spawn().

Here is the call graph for this function:



Here is the caller graph for this function:



4.29.1.3 nice_process()

```
pid_t nice_process (
        void *(*)(void *) func,
        char * argv[],
        int fd0,
        int fd1,
        int priority,
        bool bg )
```

Spawn a process with a specific priority level.

Wrapper around s_spawn() that sets the process priority after creation using s_nice(). Used by the shell to implement priority specification via the 'nice' prefix.

Parameters

func	Function to execute in the new process
argv	Null-terminated argument array
fd0	Input file descriptor
fd1	Output file descriptor
priority	Priority level (0=high, 1=medium, 2=low)
bg	Whether to run in background

Returns

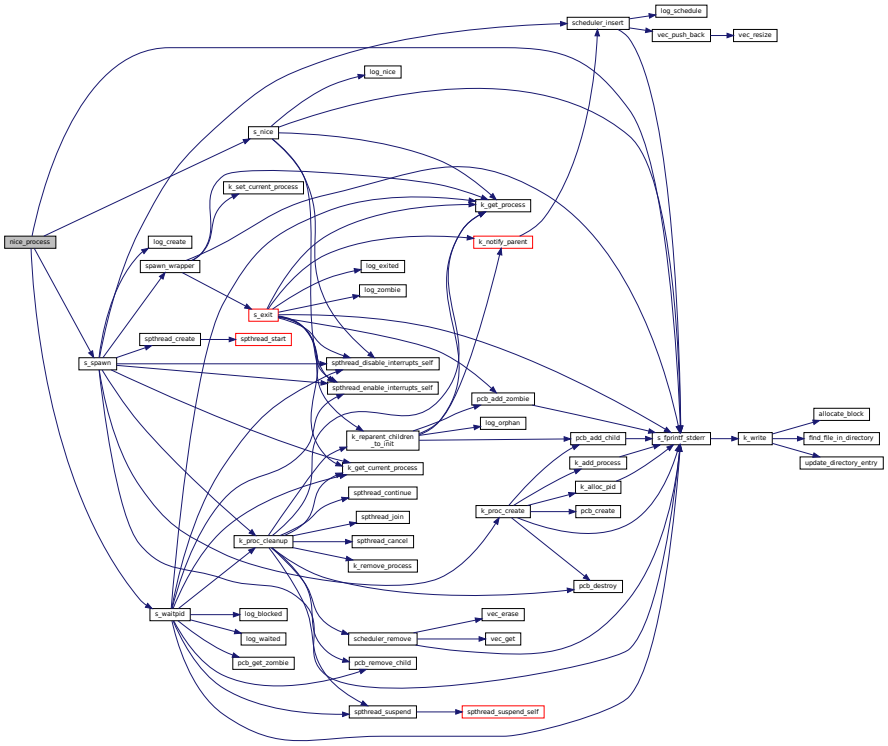
PID of spawned process, or -1 on error

Definition at line 211 of file user_processes.c.

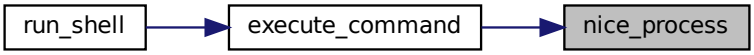
References s_fprintf_stderr(), s_nice(), s_spawn(), and s_waitpid().

Referenced by execute_command().

Here is the call graph for this function:



Here is the caller graph for this function:



4.29.1.4 orphan_child() `void* orphan_child ( void * arg )`

Child process for orphan testing.

Sleeps for a period to allow parent to exit first, creating an orphan.

Parameters

arg	Command-line arguments (unused)

Returns

NULL

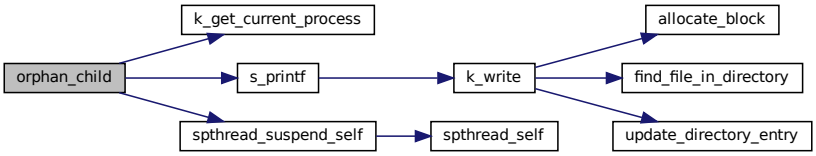
Child process for orphan testing.

Definition at line 103 of file user_processes.c.

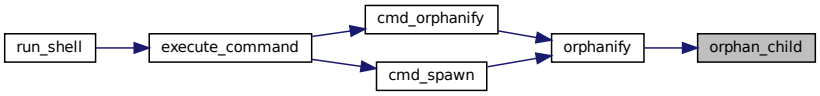
References k_get_current_process(), s_printf(), and spthread_suspend_self().

Referenced by orphanify().

Here is the call graph for this function:



Here is the caller graph for this function:



4.29.1.5 orphanify() `void* orphanify ( void * arg )`

Parent process that creates a child then immediately exits.

Used for testing orphan process handling and reparenting to INIT.

Parameters

arg	Command-line arguments (unused)

Returns

NULL

Parent process that creates a child then immediately exits.

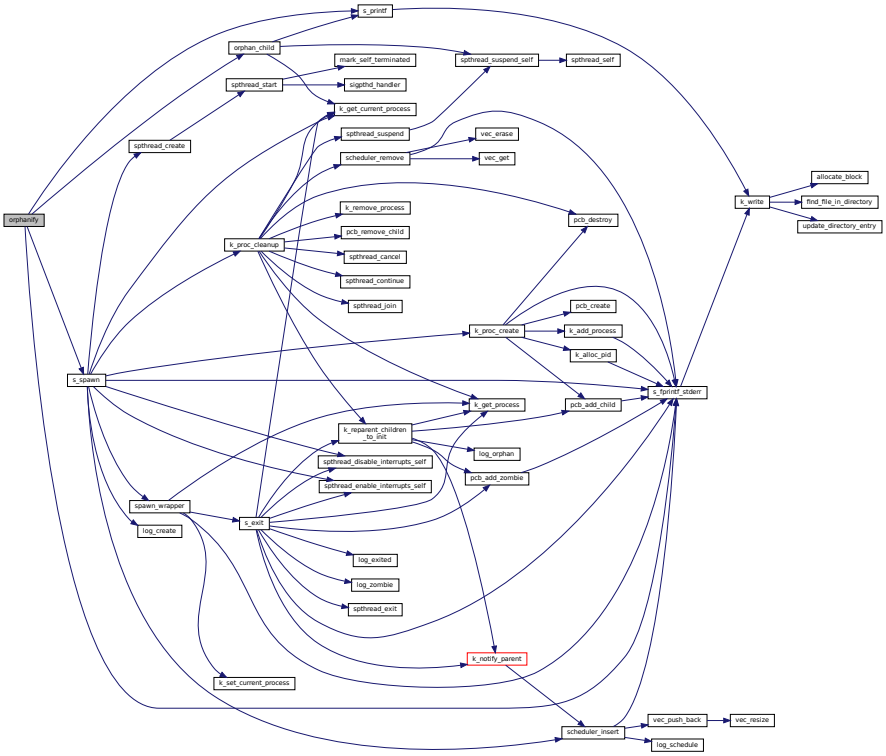
Example Usage: orphanify

Definition at line 144 of file user_processes.c.

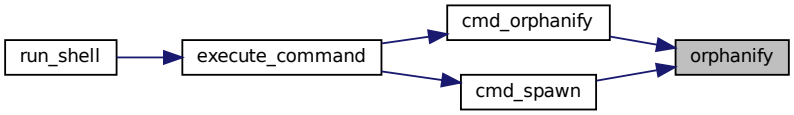
References orphan_child(), s_fprintf_stderr(), s_printf(), and s_spawn().

Referenced by cmd_orphanify(), and cmd_spawn().

Here is the call graph for this function:



Here is the caller graph for this function:



4.29.1.6 ps_process() `void* ps_process ( void * arg )`

Process that displays all running processes (ps command)

Prints a formatted table showing PID, PPID, priority, and state for all processes in the system.

Parameters

arg	Command-line arguments (unused)

Returns

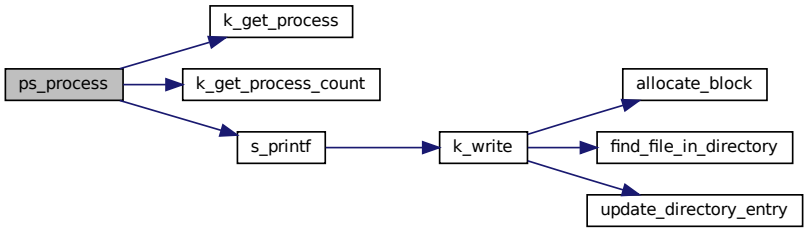
NULL

Definition at line 63 of file user_processes.c.

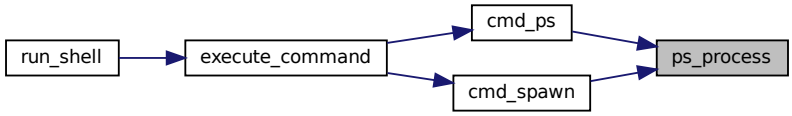
References BLOCKED, k_get_process(), k_get_process_count(), MAX_PROCESSES, pcb::name, pcb::pid, pcb::ppid, pcb::priority, RUNNING, s_printf(), pcb::state, STOPPED, and ZOMBIE.

Referenced by cmd_ps(), and cmd_spawn().

Here is the call graph for this function:



Here is the caller graph for this function:



4.29.1.7 user_busy() `void* user_busy ( void * arg )`

Busy-wait process for CPU stress testing.

Executes an infinite loop performing meaningless computation to consume CPU time. Used for testing scheduler behavior under load.

Parameters

arg	busy_args_t* containing process name and background flag

Returns

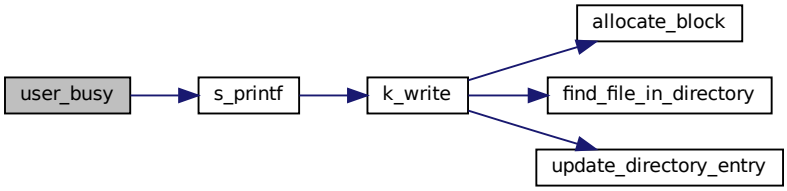
NULL (never returns normally)

Definition at line 247 of file user_processes.c.

References s_printf().

Referenced by execute_command().

Here is the call graph for this function:



Here is the caller graph for this function:



4.29.1.8 user_cat() `void* user_cat ( void * arg )`

User-space cat command - concatenate and display files.

Parameters

arg	Command-line arguments (filenames)

Returns

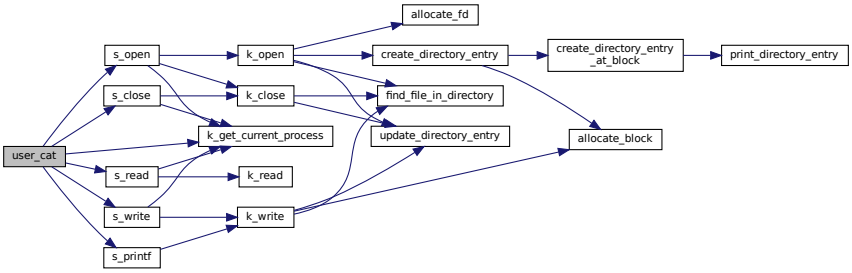
NULL

Definition at line 255 of file user_processes.c.

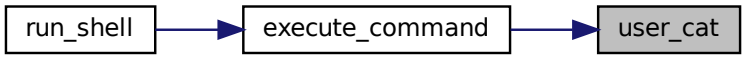
References F_APPEND, F_READ, F_WRITE, FILE_NOT_FOUND, FS_NOT_MOUNTED, k_get_current_process(), s↩ _close(), s_open(), s_printf(), s_read(), and s_write().

Referenced by execute_command().

Here is the call graph for this function:



Here is the caller graph for this function:



4.29.1.9 user_chmod() `void user_chmod ( char ** arg )`

User-space chmod command - change file permissions.

Parameters

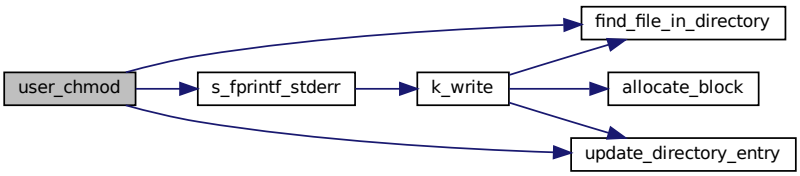
arg	Command-line arguments (permissions, filename)

Definition at line 818 of file user_processes.c.

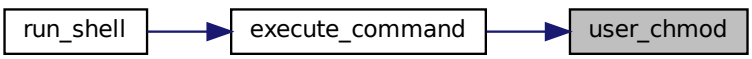
References find_file_in_directory(), directory_entry_t::perm, s_fprintf_stderr(), and update_directory_entry().

Referenced by execute_command().

Here is the call graph for this function:



Here is the caller graph for this function:



4.29.1.10 user_cp() `void* user_cp ( void * arg )`

User-space cp command - copy file.

Parameters

arg	Command-line arguments (source, destination)

Returns

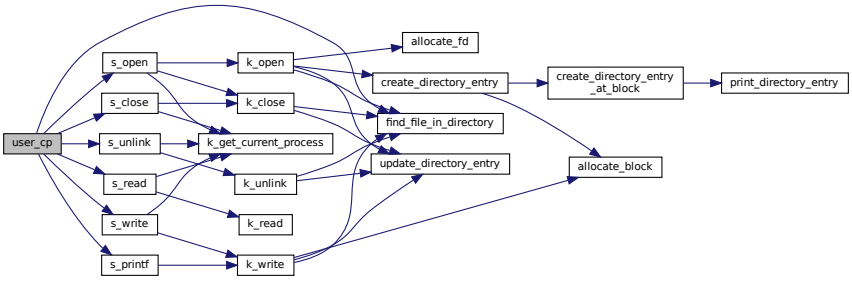
NULL

Definition at line 609 of file user_processes.c.

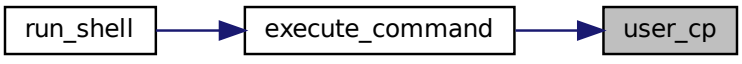
References F_READ, F_WRITE, FILE_NOT_FOUND, find_file_in_directory(), FS_NOT_MOUNTED, s_close(), s_open(), s_printf(), s_read(), s_unlink(), and s_write().

Referenced by execute_command().

Here is the call graph for this function:



Here is the caller graph for this function:



4.29.1.11 user_echo()

```
void* user_echo (
        void * arg )
```

User-space echo command - display text.

Parameters

arg	Command-line arguments (text to echo)

Returns

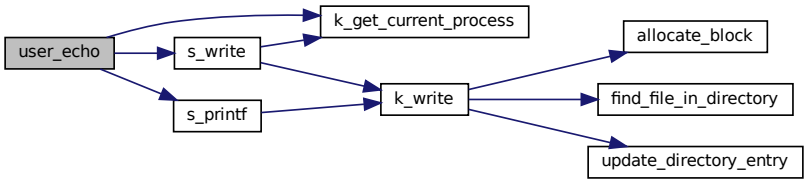
NULL

Definition at line 486 of file user_processes.c.

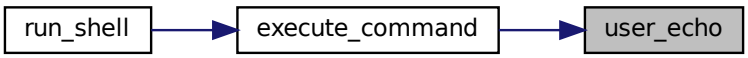
References pcb::fd_table, k_get_current_process(), s_printf(), and s_write().

Referenced by execute_command().

Here is the call graph for this function:



Here is the caller graph for this function:



4.29.1.12 user_ls() `void* user_ls ( void * arg )`

User-space ls command - list directory contents.

Parameters

arg	Command-line arguments (directory path, optional)

Returns

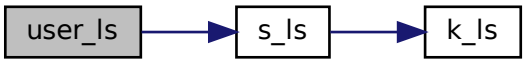
NULL

Definition at line 517 of file user_processes.c.

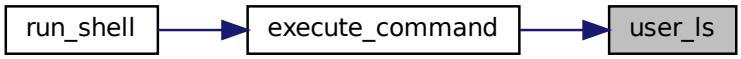
References s_ls().

Referenced by execute_command().

Here is the call graph for this function:



Here is the caller graph for this function:



4.29.1.13 user_man() `void* user_man ( void * arg )`

User-space man command - display command manual.

Parameters

arg	Command-line arguments (command name)

Returns

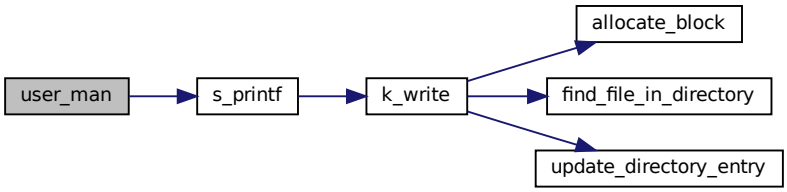
NULL

Definition at line 774 of file user_processes.c.

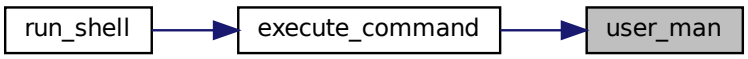
References s_printf().

Referenced by execute_command().

Here is the call graph for this function:



Here is the caller graph for this function:



4.29.1.14 user_mv() `void* user_mv ( void * arg )`

User-space mv command - move/rename file.

Parameters

arg	Command-line arguments (source, destination)

Returns

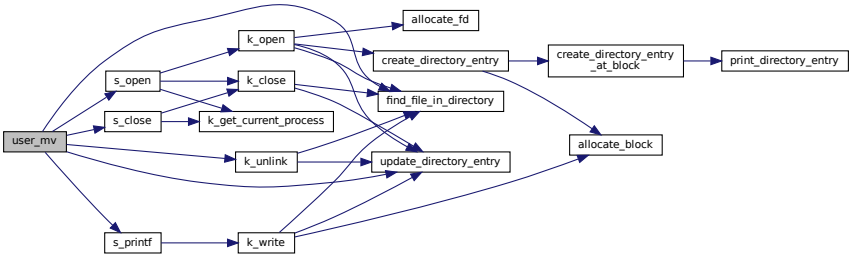
NULL

Definition at line 549 of file user_processes.c.

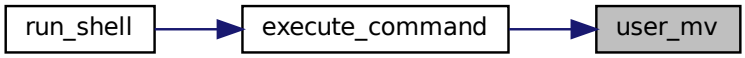
References F_READ, FILE_NOT_FOUND, find_file_in_directory(), FS_NOT_MOUNTED, k_unlink(), directory_entry↩
_t::name, s_close(), s_open(), s_printf(), and update_directory_entry().

Referenced by execute_command().

Here is the call graph for this function:



Here is the caller graph for this function:



4.29.1.15 user_rm() `void* user_rm ( void * arg )`

User-space rm command - remove file.

Parameters

arg	Command-line arguments (filename)

Returns

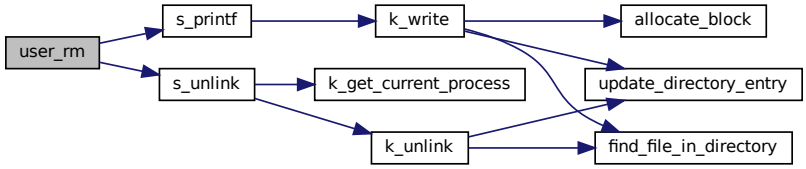
NULL

Definition at line 755 of file user_processes.c.

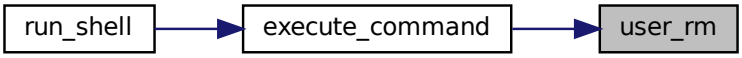
References FILE_NOT_FOUND, FS_NOT_MOUNTED, s_printf(), and s_unlink().

Referenced by execute_command().

Here is the call graph for this function:



Here is the caller graph for this function:



4.29.1.16 user_sleep() `void* user_sleep ( void * arg )`

Process that sleeps for a specified number of ticks.

Calls s_sleep() with the tick count specified in argv[1].

Parameters

arg	Command-line arguments (expects tick count as argv[1])

Returns

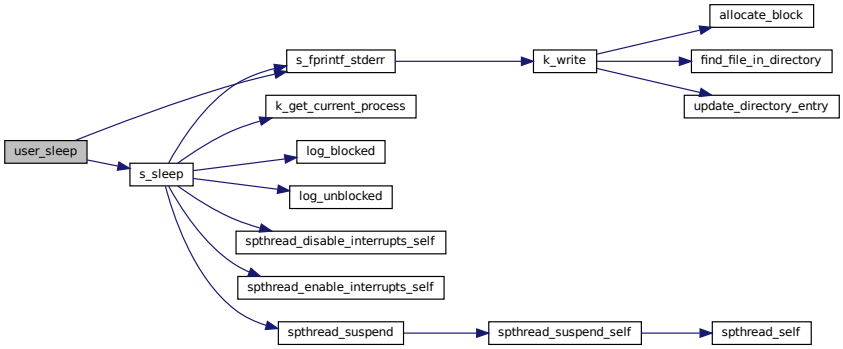
NULL

Definition at line 36 of file user_processes.c.

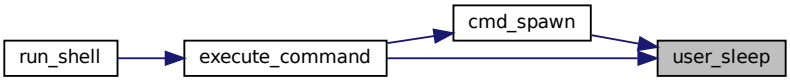
References s_fprintf_stderr(), and s_sleep().

Referenced by cmd_spawn(), and execute_command().

Here is the call graph for this function:



Here is the caller graph for this function:



4.29.1.17 user_touch() `void* user_touch ( void * arg )`

User-space touch command - create empty file.

Parameters

arg	Command-line arguments (filename)

Returns

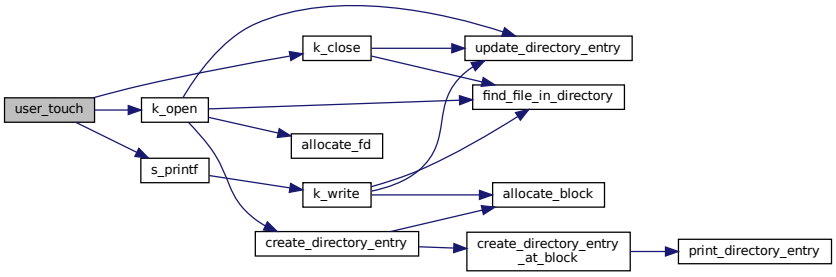
NULL

Definition at line 527 of file user_processes.c.

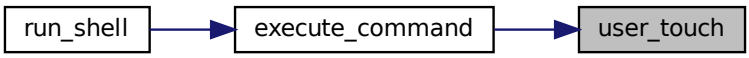
References F_APPEND, FS_NOT_MOUNTED, k_close(), k_open(), and s_printf().

Referenced by execute_command().

Here is the call graph for this function:



Here is the caller graph for this function:



4.29.1.18 zombie_process() `void* zombie_process ( void * arg )`

Test process that immediately exits to become a zombie.

Used for testing zombie process handling and s_waitpid().

Parameters

arg	Command-line arguments (unused)

Returns

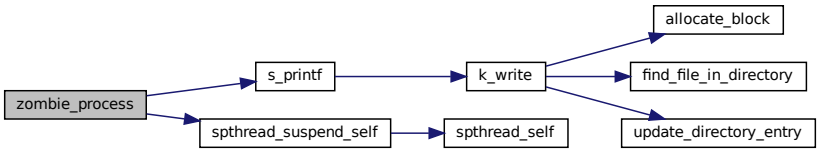
NULL

Definition at line 56 of file user_processes.c.

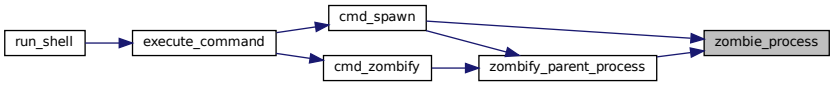
References s_printf(), and spthread_suspend_self().

Referenced by cmd_spawn(), and zombify_parent_process().

Here is the call graph for this function:



Here is the caller graph for this function:



4.29.1.19 zombify_parent_process() `void* zombify_parent_process ( void * arg )`

Process that creates a zombie child then exits (orphaning the zombie)

Used for testing orphaned zombie handling and INIT reparenting.

Parameters

arg	Command-line arguments (unused)

Returns

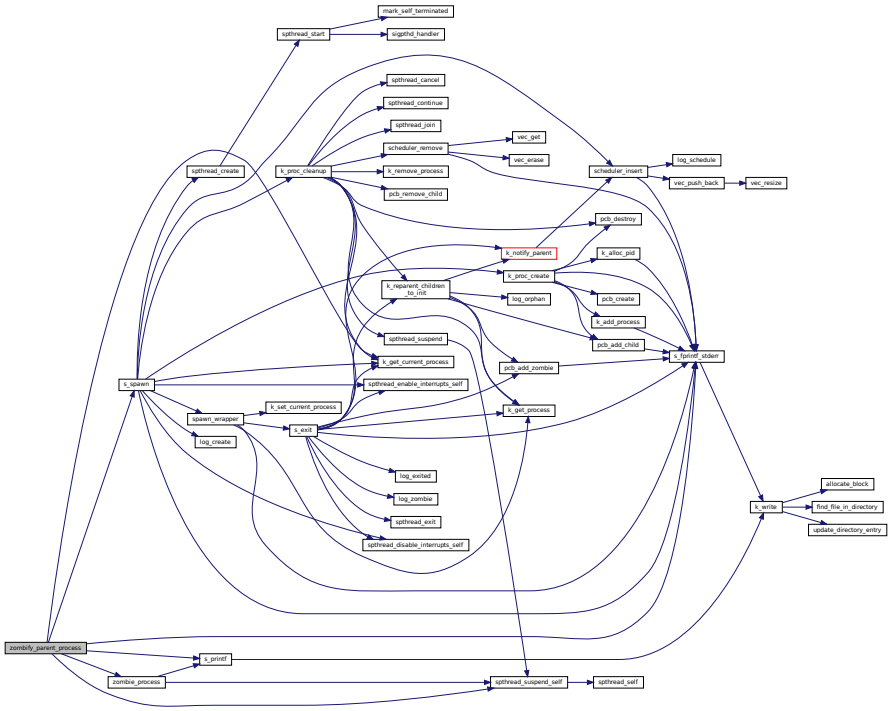
NULL

Definition at line 169 of file user_processes.c.

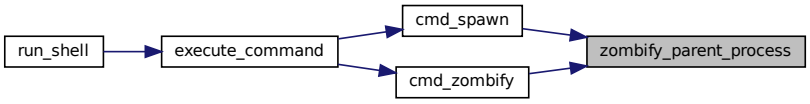
References k_get_current_process(), s_fprintf_stderr(), s_printf(), s_spawn(), spthread_suspend_self(), ZOMBIE, and zombie_process().

Referenced by cmd_spawn(), and cmd_zombify().

Here is the call graph for this function:



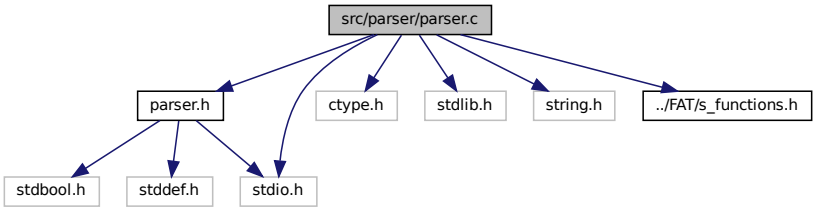
Here is the caller graph for this function:



4.30 src/parser/parser.c File Reference

```
#include "parser.h"
#include <ctype.h>
#include <stdlib.h>
#include <string.h>
#include "../FAT/s_functions.h"
#include <stdio.h>
```

Include dependency graph for parser.c:



Macros

- #define JUMP_OUT(code)

Functions

- static void skip_word (const char **const cur, const char *const end)
- static void skip_space (const char **const cur, const char *const end)
- int parse_command (const char *const cmd_line, struct parsed_command **const result)
- void print_parsed_command (const struct parsed_command *const cmd)
- void print_parser_errcode (FILE *output, int err_code)

4.30.1 Macro Definition Documentation

4.30.1.1 JUMP_OUT `#define JUMP_OUT( code )`

Value:

```
do {                    \
  ret_code = code;      \
  goto PROCESS_ERROR; \
} while (0)
```

4.30.2 Function Documentation

4.30.2.1 parse_command()

```
int parse_command (
    const char * cmd_line,
    struct parsed_command ** result )
```

Arguments: cmd_line: a null-terminated string that is the command line result: a non-null pointer to a `struct parsed_command *`

Return value (int): an error code which can be, 0: parser finished succesfully -1: parser encountered a system call error 1-7: parser specific error, see error type above

This function will parse the given `cmd_line` and store the parsed information into a `struct parsed_command`. The memory needed for the struct will be allocated by this function, and the pointer to the memory will be stored into the given `*result`.

You can directly use the result in system calls. See demo for more information.

If the function returns a successful value (0), a `struct parsed_command` is guareenteed to be allocated and stored in the given `*result`. It is the caller's responsibility to free the given pointer using `free(3)`.

Otherwise, no `struct parsed_command` is allocated and `*result` is unchanged. If a system call error (-1) is returned, the caller can use `errno(3)` or `perror(3)` to gain more information about the error. layout of memory for `struct parsed_command` bool is_background; bool is_file_append;

const char *stdin_file; const char *stdout_file;

size_t num_commands;

commands are pointers to `arguments` char **commands[num_commands];

below are hidden in memory **

arguments are pointers to `original_string + num_commands` because all argv are null-terminated char *arguments[total_strings + num_commands];

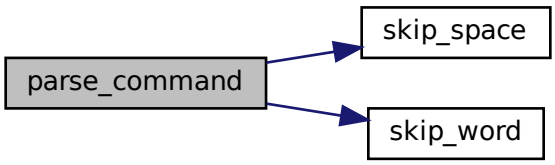
original_string is a copy of the cmdline but with each token null-terminated char *original_string;

Definition at line 19 of file parser.c.

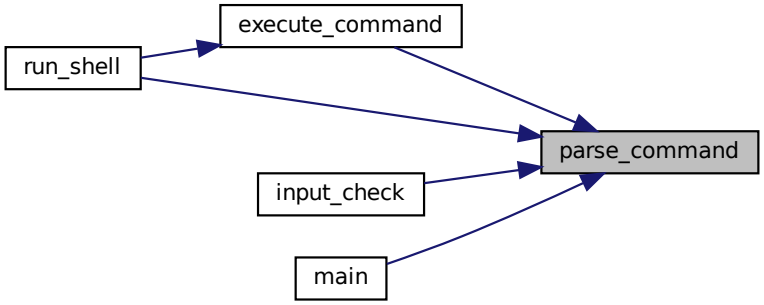
References parsed_command::commands, EXPECT_COMMANDS, EXPECT_INPUT_FILENAME, EXPECT_↩ OUTPUT_FILENAME, parsed_command::is_background, parsed_command::is_file_append, JUMP_OUT, parsed_↩ command::num_commands, skip_space(), skip_word(), parsed_command::stdin_file, parsed_command::stdout_file, UNEXPECTED_AMPERSAND, UNEXPECTED_FILE_INPUT, UNEXPECTED_FILE_OUTPUT, and UNEXPECTED↩ _PIPELINE.

Referenced by execute_command(), input_check(), main(), and run_shell().

Here is the call graph for this function:



Here is the caller graph for this function:

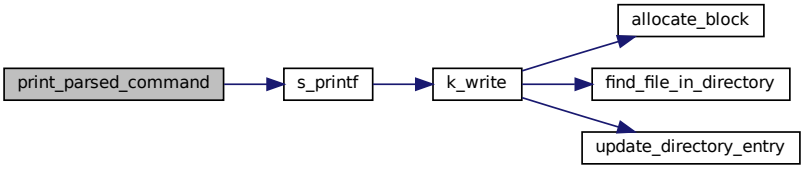


4.30.2.2 print_parsed_command() `void print_parsed_command ( const struct parsed_command *const cmd )`

Definition at line 238 of file parser.c.

References parsed_command::commands, parsed_command::is_background, parsed_command::is_file_append, parsed_command::num_commands, s_printf(), parsed_command::stdin_file, and parsed_command::stdout_file.

Here is the call graph for this function:



4.30.2.3 print_parser_errcode()

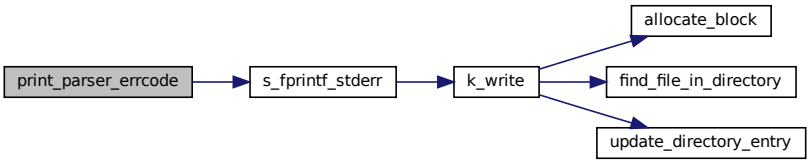
```
void print_parser_errcode (
        FILE * output,
        int err_code )
```

Definition at line 255 of file parser.c.

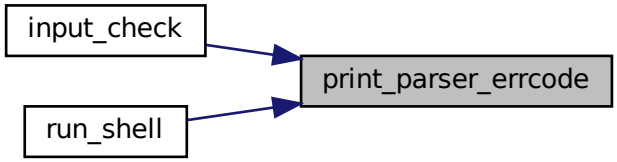
References EXPECT_COMMANDS, EXPECT_INPUT_FILENAME, EXPECT_OUTPUT_FILENAME, s_fprintf_stderr(), UNEXPECTED_AMPERSAND, UNEXPECTED_FILE_INPUT, UNEXPECTED_FILE_OUTPUT, and UNEXPECTED↩ _PIPELINE.

Referenced by input_check(), and run_shell().

Here is the call graph for this function:



Here is the caller graph for this function:



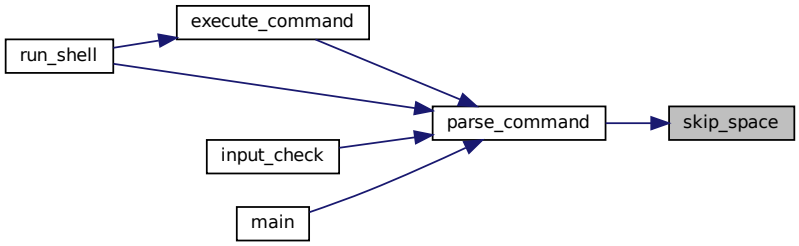
4.30.2.4 skip_space()

```
static void skip_space (
        const char **const cur,
        const char *const end )  [inline], [static]
```

Definition at line 14 of file parser.c.

Referenced by parse_command().

Here is the caller graph for this function:



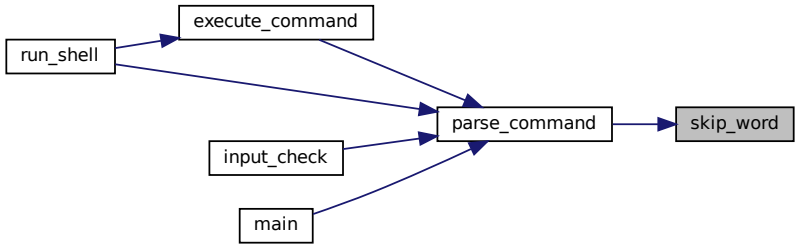
4.30.2.5 skip_word() `static void skip_word (`

```
const char **const cur,
const char *const end )  [inline], [static]
```

Definition at line 8 of file parser.c.

Referenced by parse_command().

Here is the caller graph for this function:

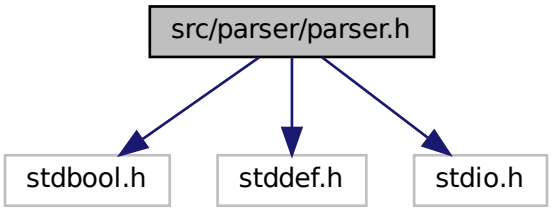


4.31 src/parser/parser.h File Reference

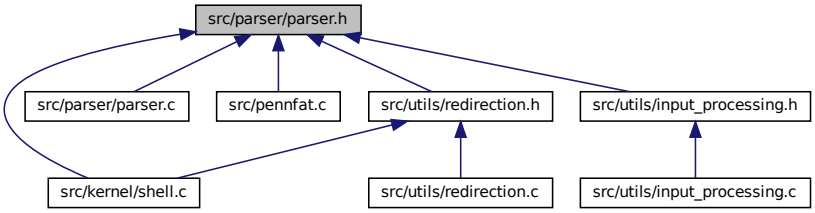
```
#include <stdbool.h>
#include <stddef.h>
```

```
#include <stdio.h>
```

Include dependency graph for parser.h:



This graph shows which files directly or indirectly include this file:



Data Structures

- struct parsed_command

Macros

- #define UNEXPECTED_FILE_INPUT 1
- #define UNEXPECTED_FILE_OUTPUT 2
- #define UNEXPECTED_PIPELINE 3
- #define UNEXPECTED_AMPERSAND 4
- #define EXPECT_INPUT_FILENAME 5
- #define EXPECT_OUTPUT_FILENAME 6
- #define EXPECT_COMMANDS 7

Functions

- int parse_command (const char ∗cmd_line, struct parsed_command ∗∗result)
- void print_parsed_command (const struct parsed_command ∗cmd)
- void print_parser_errcode (FILE ∗output, int err_code)

4.31.1 Macro Definition Documentation

4.31.1.1 EXPECT_COMMANDS `#define EXPECT_COMMANDS 7`

Definition at line 30 of file parser.h.

4.31.1.2 EXPECT_INPUT_FILENAME `#define EXPECT_INPUT_FILENAME 5`

Definition at line 24 of file parser.h.

4.31.1.3 EXPECT_OUTPUT_FILENAME `#define EXPECT_OUTPUT_FILENAME 6`

Definition at line 27 of file parser.h.

4.31.1.4 UNEXPECTED_AMPERSAND `#define UNEXPECTED_AMPERSAND 4`

Definition at line 21 of file parser.h.

4.31.1.5 UNEXPECTED_FILE_INPUT `#define UNEXPECTED_FILE_INPUT 1`

Definition at line 12 of file parser.h.

4.31.1.6 UNEXPECTED_FILE_OUTPUT `#define UNEXPECTED_FILE_OUTPUT 2`

Definition at line 15 of file parser.h.

4.31.1.7 UNEXPECTED_PIPELINE `#define UNEXPECTED_PIPELINE 3`

Definition at line 18 of file parser.h.

4.31.2 Function Documentation

4.31.2.1 parse_command()

```
int parse_command (
    const char * cmd_line,
    struct parsed_command ** result )
```

Arguments: cmd_line: a null-terminated string that is the command line result: a non-null pointer to a `struct parsed_command *`

Return value (int): an error code which can be, 0: parser finished succesfully -1: parser encountered a system call error 1-7: parser specific error, see error type above

This function will parse the given `cmd_line` and store the parsed information into a `struct parsed_command`. The memory needed for the struct will be allocated by this function, and the pointer to the memory will be stored into the given `*result`.

You can directly use the result in system calls. See demo for more information.

If the function returns a successful value (0), a `struct parsed_command` is guareenteed to be allocated and stored in the given `*result`. It is the caller's responsibility to free the given pointer using `free(3)`.

Otherwise, no `struct parsed_command` is allocated and `*result` is unchanged. If a system call error (-1) is returned, the caller can use `errno(3)` or `perror(3)` to gain more information about the error. layout of memory for `struct parsed_command` bool is_background; bool is_file_append;

const char *stdin_file; const char *stdout_file;

size_t num_commands;

commands are pointers to `arguments` char **commands[num_commands];

below are hidden in memory **

arguments are pointers to `original_string + num_commands` because all argv are null-terminated char *arguments[total_strings + num_commands];

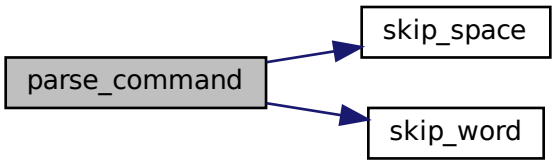
original_string is a copy of the cmdline but with each token null-terminated char *original_string;

Definition at line 19 of file parser.c.

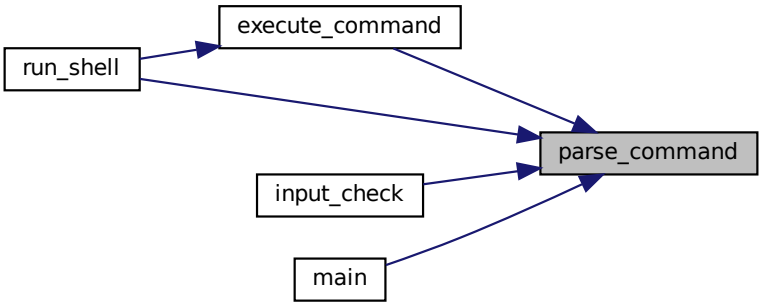
References parsed_command::commands, EXPECT_COMMANDS, EXPECT_INPUT_FILENAME, EXPECT_↩ OUTPUT_FILENAME, parsed_command::is_background, parsed_command::is_file_append, JUMP_OUT, parsed_↩ command::num_commands, skip_space(), skip_word(), parsed_command::stdin_file, parsed_command::stdout_file, UNEXPECTED_AMPERSAND, UNEXPECTED_FILE_INPUT, UNEXPECTED_FILE_OUTPUT, and UNEXPECTED↩ _PIPELINE.

Referenced by execute_command(), input_check(), main(), and run_shell().

Here is the call graph for this function:



Here is the caller graph for this function:



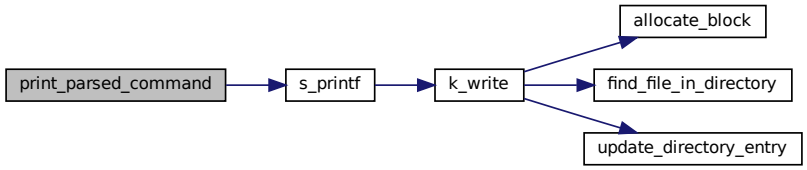
4.31.2.2 print_parsed_command()

```
void print_parsed_command (
        const struct parsed_command * cmd )
```

Definition at line 238 of file parser.c.

References parsed_command::commands, parsed_command::is_background, parsed_command::is_file_append, parsed_command::num_commands, s_printf(), parsed_command::stdin_file, and parsed_command::stdout_file.

Here is the call graph for this function:



4.31.2.3 print_parser_errcode()

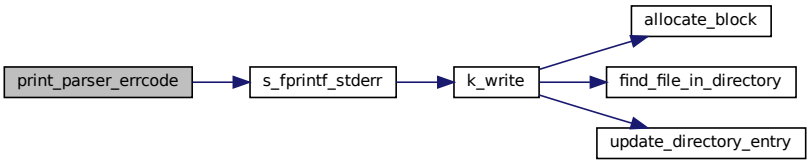
```
void print_parser_errcode (
    FILE * output,
    int err_code )
```

Definition at line 255 of file parser.c.

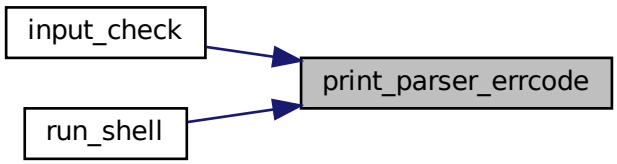
References EXPECT_COMMANDS, EXPECT_INPUT_FILENAME, EXPECT_OUTPUT_FILENAME, s_fprintf_stderr(), UNEXPECTED_AMPERSAND, UNEXPECTED_FILE_INPUT, UNEXPECTED_FILE_OUTPUT, and UNEXPECTED↩ _PIPELINE.

Referenced by input_check(), and run_shell().

Here is the call graph for this function:



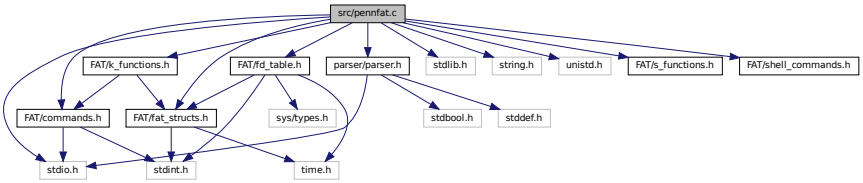
Here is the caller graph for this function:



4.32 src/pennfat.c File Reference

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include "FAT/commands.h"
#include "FAT/fat_structs.h"
#include "FAT/fd_table.h"
#include "FAT/k_functions.h"
#include "FAT/s_functions.h"
#include "FAT/shell_commands.h"
#include "parser/parser.h"
```

Include dependency graph for pennfat.c:



Macros

- #define MAX_LINE_LENGTH 200
- #define PROMPT "pennfat> "

Functions

- int main (int argc, char *argv[ ])

4.32.1 Macro Definition Documentation

4.32.1.1 MAX_LINE_LENGTH `#define MAX_LINE_LENGTH 200`

Definition at line 16 of file pennfat.c.

4.32.1.2 PROMPT `#define PROMPT "pennfat> "`

Definition at line 17 of file pennfat.c.

4.32.2 Function Documentation

4.32.2.1 main()

```
int main (
        int argc,
        char * argv[] )
```

< Write mode - truncate file if exists, create if not

< Read mode - file must exist

< Requested file does not exist

< Requested file does not exist

< Write mode - truncate file if exists, create if not

< File is already open

< File is already open

< Append mode - create if not exists, seek to end

< Read mode - file must exist

< Write mode - truncate file if exists, create if not

< Write mode - truncate file if exists, create if not

< Read mode - file must exist

< Write mode - truncate file if exists, create if not

< Read mode - file must exist

< Read mode - file must exist

< Seek from beginning of file

< Append mode - create if not exists, seek to end

< Read mode - file must exist

< Write mode - truncate file if exists, create if not

< Read mode - file must exist

< Write mode - truncate file if exists, create if not

< Read mode - file must exist

< Write mode - truncate file if exists, create if not

< Write mode - truncate file if exists, create if not

< Read mode - file must exist

< Write mode - truncate file if exists, create if not

< Read mode - file must exist

< Write mode - truncate file if exists, create if not

< Write mode - truncate file if exists, create if not

< Write mode - truncate file if exists, create if not

< Write mode - truncate file if exists, create if not

< Write mode - truncate file if exists, create if not

< Write mode - truncate file if exists, create if not

< Write mode - truncate file if exists, create if not

< Read mode - file must exist

< Requested file does not exist

< Write mode - truncate file if exists, create if not

< Read mode - file must exist

< Write mode - truncate file if exists, create if not

< Write mode - truncate file if exists, create if not

< Write mode - truncate file if exists, create if not

< Read mode - file must exist

< Read mode - file must exist

< Write mode - truncate file if exists, create if not

< Read mode - file must exist

< Write mode - truncate file if exists, create if not

< Write mode - truncate file if exists, create if not

< Read mode - file must exist

< Write mode - truncate file if exists, create if not

< Write mode - truncate file if exists, create if not

< Read mode - file must exist

< Read mode - file must exist

< Write mode - truncate file if exists, create if not

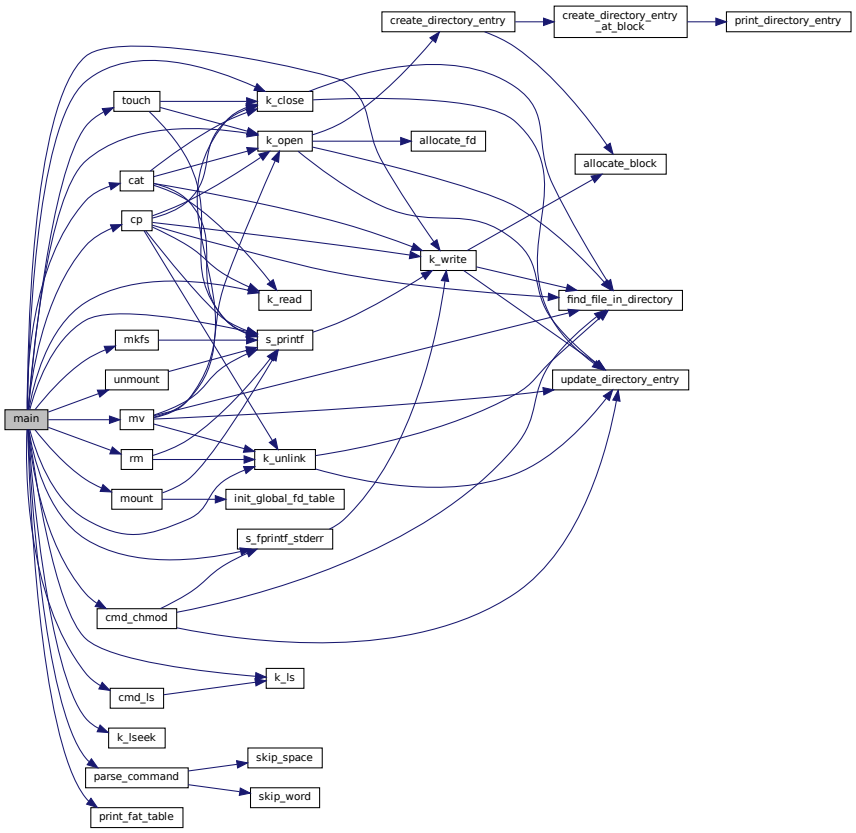
< Write mode - truncate file if exists, create if not

< Write mode - truncate file if exists, create if not

Definition at line 19 of file pennfat.c.

References _current_mount, cat(), cmd_chmod(), cmd_ls(), parsed_command::commands, cp(), F_APPEND, F_READ, F_SEEK_SET, F_WRITE, FILE_ALREADY_OPEN, FILE_NOT_FOUND, global_fd_table, k_close(), k_ls(), k_lseek(), k_open(), k_read(), k_unlink(), k_write(), MAX_LINE_LENGTH, mkfs(), mount(), mv(), parse_command(), print_fat_↩ table(), PROMPT, rm(), s_fprintf_stderr(), s_printf(), touch(), and unmount().

Here is the call graph for this function:

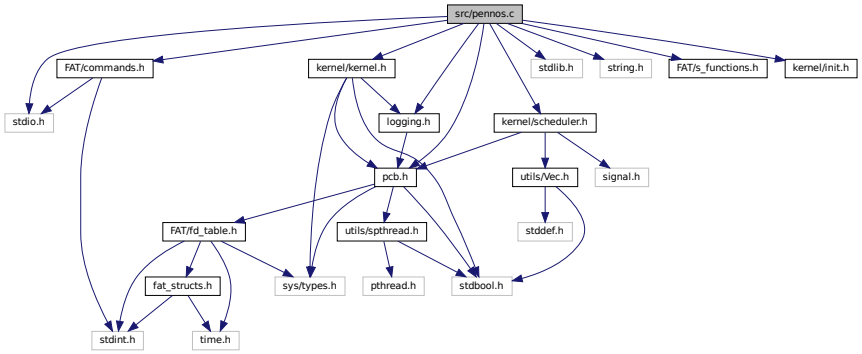


4.33 src/pennos.c File Reference

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
```

```
#include "FAT/commands.h"
#include "FAT/s_functions.h"
#include "kernel/init.h"
#include "kernel/kernel.h"
#include "kernel/logging.h"
#include "kernel/pcb.h"
#include "kernel/scheduler.h"
```

Include dependency graph for pennos.c:



Functions

- int main (int argc, char ∗argv[ ])

Variables

- static pcb_t ∗ init_process = NULL

4.33.1 Function Documentation

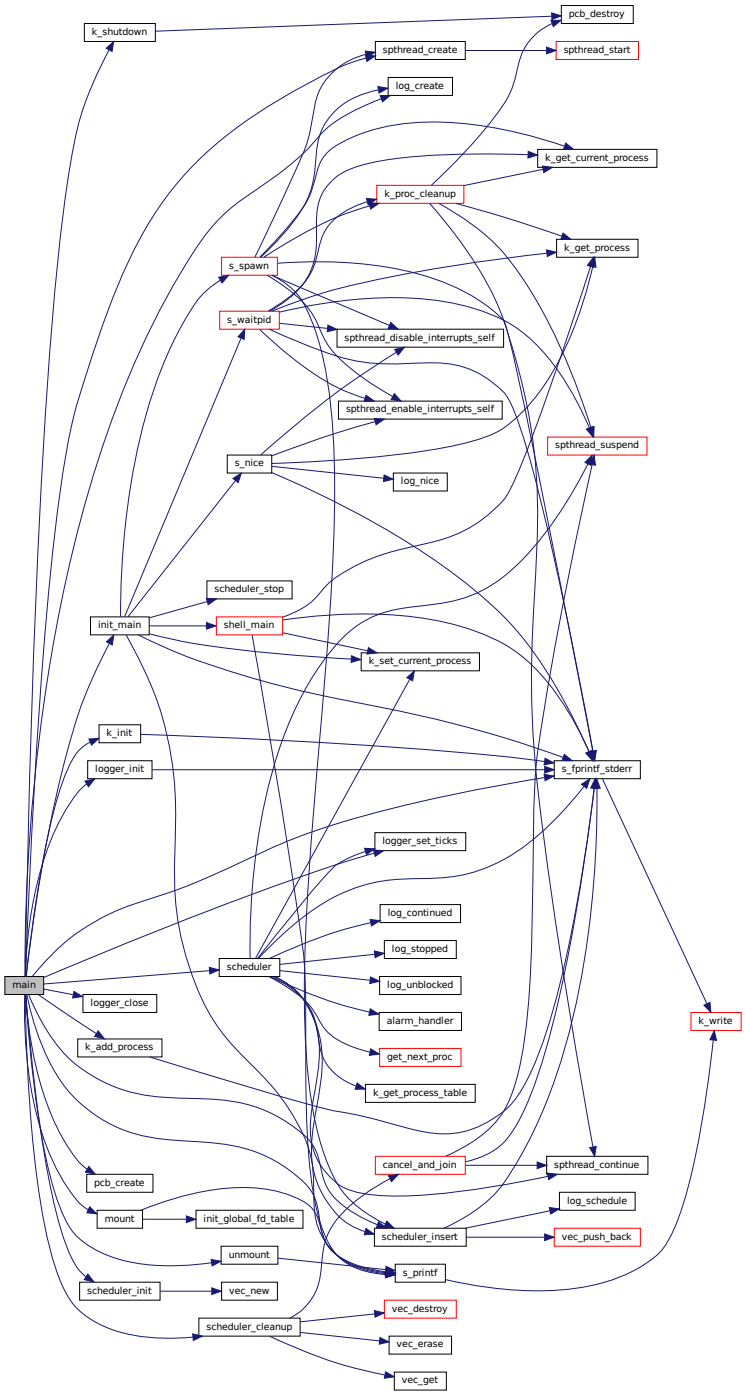
4.33.1.1 main()

```
int main (
        int argc,
        char * argv[] )
```

Definition at line 14 of file pennos.c.

References init_main(), init_process, k_add_process(), k_init(), k_shutdown(), log_create(), logger_close(), logger↩ _init(), logger_set_ticks(), mount(), pcb::name, pcb_create(), pcb::pid, pcb::priority, RUNNING, s_fprintf_stderr(), s↩ _printf(), scheduler(), scheduler_cleanup(), scheduler_init(), scheduler_insert(), spthread_create(), pcb::state, pcb↩ ::thread, pcb::thread_created, and unmount().

Here is the call graph for this function:



4.33.2 Variable Documentation

4.33.2.1 init_process `pcb_t* init_process = NULL [static]`

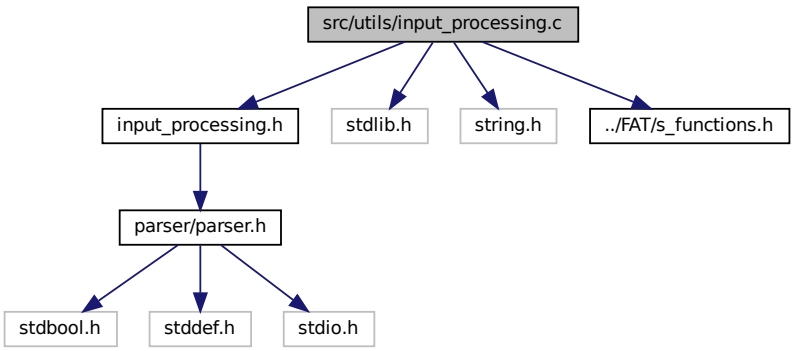
Definition at line 12 of file pennos.c.

Referenced by main().

4.34 src/utils/input_processing.c File Reference

```
#include "input_processing.h"
#include <stdlib.h>
#include <string.h>
#include "../FAT/s_functions.h"
```

Include dependency graph for input_processing.c:



Macros

- #define _GNU_SOURCE

Functions

- bool input_check (struct parsed_command **cmd, char *input)

  Helper function for parsing command. It calls parse_command() in parser.c with given input. It stores the parsed command to given pointer to cmd. Then it error checks if parse_command() is done successfully.

4.34.1 Macro Definition Documentation

4.34.1.1 _GNU_SOURCE `#define _GNU_SOURCE`

Definition at line 1 of file input_processing.c.

4.34.2 Function Documentation

4.34.2.1 input_check() `bool input_check ( struct parsed_command ** cmd, char * input )`

Helper function for parsing command. It calls parse_command() in parser.c with given input. It stores the parsed command to given pointer to cmd. Then it error checks if parse_command() is done successfully.

Parameters

cmd	pointer to struct parsed_command*
input	input string that needs to be parsed

Returns

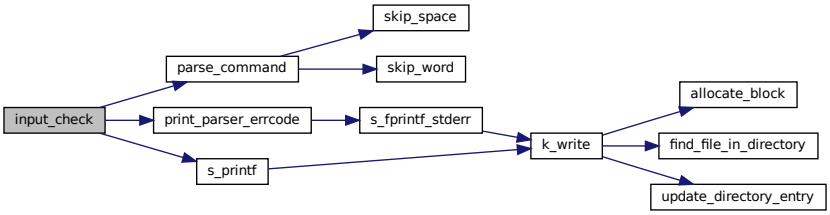
true if command parsed successfully without error

false if there was error

Definition at line 7 of file input_processing.c.

References parse_command(), print_parser_errcode(), s_printf(), UNEXPECTED_FILE_INPUT, and UNEXPECTED↩ _FILE_OUTPUT.

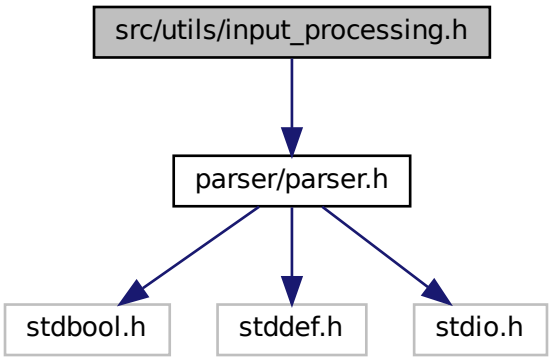
Here is the call graph for this function:



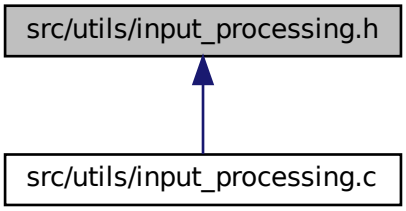
4.35 src/utils/input_processing.h File Reference

```
#include "parser/parser.h"
```

Include dependency graph for input_processing.h:



This graph shows which files directly or indirectly include this file:



Functions

- bool input_check (struct parsed_command **cmd, char *input)

  Helper function for parsing command. It calls parse_command() in parser.c with given input. It stores the parsed command to given pointer to cmd. Then it error checks if parse_command() is done successfully.

4.35.1 Function Documentation

4.35.1.1 input_check()

```
bool input_check (
        struct parsed_command ** cmd,
        char * input )
```

Helper function for parsing command. It calls parse_command() in parser.c with given input. It stores the parsed command to given pointer to cmd. Then it error checks if parse_command() is done successfully.

Parameters

cmd	pointer to struct parsed_command*
input	input string that needs to be parsed

Returns

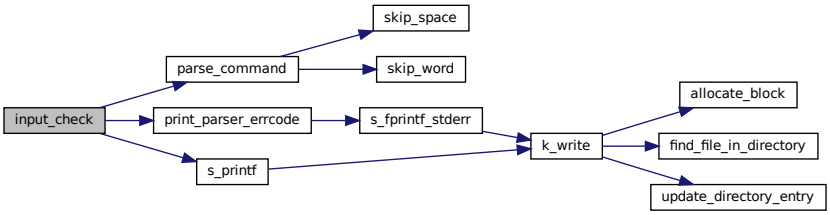
true if command parsed successfully without error

false if there was error

Definition at line 7 of file input_processing.c.

References parse_command(), print_parser_errcode(), s_printf(), UNEXPECTED_FILE_INPUT, and UNEXPECTED↩ _FILE_OUTPUT.

Here is the call graph for this function:

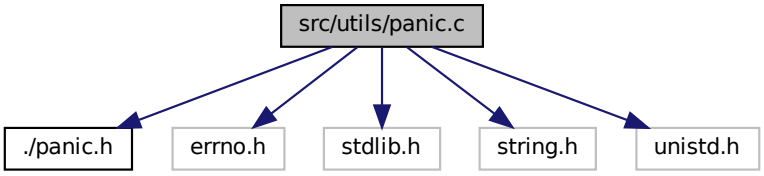


4.36 src/utils/panic.c File Reference

```
#include "./panic.h"
#include <errno.h>
#include <stdlib.h>
#include <string.h>
```

```
#include <unistd.h>
```

Include dependency graph for panic.c:



Functions

- void print_and_abort (const char *error_message)

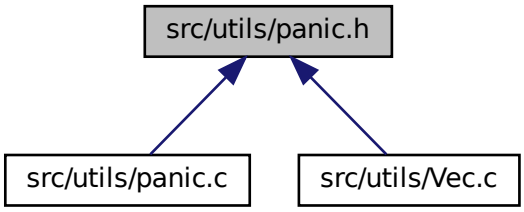
4.36.1 Function Documentation

4.36.1.1 print_and_abort() `void print_and_abort ( const char * error_message )`

Definition at line 7 of file panic.c.

4.37 src/utils/panic.h File Reference

This graph shows which files directly or indirectly include this file:



Macros

- #define panic(error_message) print_and_abort(error_message)

Functions

- void print_and_abort (const char *error_message)

4.37.1 Macro Definition Documentation

4.37.1.1 panic `#define panic( error_message ) print_and_abort(error_message)`

Definition at line 13 of file panic.h.

4.37.2 Function Documentation

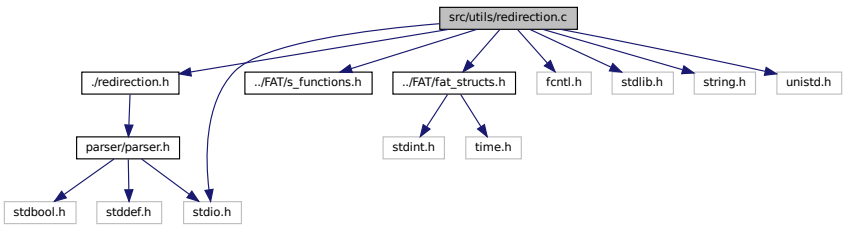
4.37.2.1 print_and_abort() `void print_and_abort ( const char * error_message )`

Definition at line 7 of file panic.c.

4.38 src/utils/redirection.c File Reference

```
#include "./redirection.h"
#include "../FAT/s_functions.h"
#include "../FAT/fat_structs.h"
#include <fcntl.h>
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
```

Include dependency graph for redirection.c:



Functions

- void get_redirected_fds (struct parsed_command ∗cmd, int ∗fd_in, int ∗fd_out)

  Get redirected file descriptors for stdin/stdout Opens files specified in redirection operators and returns their file descriptors.

4.38.1 Function Documentation

4.38.1.1 get_redirected_fds()

```
void get_redirected_fds (
        struct parsed_command * cmd,
        int * fd_in,
        int * fd_out )
```

Get redirected file descriptors for stdin/stdout Opens files specified in redirection operators and returns their file descriptors.

Parameters

cmd	parsed command with redirection info
fd_in	pointer to store input file descriptor
fd_out	pointer to store output file descriptor

< Write mode - truncate file if exists, create if not

< Read mode - file must exist

< Read mode - file must exist

< Append mode - create if not exists, seek to end

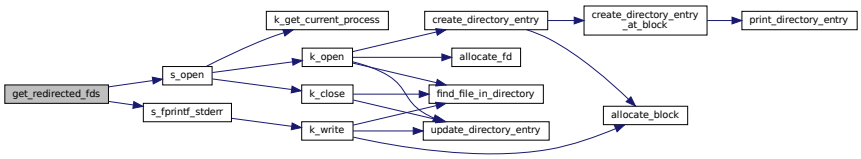
< Write mode - truncate file if exists, create if not

Definition at line 11 of file redirection.c.

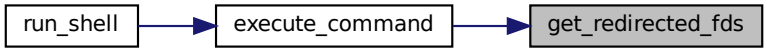
References F_APPEND, F_READ, F_WRITE, parsed_command::is_file_append, s_fprintf_stderr(), s_open(), parsed↩ _command::stdin_file, and parsed_command::stdout_file.

Referenced by execute_command().

Here is the call graph for this function:



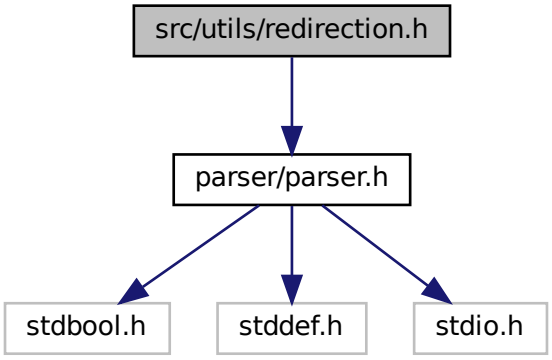
Here is the caller graph for this function:



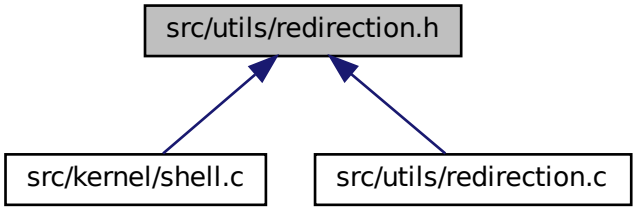
4.39 src/utils/redirection.h File Reference

```
#include "parser/parser.h"
```

Include dependency graph for redirection.h:



This graph shows which files directly or indirectly include this file:



Functions

- void get_redirected_fds (struct parsed_command ∗cmd, int ∗fd_in, int ∗fd_out)

  Get redirected file descriptors for stdin/stdout Opens files specified in redirection operators and returns their file descriptors.

4.39.1 Function Documentation

4.39.1.1 get_redirected_fds()

```
void get_redirected_fds (
        struct parsed_command * cmd,
        int * fd_in,
        int * fd_out )
```

Get redirected file descriptors for stdin/stdout Opens files specified in redirection operators and returns their file descriptors.

Parameters

cmd	parsed command with redirection info
fd_in	pointer to store input file descriptor
fd_out	pointer to store output file descriptor

< Write mode - truncate file if exists, create if not

< Read mode - file must exist

< Read mode - file must exist

< Append mode - create if not exists, seek to end

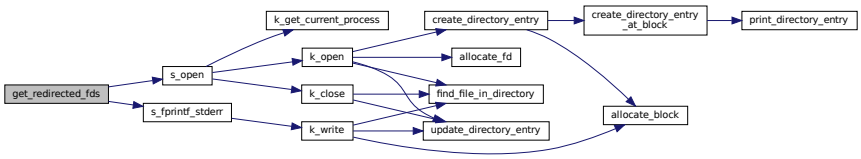
< Write mode - truncate file if exists, create if not

Definition at line 11 of file redirection.c.

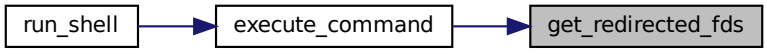
References F_APPEND, F_READ, F_WRITE, parsed_command::is_file_append, s_fprintf_stderr(), s_open(), parsed↩ _command::stdin_file, and parsed_command::stdout_file.

Referenced by execute_command().

Here is the call graph for this function:



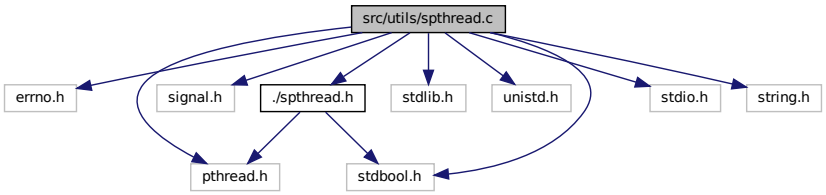
Here is the caller graph for this function:



4.40 src/utils/spthread.c File Reference

```
#include <errno.h>
#include <pthread.h>
#include <signal.h>
#include <stdbool.h>
#include <stdlib.h>
#include <unistd.h>
#include "./spthread.h"
#include <stdio.h>
#include <string.h>
```

Include dependency graph for spthread.c:



Data Structures

- struct spthread_fwd_args_st
- struct spthread_signal_args_st
- struct spthread_meta_st

Macros

- #define _GNU_SOURCE
- #define MILISEC_IN_NANO 100000
- #define SI_TKILL -6
- #define SPTHREAD_RUNNING_STATE 0
- #define SPTHREAD_SUSPENDED_STATE 1
- #define SPTHREAD_TERMINATED_STATE 2
- #define SPTHREAD_SIG_SUSPEND -1
- #define SPTHREAD_SIG_CONTINUE -2

Typedefs

- typedef void *(* pthread_fn) (void *)
- typedef struct spthread_fwd_args_st spthread_fwd_args
- typedef struct spthread_signal_args_st spthread_signal_args
- typedef struct spthread_meta_st spthread_meta_t

Functions

- static void sigpthd_handler (int signum, siginfo_t *, void *)
- static void * spthread_start (void *arg)
- static void mark_self_terminated (void *arg)
- int spthread_create (spthread_t *thread, const pthread_attr_t *attr, pthread_fn start_routine, void *arg)
- int spthread_suspend (spthread_t thread)
- int spthread_suspend_self ()
- int spthread_continue (spthread_t thread)
- int spthread_cancel (spthread_t thread)
- bool spthread_self (spthread_t *thread)
- int spthread_join (spthread_t thread, void **retval)
- void spthread_exit (void *status)
- bool spthread_equal (spthread_t first, spthread_t second)
- int spthread_disable_interrupts_self ()
- int spthread_enable_interrupts_self ()

Variables

- static _Thread_local spthread_meta_t * my_meta = NULL

4.40.1 Macro Definition Documentation

4.40.1.1 _GNU_SOURCE `#define _GNU_SOURCE`

Definition at line 1 of file spthread.c.

4.40.1.2 MILISEC_IN_NANO `#define MILISEC_IN_NANO 100000`

Definition at line 12 of file spthread.c.

4.40.1.3 SI_TKILL `#define SI_TKILL -6`

Definition at line 15 of file spthread.c.

4.40.1.4 SPTHREAD_RUNNING_STATE `#define SPTHREAD_RUNNING_STATE 0`

Definition at line 80 of file spthread.c.

4.40.1.5 SPTHREAD_SIG_CONTINUE `#define SPTHREAD_SIG_CONTINUE -2`

Definition at line 89 of file spthread.c.

4.40.1.6 SPTHREAD_SIG_SUSPEND `#define SPTHREAD_SIG_SUSPEND -1`

Definition at line 88 of file spthread.c.

4.40.1.7 SPTHREAD_SUSPENDED_STATE `#define SPTHREAD_SUSPENDED_STATE 1`

Definition at line 81 of file spthread.c.

4.40.1.8 SPTHREAD_TERMINATED_STATE `#define SPTHREAD_TERMINATED_STATE 2`

Definition at line 82 of file spthread.c.

4.40.2 Typedef Documentation

4.40.2.1 pthread_fn `typedef void*(* pthread_fn) (void *)`

Definition at line 24 of file spthread.c.

4.40.2.2 spthread_fwd_args typedef struct spthread_fwd_args_st spthread_fwd_args

4.40.2.3 spthread_meta_t typedef struct spthread_meta_st spthread_meta_t

4.40.2.4 spthread_signal_args typedef struct spthread_signal_args_st spthread_signal_args

4.40.3 Function Documentation

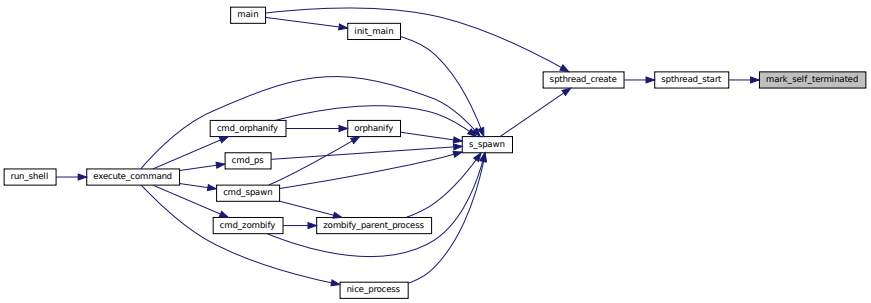
4.40.3.1 mark_self_terminated() static void mark_self_terminated (void * *arg*) [static]

Definition at line 467 of file spthread.c.

References my_meta, SIGPTHD, SPTHREAD_TERMINATED_STATE, and spthread_meta_st::state.

Referenced by spthread_start().

Here is the caller graph for this function:



4.40.3.2 sigpthd_handler()

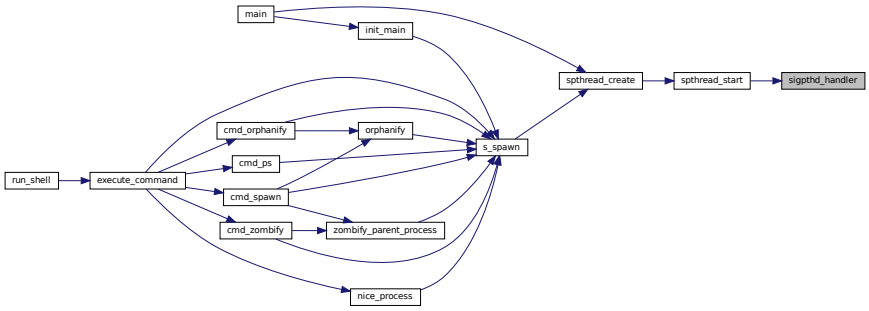
```
static void sigpthd_handler (
        int signum,
        siginfo_t * info,
        void * ucontext )  [static]
```

Definition at line 370 of file spthread.c.

References spthread_signal_args_st::ack, my_meta, spthread_signal_args_st::shutup_mutex, SI_TKILL, spthread_signal_args_st::signal, SIGPTHD, SPTHREAD_RUNNING_STATE, SPTHREAD_SIG_CONTINUE, SPTHREAD_SIG_SUSPEND, SPTHREAD_SUSPENDED_STATE, spthread_meta_st::state, and spthread_meta_st::suspend_set.

Referenced by spthread_start().

Here is the caller graph for this function:



4.40.3.3 spthread_cancel()

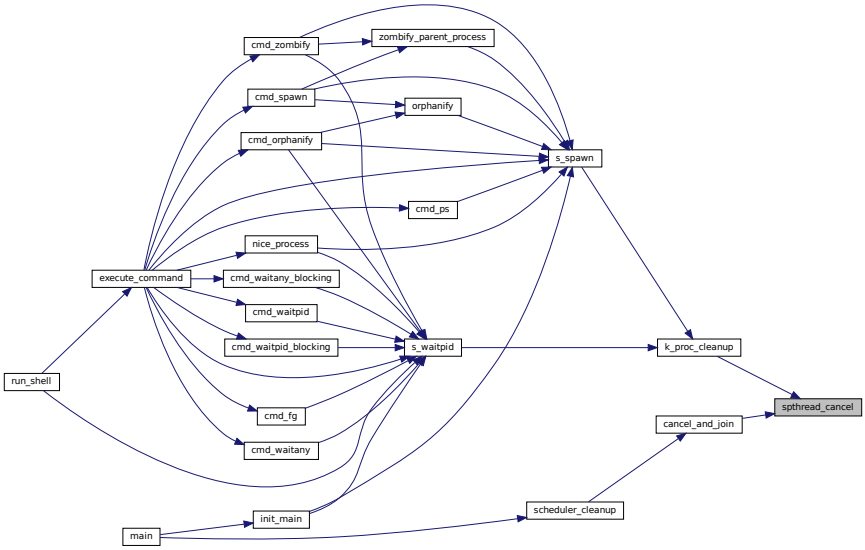
```
int spthread_cancel (
        spthread_t thread )
```

Definition at line 294 of file spthread.c.

References spthread_st::thread.

Referenced by cancel_and_join(), and k_proc_cleanup().

Here is the caller graph for this function:



4.40.3.4 spthread_continue() `int spthread_continue (`

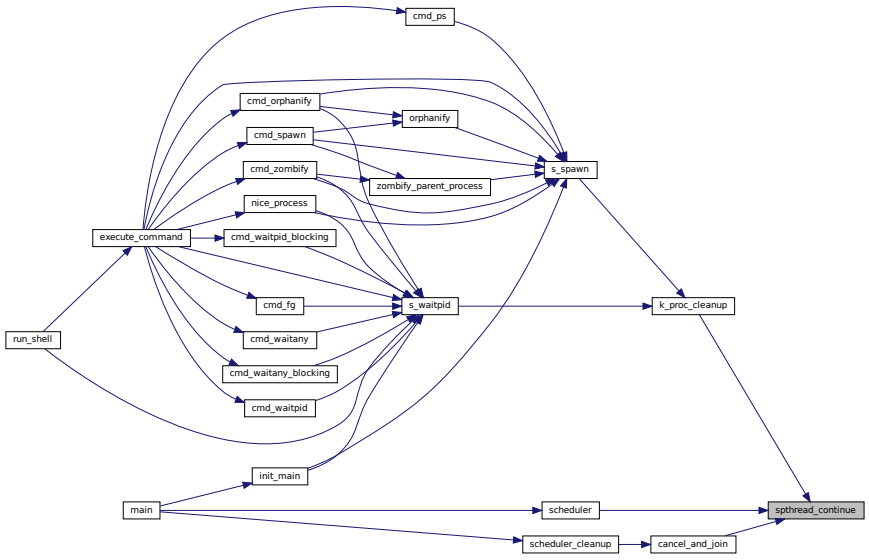
`spthread_t thread )`

Definition at line 243 of file spthread.c.

References spthread_signal_args_st::ack, spthread_st::meta, MILISEC_IN_NANO, my_meta, spthread_signal_args_st::shutup_mutex, spthread_signal_args_st::signal, SIGPTHD, SPTHREAD_RUNNING_STATE, SPTHREAD_SIG_CONTINUE, SPTHREAD_TERMINATED_STATE, spthread_meta_st::state, and spthread_st::thread.

Referenced by cancel_and_join(), k_proc_cleanup(), and scheduler().

Here is the caller graph for this function:



4.40.3.5 spthread_create()

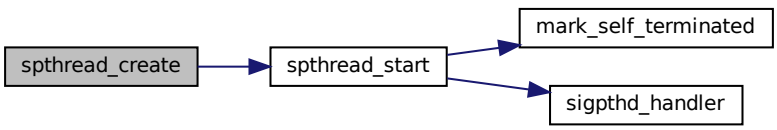
```
int spthread_create (
        spthread_t * thread,
        const pthread_attr_t * attr,
        pthread_fn start_routine,
        void * arg )
```

Definition at line 118 of file spthread.c.

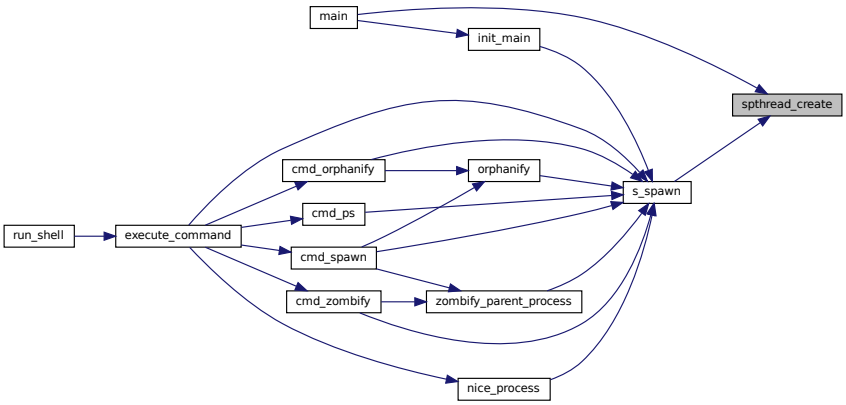
References spthread_fwd_args_st::actual_routine, spthread_fwd_args_st::setup_cond, spthread_fwd_args_st::setup_done, spthread_fwd_args_st::setup_mutex, spthread_start(), and spthread_st::thread.

Referenced by main(), and s_spawn().

Here is the call graph for this function:



Here is the caller graph for this function:



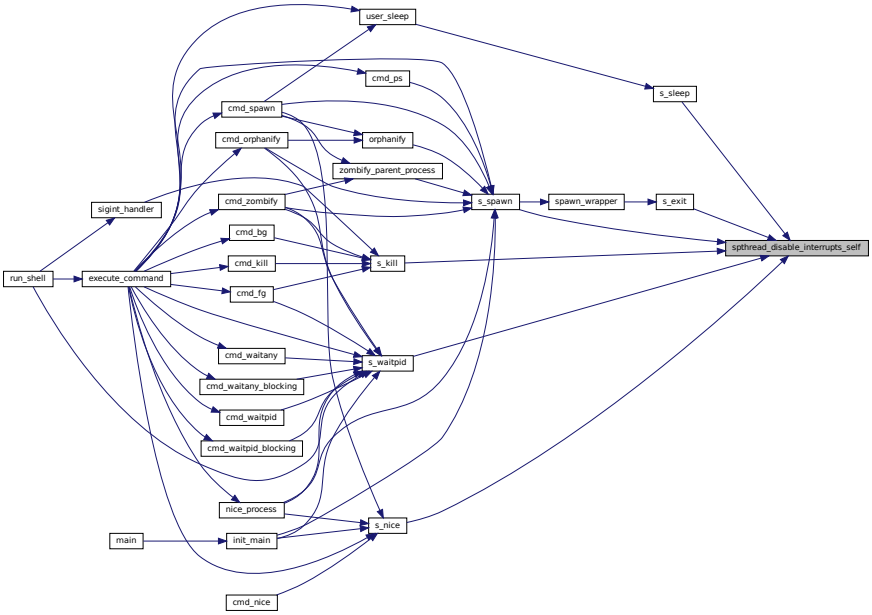
4.40.3.6 spthread_disable_interrupts_self() `int spthread_disable_interrupts_self ( )`

Definition at line 328 of file spthread.c.

References SIGPTHD.

Referenced by s_exit(), s_kill(), s_nice(), s_sleep(), s_spawn(), and s_waitpid().

Here is the caller graph for this function:



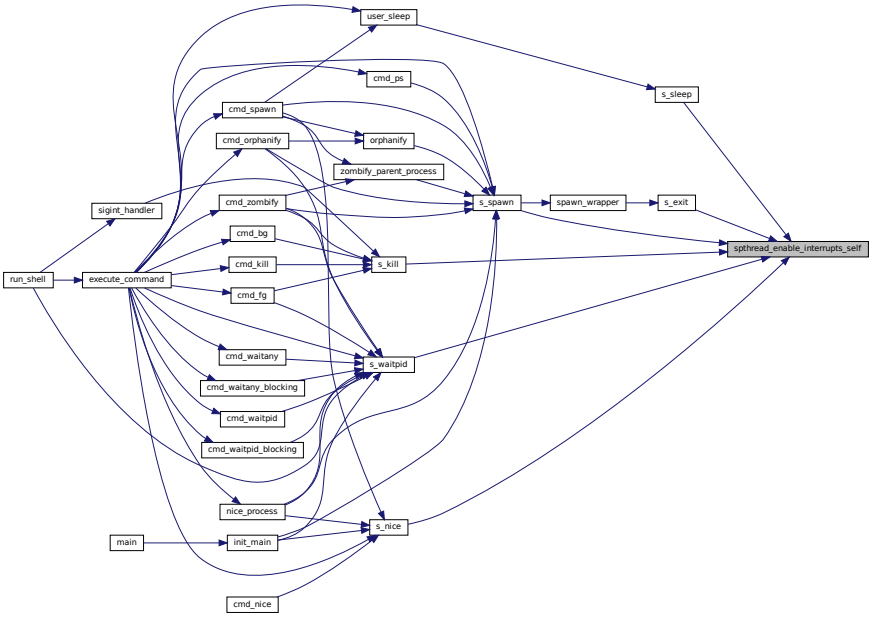
4.40.3.7 spthread_enable_interrupts_self() `int spthread_enable_interrupts_self ( )`

Definition at line 347 of file spthread.c.

References SIGPTHD.

Referenced by s_exit(), s_kill(), s_nice(), s_sleep(), s_spawn(), and s_waitpid().

Here is the caller graph for this function:



4.40.3.8 spthread_equal() `bool spthread_equal (`

```
spthread_t first,
spthread_t second )
```

Definition at line 323 of file spthread.c.

References spthread_st::meta, and spthread_st::thread.

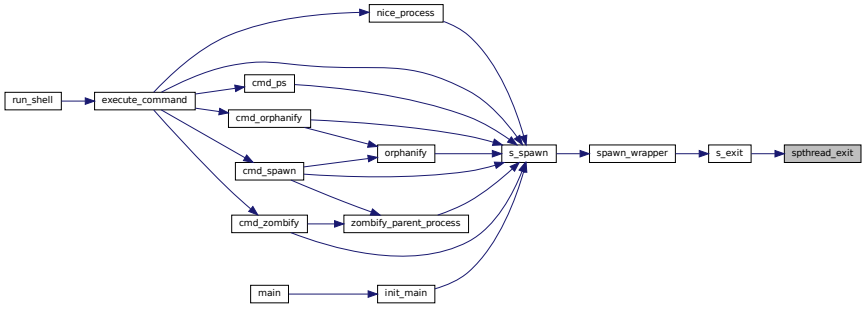
4.40.3.9 spthread_exit() `void spthread_exit (`

```
void * status )
```

Definition at line 316 of file spthread.c.

Referenced by s_exit().

Here is the caller graph for this function:



4.40.3.10 spthread_join() `int spthread_join (`

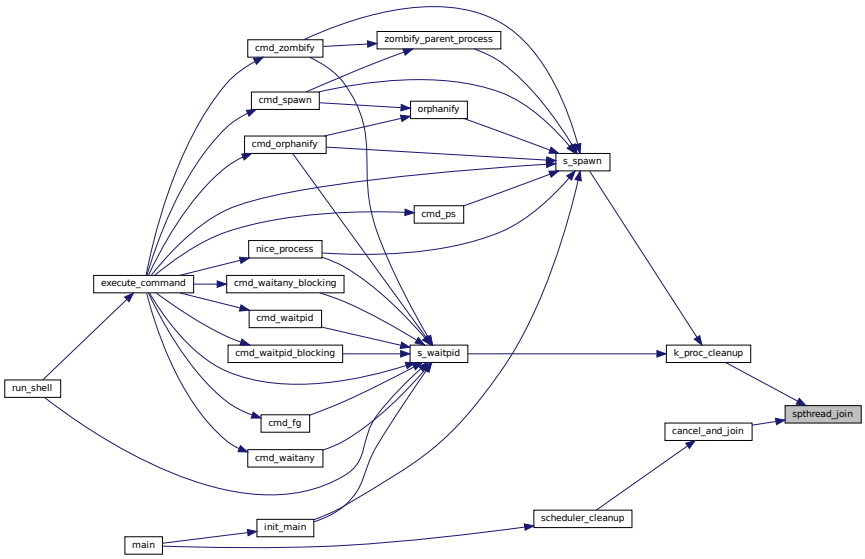
```
spthread_t thread,
void ** retval )
```

Definition at line 309 of file spthread.c.

References spthread_st::meta, spthread_meta_st::meta_mutex, and spthread_st::thread.

Referenced by cancel_and_join(), and k_proc_cleanup().

Here is the caller graph for this function:



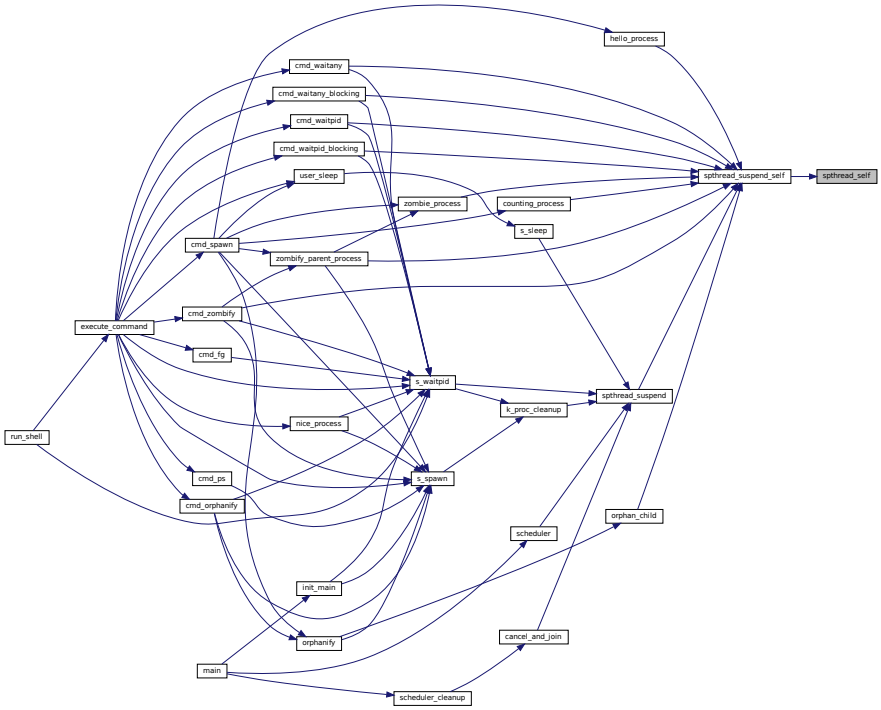
4.40.3.11 spthread_self() `bool spthread_self ( spthread_t * thread )`

Definition at line 298 of file spthread.c.

References my_meta, and spthread_st::thread.

Referenced by spthread_suspend_self().

Here is the caller graph for this function:



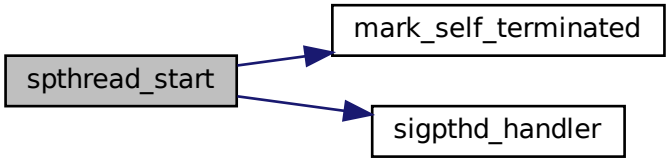
4.40.3.12 spthread_start() `static void * spthread_start ( void * arg ) [static]`

Definition at line 420 of file spthread.c.

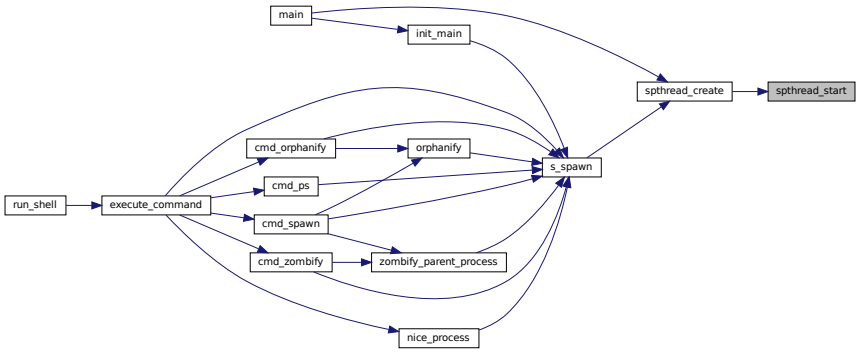
References spthread_fwd_args_st::actual_arg, spthread_fwd_args_st::actual_routine, spthread_fwd_args_st::child↩ _meta, mark_self_terminated(), spthread_meta_st::meta_mutex, my_meta, spthread_fwd_args_st::setup_cond, spthread_fwd_args_st::setup_done, spthread_fwd_args_st::setup_mutex, SIGPTHD, sigpthd_handler(), SPTHREAD↩ _SUSPENDED_STATE, spthread_meta_st::state, and spthread_meta_st::suspend_set.

Referenced by spthread_create().

Here is the call graph for this function:



Here is the caller graph for this function:



4.40.3.13 spthread_suspend() `int spthread_suspend (`

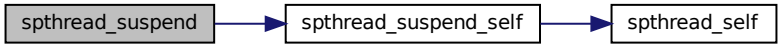
`spthread_t thread )`

Definition at line 175 of file spthread.c.

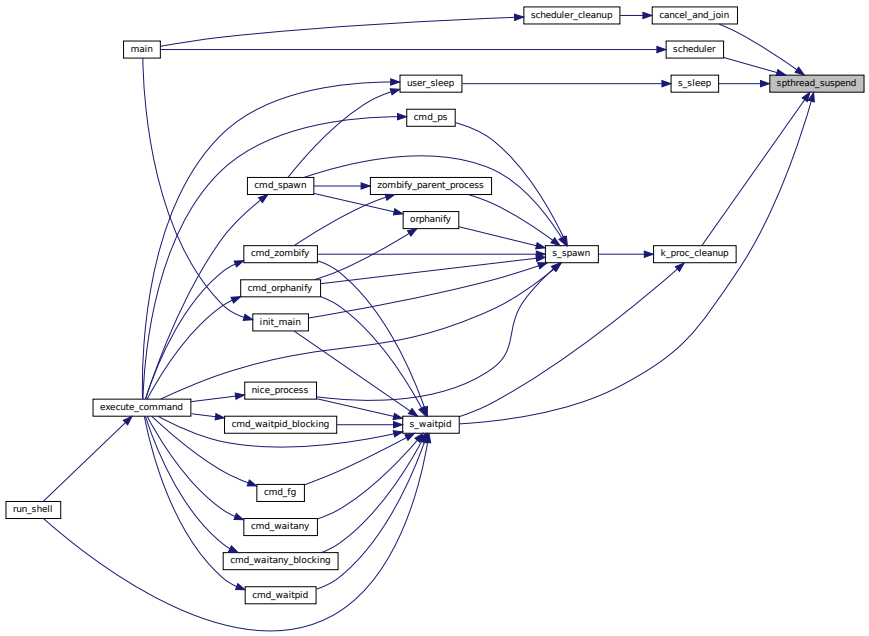
References spthread_signal_args_st::ack, spthread_st::meta, MILISEC_IN_NANO, spthread_signal_args_st::shutup_mutex, spthread_signal_args_st::signal, SIGPTHD, SPTHREAD_SIG_SUSPEND, spthread_suspend_self(), SPTHREAD_TERMINATED_STATE, spthread_meta_st::state, and spthread_st::thread.

Referenced by cancel_and_join(), k_proc_cleanup(), s_sleep(), s_waitpid(), and scheduler().

Here is the call graph for this function:



Here is the caller graph for this function:



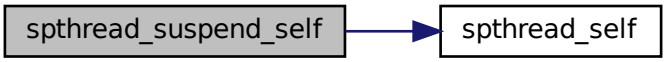
4.40.3.14 spthread_suspend_self() `int spthread_suspend_self ( )`

Definition at line 227 of file spthread.c.

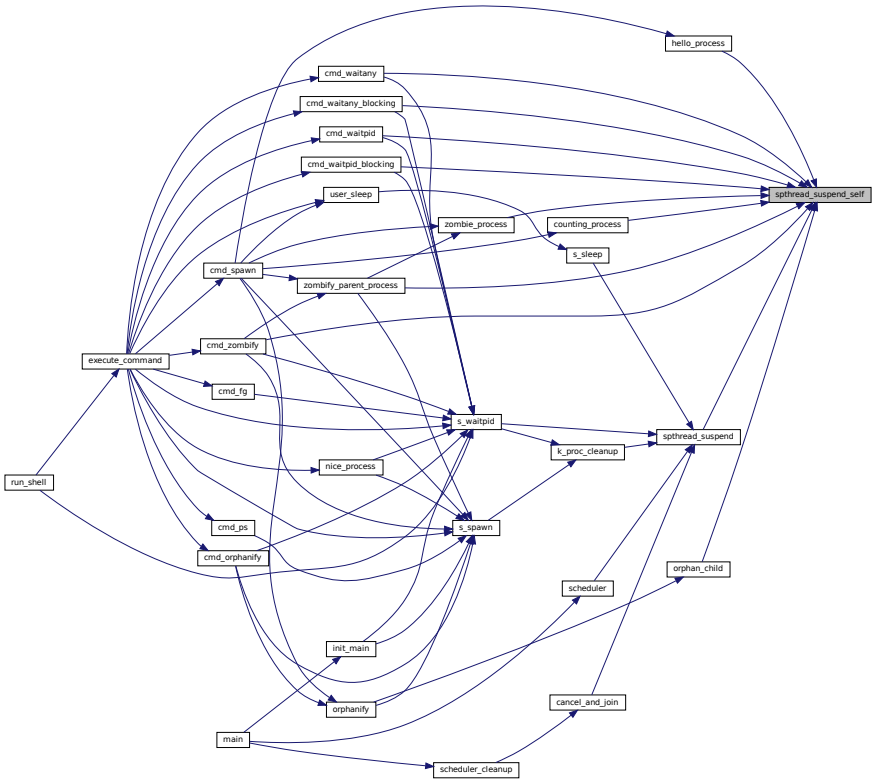
References my_meta, spthread_self(), SPTHREAD_SUSPENDED_STATE, spthread_meta_st::state, and spthread_↩ meta_st::suspend_set.

Referenced by cmd_waitany(), cmd_waitany_blocking(), cmd_waitpid(), cmd_waitpid_blocking(), cmd_zombify(), counting_process(), hello_process(), orphan_child(), spthread_suspend(), zombie_process(), and zombify_parent_↩ process().

Here is the call graph for this function:



Here is the caller graph for this function:



4.40.4 Variable Documentation

4.40.4.1 my_meta `_Thread_local spthread_meta_t* my_meta = NULL [static]`

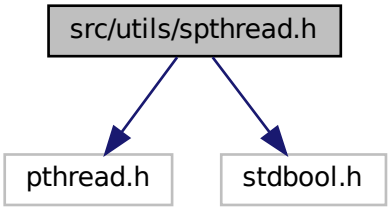
Definition at line 96 of file spthread.c.

Referenced by mark_self_terminated(), sigpthd_handler(), spthread_continue(), spthread_self(), spthread_start(), and spthread_suspend_self().

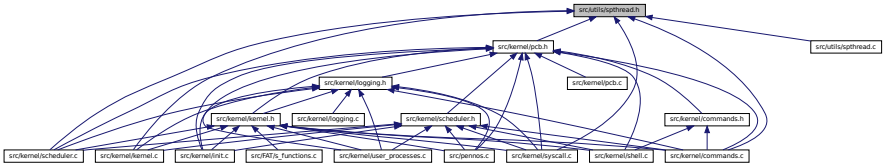
4.41 src/utils/spthread.h File Reference

```
#include <pthread.h>
#include <stdbool.h>
```

Include dependency graph for spthread.h:



This graph shows which files directly or indirectly include this file:



Data Structures

- struct spthread_st

Macros

- #define SIGPTHD SIGUSR1

Typedefs

- typedef struct spthread_meta_st spthread_meta_t
- typedef struct spthread_st spthread_t

Functions

- int spthread_create (spthread_t *thread, const pthread_attr_t *attr, void *(*start_routine)(void *), void *arg)
- int spthread_suspend (spthread_t thread)
- int spthread_suspend_self ()
- int spthread_continue (spthread_t thread)
- int spthread_cancel (spthread_t thread)
- bool spthread_self (spthread_t *thread)
- int spthread_join (spthread_t thread, void **retval)
- void spthread_exit (void *status)
- bool spthread_equal (spthread_t first, spthread_t second)
- int spthread_disable_interrupts_self ()
- int spthread_enable_interrupts_self ()

4.41.1 Macro Definition Documentation

4.41.1.1 SIGPTHD `#define SIGPTHD SIGUSR1`

Definition at line 19 of file spthread.h.

4.41.2 Typedef Documentation

4.41.2.1 spthread_meta_t `typedef struct spthread_meta_st spthread_meta_t`

Definition at line 1 of file spthread.h.

4.41.2.2 spthread_t `typedef struct spthread_st spthread_t`

4.41.3 Function Documentation

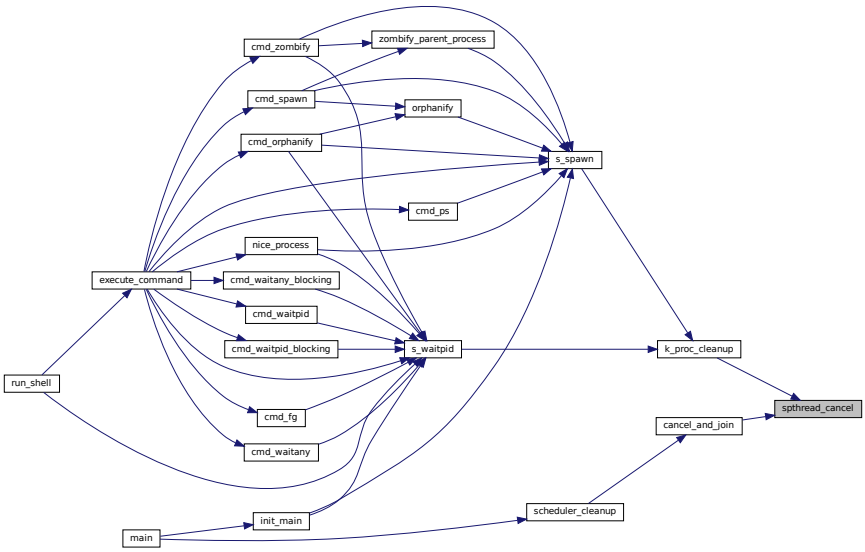
4.41.3.1 spthread_cancel() int spthread_cancel (spthread_t thread)

Definition at line 294 of file spthread.c.

References spthread_st::thread.

Referenced by cancel_and_join(), and k_proc_cleanup().

Here is the caller graph for this function:



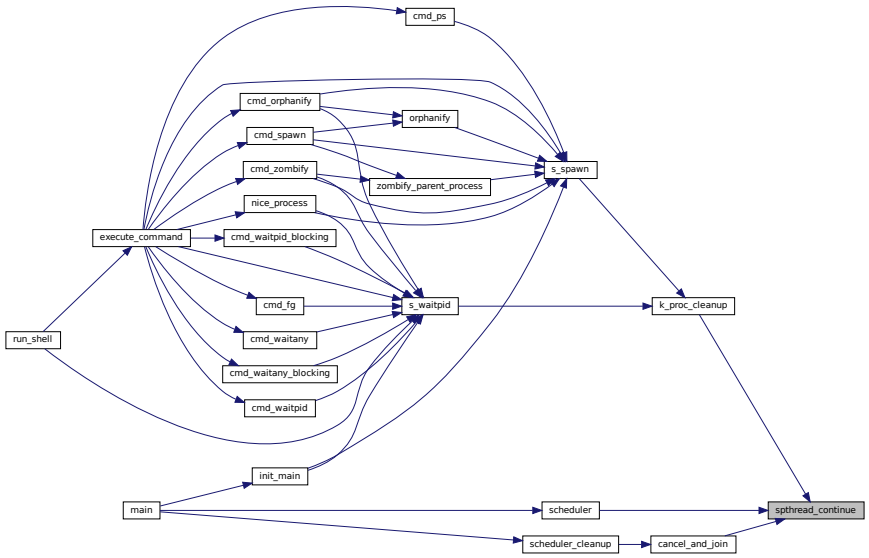
4.41.3.2 spthread_continue() int spthread_continue (spthread_t thread)

Definition at line 243 of file spthread.c.

References spthread_signal_args_st::ack, spthread_st::meta, MILISEC_IN_NANO, my_meta, spthread_signal_args↩ _st::shutup_mutex, spthread_signal_args_st::signal, SIGPTHD, SPTHREAD_RUNNING_STATE, SPTHREAD_SIG_↩ CONTINUE, SPTHREAD_TERMINATED_STATE, spthread_meta_st::state, and spthread_st::thread.

Referenced by cancel_and_join(), k_proc_cleanup(), and scheduler().

Here is the caller graph for this function:



4.41.3.3 spthread_create()

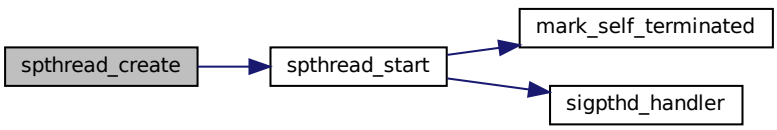
```
int spthread_create (
        spthread_t * thread,
        const pthread_attr_t * attr,
        void *(*)(void *) start_routine,
        void * arg )
```

Definition at line 118 of file spthread.c.

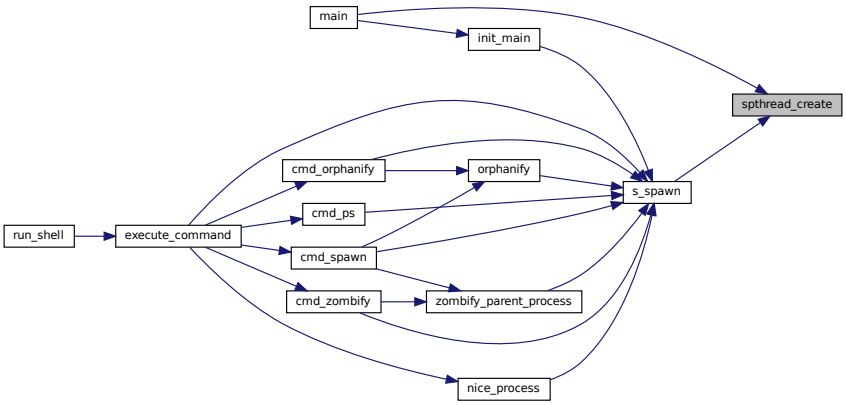
References spthread_fwd_args_st::actual_routine, spthread_fwd_args_st::setup_cond, spthread_fwd_args_st::setup_done, spthread_fwd_args_st::setup_mutex, spthread_start(), and spthread_st::thread.

Referenced by main(), and s_spawn().

Here is the call graph for this function:



Here is the caller graph for this function:



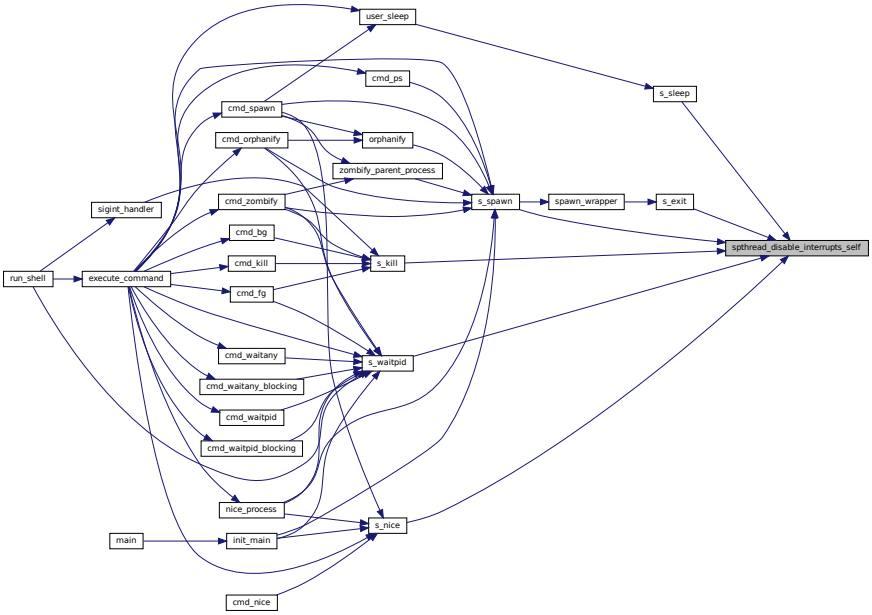
4.41.3.4 spthread_disable_interrupts_self() `int spthread_disable_interrupts_self ( )`

Definition at line 328 of file spthread.c.

References SIGPTHD.

Referenced by s_exit(), s_kill(), s_nice(), s_sleep(), s_spawn(), and s_waitpid().

Here is the caller graph for this function:



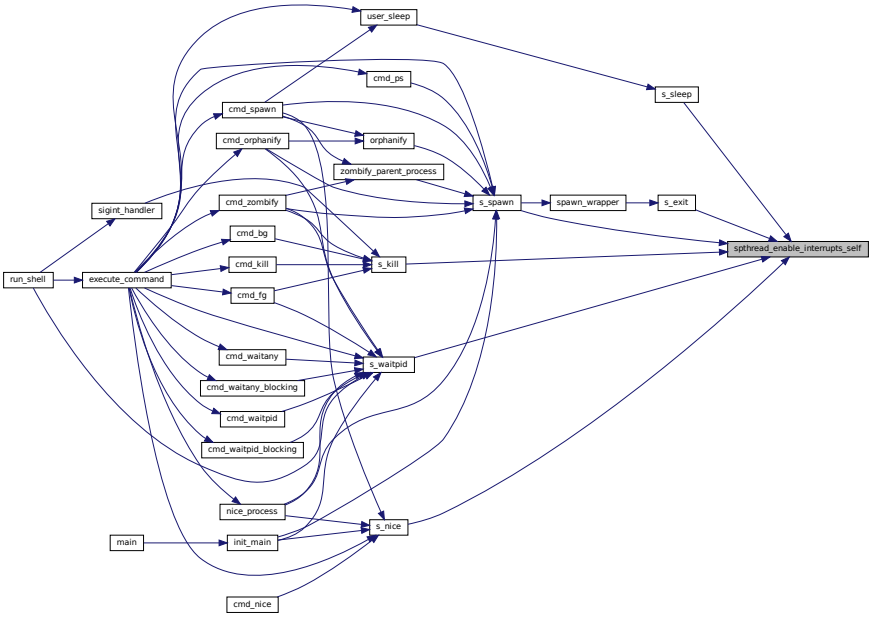
4.41.3.5 spthread_enable_interrupts_self() `int spthread_enable_interrupts_self ( )`

Definition at line 347 of file spthread.c.

References SIGPTHD.

Referenced by s_exit(), s_kill(), s_nice(), s_sleep(), s_spawn(), and s_waitpid().

Here is the caller graph for this function:



4.41.3.6 spthread_equal() `bool spthread_equal (`

```
spthread_t first,
spthread_t second )
```

Definition at line 323 of file spthread.c.

References spthread_st::meta, and spthread_st::thread.

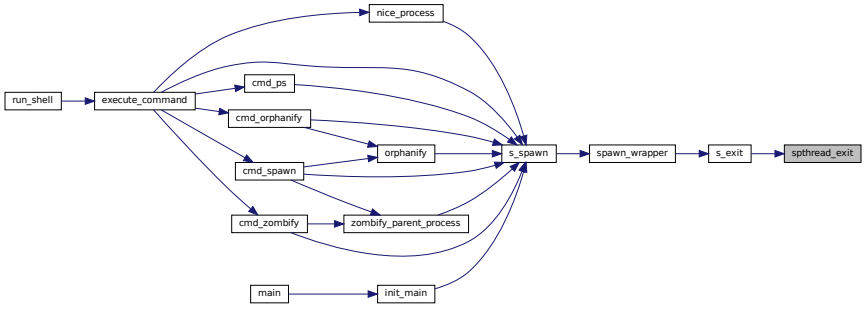
4.41.3.7 spthread_exit() `void spthread_exit (`

```
void * status )
```

Definition at line 316 of file spthread.c.

Referenced by s_exit().

Here is the caller graph for this function:



4.41.3.8 spthread_join() `int spthread_join (`

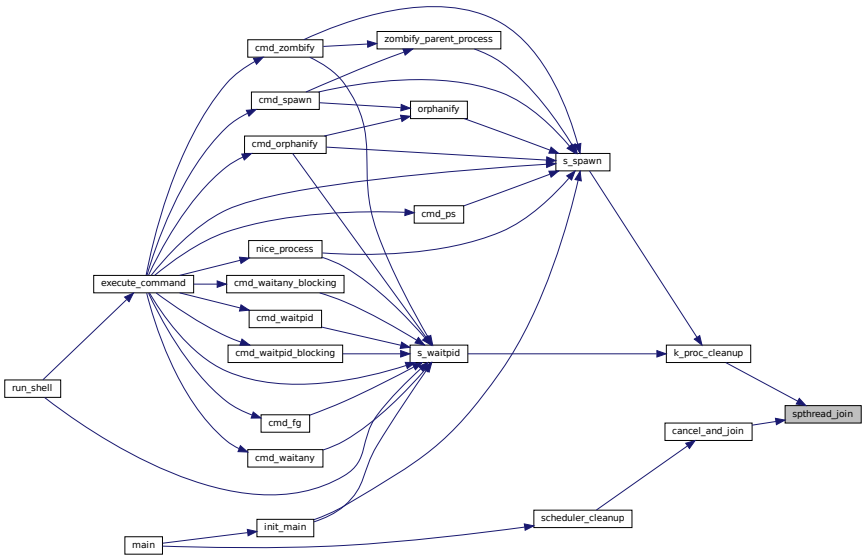
```
spthread_t thread,
void ** retval )
```

Definition at line 309 of file spthread.c.

References spthread_st::meta, spthread_meta_st::meta_mutex, and spthread_st::thread.

Referenced by cancel_and_join(), and k_proc_cleanup().

Here is the caller graph for this function:



4.41.3.9 spthread_self() `bool spthread_self (`

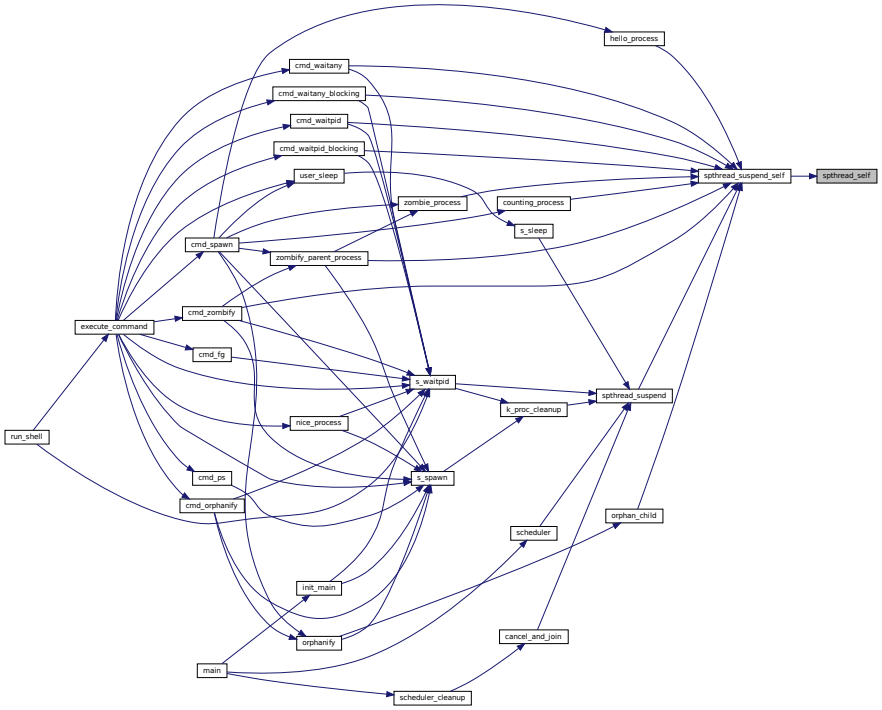
`spthread_t * thread )`

Definition at line 298 of file spthread.c.

References my_meta, and spthread_st::thread.

Referenced by spthread_suspend_self().

Here is the caller graph for this function:



4.41.3.10 spthread_suspend() `int spthread_suspend (`

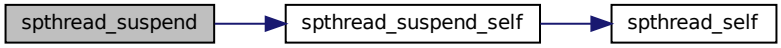
`spthread_t thread )`

Definition at line 175 of file spthread.c.

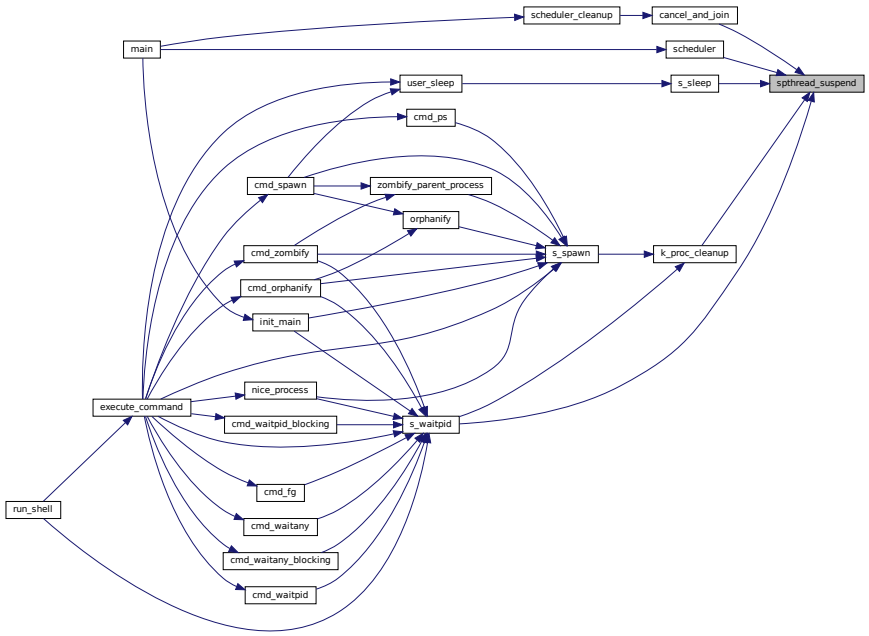
References spthread_signal_args_st::ack, spthread_st::meta, MILISEC_IN_NANO, spthread_signal_args_st::shutup_mutex, spthread_signal_args_st::signal, SIGPTHD, SPTHREAD_SIG_SUSPEND, spthread_suspend_self(), SPTHREAD_TERMINATED_STATE, spthread_meta_st::state, and spthread_st::thread.

Referenced by cancel_and_join(), k_proc_cleanup(), s_sleep(), s_waitpid(), and scheduler().

Here is the call graph for this function:



Here is the caller graph for this function:



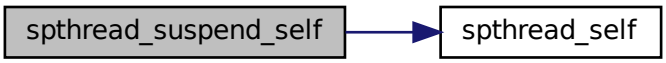
4.41.3.11 spthread_suspend_self() `int spthread_suspend_self ( )`

Definition at line 227 of file spthread.c.

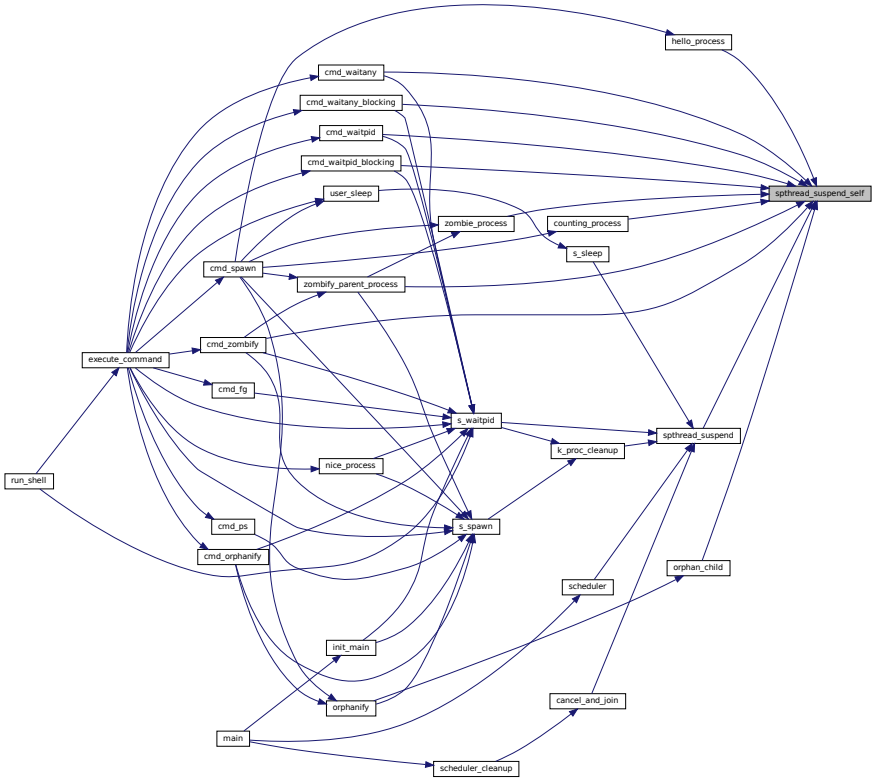
References my_meta, spthread_self(), SPTHREAD_SUSPENDED_STATE, spthread_meta_st::state, and spthread_↩ meta_st::suspend_set.

Referenced by cmd_waitany(), cmd_waitany_blocking(), cmd_waitpid(), cmd_waitpid_blocking(), cmd_zombify(), counting_process(), hello_process(), orphan_child(), spthread_suspend(), zombie_process(), and zombify_parent_↩ process().

Here is the call graph for this function:



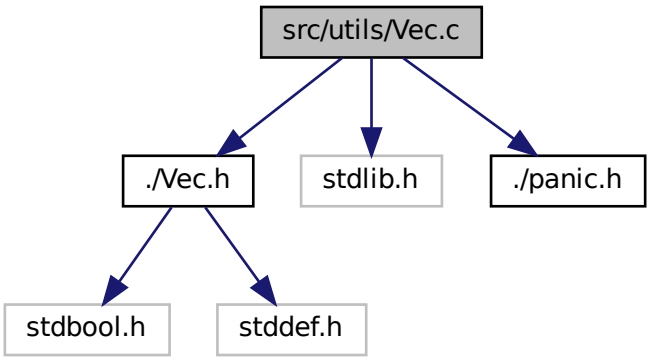
Here is the caller graph for this function:



4.42 src/utils/Vec.c File Reference

```
#include "./Vec.h"
#include <stdlib.h>
#include "./panic.h"
```

Include dependency graph for Vec.c:



Functions

- Vec vec_new (size_t initial_capacity, ptr_dtor_fn ele_dtor_fn)
- ptr_t vec_get (Vec *self, size_t index)
- void vec_set (Vec *self, size_t index, ptr_t new_ele)
- void vec_push_back (Vec *self, ptr_t new_ele)
- bool vec_pop_back (Vec *self)
- void vec_insert (Vec *self, size_t index, ptr_t new_ele)
- void vec_erase (Vec *self, size_t index)
- void vec_resize (Vec *self, size_t new_capacity)
- void vec_clear (Vec *self)
- void vec_destroy (Vec *self)

4.42.1 Function Documentation

4.42.1.1 vec_clear() `void vec_clear ( Vec * self )`

Definition at line 137 of file Vec.c.

Referenced by vec_destroy().

Here is the caller graph for this function:



4.42.1.2 vec_destroy() `void vec_destroy ( Vec * self )`

Definition at line 148 of file Vec.c.

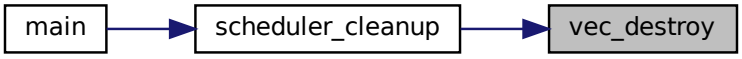
References vec_clear().

Referenced by scheduler_cleanup().

Here is the call graph for this function:



Here is the caller graph for this function:



4.42.1.3 vec_erase()

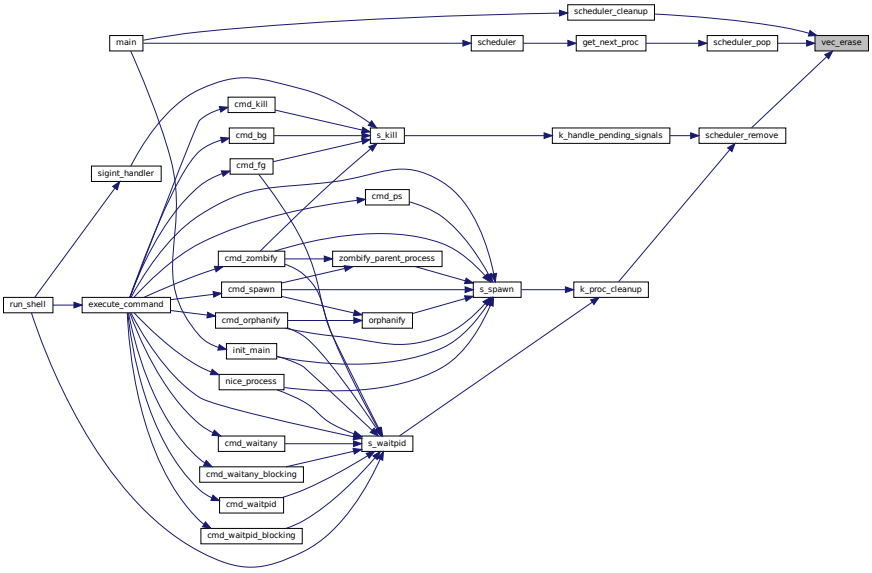
```
void vec_erase (
        Vec * self,
        size_t index )
```

Definition at line 101 of file Vec.c.

References panic.

Referenced by scheduler_cleanup(), scheduler_pop(), and scheduler_remove().

Here is the caller graph for this function:



4.42.1.4 vec_get()

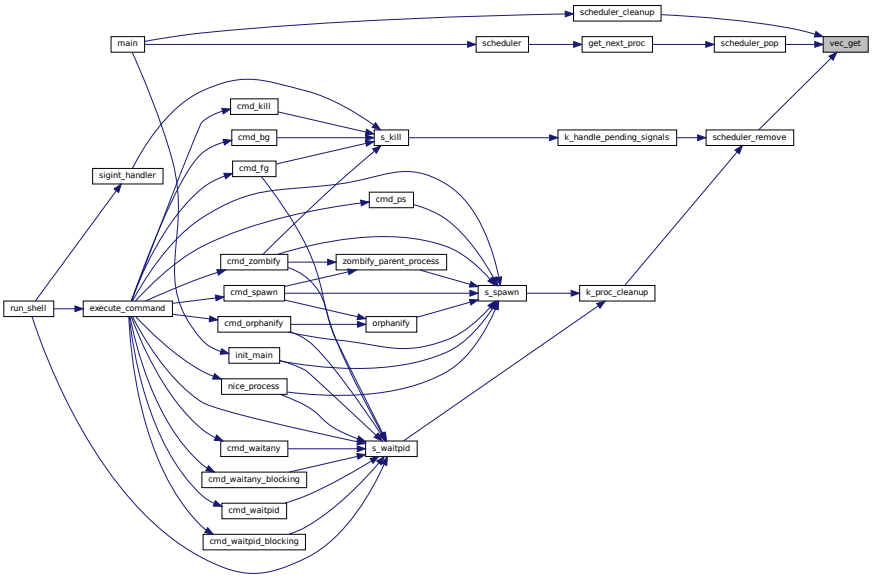
```
ptr_t vec_get (
        Vec * self,
        size_t index )
```

Definition at line 27 of file Vec.c.

References panic.

Referenced by scheduler_cleanup(), scheduler_pop(), and scheduler_remove().

Here is the caller graph for this function:



4.42.1.5 vec_insert()

```
void vec_insert (
        Vec * self,
        size_t index,
        ptr_t new_ele )
```

Definition at line 78 of file Vec.c.

References panic, and vec_resize().

Here is the call graph for this function:



4.42.1.6 vec_new()

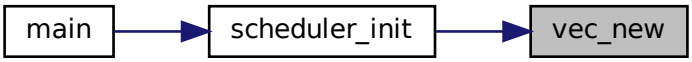
```
Vec vec_new (
        size_t initial_capacity,
        ptr_dtor_fn ele_dtor_fn )
```

Definition at line 10 of file Vec.c.

References vec_st::capacity, vec_st::data, vec_st::ele_dtor_fn, vec_st::length, and panic.

Referenced by scheduler_init().

Here is the caller graph for this function:



4.42.1.7 vec_pop_back()

```
bool vec_pop_back (
        Vec * self )
```

Definition at line 65 of file Vec.c.

4.42.1.8 vec_push_back()

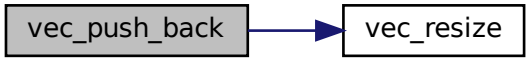
```
void vec_push_back (
        Vec * self,
        ptr_t new_ele )
```

Definition at line 50 of file Vec.c.

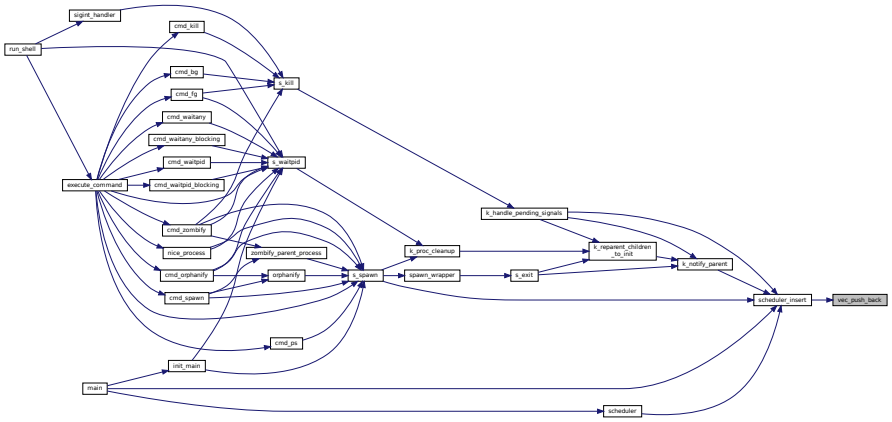
References vec_resize().

Referenced by scheduler_insert().

Here is the call graph for this function:



Here is the caller graph for this function:



4.42.1.9 vec_resize() `void vec_resize (`

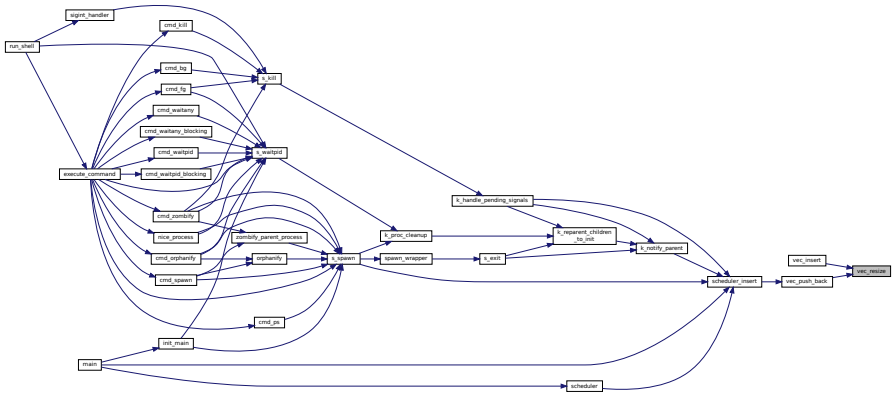
```
Vec * self,
size_t new_capacity )
```

Definition at line 120 of file Vec.c.

References panic.

Referenced by vec_insert(), and vec_push_back().

Here is the caller graph for this function:



4.42.1.10 vec_set()

```
void vec_set (
        Vec * self,
        size_t index,
        ptr_t new_ele )
```

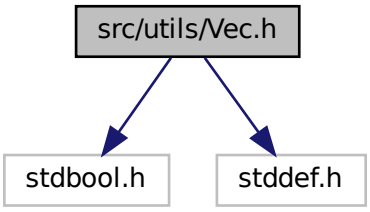
Definition at line 35 of file Vec.c.

References panic.

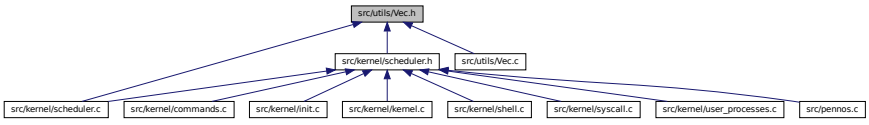
4.43 src/utils/Vec.h File Reference

```
#include <stdbool.h>
#include <stddef.h>
```

Include dependency graph for Vec.h:



This graph shows which files directly or indirectly include this file:



Data Structures

- struct vec_st

Macros

- #define vec_capacity(vec) ((vec)->capacity)
- #define vec_len(vec) ((vec)->length)
- #define vec_is_empty(vec) ((vec)->length == 0)

Typedefs

- typedef void * ptr_t
- typedef void(* ptr_dtor_fn) (ptr_t)
- typedef struct vec_st Vec

Functions

- Vec vec_new (size_t initial_capacity, ptr_dtor_fn ele_dtor_fn)
- ptr_t vec_get (Vec *self, size_t index)
- void vec_set (Vec *self, size_t index, ptr_t new_ele)
- void vec_push_back (Vec *self, ptr_t new_ele)
- bool vec_pop_back (Vec *self)
- void vec_insert (Vec *self, size_t index, ptr_t new_ele)
- void vec_erase (Vec *self, size_t index)
- void vec_resize (Vec *self, size_t new_capacity)
- void vec_clear (Vec *self)
- void vec_destroy (Vec *self)

4.43.1 Macro Definition Documentation

4.43.1.1 vec_capacity `#define vec_capacity( vec ) ((vec)->capacity)`

Definition at line 40 of file Vec.h.

4.43.1.2 vec_is_empty `#define vec_is_empty( vec ) ((vec)->length == 0)`

Definition at line 54 of file Vec.h.

4.43.1.3 vec_len `#define vec_len( vec ) ((vec)->length)`

Definition at line 47 of file Vec.h.

4.43.2 Typedef Documentation

4.43.2.1 ptr_dtor_fn `typedef void(* ptr_dtor_fn) (ptr_t)`

Definition at line 8 of file Vec.h.

4.43.2.2 ptr_t `typedef void* ptr_t`

Definition at line 7 of file Vec.h.

4.43.2.3 Vec `typedef struct vec_st Vec`

4.43.3 Function Documentation

4.43.3.1 vec_clear() `void vec_clear ( Vec * self )`

Definition at line 137 of file Vec.c.

Referenced by vec_destroy().

Here is the caller graph for this function:



4.43.3.2 vec_destroy() `void vec_destroy (`

```
Vec * self )
```

Definition at line 148 of file Vec.c.

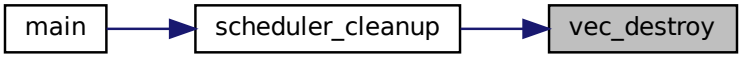
References vec_clear().

Referenced by scheduler_cleanup().

Here is the call graph for this function:



Here is the caller graph for this function:



4.43.3.3 vec_erase() `void vec_erase (`

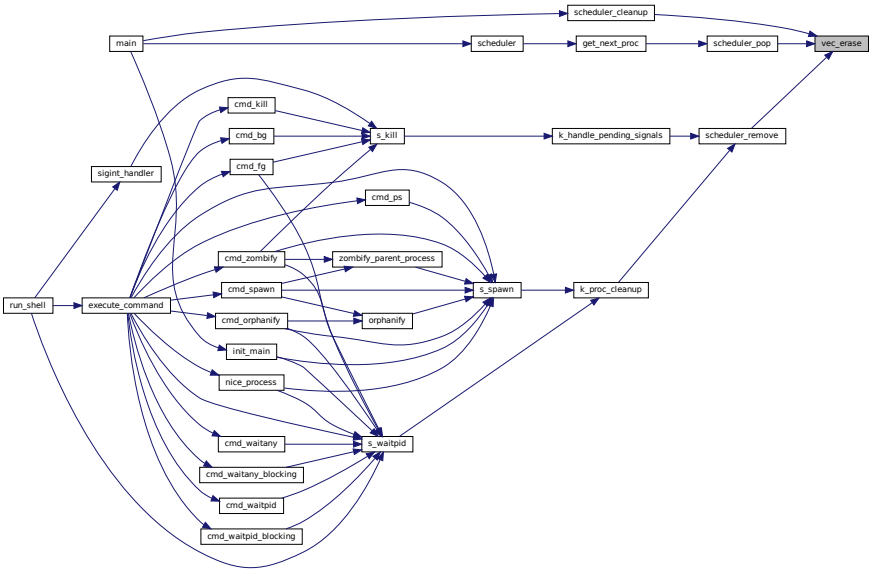
```
Vec * self,
size_t index )
```

Definition at line 101 of file Vec.c.

References panic.

Referenced by scheduler_cleanup(), scheduler_pop(), and scheduler_remove().

Here is the caller graph for this function:



4.43.3.4 vec_get()

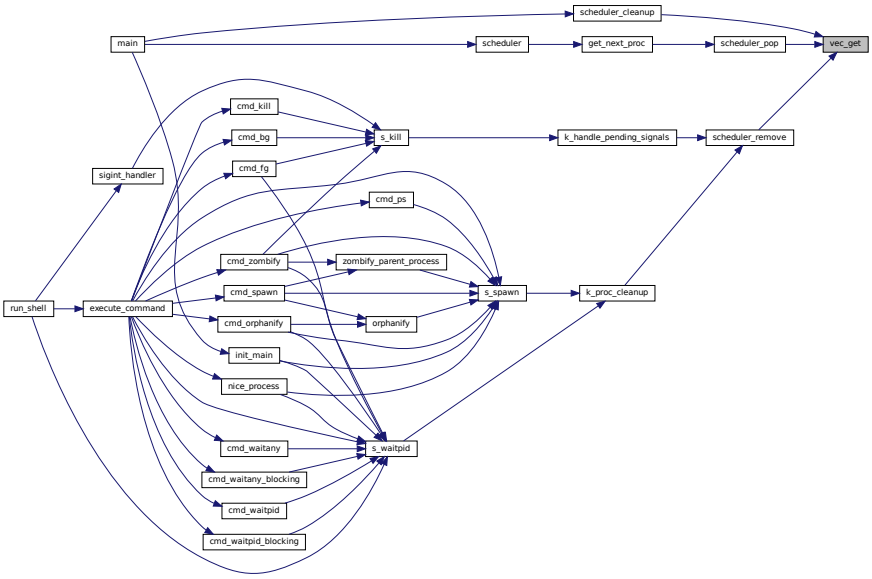
```
ptr_t vec_get (
        Vec * self,
        size_t index )
```

Definition at line 27 of file Vec.c.

References panic.

Referenced by scheduler_cleanup(), scheduler_pop(), and scheduler_remove().

Here is the caller graph for this function:



4.43.3.5 vec_insert()

```
void vec_insert (
        Vec * self,
        size_t index,
        ptr_t new_ele )
```

Definition at line 78 of file Vec.c.

References panic, and vec_resize().

Here is the call graph for this function:



4.43.3.6 vec_new()

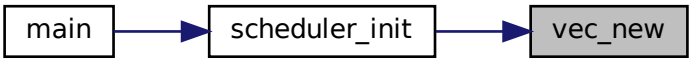
```
Vec vec_new (
        size_t initial_capacity,
        ptr_dtor_fn ele_dtor_fn )
```

Definition at line 10 of file Vec.c.

References vec_st::capacity, vec_st::data, vec_st::ele_dtor_fn, vec_st::length, and panic.

Referenced by scheduler_init().

Here is the caller graph for this function:



4.43.3.7 vec_pop_back()

```
bool vec_pop_back (
        Vec * self )
```

Definition at line 65 of file Vec.c.

4.43.3.8 vec_push_back()

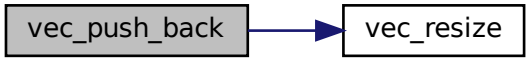
```
void vec_push_back (
        Vec * self,
        ptr_t new_ele )
```

Definition at line 50 of file Vec.c.

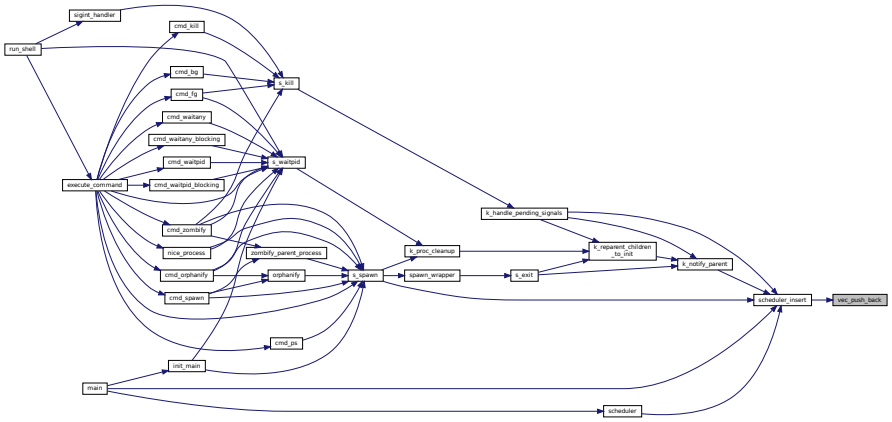
References vec_resize().

Referenced by scheduler_insert().

Here is the call graph for this function:



Here is the caller graph for this function:



4.43.3.9 vec_resize() `void vec_resize (`

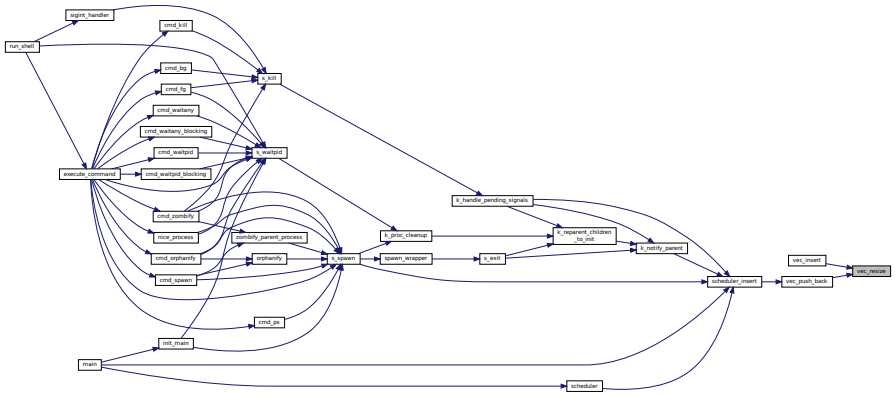
```
Vec * self,
size_t new_capacity )
```

Definition at line 120 of file Vec.c.

References panic.

Referenced by vec_insert(), and vec_push_back().

Here is the caller graph for this function:



4.43.3.10 vec_set()

```
void vec_set (
    Vec * self,
    size_t index,
    ptr_t new_ele )
```

Definition at line 35 of file Vec.c.

References panic.

Index

_DEFAULT_SOURCE
  scheduler.c, 264
_GNU_SOURCE
  input_processing.c, 373
  scheduler.c, 264
  spthread.c, 383
_POSIX_C_SOURCE
  scheduler.c, 264
_block_size
  commands.c, 27
  commands.h, 52
_current_mount
  commands.c, 27
  commands.h, 52
_fat
  commands.c, 28
  commands.h, 53
_fat_size
  commands.c, 28
  commands.h, 53
_fs_fd
  commands.c, 28
  commands.h, 53

ack
  spthread_signal_args_st, 20
actual_arg
  spthread_fwd_args_st, 17
actual_routine
  spthread_fwd_args_st, 18
alarm_handler
  scheduler.c, 264
allocate_block
  k_functions.c, 91
  k_functions.h, 112
allocate_fd
  k_functions.c, 92
  k_functions.h, 113
argv
  spawn_args_t, 16

BLOCK_SIZES
  commands.c, 28
BLOCKED
  pcb.h, 253
busy_args_t, 3
  is_background, 4
  name, 4

cancel_and_join
  scheduler.c, 264
capacity
  vec_st, 22
cat
  shell_commands.c, 152
  shell_commands.h, 162
child_meta
  spthread_fwd_args_st, 18
children
  pcb, 11
cmd_bg
  commands.c, 30
  commands.h, 55
cmd_chmod
  shell_commands.c, 153
  shell_commands.h, 163
cmd_fg
  commands.c, 31
  commands.h, 56
cmd_info
  commands.c, 32
  commands.h, 57
cmd_jobs
  commands.c, 33
  commands.h, 58
cmd_kill
  commands.c, 34
  commands.h, 59
cmd_logout
  commands.c, 35
  commands.h, 60
cmd_ls
  shell_commands.c, 154
  shell_commands.h, 164
cmd_nice
  commands.c, 36
  commands.h, 61
cmd_orphanify
  commands.c, 37
  commands.h, 62
cmd_ps
  commands.c, 38
  commands.h, 63
cmd_spawn
  commands.c, 39
  commands.h, 64
cmd_waitany
  commands.c, 40
  commands.h, 65
cmd_waitany_blocking
  commands.c, 41
  commands.h, 66
cmd_waitpid

commands.c, 42
commands.h, 67
cmd_waitpid_blocking
commands.c, 43
commands.h, 68
cmd_zombify
commands.c, 44
commands.h, 69
commands
parsed_command, 9
commands.c
_block_size, 27
_current_mount, 27
_fat, 28
_fat_size, 28
_fs_fd, 28
BLOCK_SIZES, 28
cmd_bg, 30
cmd_fg, 31
cmd_info, 32
cmd_jobs, 33
cmd_kill, 34
cmd_logout, 35
cmd_nice, 36
cmd_orphanify, 37
cmd_ps, 38
cmd_spawn, 39
cmd_waitany, 40
cmd_waitany_blocking, 41
cmd_waitpid, 42
cmd_waitpid_blocking, 43
cmd_zombify, 44
foreground_pid, 47
mkfs, 24
mount, 25
print_help, 45
sigint_handler, 46
sigint_received, 47
unmount, 26
commands.h
_block_size, 52
_current_mount, 52
_fat, 53
_fat_size, 53
_fs_fd, 53
cmd_bg, 55
cmd_fg, 56
cmd_info, 57
cmd_jobs, 58
cmd_kill, 59
cmd_logout, 60
cmd_nice, 61
cmd_orphanify, 62
cmd_ps, 63
cmd_spawn, 64
cmd_waitany, 65
cmd_waitany_blocking, 66
cmd_waitpid, 67
cmd_waitpid_blocking, 68
cmd_zombify, 69
foreground_pid, 77
mkfs, 49
mount, 50
print_help, 71
run_shell, 71
shell_main, 73
sigint_handler, 76
sigint_received, 77
unmount, 51
counting_process
user_processes.c, 314
user_processes.h, 335
cp
shell_commands.c, 156
shell_commands.h, 165
create_directory_entry
k_functions.c, 93
k_functions.h, 114
create_directory_entry_at_block
k_functions.c, 94
current_process
kernel.c, 193
current_ticks
logging.c, 225

data
vec_st, 22
decrement_refcount
fd_table.c, 84
fd_table.h, 88
directory_entry_t, 4
firstBlock, 5
mtime, 5
name, 5
perm, 5
reserved, 5
size, 6
type, 6
done
scheduler.c, 274

ele_dtor_fn
vec_st, 22
execute_command
shell.c, 285
exit_status
pcb, 11
exit_statuses
pcb, 12

EXPECT_COMMANDS
parser.h, 363
EXPECT_INPUT_FILENAME
parser.h, 363
EXPECT_OUTPUT_FILENAME
parser.h, 363

F_APPEND
fat_structs.h, 79
fd_table.h, 87
F_READ
fat_structs.h, 79
fd_table.h, 87
F_SEEK_CUR
fat_structs.h, 79
F_SEEK_END
fat_structs.h, 80
F_SEEK_SET
fat_structs.h, 80
F_WRITE
fat_structs.h, 80
fd_table.h, 88
fat_structs.h
F_APPEND, 79
F_READ, 79
F_SEEK_CUR, 79
F_SEEK_END, 80
F_SEEK_SET, 80
F_WRITE, 80
FILE_ALREADY_OPEN, 80
FILE_NOT_FOUND, 80
FS_NOT_MOUNTED, 80
INVALID_MODE, 81
MAX_OPEN_FILES, 81
NO_FREE_FD, 81
PERM_ALL, 81
PERM_NONE, 81
PERM_READ, 81
PERM_READ_EXEC, 82
PERM_READ_WRITE, 82
PERM_WRITE, 82
TYPE_DIRECTORY, 82
TYPE_REGULAR, 82
TYPE_SYMLINK, 82
TYPE_UNKNOWN, 83
fd_table
pcb, 12
fd_table.c
decrement_refcount, 84
global_fd_table, 85
increment_refcount, 84
init_global_fd_table, 85
fd_table.h
decrement_refcount, 88
F_APPEND, 87
F_READ, 87
F_WRITE, 88
global_fd_table, 90
increment_refcount, 89
init_global_fd_table, 89
IS_FD_IN_USE, 88
MAX_FDS, 88
FILE_ALREADY_OPEN
fat_structs.h, 80
FILE_NOT_FOUND
fat_structs.h, 80
filename
global_fd_entry_t, 7
find_file_in_directory
k_functions.c, 95
k_functions.h, 115
first_block
global_fd_entry_t, 7
firstBlock
directory_entry_t, 5
foreground_pid
commands.c, 47
commands.h, 77
FS_NOT_MOUNTED
fat_structs.h, 80
func
spawn_args_t, 16

get_next_proc
scheduler.c, 265
scheduler.h, 276
get_redirected_fds
redirection.c, 379
redirection.h, 381
global_fd_entry_t, 6
filename, 7
first_block, 7
mode, 7
offset, 7
perm, 8
refcount, 8
size, 8
global_fd_table
fd_table.c, 85
fd_table.h, 90

hello_process
user_processes.c, 315
user_processes.h, 336

increment_refcount
fd_table.c, 84
fd_table.h, 89
init.c

init_main, 170
init_shutdown, 172
system_running, 173
init.h
init_main, 174
init_shutdown, 175
init_global_fd_table
fd_table.c, 85
fd_table.h, 89
init_main
init.c, 170
init.h, 174
INIT_PID
kernel.h, 195
init_process
kernel.c, 193
pennos.c, 372
init_shutdown
init.c, 172
init.h, 175
input_check
input_processing.c, 374
input_processing.h, 375
input_processing.c
_GNU_SOURCE, 373
input_check, 374
input_processing.h
input_check, 375
INVALID_MODE
fat_structs.h, 81
is_background
busy_args_t, 4
parsed_command, 9
pcb, 12
IS_FD_IN_USE
fd_table.h, 88
is_file_append
parsed_command, 9
is_waiting
pcb, 12

JUMP_OUT
parser.c, 357

k_add_process
kernel.c, 177
kernel.h, 196
k_alloc_pid
kernel.c, 178
kernel.h, 197
k_close
k_functions.c, 97
k_functions.h, 117
k_functions.c
allocate_block, 91
allocate_fd, 92
create_directory_entry, 93
create_directory_entry_at_block, 94
find_file_in_directory, 95
k_close, 97
k_ls, 99
k_lseek, 100
k_open, 101
k_read, 103
k_unlink, 104
k_write, 105
print_directory_entry, 107
print_fat_table, 108
update_directory_entry, 108
k_functions.h
allocate_block, 112
allocate_fd, 113
create_directory_entry, 114
find_file_in_directory, 115
k_close, 117
k_ls, 119
k_lseek, 120
k_open, 121
k_read, 123
k_unlink, 124
k_write, 125
print_directory_entry, 127
print_fat_table, 128
update_directory_entry, 128
k_get_current_process
kernel.c, 179
kernel.h, 197
k_get_init_process
kernel.c, 180
kernel.h, 198
k_get_process
kernel.c, 181
kernel.h, 199
k_get_process_count
kernel.c, 182
kernel.h, 200
k_get_process_table
kernel.c, 182
kernel.h, 200
k_handle_pending_signals
kernel.c, 183
kernel.h, 201
k_init
kernel.c, 184
kernel.h, 202
k_ls
k_functions.c, 99
k_functions.h, 119
k_lseek

k_functions.c, 100
k_functions.h, 120
k_notify_parent
kernel.c, 185
kernel.h, 203
k_open
k_functions.c, 101
k_functions.h, 121
k_proc_cleanup
kernel.c, 186
kernel.h, 204
k_proc_create
kernel.c, 188
kernel.h, 206
k_read
k_functions.c, 103
k_functions.h, 123
k_remove_process
kernel.c, 189
kernel.h, 207
k_reparent_children_to_init
kernel.c, 190
kernel.h, 209
k_set_current_process
kernel.c, 191
kernel.h, 210
k_shutdown
kernel.c, 192
kernel.h, 211
k_unlink
k_functions.c, 104
k_functions.h, 124
k_write
k_functions.c, 105
k_functions.h, 125
kernel.c
current_process, 193
init_process, 193
k_add_process, 177
k_alloc_pid, 178
k_get_current_process, 179
k_get_init_process, 180
k_get_process, 181
k_get_process_count, 182
k_get_process_table, 182
k_handle_pending_signals, 183
k_init, 184
k_notify_parent, 185
k_proc_cleanup, 186
k_proc_create, 188
k_remove_process, 189
k_reparent_children_to_init, 190
k_set_current_process, 191
k_shutdown, 192
kernel_initialized, 193
next_pid, 193
process_table, 193
kernel.h
INIT_PID, 195
k_add_process, 196
k_alloc_pid, 197
k_get_current_process, 197
k_get_init_process, 198
k_get_process, 199
k_get_process_count, 200
k_get_process_table, 200
k_handle_pending_signals, 201
k_init, 202
k_notify_parent, 203
k_proc_cleanup, 204
k_proc_create, 206
k_remove_process, 207
k_reparent_children_to_init, 209
k_set_current_process, 210
k_shutdown, 211
MAX_PROCESSES, 195
kernel_initialized
kernel.c, 193

length
vec_st, 23
log_blocked
logging.c, 213
logging.h, 227
log_continued
logging.c, 214
logging.h, 228
log_create
logging.c, 215
logging.h, 229
log_exited
logging.c, 216
logging.h, 230
log_fd
logging.c, 226
log_nice
logging.c, 216
logging.h, 231
log_orphan
logging.c, 217
logging.h, 232
log_schedule
logging.c, 218
logging.h, 233
log_signaled
logging.c, 219
logging.h, 234
log_stopped

logging.c, 220
logging.h, 235
log_unblocked
logging.c, 220
logging.h, 235
log_waited
logging.c, 221
logging.h, 236
log_zombie
logging.c, 222
logging.h, 237
logger_close
logging.c, 223
logging.h, 238
logger_get_ticks
logging.c, 223
logging.h, 238
logger_init
logging.c, 224
logging.h, 239
logger_set_ticks
logging.c, 225
logging.h, 240
logging.c
current_ticks, 225
log_blocked, 213
log_continued, 214
log_create, 215
log_exited, 216
log_fd, 226
log_nice, 216
log_orphan, 217
log_schedule, 218
log_signaled, 219
log_stopped, 220
log_unblocked, 220
log_waited, 221
log_zombie, 222
logger_close, 223
logger_get_ticks, 223
logger_init, 224
logger_set_ticks, 225
logging.h
log_blocked, 227
log_continued, 228
log_create, 229
log_exited, 230
log_nice, 231
log_orphan, 232
log_schedule, 233
log_signaled, 234
log_stopped, 235
log_unblocked, 235
log_waited, 236
log_zombie, 237
logger_close, 238
logger_get_ticks, 238
logger_init, 239
logger_set_ticks, 240

main
pennfat.c, 368
pennos.c, 371
mark_self_terminated
spthread.c, 385
MAX_CHILDREN
pcb.h, 251
MAX_FDS
fd_table.h, 88
MAX_LINE_LENGTH
pennfat.c, 367
MAX_OPEN_FILES
fat_structs.h, 81
MAX_PROCESSES
kernel.h, 195
meta
spthread_st, 21
meta_mutex
spthread_meta_st, 19
MILISEC_IN_NANO
spthread.c, 383
mkfs
commands.c, 24
commands.h, 49
mode
global_fd_entry_t, 7
mount
commands.c, 25
commands.h, 50
mtime
directory_entry_t, 5
mv
shell_commands.c, 157
shell_commands.h, 166
my_meta
spthread.c, 395

name
busy_args_t, 4
directory_entry_t, 5
pcb, 12
next_pid
kernel.c, 193
nice_process
user_processes.c, 316
user_processes.h, 337
NO_FREE_FD
fat_structs.h, 81
num_children

pcb, 13
num_commands
parsed_command, 9
NUM_PRIORITY
scheduler.h, 276
num_stopped
pcb, 13
num_zombies
pcb, 13

offset
global_fd_entry_t, 7
orphan_child
user_processes.c, 317
user_processes.h, 338
orphanify
user_processes.c, 318
user_processes.h, 340

P_EXITSTATUS
pcb.h, 251
P_SIGCONT
pcb.h, 251
P_SIGSTOP
pcb.h, 251
P_SIGTERM
pcb.h, 251
P_WIFEXITED
pcb.h, 251
P_WIFSIGNALED
pcb.h, 252
P_WIFSTOPPED
pcb.h, 252
panic
panic.h, 378
panic.c
print_and_abort, 377
panic.h
panic, 378
print_and_abort, 378
parse_command
parser.c, 357
parser.h, 364
parsed_command, 8
commands, 9
is_background, 9
is_file_append, 9
num_commands, 9
stdin_file, 10
stdout_file, 10
parser.c
JUMP_OUT, 357
parse_command, 357
print_parsed_command, 359
print_parser_errcode, 359
skip_space, 360
skip_word, 361
parser.h
EXPECT_COMMANDS, 363
EXPECT_INPUT_FILENAME, 363
EXPECT_OUTPUT_FILENAME, 363
parse_command, 364
print_parsed_command, 365
print_parser_errcode, 366
UNEXPECTED_AMPERSAND, 363
UNEXPECTED_FILE_INPUT, 363
UNEXPECTED_FILE_OUTPUT, 363
UNEXPECTED_PIPELINE, 363
pcb, 10
children, 11
exit_status, 11
exit_statuses, 12
fd_table, 12
is_background, 12
is_waiting, 12
name, 12
num_children, 13
num_stopped, 13
num_zombies, 13
pending_signal, 13
pid, 13
ppid, 14
priority, 14
sleep_ticks_remaining, 14
state, 14
stopped_children, 15
thread, 15
thread_created, 15
waiting_for_pid, 15
zombie_children, 15
pcb.c
pcb_add_child, 241
pcb_add_stopped_child, 242
pcb_add_zombie, 243
pcb_create, 244
pcb_destroy, 245
pcb_get_zombie, 246
pcb_remove_child, 247
pcb_remove_stopped_child, 248
pcb.h
BLOCKED, 253
MAX_CHILDREN, 251
P_EXITSTATUS, 251
P_SIGCONT, 251
P_SIGSTOP, 251
P_SIGTERM, 251
P_WIFEXITED, 251
P_WIFSIGNALED, 252
P_WIFSTOPPED, 252

pcb_add_child, 253
pcb_add_stopped_child, 254
pcb_add_zombie, 255
pcb_create, 256
pcb_destroy, 258
pcb_get_zombie, 259
pcb_remove_child, 260
pcb_remove_stopped_child, 261
pcb_t, 252
process_state_t, 252
RUNNING, 253
STOPPED, 253
ZOMBIE, 253
pcb_add_child
pcb.c, 241
pcb.h, 253
pcb_add_stopped_child
pcb.c, 242
pcb.h, 254
pcb_add_zombie
pcb.c, 243
pcb.h, 255
pcb_create
pcb.c, 244
pcb.h, 256
pcb_destroy
pcb.c, 245
pcb.h, 258
pcb_get_zombie
pcb.c, 246
pcb.h, 259
pcb_remove_child
pcb.c, 247
pcb.h, 260
pcb_remove_stopped_child
pcb.c, 248
pcb.h, 261
pcb_t
pcb.h, 252
pending_signal
pcb, 13
pennfat.c
main, 368
MAX_LINE_LENGTH, 367
PROMPT, 367
pennos.c
init_process, 372
main, 371
perm
directory_entry_t, 5
global_fd_entry_t, 8
PERM_ALL
fat_structs.h, 81
PERM_NONE
fat_structs.h, 81
PERM_READ
fat_structs.h, 81
PERM_READ_EXEC
fat_structs.h, 82
PERM_READ_WRITE
fat_structs.h, 82
PERM_WRITE
fat_structs.h, 82
pid
pcb, 13
spawn_args_t, 16
ppid
pcb, 14
print_and_abort
panic.c, 377
panic.h, 378
print_directory_entry
k_functions.c, 107
k_functions.h, 127
print_fat_table
k_functions.c, 108
k_functions.h, 128
print_help
commands.c, 45
commands.h, 71
print_parsed_command
parser.c, 359
parser.h, 365
print_parser_errcode
parser.c, 359
parser.h, 366
priority
pcb, 14
priority_counter
scheduler.c, 274
priority_max_count
scheduler.c, 274
process_state_t
pcb.h, 252
process_table
kernel.c, 193
processes_queue
scheduler.c, 274
PROMPT
pennfat.c, 367
ps_process
user_processes.c, 319
user_processes.h, 342
pthread_fn
spthread.c, 384
ptr_dtor_fn
Vec.h, 413
ptr_t

Vec.h, 414

quantum
scheduler.c, 274

redirection.c
get_redirected_fds, 379
redirection.h
get_redirected_fds, 381
refcount
global_fd_entry_t, 8
reserved
directory_entry_t, 5
rm
shell_commands.c, 159
shell_commands.h, 167
run_shell
commands.h, 71
shell.c, 288
shell.h, 292
RUNNING
pcb.h, 253

s_close
s_functions.c, 131
s_functions.h, 143
s_exit
syscall.c, 295
syscall.h, 304
s_fprintf_stderr
s_functions.c, 132
s_functions.h, 144
s_functions.c
s_close, 131
s_fprintf_stderr, 132
s_ls, 134
s_lseek, 135
s_open, 136
s_printf, 137
s_read, 139
s_unlink, 140
s_write, 141
s_functions.h
s_close, 143
s_fprintf_stderr, 144
s_ls, 145
s_lseek, 146
s_open, 146
s_printf, 147
s_read, 148
s_unlink, 149
s_write, 150
s_kill
syscall.c, 296
syscall.h, 305
s_ls
s_functions.c, 134
s_functions.h, 145
s_lseek
s_functions.c, 135
s_functions.h, 146
s_nice
syscall.c, 297
syscall.h, 307
s_open
s_functions.c, 136
s_functions.h, 146
s_printf
s_functions.c, 137
s_functions.h, 147
s_read
s_functions.c, 139
s_functions.h, 148
s_sleep
syscall.c, 298
syscall.h, 308
s_spawn
syscall.c, 299
syscall.h, 309
s_unlink
s_functions.c, 140
s_functions.h, 149
s_waitpid
syscall.c, 301
syscall.h, 311
s_write
s_functions.c, 141
s_functions.h, 150
scheduler
scheduler.c, 266
scheduler.h, 277
scheduler.c
_DEFAULT_SOURCE, 264
_GNU_SOURCE, 264
_POSIX_C_SOURCE, 264
alarm_handler, 264
cancel_and_join, 264
done, 274
get_next_proc, 265
priority_counter, 274
priority_max_count, 274
processes_queue, 274
quantum, 274
scheduler, 266
scheduler_cleanup, 268
scheduler_init, 269
scheduler_insert, 269
scheduler_pop, 271
scheduler_remove, 271

scheduler_stop, 273
tick_count, 274
scheduler.h
get_next_proc, 276
NUM_PRIORITY, 276
scheduler, 277
scheduler_cleanup, 279
scheduler_init, 280
scheduler_insert, 280
scheduler_pop, 282
scheduler_remove, 282
scheduler_stop, 284
scheduler_cleanup
scheduler.c, 268
scheduler.h, 279
scheduler_init
scheduler.c, 269
scheduler.h, 280
scheduler_insert
scheduler.c, 269
scheduler.h, 280
scheduler_pop
scheduler.c, 271
scheduler.h, 282
scheduler_remove
scheduler.c, 271
scheduler.h, 282
scheduler_stop
scheduler.c, 273
scheduler.h, 284
setup_cond
spthread_fwd_args_st, 18
setup_done
spthread_fwd_args_st, 18
setup_mutex
spthread_fwd_args_st, 18
shell.c
execute_command, 285
run_shell, 288
shell_main, 290
shell.h
run_shell, 292
shell_main, 293
shell_commands.c
cat, 152
cmd_chmod, 153
cmd_ls, 154
cp, 156
mv, 157
rm, 159
touch, 160
shell_commands.h
cat, 162
cmd_chmod, 163
cmd_ls, 164
cp, 165
mv, 166
rm, 167
touch, 168
shell_main
commands.h, 73
shell.c, 290
shell.h, 293
shutup_mutex
spthread_signal_args_st, 20
SI_TKILL
spthread.c, 383
sigint_handler
commands.c, 46
commands.h, 76
sigint_received
commands.c, 47
commands.h, 77
signal
spthread_signal_args_st, 20
SIGPTHD
spthread.h, 397
sigpthd_handler
spthread.c, 385
size
directory_entry_t, 6
global_fd_entry_t, 8
skip_space
parser.c, 360
skip_word
parser.c, 361
sleep_ticks_remaining
pcb, 14
spawn_args_t, 16
argv, 16
func, 16
pid, 16
spawn_wrapper
syscall.c, 303
spthread.c
_GNU_SOURCE, 383
mark_self_terminated, 385
MILISEC_IN_NANO, 383
my_meta, 395
pthread_fn, 384
SI_TKILL, 383
sigpthd_handler, 385
spthread_cancel, 386
spthread_continue, 387
spthread_create, 388
spthread_disable_interrupts_self, 389
spthread_enable_interrupts_self, 389
spthread_equal, 390

spthread_exit, 390
spthread_fwd_args, 384
spthread_join, 391
spthread_meta_t, 385
SPTHREAD_RUNNING_STATE, 384
spthread_self, 392
SPTHREAD_SIG_CONTINUE, 384
SPTHREAD_SIG_SUSPEND, 384
spthread_signal_args, 385
spthread_start, 392
spthread_suspend, 393
spthread_suspend_self, 394
SPTHREAD_SUSPENDED_STATE, 384
SPTHREAD_TERMINATED_STATE, 384
spthread.h
SIGPTHD, 397
spthread_cancel, 397
spthread_continue, 398
spthread_create, 399
spthread_disable_interrupts_self, 400
spthread_enable_interrupts_self, 400
spthread_equal, 401
spthread_exit, 401
spthread_join, 402
spthread_meta_t, 397
spthread_self, 403
spthread_suspend, 403
spthread_suspend_self, 404
spthread_t, 397
spthread_cancel
spthread.c, 386
spthread.h, 397
spthread_continue
spthread.c, 387
spthread.h, 398
spthread_create
spthread.c, 388
spthread.h, 399
spthread_disable_interrupts_self
spthread.c, 389
spthread.h, 400
spthread_enable_interrupts_self
spthread.c, 389
spthread.h, 400
spthread_equal
spthread.c, 390
spthread.h, 401
spthread_exit
spthread.c, 390
spthread.h, 401
spthread_fwd_args
spthread.c, 384
spthread_fwd_args_st, 17
actual_arg, 17
actual_routine, 18
child_meta, 18
setup_cond, 18
setup_done, 18
setup_mutex, 18
spthread_join
spthread.c, 391
spthread.h, 402
spthread_meta_st, 19
meta_mutex, 19
state, 19
suspend_set, 19
spthread_meta_t
spthread.c, 385
spthread.h, 397
SPTHREAD_RUNNING_STATE
spthread.c, 384
spthread_self
spthread.c, 392
spthread.h, 403
SPTHREAD_SIG_CONTINUE
spthread.c, 384
SPTHREAD_SIG_SUSPEND
spthread.c, 384
spthread_signal_args
spthread.c, 385
spthread_signal_args_st, 20
ack, 20
shutup_mutex, 20
signal, 20
spthread_st, 21
meta, 21
thread, 21
spthread_start
spthread.c, 392
spthread_suspend
spthread.c, 393
spthread.h, 403
spthread_suspend_self
spthread.c, 394
spthread.h, 404
SPTHREAD_SUSPENDED_STATE
spthread.c, 384
spthread_t
spthread.h, 397
SPTHREAD_TERMINATED_STATE
spthread.c, 384
src/FAT/commands.c, 23
src/FAT/commands.h, 48
src/FAT/fat_structs.h, 78
src/FAT/fd_table.c, 83
src/FAT/fd_table.h, 86
src/FAT/k_functions.c, 90
src/FAT/k_functions.h, 110

src/FAT/s_functions.c, 130
src/FAT/s_functions.h, 143
src/FAT/shell_commands.c, 151
src/FAT/shell_commands.h, 161
src/kernel/commands.c, 29
src/kernel/commands.h, 54
src/kernel/init.c, 170
src/kernel/init.h, 173
src/kernel/kernel.c, 176
src/kernel/kernel.h, 194
src/kernel/logging.c, 212
src/kernel/logging.h, 226
src/kernel/pcb.c, 240
src/kernel/pcb.h, 249
src/kernel/scheduler.c, 262
src/kernel/scheduler.h, 275
src/kernel/shell.c, 285
src/kernel/shell.h, 292
src/kernel/syscall.c, 294
src/kernel/syscall.h, 304
src/kernel/user_processes.c, 313
src/kernel/user_processes.h, 333
src/parser/parser.c, 356
src/parser/parser.h, 361
src/pennfat.c, 367
src/pennos.c, 370
src/utils/input_processing.c, 373
src/utils/input_processing.h, 374
src/utils/panic.c, 376
src/utils/panic.h, 377
src/utils/redirection.c, 378
src/utils/redirection.h, 380
src/utils/spthread.c, 382
src/utils/spthread.h, 396
src/utils/Vec.c, 405
src/utils/Vec.h, 412
state
    pcb, 14
    spthread_meta_st, 19
stdin_file
    parsed_command, 10
stdout_file
    parsed_command, 10
STOPPED
    pcb.h, 253
stopped_children
    pcb, 15
suspend_set
    spthread_meta_st, 19
syscall.c
    s_exit, 295
    s_kill, 296
    s_nice, 297
    s_sleep, 298
    s_spawn, 299
    s_waitpid, 301
    spawn_wrapper, 303
syscall.h
    s_exit, 304
    s_kill, 305
    s_nice, 307
    s_sleep, 308
    s_spawn, 309
    s_waitpid, 311
system_running
    init.c, 173

thread
    pcb, 15
    spthread_st, 21
thread_created
    pcb, 15
tick_count
    scheduler.c, 274
touch
    shell_commands.c, 160
    shell_commands.h, 168
type
    directory_entry_t, 6
TYPE_DIRECTORY
    fat_structs.h, 82
TYPE_REGULAR
    fat_structs.h, 82
TYPE_SYMLINK
    fat_structs.h, 82
TYPE_UNKNOWN
    fat_structs.h, 83

UNEXPECTED_AMPERSAND
    parser.h, 363
UNEXPECTED_FILE_INPUT
    parser.h, 363
UNEXPECTED_FILE_OUTPUT
    parser.h, 363
UNEXPECTED_PIPELINE
    parser.h, 363
unmount
    commands.c, 26
    commands.h, 51
update_directory_entry
    k_functions.c, 108
    k_functions.h, 128
user_busy
    user_processes.c, 320
    user_processes.h, 343
user_cat
    user_processes.c, 321
    user_processes.h, 344
user_chmod

user_processes.c, 322
user_processes.h, 345
user_cp
user_processes.c, 323
user_processes.h, 346
user_echo
user_processes.c, 324
user_processes.h, 346
user_ls
user_processes.c, 325
user_processes.h, 348
user_man
user_processes.c, 326
user_processes.h, 349
user_mv
user_processes.c, 327
user_processes.h, 350
user_processes.c
counting_process, 314
hello_process, 315
nice_process, 316
orphan_child, 317
orphanify, 318
ps_process, 319
user_busy, 320
user_cat, 321
user_chmod, 322
user_cp, 323
user_echo, 324
user_ls, 325
user_man, 326
user_mv, 327
user_rm, 328
user_sleep, 329
user_touch, 330
zombie_process, 331
zombify_parent_process, 332
user_processes.h
counting_process, 335
hello_process, 336
nice_process, 337
orphan_child, 338
orphanify, 340
ps_process, 342
user_busy, 343
user_cat, 344
user_chmod, 345
user_cp, 346
user_echo, 346
user_ls, 348
user_man, 349
user_mv, 350
user_rm, 351
user_sleep, 352
user_touch, 353
zombie_process, 354
zombify_parent_process, 355
user_rm
user_processes.c, 328
user_processes.h, 351
user_sleep
user_processes.c, 329
user_processes.h, 352
user_touch
user_processes.c, 330
user_processes.h, 353

Vec
Vec.h, 414
Vec.c
vec_clear, 406
vec_destroy, 407
vec_erase, 407
vec_get, 408
vec_insert, 409
vec_new, 409
vec_pop_back, 410
vec_push_back, 410
vec_resize, 411
vec_set, 411
Vec.h
ptr_dtor_fn, 413
ptr_t, 414
Vec, 414
vec_capacity, 413
vec_clear, 414
vec_destroy, 414
vec_erase, 415
vec_get, 416
vec_insert, 417
vec_is_empty, 413
vec_len, 413
vec_new, 417
vec_pop_back, 418
vec_push_back, 418
vec_resize, 419
vec_set, 419
vec_capacity
Vec.h, 413
vec_clear
Vec.c, 406
Vec.h, 414
vec_destroy
Vec.c, 407
Vec.h, 414
vec_erase
Vec.c, 407
Vec.h, 415

vec_get
    Vec.c, 408
    Vec.h, 416
vec_insert
    Vec.c, 409
    Vec.h, 417
vec_is_empty
    Vec.h, 413
vec_len
    Vec.h, 413
vec_new
    Vec.c, 409
    Vec.h, 417
vec_pop_back
    Vec.c, 410
    Vec.h, 418
vec_push_back
    Vec.c, 410
    Vec.h, 418
vec_resize
    Vec.c, 411
    Vec.h, 419
vec_set
    Vec.c, 411
    Vec.h, 419
vec_st, 22
    capacity, 22
    data, 22
    ele_dtor_fn, 22
    length, 23

waiting_for_pid
    pcb, 15

ZOMBIE
    pcb.h, 253
zombie_children
    pcb, 15
zombie_process
    user_processes.c, 331
    user_processes.h, 354
zombify_parent_process
    user_processes.c, 332
    user_processes.h, 355